

Skript zur Veranstaltung

84171 Visualisierung

mit Type-/JavaScript und D3

von

Prof. Jürgen Singer, PhD/USA

im Bachelor-Studiengang

Medieninformatik



im Fachbereich Automatisierung und Informatik

▲ Hochschule Harz

Stand: Frühjahr 2026

Inhaltsverzeichnis

I. Grundlagen der Visualisierung	1
1. Einführung	1
1.1. Was ist Visualisierung	1
1.2. Gründe für Visualisierung	2
1.2.1. Datenexplosion	3
1.2.2. Industrielle Datenrevolution	4
1.2.3. Kognitive Beschränkung	4
1.2.4. Ziele von Visualisierung	5
1.3. Beispielvideo	8
2. Entwurf	9
2.1. Grafische Integrität	9
2.2. Entwurfsprinzipien	14
2.3. Subjektive Dimensionen	15
2.4. Entwurfselemente	19
2.4.1. Kleine Vielfache	19
2.4.2. Dimensionenreduktion	20
2.4.3. Räumlich- und Zeitliche Narration	21
2.5. Analyse der Darstellung	21
2.6. Kategorisierung von Variablen	22
2.6.1. Modellbildung	22
2.6.2. Semiotik	22
2.6.3. Designraum	23
2.6.4. Datentypen	23
2.6.5. Kanalrangfolge	24
2.6.6. Taxinomie	24
2.6.7. Datenmodell vs. konzeptionelles Modell	25
2.7. Beispielvideo	27
3. Wahrnehmung	29
3.1. Das Auge	29
3.2. Wahrnehmung	32
3.2.1. Folgen der Anordnung der Sehrezeptoren	32
3.2.2. Mustererkennung	35
3.3. Visuelle Pfade	37
3.3.1. Gehirregionen	38
3.3.2. Auge und Gehirn	41
3.4. Preattentive Processing	43
3.4.1. Form- und Farbauswahl	43
3.4.2. Mehrfachkanäle	44
3.5. Multiskaleneffekte	46
3.5.1. Visuelle Suchstrategien	46
3.5.2. Suchhierarchie	47
3.5.3. Merkmalskontrast	48
3.5.4. Wahrnehmungshierarchie	48
3.6. Beispielvideos	49

II. Full-Stack JavaScript	50
4. Webprogrammierung	50
4.1. Klassisches Backend	50
4.2. Frontend-Technologien	51
4.3. Single-Page-Applications	51
5. Einführung in Node/Bun	52
5.1. Installation	52
5.2. Module	53
5.2.1. CommonJS-Module	54
5.2.2. Core-Module	54
5.2.3. Eigene ES6-Module	55
5.2.4. Externe Module	56
5.2.5. Infrastruktur	57
5.3. Bun-Spezifika	59
5.4. Bundler	59
5.5. Das Vite-Modul	60
5.6. Das ViewModel Framework Vue	61
5.6.1. Scaffolding	62
5.6.2. Die Dateien einer Vue-Anwendung	62
6. JavaScript und TypeScript Grundlagen	65
6.1. Datentypen	65
6.2. Klassen	74
6.3. Ausnahmen	75
6.4. Funktionen	77
6.4.1. Reguläre Funktionen	77
6.4.2. Closures	77
6.4.3. Anonyme Funktionen	78
6.4.4. Dynamische Erzeugung von Funktionen	79
6.4.5. Iteratoren	80
6.4.6. Generatoren	82
6.5. Asynchrones JavaScript	83
6.5.1. Callbacks	83
6.5.2. Promises	84
6.5.3. async/await	85
6.6. Globale Umgebungen für JavaScript	86
6.6.1. Browser und das Document Object Model	88
6.6.2. Virtuelles DOM	91
7. JavaScript-Backend	92
7.1. Das Express-Modul	92
7.1.1. Erste Express Installation	92
7.1.2. Starten des Servers	93
7.1.3. Server-Konfiguration	94
7.2. Nuxt.js	95

8. Frontend-Entwicklung mit Vue	97
8.1. Entwicklungsumgebung	97
8.1.1. Vue CLI	97
8.1.2. Vite	97
8.1.3. Visual Studio Code	97
8.2. Aufsetzen eines Vue-Projekts	97
8.3. Vue-Komponenten	99
8.3.1. Erstellung	99
8.3.2. Vue-Hooks und weitere Direktiven	103
8.4. Vue Router	105
8.4.1. Installation	105
8.4.2. Grundlagen	106
8.5. D3 in Vue-Komponenten	109
9. Full-Stack Vue mit Nuxt	111
9.1. Installation	111
9.2. Nuxt-Ordnerstruktur	111
9.3. Zustandsverwaltung mit Pinia	112
9.3.1. Vorteile eines Store-Plugin	112
9.3.2. Installation in einem Vue-Projekt	113
9.3.3. Konfiguration in einem Nuxt-Projekt	113
9.3.4. Erstellen eines Stores	113
9.3.5. Verwendung des Stores in Komponenten	114
9.4. HTTP-Anfragen mit Axios	114
9.4.1. Axios vs. fetch	114
9.4.2. Installation in einem Vue-Projekt	114
9.4.3. Verwendung in Komponenten	115
9.4.4. Installation in einem Nuxt-Projekt	115
9.4.5. Verwendung in Komponenten	115
9.4.6. Verwendung von Interceptors	116
III. 2d-Visualisierung im Frontend mit D3	117
10. Einführung in D3.js	117
10.1. Kreis mit D3 in einer Nuxt-Anwendung	117
10.2. Balkendiagramm mit D3	120
11. Umgang mit Daten in D3	127
11.1. D3-Daten und das DOM	127
11.2. Dynamische Visualisierungen	128
11.2.1. Dynamik mit enter und exit	128
11.2.2. Dynamik mit join	128
11.3. Ausführliches komponentenbasiertes Beispielprojekt	129
11.3.1. Erstellung der Layouts	129
11.3.2. Erstellung der Indexseite	132
11.3.3. Die Shuffle-Komponenten	134
11.4. Animation mit Transitions	146
11.4.1. Kontinuierliche Übergänge	146
11.4.2. Chart-Auswahl	146

11.4.3. Template für Chart-Komponenten	149
11.4.4. Styles für die Chart-Komponenten	150
11.4.5. Basisklasse zur Chart-Animation	152
11.4.6. Funktion zur Erzeugung der Chart-Komponenten	154
11.5. Diagramm-Implementierung	155
11.5.1. Animiertes Balkendiagramm	155
11.5.2. Animiertes Liniendiagramm	160
11.5.3. Animiertes Tortendiagramm	167
11.5.4. Animierter Donut Chart	172
11.6. Scatterplots	177
11.6.1. Einbindung	177
11.6.2. Einfacher Scatterplot	180
11.6.3. Bubblechart	185
11.6.4. Scattermatrix	191
11.7. Darstellung hierarchischer Daten	195
11.7.1. Einbindung	195
11.7.2. Einfache Baumstruktur mit Zoom und Panning	196
11.7.3. Radialer Cluster	202
11.7.4. Treemap-Layout	205
11.7.5. Circle Packing	208
11.7.6. Force-Layout	210
11.7.7. Sunburst-Diagramm	215
11.8. Darstellung von Landkarten	220
11.8.1. Einbindung	220
11.8.2. Projektionen	221
11.8.3. Choropleth-Karten	226
11.8.4. Bubble-Maps	231
11.8.5. Deutsche Landkreise	236
Literatur	239
Anhang	243
A. Scalable Vector Graphics	243
A.1. Grundstruktur von SVG	243
A.2. Gestaltung mit CSS	243
A.3. Gruppierungen und Transformationen	244
A.4. Fortgeschrittene SVG-Techniken	245
A.4.1. Lineare Gradienten	245
A.4.2. Radiale Gradienten	245
A.4.3. Masken	246
A.4.4. Filter	247
A.5. Erstellung von Linienzügen	247
A.5.1. Polyline	247
A.5.2. Pfade	247
A.5.3. Kreissegment	248
A.5.4. Ellipse	249
A.5.5. Kreisbögen mit Arc-Anweisungen	249
A.5.6. Splines und Bézierkurven	250

A.5.7. Quadratische Bézierkurven	250
A.5.8. Geschlossene und gefüllte Linienzüge	251
B. Web Scraping mit Puppeteer	252
B.1. Einführung	252
B.1.1. Definiton und Zweck von Web-Scraping	252
B.1.2. Ethische und rechtliche Aspekte	252
B.2. Was ist Puppeteer?	252
B.2.1. Vorteile und Unterschied	253
B.2.2. Installation und Setup	253
B.2.3. Erste Schritte und grundlegende Konfiguration	253
B.2.4. Grundlagen von Puppeteer	253
B.2.5. Web Scraping mit Puppeteer	254
B.3. Datenaufbereitung für Visualisierungen	254
C. Testing	257
C.1. JavaScript Testing Frameworks	257
D. Geodaten in D3.js	258
D.1. GIS Dateiformate	258
D.1.1. GeoJSON	258
D.1.2. TopoJSON	258
D.1.3. Shapefile	259
D.2. GDAL	260
D.3. Umwandlung GeoJSON nach TopoJSON	260

Teil I.

Grundlagen der Visualisierung

1. Einführung

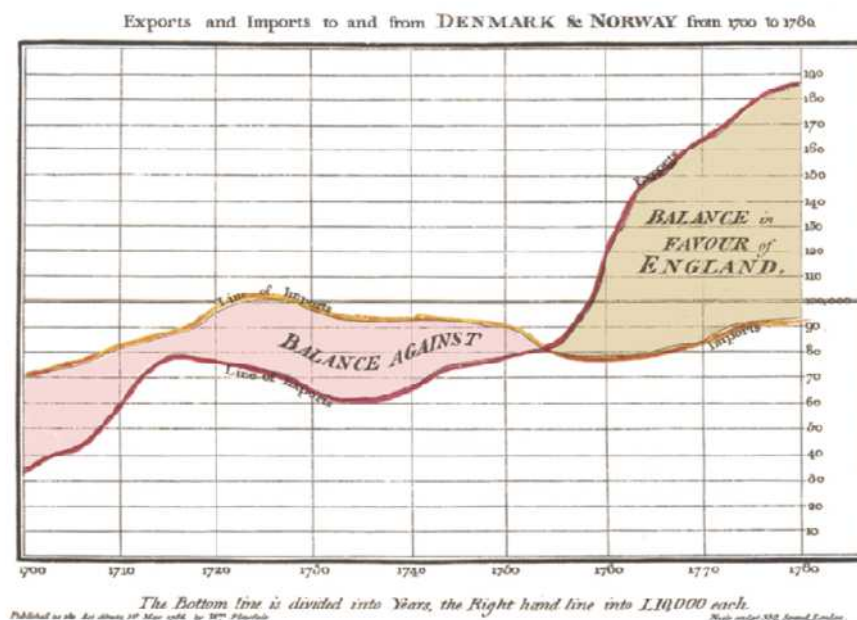
1.1. Was ist Visualisierung

Was ist Visualisierung

Visualisierung

1. Ein mentales Bild formen
2. Sichtbar machen
3. Information durch visuelle Darstellung vermitteln

Beispiel 1: William Playfair Zeitreihe



Beispiele



1.2.1. Datenexplosion



1981



2020



1.2.2. Industrielle Datenrevolution

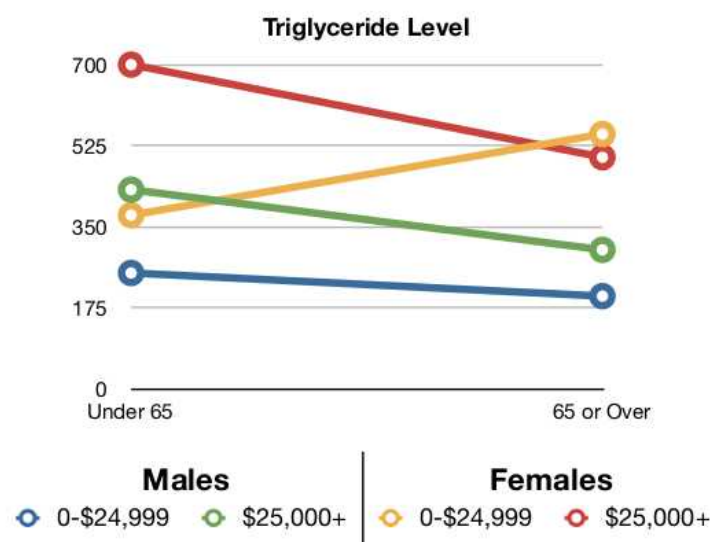


1.2.3. Kognitive Beschränkung

Welche Einkommens- oder Geschlechtergruppe zeigt unterschiedliche Alterseffekte hinsichtlich der Triglycerid-Werte?

Triglycerid-Werte	Männer		Frauen	
Einkommen	unter 65	über 65	unter 65	über 65
unter 25000 \$	250	200	375	550
über 25000 \$	430	300	700	500

Visuelle Darstellung



Visualisierung

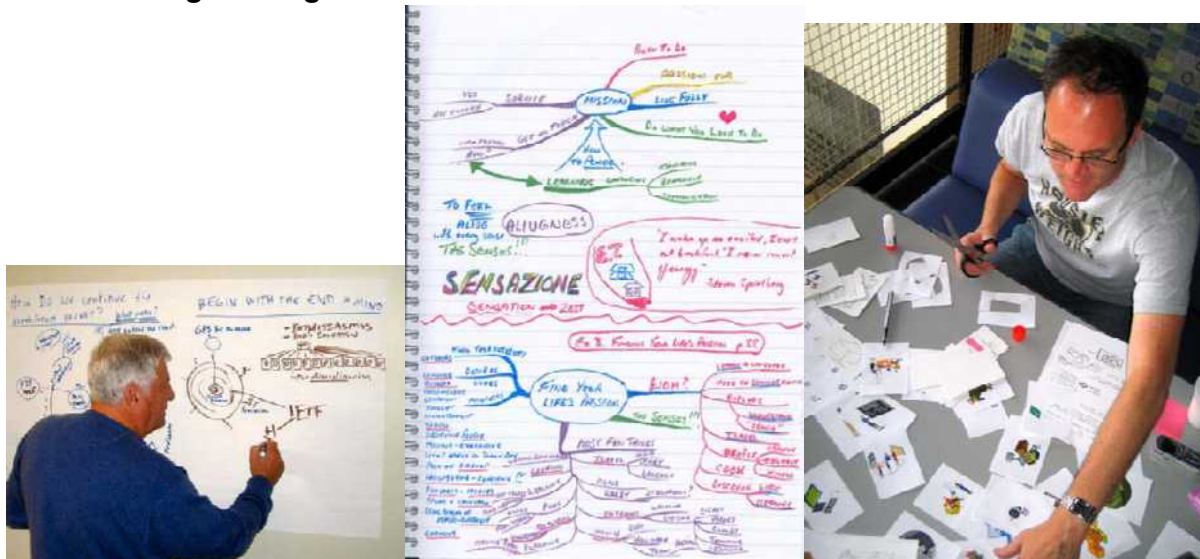
- hilft beim denken
- vermindert die Gedächtnislast
- verringert den Kognitionsbedarf
- nutzt menschliche Wahrnehmung

1.2.4. Ziele von Visualisierung

Visualisierung

- unterstützt die Kognition
- beantwortet Fragen
- deckt Muster auf
- speichert
- stellt klar
- kommuniziert
- inspiriert

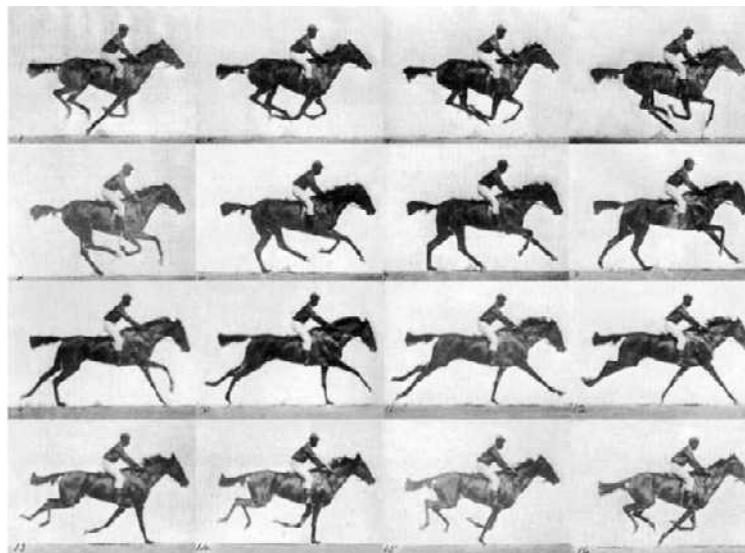
Unterstützung der Kognition



IBM Visualisation Tool ManyEyes (geschlossen 2015)

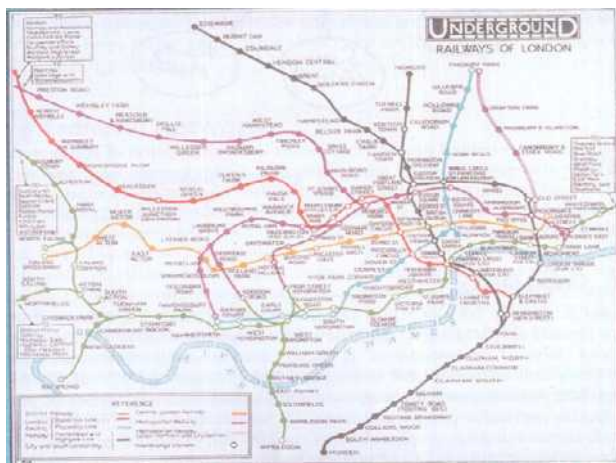


Speicherung



Edward Muybridge, 1878

Klarstellung



Plan der Londoner U-Bahn 1927



Plan der Londoner U-Bahn 1933 (Harry Beck)

1.3. Beispielvideo

Inspiration



Hans Rosling, TED 2006 (YouTube) <https://youtu.be/RUwS1uAdUcI>

Veranstaltungsziele

- Grundlagen Entwurf und Wahrnehmung
- Auswahl an Visualisierungsmethoden
- Anwendung von Visualisierung
- Entwurf eines Interaktiven Visualisierungssystems

2. Entwurf

Inhalt

- Grafische Integrität
- Entwurfsprinzipien
- Entwurfselemente
- Modellbildung
- Semiotik
- Datentypen
- Taxinomien
- Datenmodell

2.1. Grafische Integrität

Beispiel 4: Fehlende Skalen

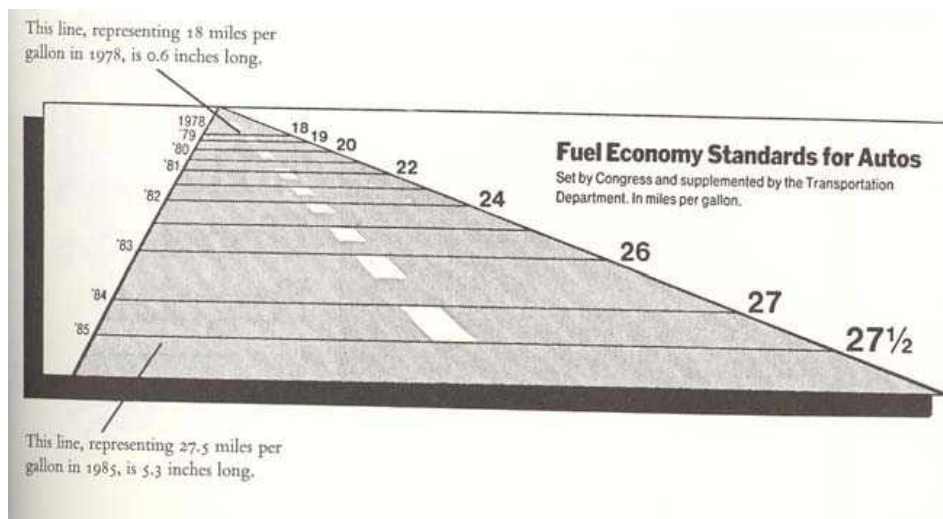
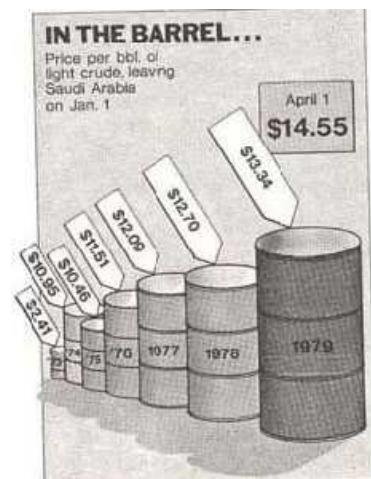
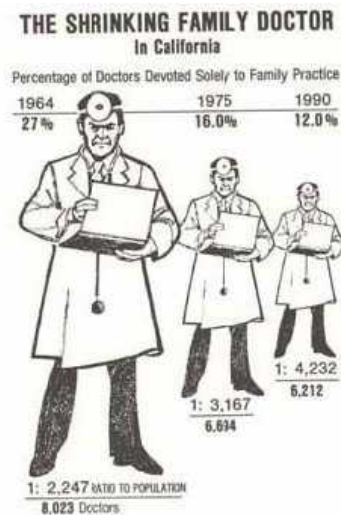


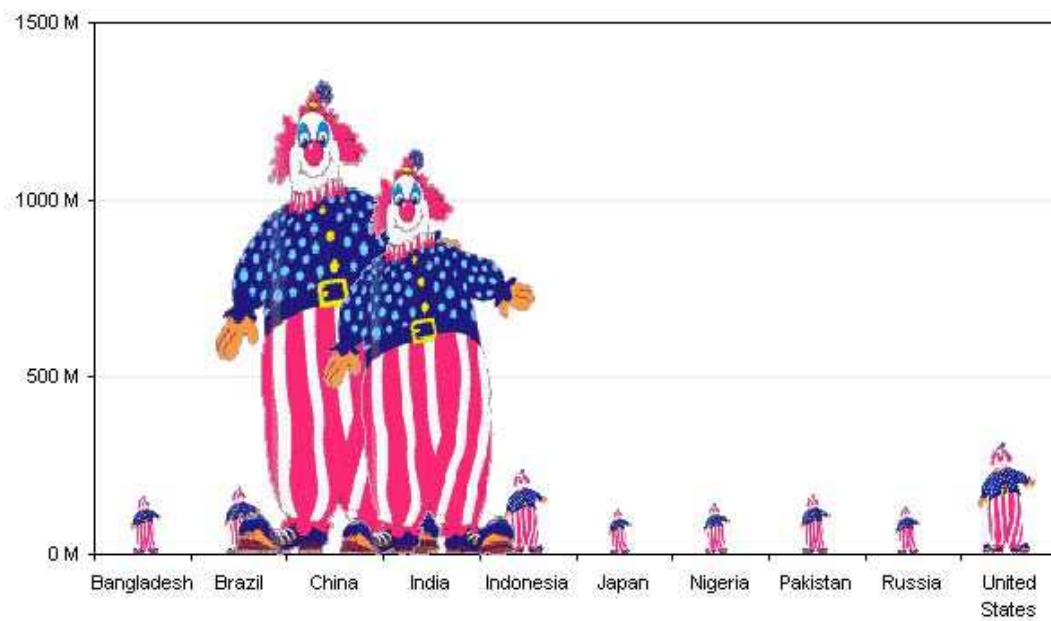
Definition 1: Der Lügenfaktor

$$Lf = \frac{\text{rel. Effekt in der Grafik}}{\text{relativer Effekt in den Daten}}$$

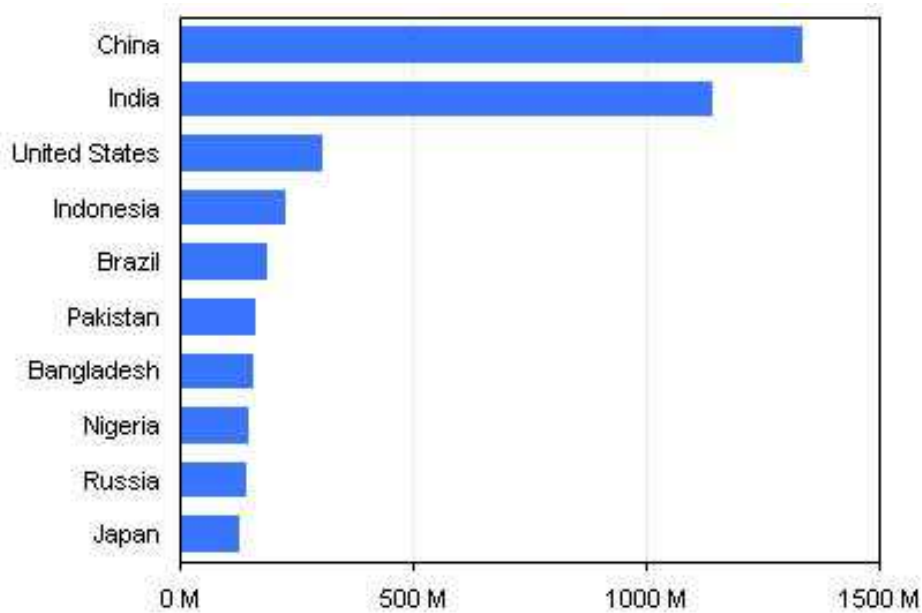
Beispiel 5: Lügenfaktor

$$Lf = \frac{\frac{5.3-0.6}{0.6}}{\frac{27.5-18}{18}} = 14.8$$

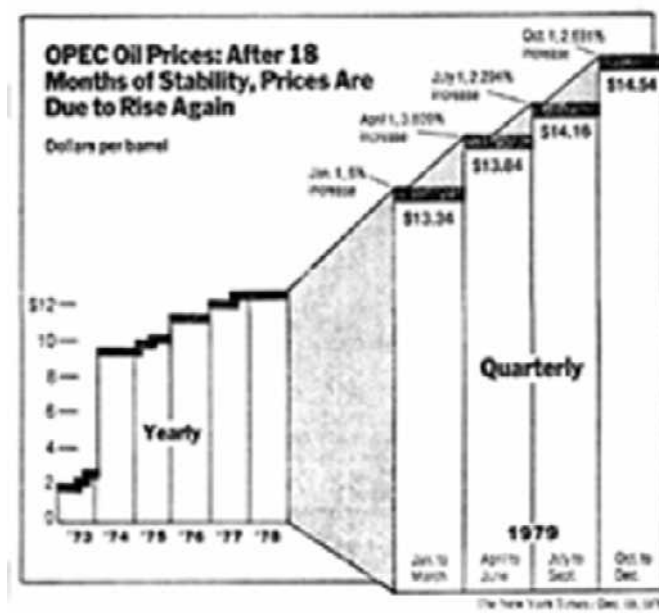
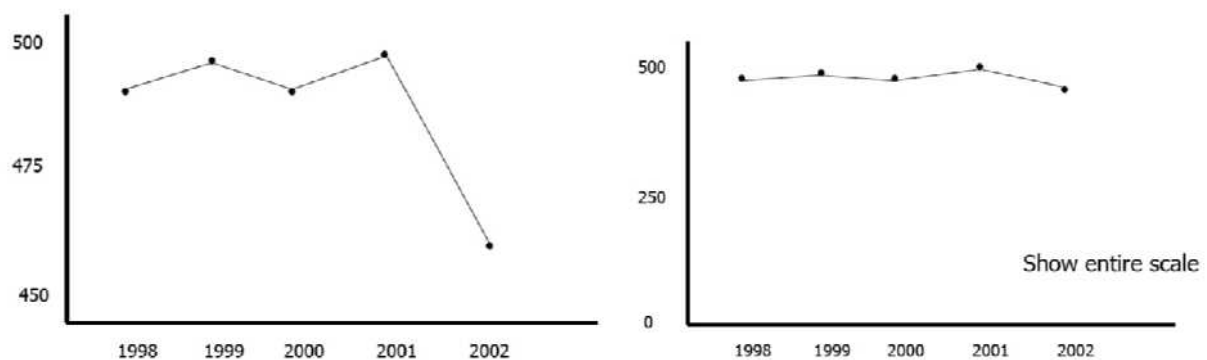
**Beispiel 6: Lügenfaktor**

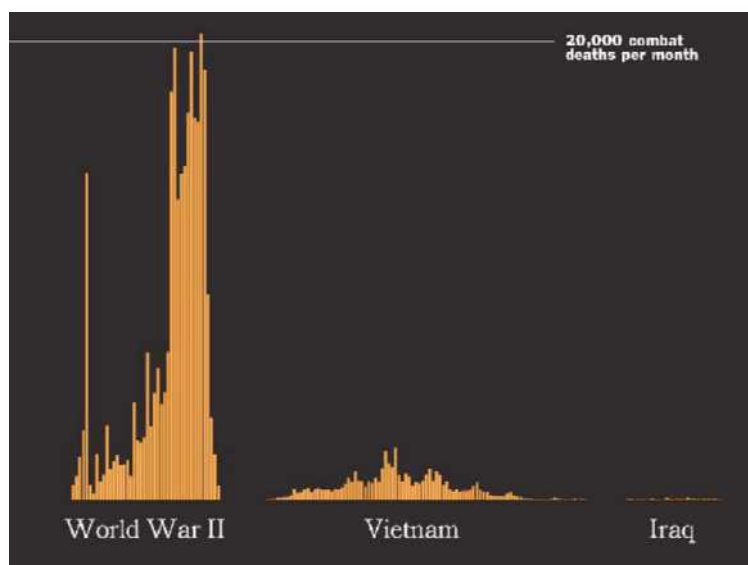
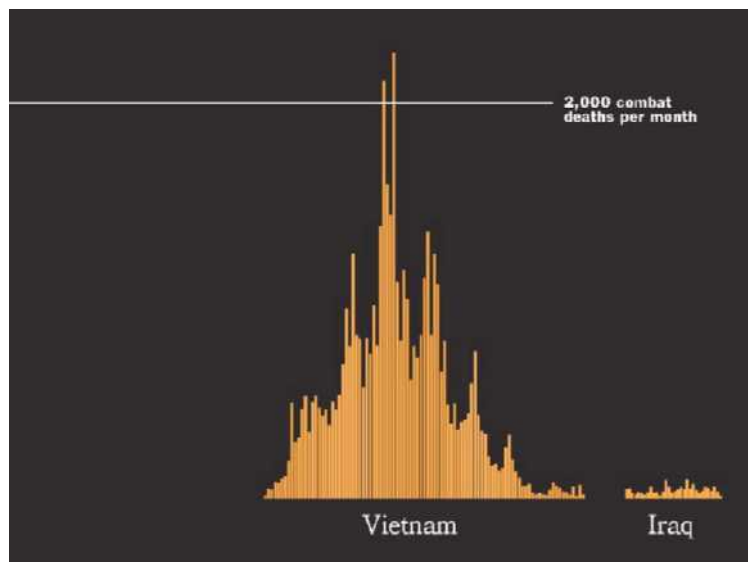
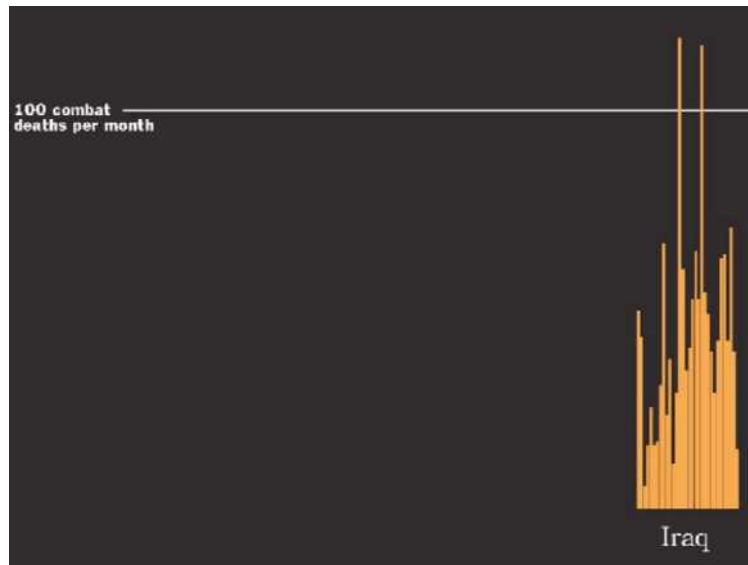
Beispiel 7: Bring in the Clowns

Weltbevölkerung 2008

Bessere Version

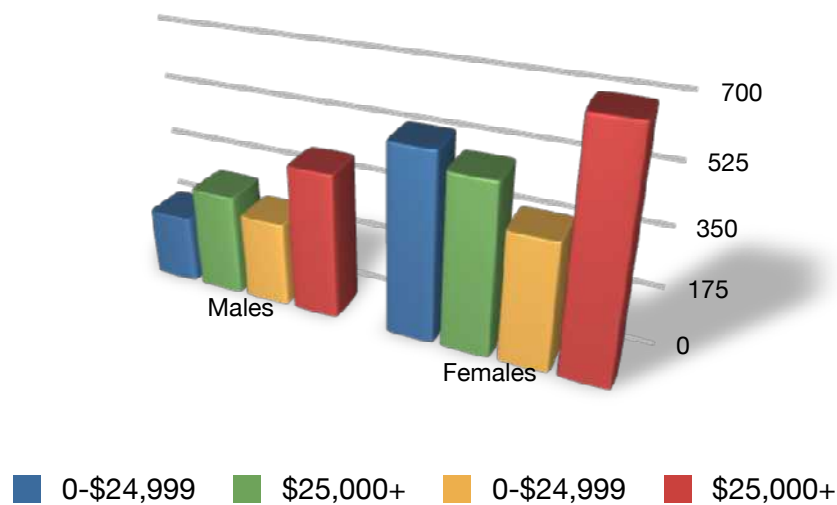
Weltbevölkerung 2008

Beispiel 8: Design-Verzerrung**Beispiel 9: Skalenverzerrung**

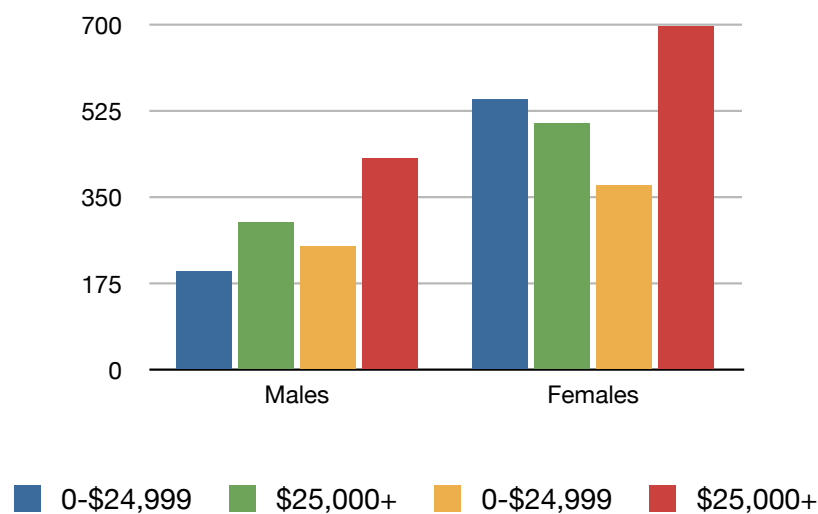
Beispiel 10: Kontext

2.2. Entwurfsprinzipien

- Vor allem Anderen zeige die Daten
- Maximiere das Daten-zu-Tinte Verhältnis
- Lösche Tinte ohne zugehörige Daten
- Lösche redundante Datentinte
- revidiere und redigiere
- Datentinte = Tinte genutzt um Daten zu zeigen
- Datentintenverhältnis = Datentinte / gesamte Tinte



- Datentinte = Tinte genutzt um Daten zu zeigen
- Datentintenverhältnis = Datentinte / gesamte Tinte



2.3. Subjektive Dimensionen

- Ästhetik: Attraktive werden als nützlicher wahrgenommen als unattraktive
- Stil: kommuniziert Marke, Prozess, Designer
- Verspieltheit: Fördert Experimente und Erforschung
- Lebhaftigkeit: prägt das Bild besser ein

Definition 2: Datendichte

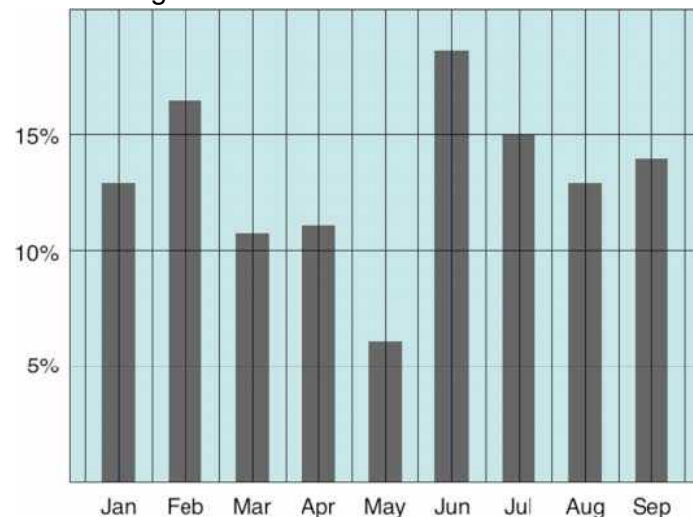
Datendichte = Anzahl der Dateneinträge / Fläche der Datengrafik

Beispiel 11: Sparklines (Tufte 1990)

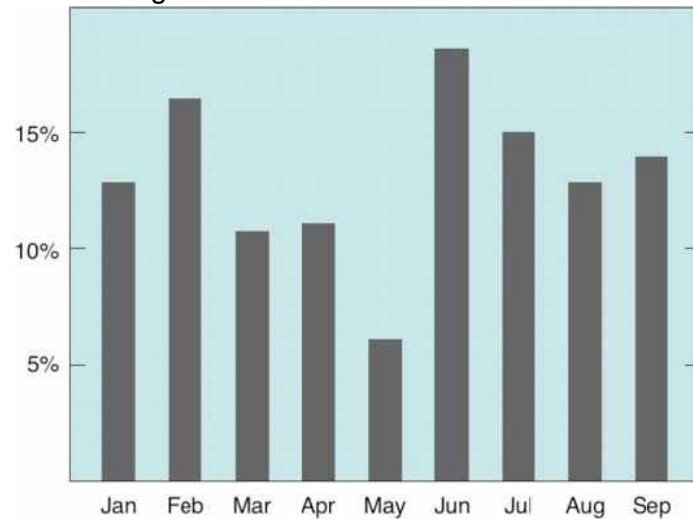


Beispiel 12: Vermeide Ausschuss

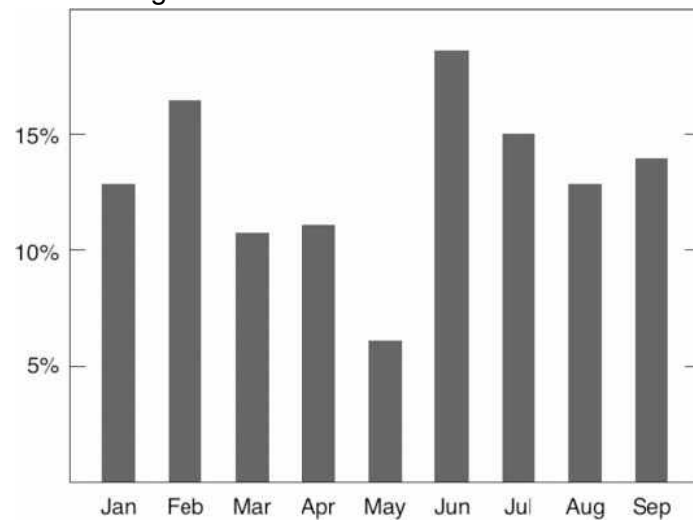
Überflüssige visuelle Elemente lenken vom Inhalt ab



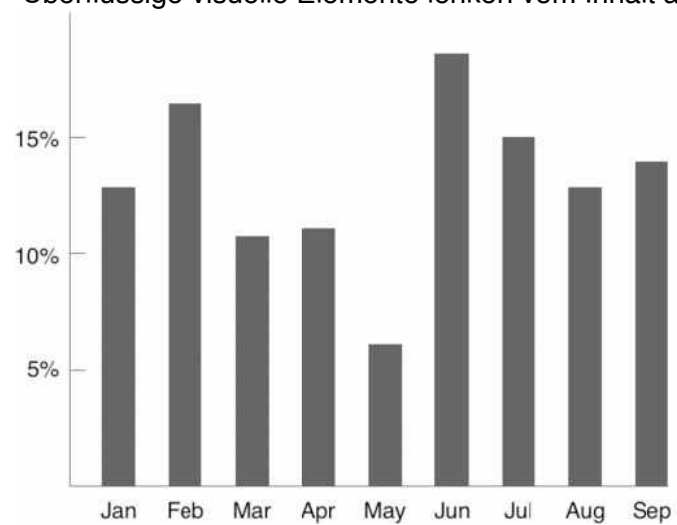
Überflüssige visuelle Elemente lenken vom Inhalt ab



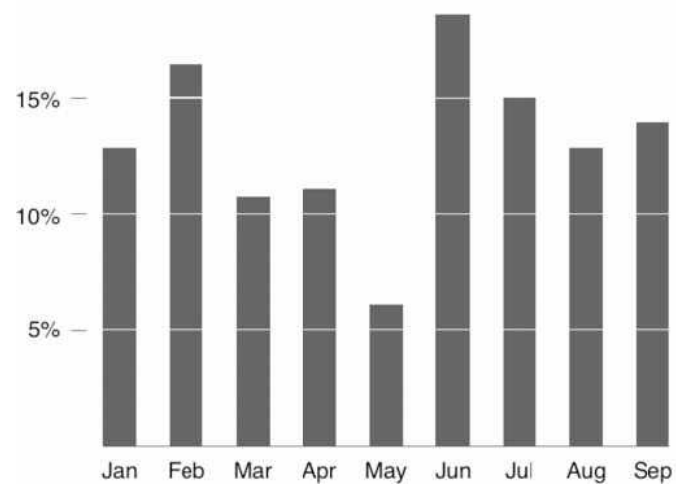
Überflüssige visuelle Elemente lenken vom Inhalt ab



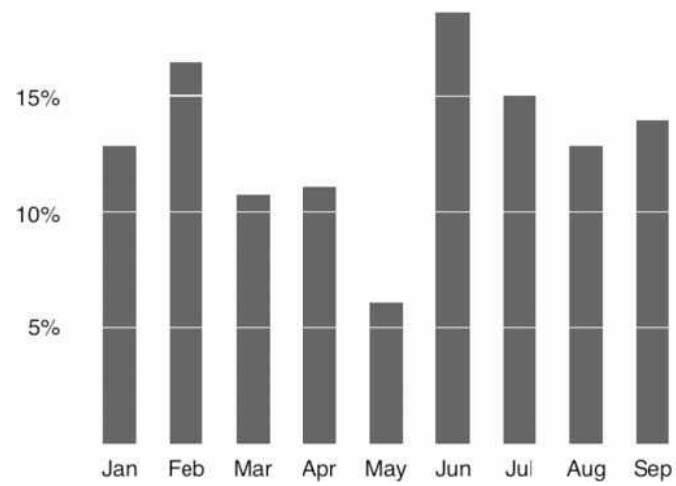
Überflüssige visuelle Elemente lenken vom Inhalt ab

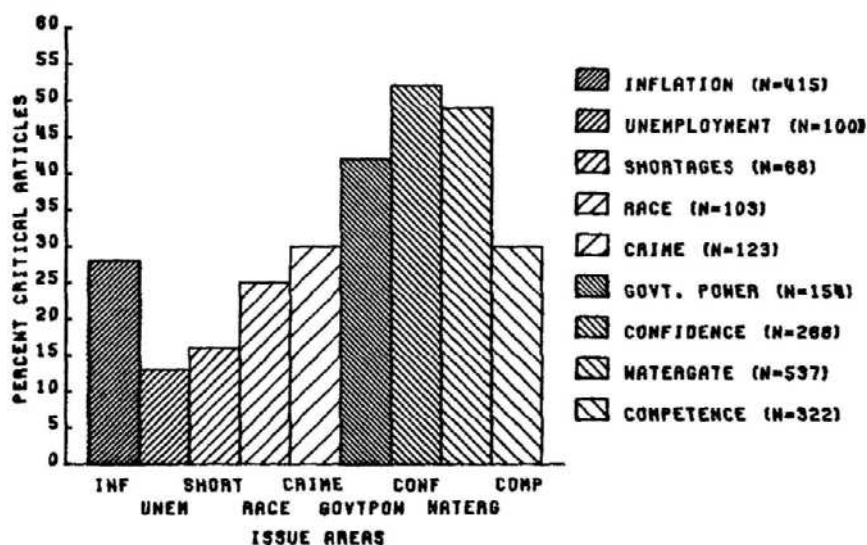


Überflüssige visuelle Elemente lenken vom Inhalt ab



Überflüssige visuelle Elemente lenken vom Inhalt ab



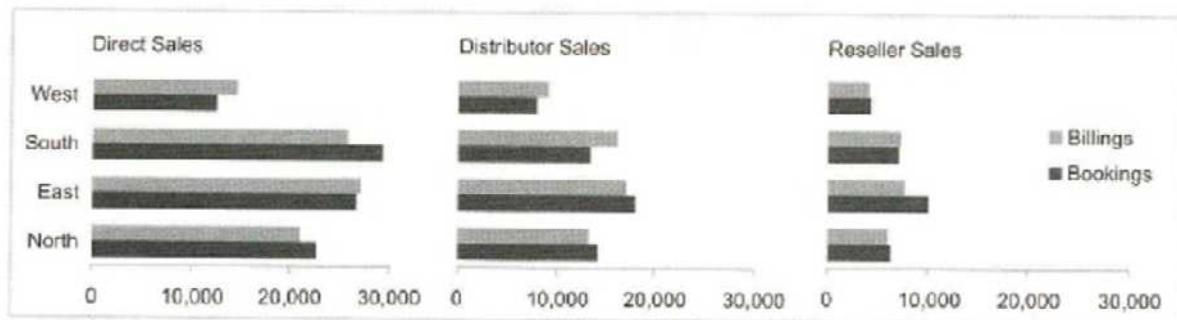
Beispiel 13: Eliminiere den Graphen

Der Graph ist überflüssig und verwirrend.

Content and tone of front-page articles in 94 U. S. newspapers, October and November, 1974	Number of articles	Percent of articles with negative criticism of specific person or policy
Watergate: defendants and prosecutors, Ford's pardon of Nixon	537	49%
Inflation, high cost of living	415	28%
Government competence: costs, quality, salaries of public employees	322	30%
Confidence in government: power of special interests, trust in political leaders, dishonesty in politics	266	52%
Government power: regulation of business, secrecy, control of CIA and FBI	154	42%
Crime	123	30%
Race	103	25%
Unemployment	100	13%
Shortages: energy, food	68	16%

2.4. Entwurfselemente

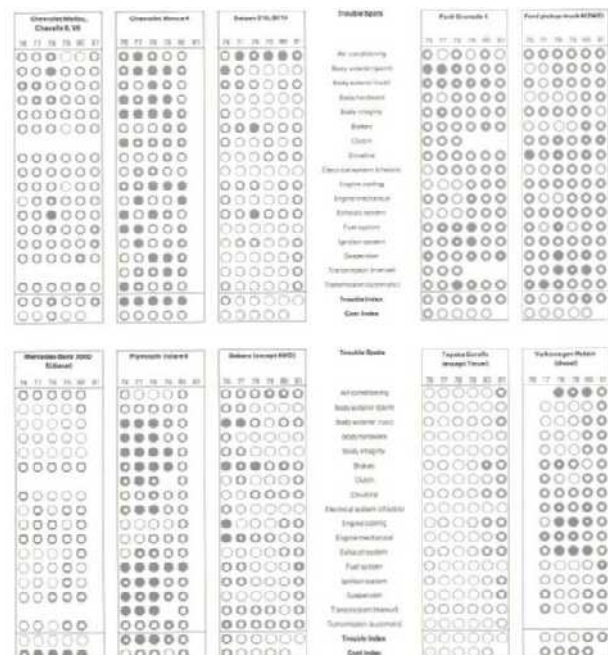
2.4.1. Kleine Vielfache



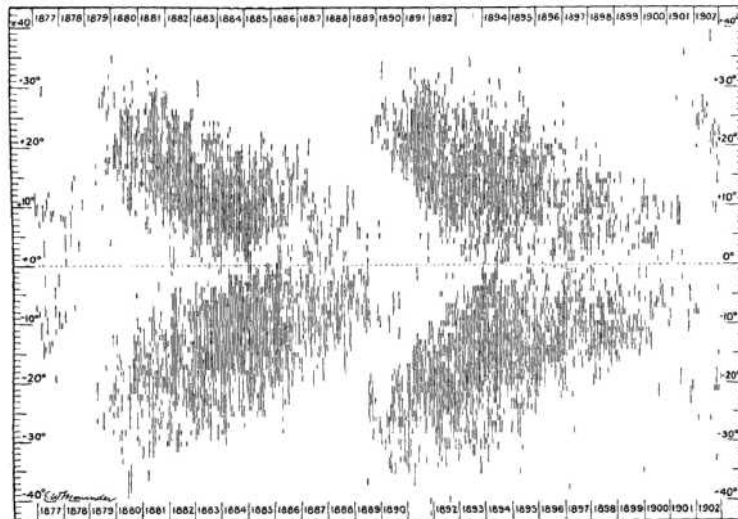
Anordnung nebeneinander erlaubt einfache Vergleiche der Daten auf der vertikalen Achse (hier Regionen).

Reperaturhäufigkeit verschiedener Komponenten von Fahrzeugen nach Automarke in den Jahren 1976-81.

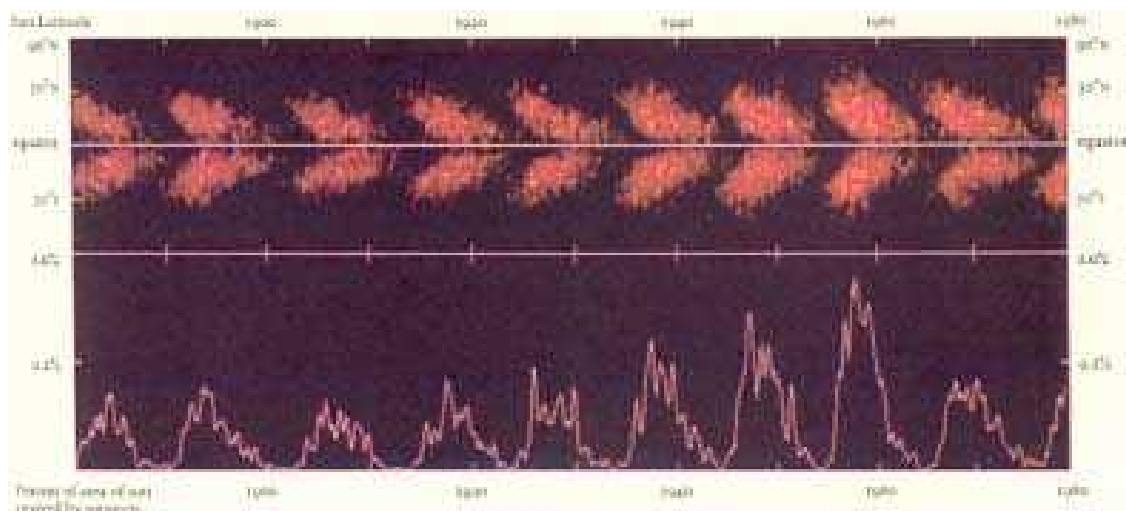
- Leerer Kreis: Viel besser als der Durchschnitt
- Kreis mit geringem Rand: Besser als der Durchschnitt
- Kreis mit mittlerem Rand: Durchschnitt
- Kreis mit dickem Rand: Schlechter als der Durchschnitt
- ausgefüllter Kreis: Viel schlechter als der Durchschnitt



2.4.2. Dimensionenreduktion

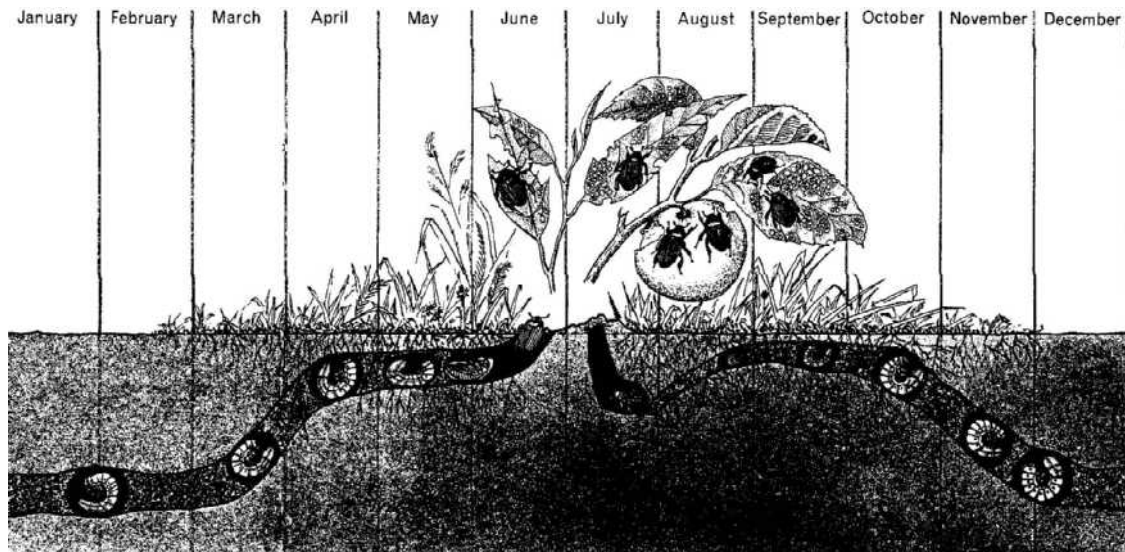


Sonnenflecken 1874 - 1902. Lediglich die vertikale Position ist notiert, die horizontale Ausdehnung wird ignoriert.



Oben: Sonnenflecken von 1880 - 1980. Die horizontale Position ist ignoriert. Farbe: Logarithmus der Fläche der Sonnenflecken. Darunter: Zeitserie durch Summation: Anteil der Sonnenflecken an der gesamten Sonnenoberfläche.

2.4.3. Räumlich- und Zeitliche Narration



Lebenszyklus des Kartoffelkäfers

2.5. Analyse der Darstellung

Grafische Darstellungen sollten ...

- die Daten zeigen
- die Aussage der Daten nicht verfälschen
- zu Vergleichen ermuntern
- die Daten auf mehreren Ebenen deutlich machen
- einem klaren Zweck dienen
- in engem Zusammenhang mit der statistischen und verbalen Beschreibung stehen

Bei der Gestaltung von Grafiken sollten folgende Fragen beantwortet werden:

- Was ist das Ziel der Darstellung?
- Welche Zielgruppe soll angesprochen werden?
- Welche Information wird dargestellt?
- Wieviele Datendimensionen werden kodiert?
- Zähle mehrere ermöglichte Aufgaben, Vergleiche, Auswertungen auf.
- Welche Prinzipien beschreiben am besten weshalb die Darstellung gut/schlecht ist?
- Sind Verbesserungen möglich?
- Weshalb mag ich die Darstellung? Weshalb nicht?

2.6. Kategorisierung von Variablen

2.6.1. Modellbildung

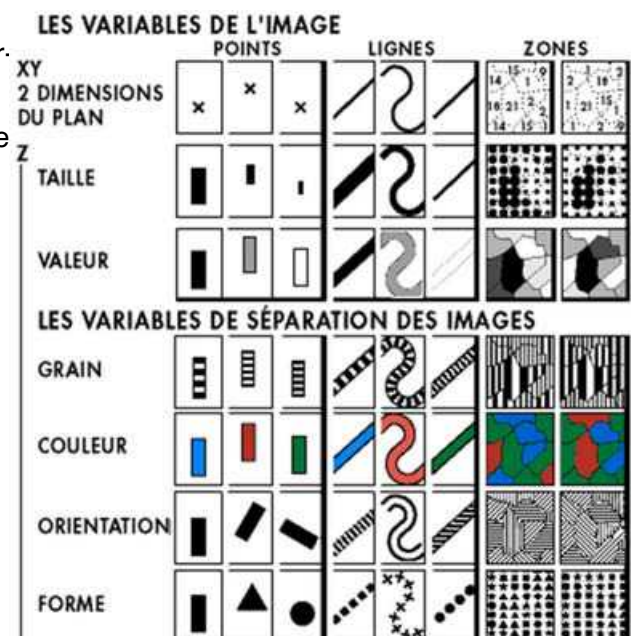
Abbildung

- Eingabe
 - Datensemantik
 - Nutze Domänenwissen
- Ausgabe
 - Visuelle Kodierung
 - * visuell/grafisch/perzeptionell
 - * Kanäle/Attribute/Dimension/Variable
 - Nutze menschliche Wahrnehmung
- Verarbeitung
 - Algorithmen
 - Berücksichtige Einschränkungen durch Rechner

2.6.2. Semiotik

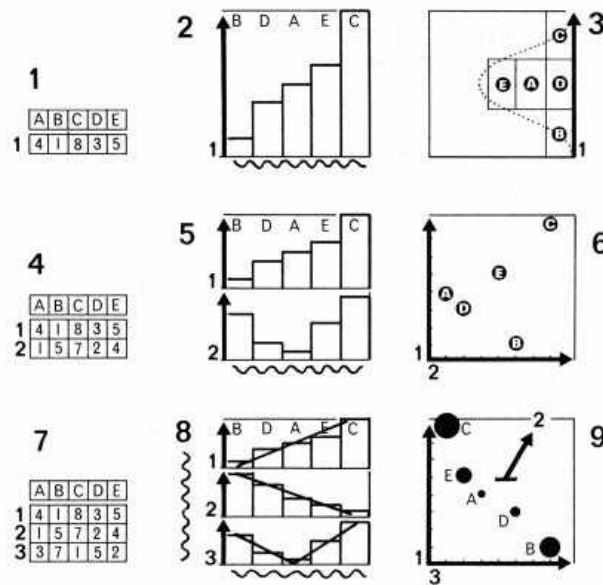
Semiotik der Grafik (nach Bertin)

- geometrische Primitive: Marker
 - Punkte, Linien, Flächen, Volumen
- Attribute: visuelle Variable
 - Parameter bestimmen Marker-Aussehen
 - Verschiedene Kanäle fließen vom Auge zum Gehirn
- x, y
 - Position
- z
 - Größe
 - Grauwert
 - Farbe
 - Textur
 - Orientierung
 - Form



2.6.3. Designraum

Designraum = Visuelle Metaphern



2.6.4. Datentypen

kontinuierlich (quantitativ)



- 10cm, 17cm, 23cm

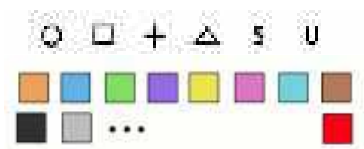
angeordnet (ordinal)

- klein, mittel, groß
- Wochentage: Sonntag, Montag,



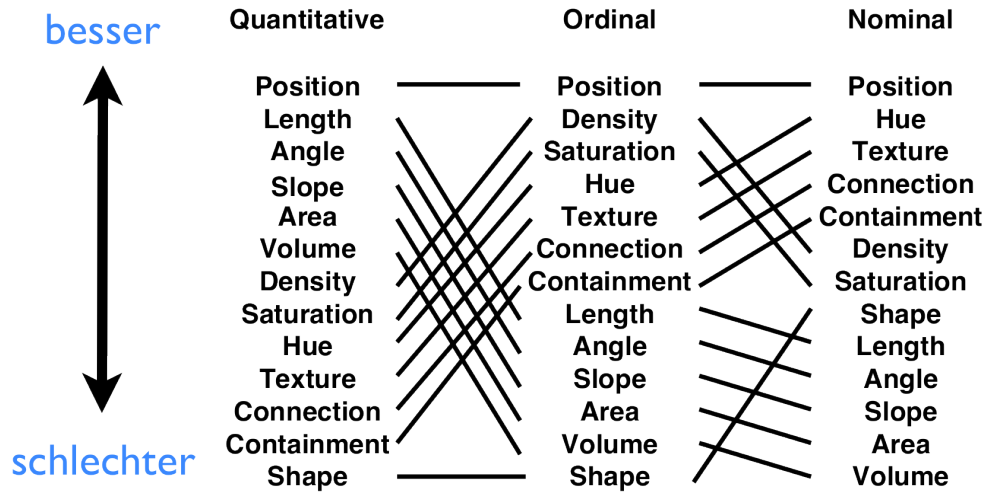
kategorisch (nominal)

- Äpfel, Orangen, Bananen



2.6.5. Kanalrangfolge

räumliche Position für alle Datentypen am Besten



[Mackinlay, Automating the Design of Graphical Presentations of Relational Information, ACM TOG 5:2, 1986]

2.6.6. Taxinomie

Taxinomie nach Mackinlay und Card (Dogma!)

Datenvariable

1d, 2d, 3d, 4d,

Datentyp

nominal, angeordnet, quantitativ

Marker

- Punkte, Linien, Fläche, Oberfläche, Volumen
- geometrische Primitive

Visuelle Kanäle

- Größe, Helligkeit, Farbe, Textur, Orientierung, Form
- Kontrollparameter für die geometrischen Primitive
- Separierbare Kanäle für Informationsfluss vom Auge zum Gehirn

Taxinomie nach Shneiderman:

Daten

- 1d, 2d, 3d, zeitlich, Bäume, Netzwerke
- Texte und Dokumente

Aufgaben

- Überblick, Zoom, Filter, Details auf Nachfrage
- Verbinden, Geschichte, Extrahieren

Daten alleine reichen nicht!

Überlege was zu tun ist?

Mantra

Zuerst der Überblick, dann Zoom und Filter, Details auf Nachfrage

2.6.7. Datenmodell vs. konzeptionelles Modell**Datenmodell: mathematische Abstraktion**

- Eine Menge mit definierten Operationen
- z.B. Ganzzahlen oder reelle Zahlen mit Operationen $*$, $+$

konzeptionelles Modell: mentale Konstruktion

- hat Semantik zur Unterstützung der Daten
- z.B. Navigation durch eine Stadt mit Orientierungspunkten

Beispiel 14: Modelbeispiel

- aus dem Datenmodell: Fließkommazahlen: 17, 25, -4, 28.6
- über das konzeptionelle Modell: Temperatur
- zum Datentyp:
 - stetig mit 4 Stellen Genauigkeit (quantitativ)
 - kalt, warm, heiß (ordinal)
 - verbrannt vs. nicht verbrannt (nominal)
- mittels Aufgaben:
 - finde Anomalien in lokalen Wettermustern
 - klassifiziere Regenfälle
 - mache Toast

Zeit

2d + t vs. 3d

- Dasselbe oder unterschiedlich? Hängt vom Standpunkt ab.
 - Zeit als Eingabedatum
 - Zeit als visuelle Kodierung
- Dasselbe
 - Zeit ist eine abstrakte Eingabedimension
- Unterschiedlich
 - Eingabesemantik
 - Visuelle Kodierung: räumliche Position vs. zeitliche Änderung
- Verarbeitung kann unterschiedlich sein
 - unterschiedliche Interpolation von Zeitschritten und Positionen

Kombinatorik der Kodierung**Herausforderung**

Wähle die beste Kodierung aus den $(n + 1)^k$ möglichen Kodierungen bei n Eingabedimensionen und k visuellen Kanälen.

Konsistenzprinzip

Die Eigenschaften des Bildes sollten die Eigenschaften der Daten wiedergeben.

Prinzip der Rangfolge

Kodiere die wichtigste Information in der effektivsten Weise

Automatisierung**Ziel**

Für gegebene Daten automatisch eine passende Visualisierung erzeugen

Möglich

mit eingeschränkten Datenmengen und eingeschränkten Visualisierungen (Scatterplot, Balkendiagramme, etc).

Heiliger Gral der Visualisierung

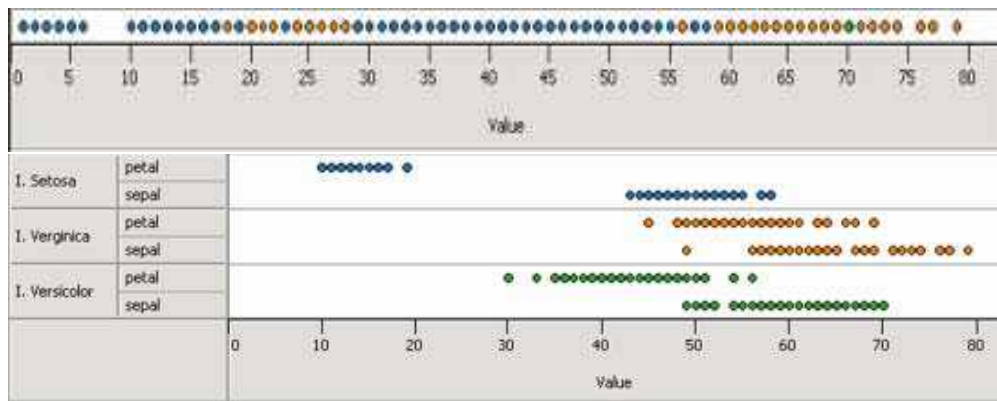
Automatische Generierung für den gesamten Parameterraum

Mackinlay's Kriterien**Ausdrucksfähigkeit**

Eine Menge von Daten kann in einer visuellen Sprache ausgedrückt werden, falls Sätze (Visualisierungen) in der Sprache alle Fakten der Daten und nur Fakten der Daten ausdrücken.

Betrachte Versagensfälle**Beispiel 15:** Versagensfall: Fakten können nicht ausgedrückt werden

Eine $1 \leftrightarrow N$ Relation kann nicht in einem einzelnen horizontalen Punkte-Diagramm ausgedrückt werden, da mehrere Tupel auf dieselbe Position abgebildet werden.



Beispiel 16: Versagensfall: Nicht vorhandene Fakten werden ausgedrückt

Länge wird als quantitatives Datum interpretiert
Daher sagt Länge die Unwahrheit über nominale Daten aus.

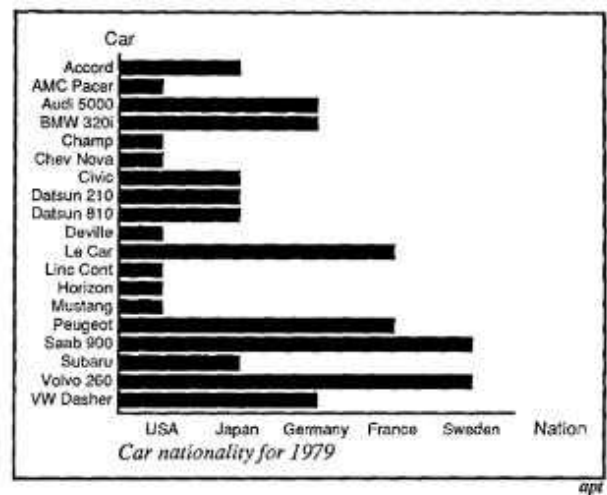


Fig. 11. Incorrect use of a bar chart for the *Nation* relation. The lengths of the bars suggest an ordering on the vertical axis, as if the USA cars were longer or better than the other cars, which is not true for the *Nation* relation.

Mackinlay's Kriterien

Ausdrucksfähigkeit

Eine Menge von Daten kann in einer visuellen Sprache ausgedrückt werden, falls Sätze (Visualisierungen) in der Sprache alle Fakten der Daten und nur Fakten der Daten ausdrücken.

Effektivität

Eine Visualisierung ist effektiver als eine andere, falls information in der Visualisierung besser vermittelt werden kann als in der anderen.

2.7. Beispielvideo

David Maccandless: Information is Beautiful

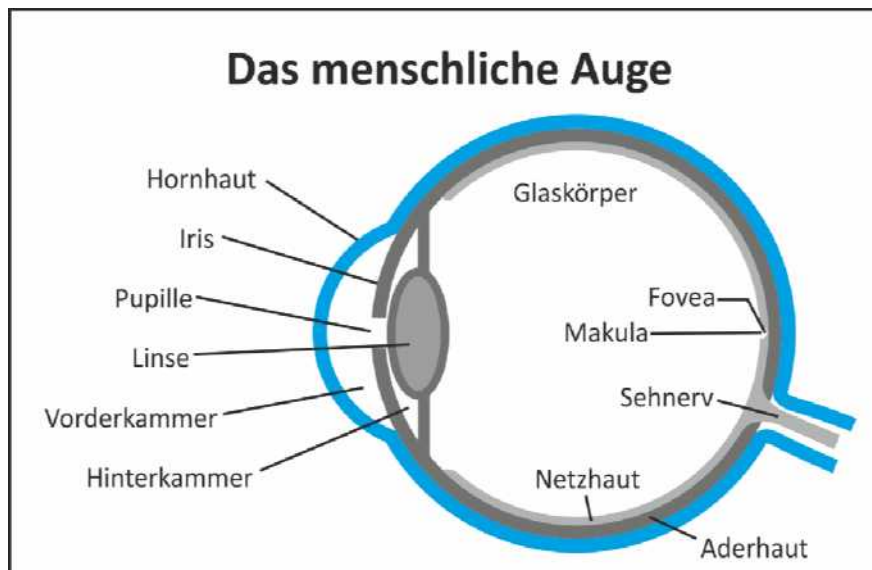


http://www.ted.com/talks/david_mccandless_the_beauty_of_data_visualization.html

3. Wahrnehmung

3.1. Das Auge

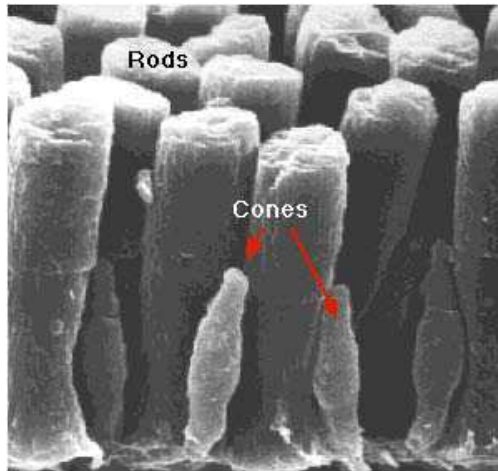
Querschnitt des Auges



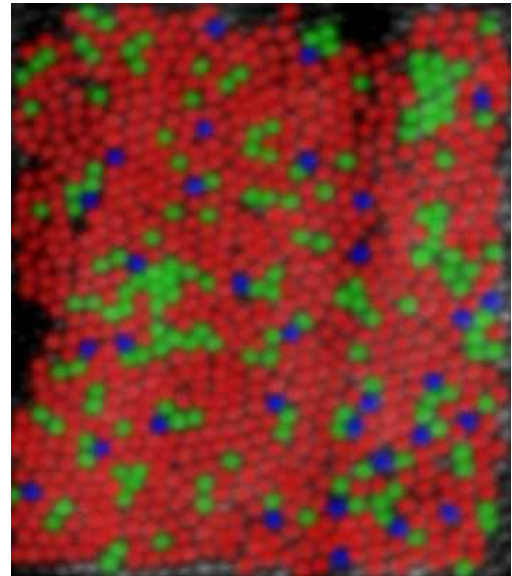
Netzhaut



Rezeptoren

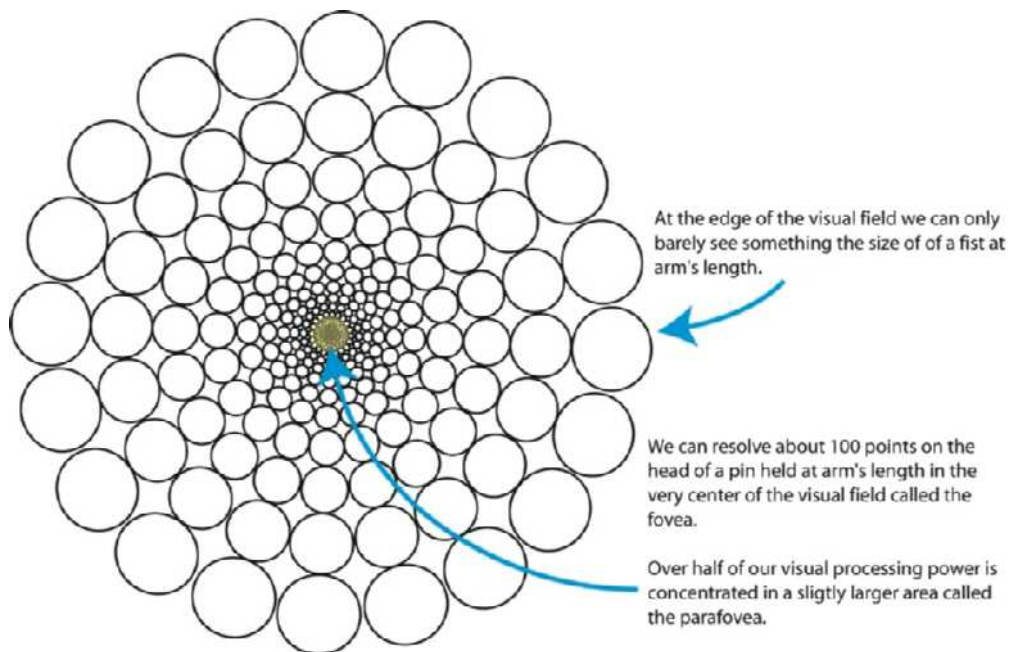


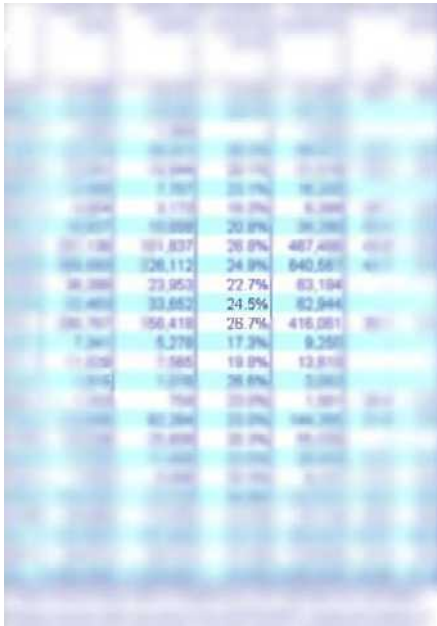
Stäbchen und Zäpfchen



Farbrezeptoren in der Fovea

Wahrnehmungsfeld



Bildschärfebereiche

Zentrum visuelles Feld

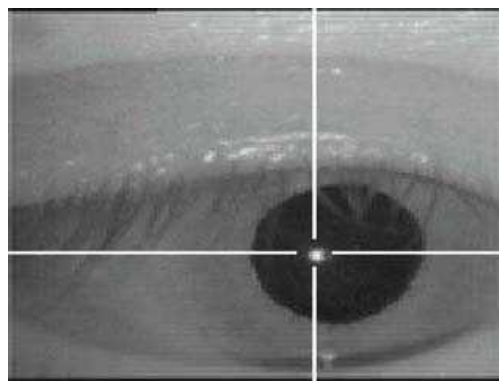


Randbereich visuelles Feld

Saccadas

sind schnelle unwillkürliche Augenbewegungen

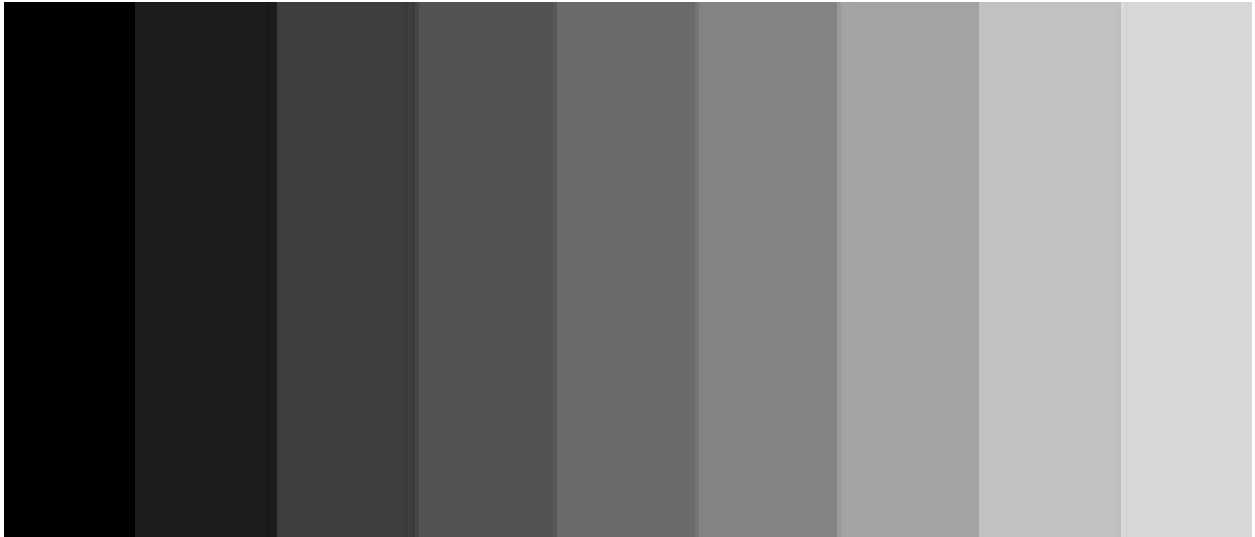
- Bewegung: 20 – 100 ms
- Fixierung: 200 – 600 ms

**Augenbewegung**

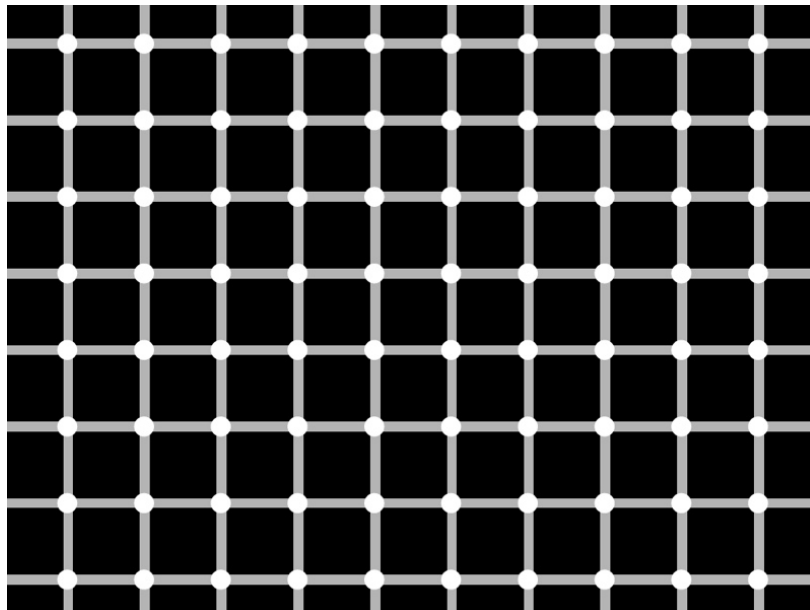
3.2. Wahrnehmung

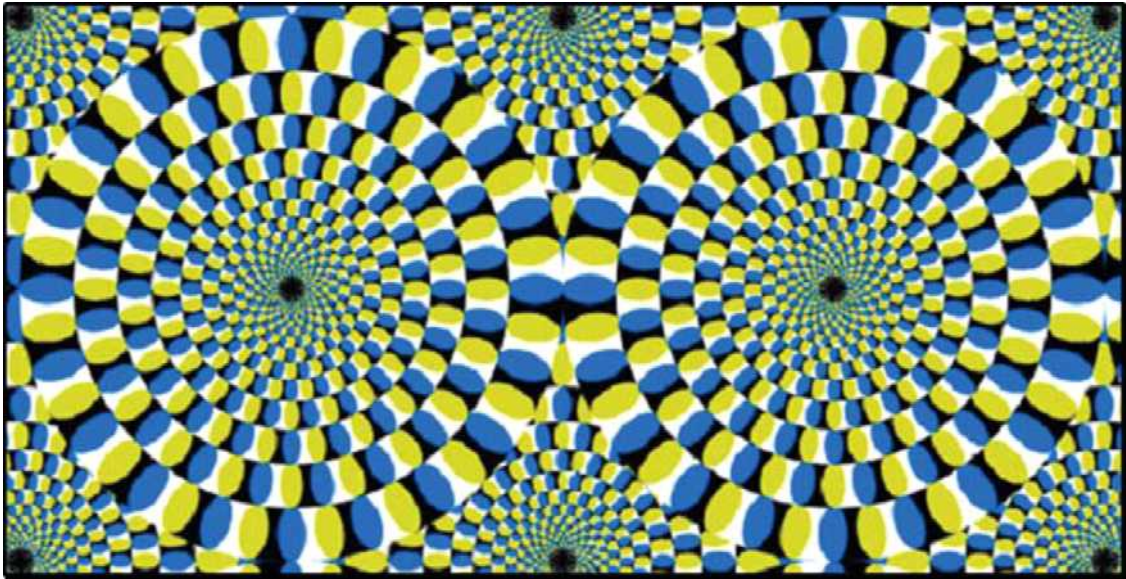
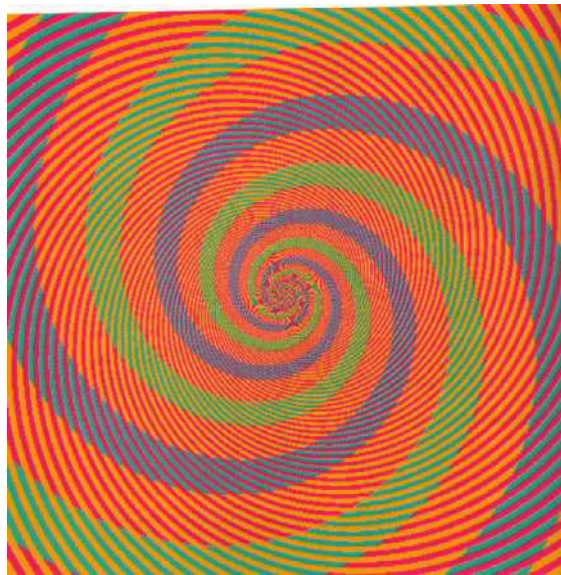
3.2.1. Folgen der Anordnung der Sehrezeptoren

Beispiel 17: Machbänder

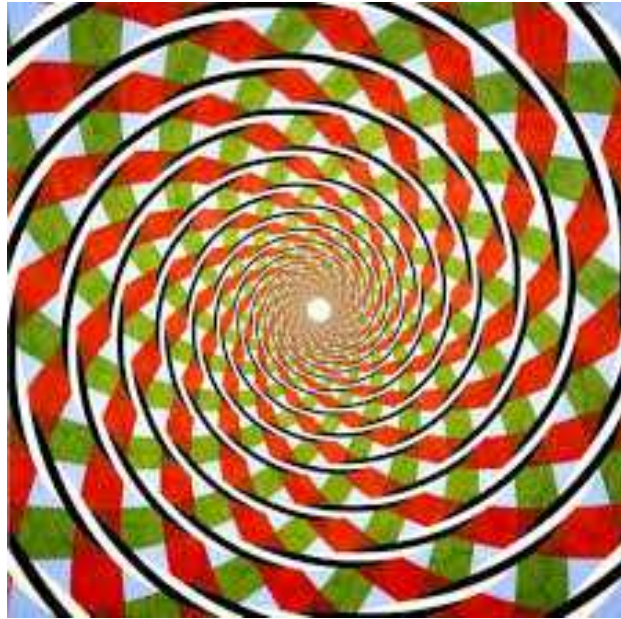


Beispiel 18: Dunkle Kreise

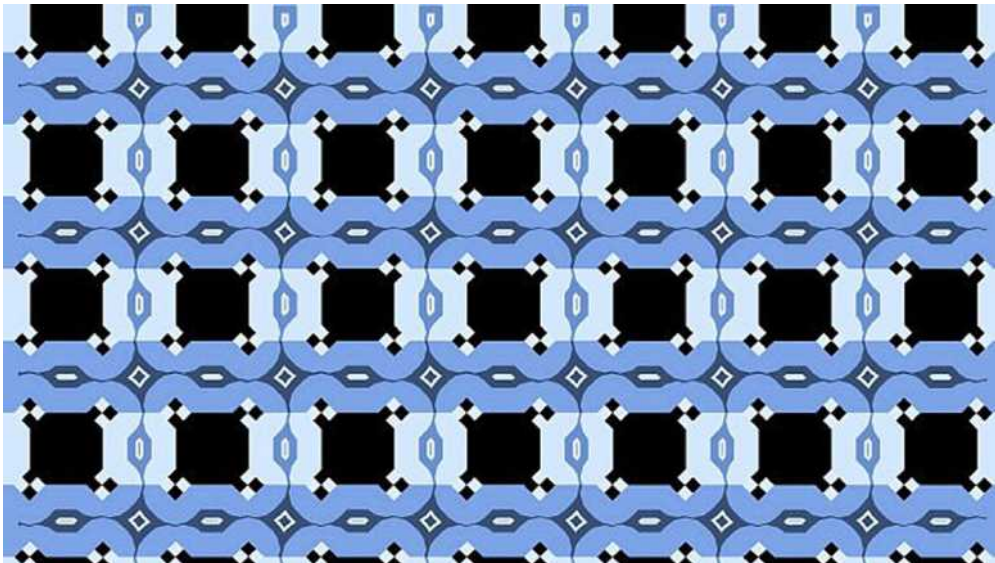


Beispiel 19: Bewegungssillusion**Beispiel 20:** Farbillusion

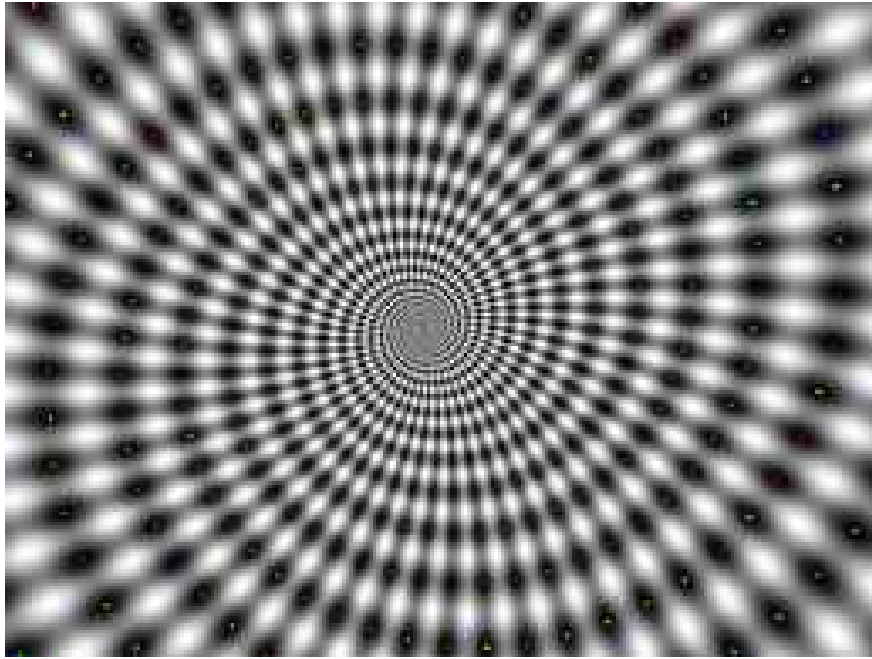
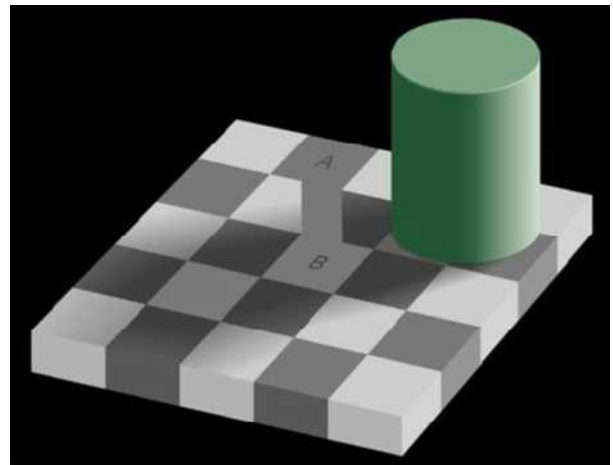
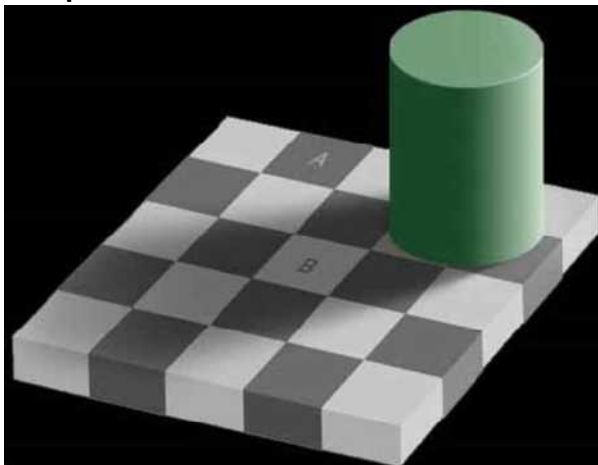
Weder grün noch blau sind vorhanden, nur türkis!

Beispiel 21: Spiralillusion

Keine Spirale, sondern konzentrische Kreise!

Beispiel 22: Linienillusion

Linien sind parallel!

Beispiel 23: Bewegungssillusion**3.2.2. Mustererkennung****Beispiel 24:** Schattenillusion

A und B haben denselben Grauwert! (Edward H. Adelson)

Beispiel 25:

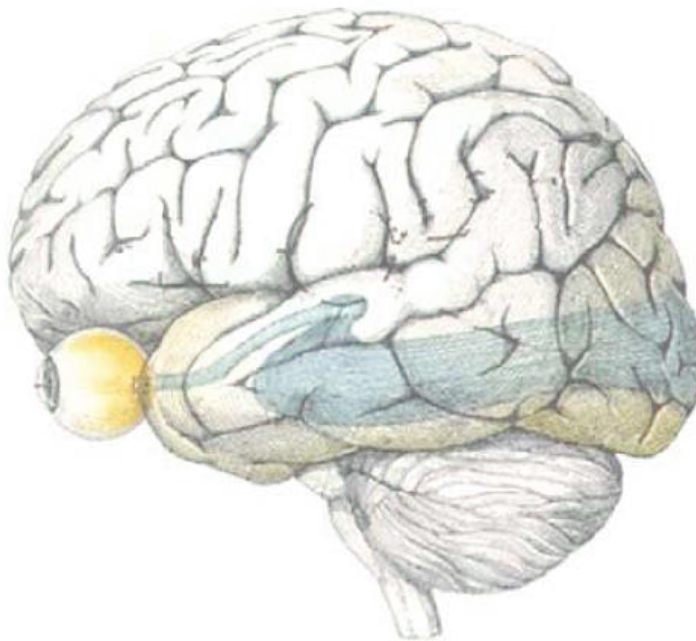
Zusatzinformation nötig: Kuh!

Beispiel 26:

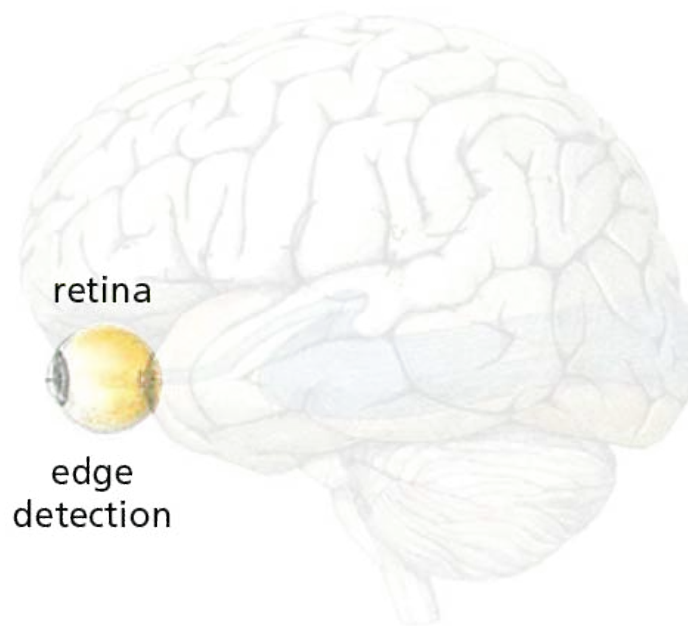
Zusatzinformation nötig: Dalmatiner!

Beispiel 27:

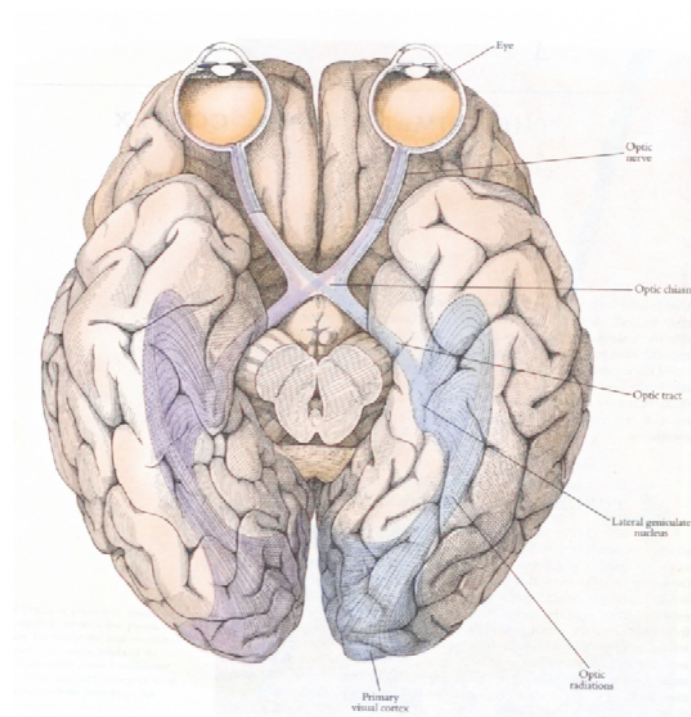
Zusatzinformation nötig: Putzfrau mit Eimer!

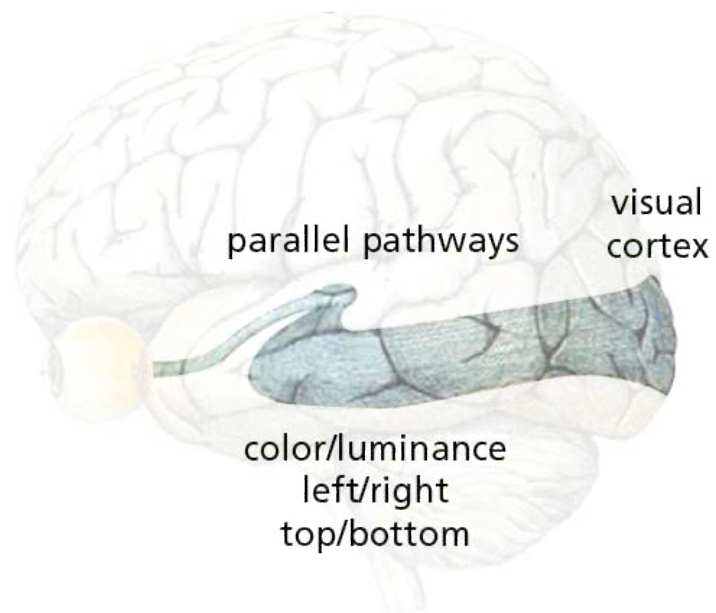
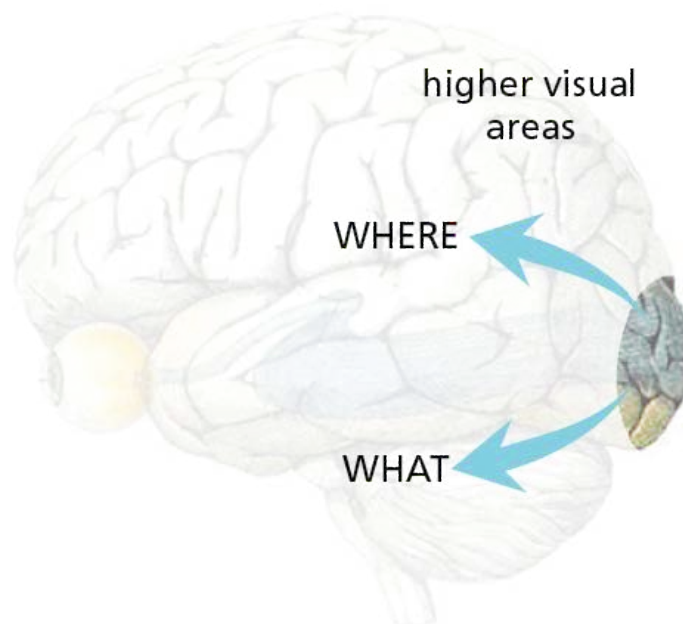
3.3. Visuelle Pfade

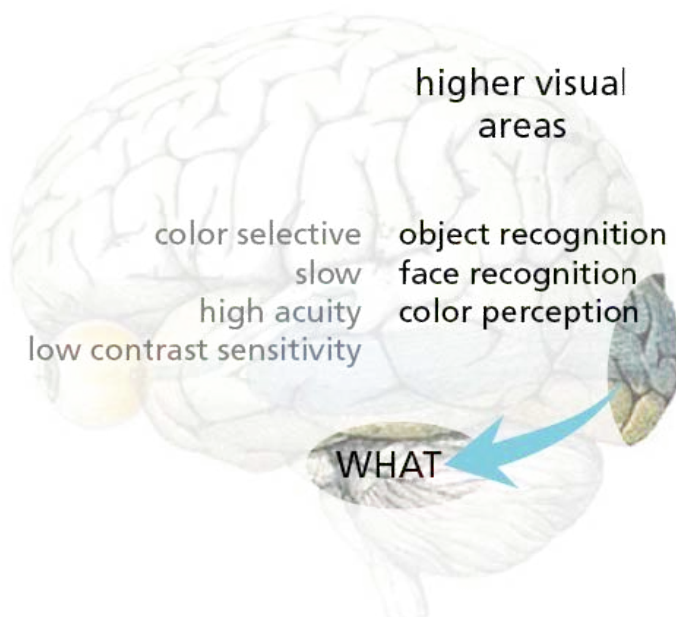
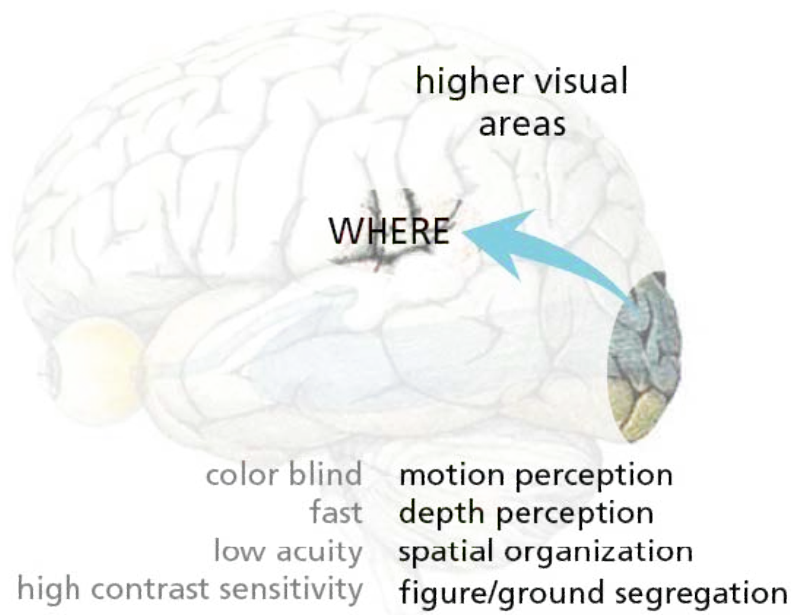
Kantendetektion findet bereits im Auge statt.



3.3.1. Gehirninregionen



Visueller Cortex**Höhere Regionen**



3.3.2. Auge und Gehirn

Was fällt auf?

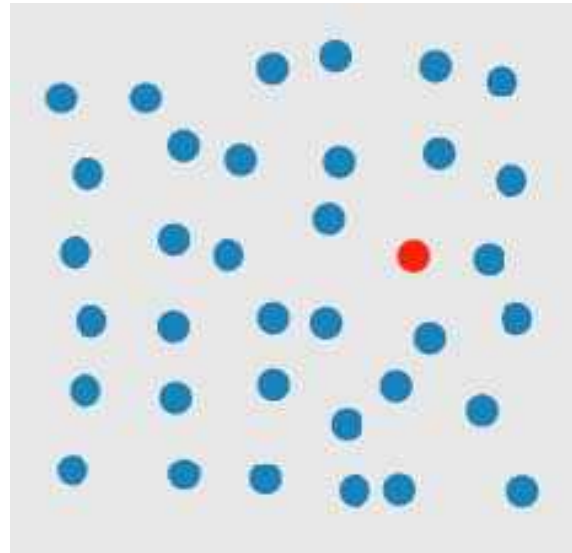
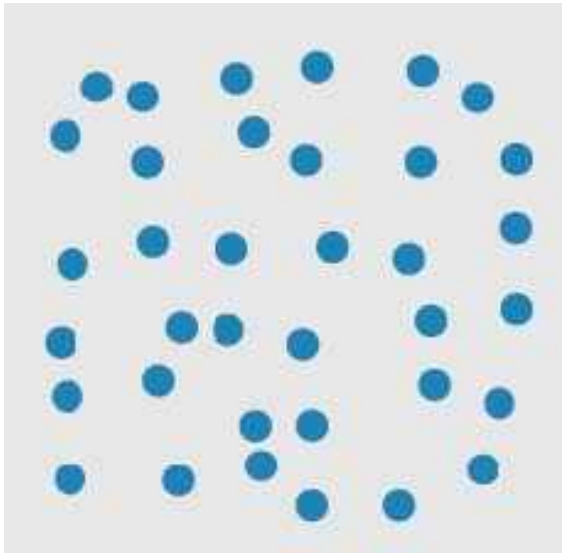
1281768756138976546984506985604982826762
9809858458224509856458945098450980943585
9091030209905959595772564675050678904567
8845789809821677654876364908560912949686

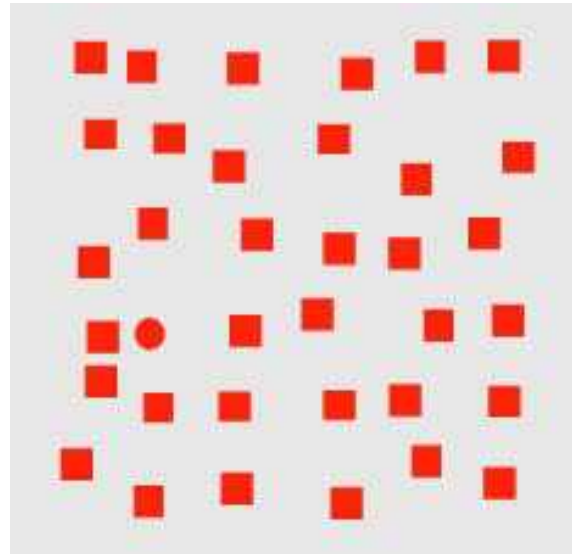
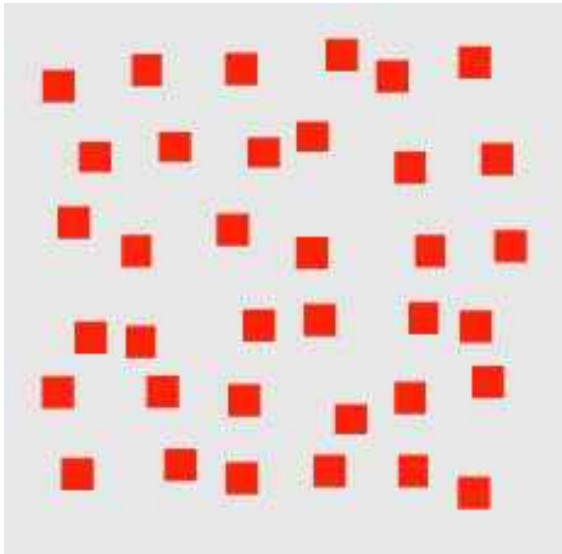
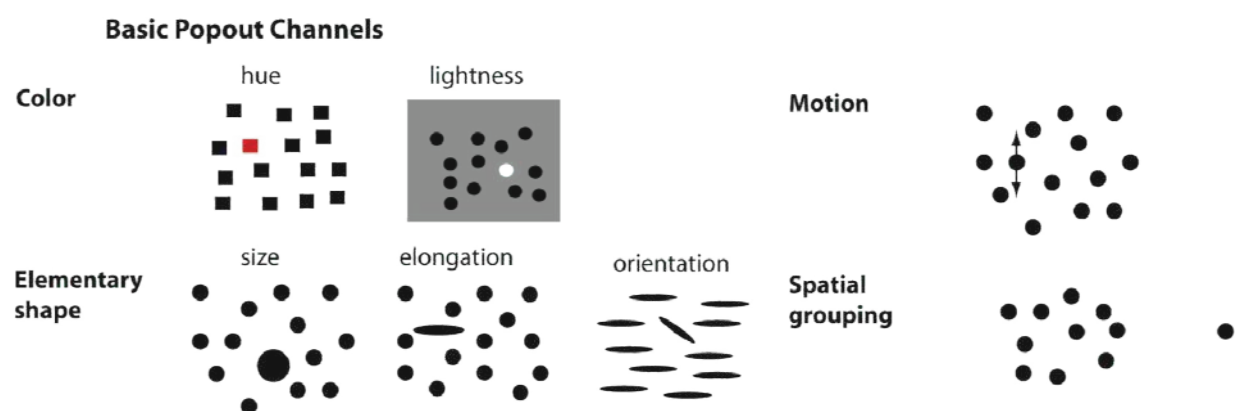
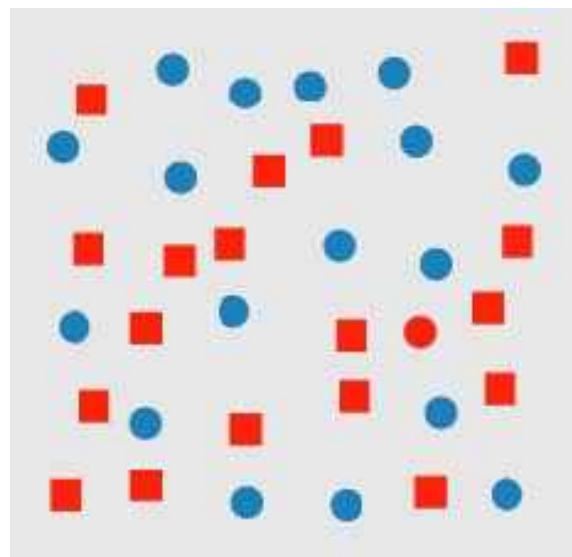
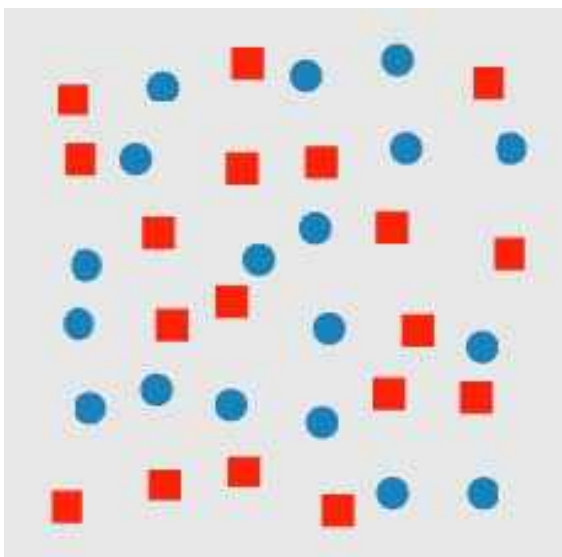
Finde die Anzahl von 3-en!

12817687561**3**8976546984506985604982826762
980985845822450985645894509845098094**3**585
90910**3**0209905959595772564675050678904567
8845789809821677654876**3**64908560912949686

Finde die Anzahl von 3-en!

Farbauswahl: Einfach – Verarbeitung im Auge



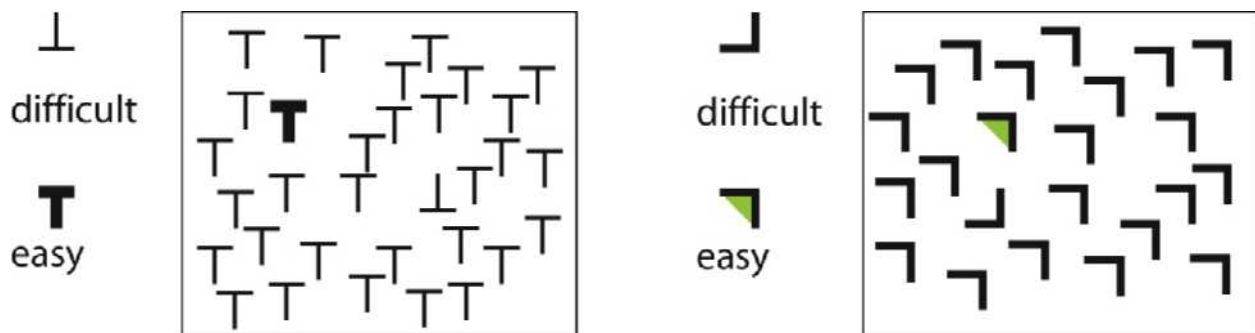
Formauswahl: Einfach – Verarbeitung im Auge**Form- und Farbauswahl: Schwieriger – Verarbeitung im Gehirn**

3.4. Preattentive Processing

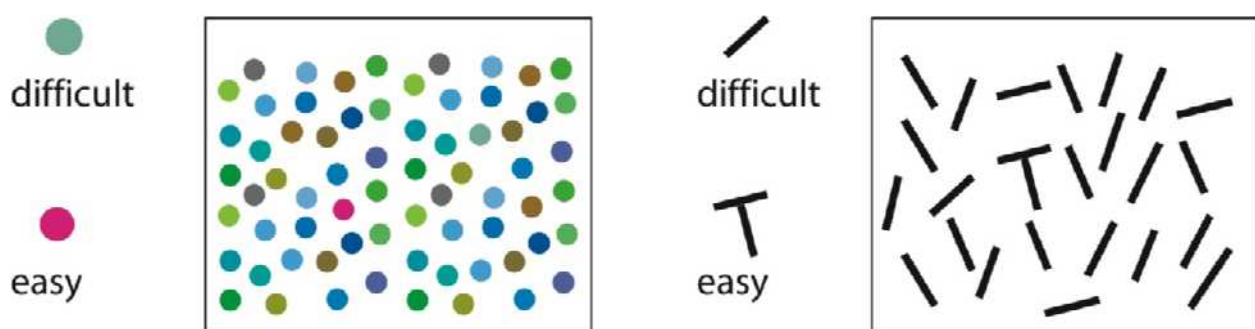
- Benötigt Aufmerksamkeit (trotz des Namens)
- sehr schnell (200 - 250 ms)
- Auffallend: Farbe, Orientierung, Größe, Bewegung, Gruppierung
- Am wichtigsten: Verschiedenheit der Eigenschaften

3.4.1. Form- und Farbauswahl

Beispiel 28:



Beispiel 29:

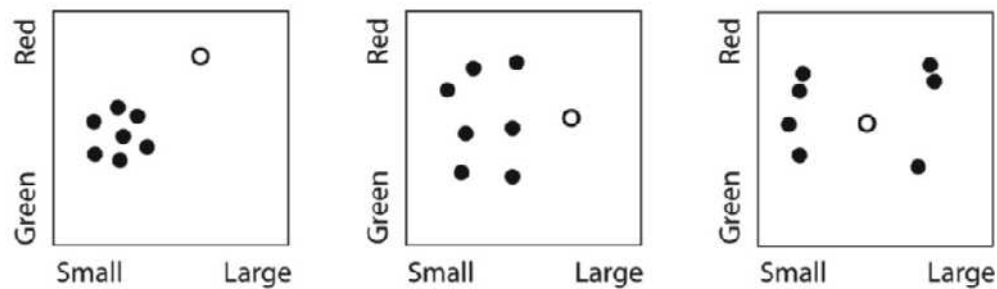
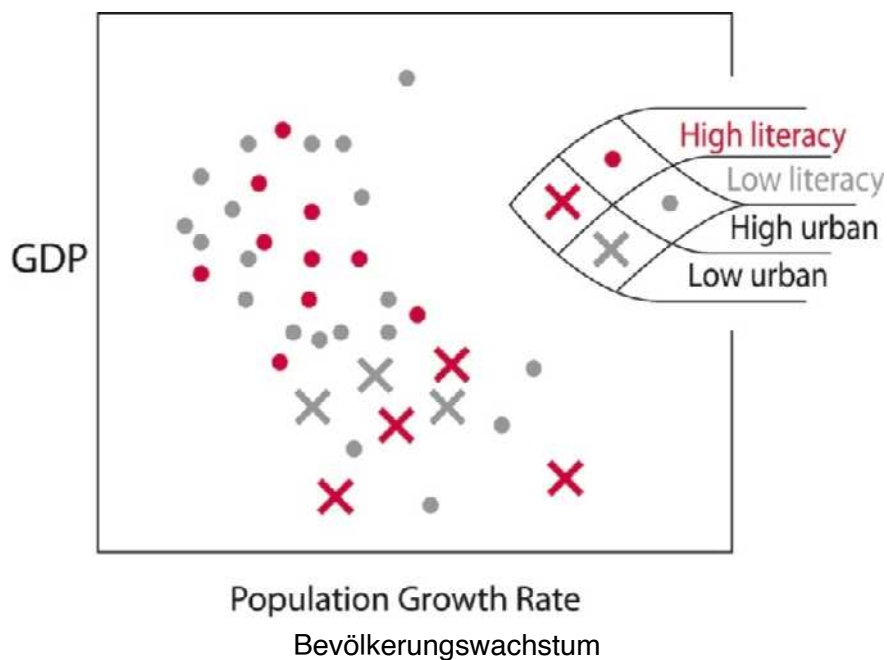


Beispiel 30:

Objects to be searched



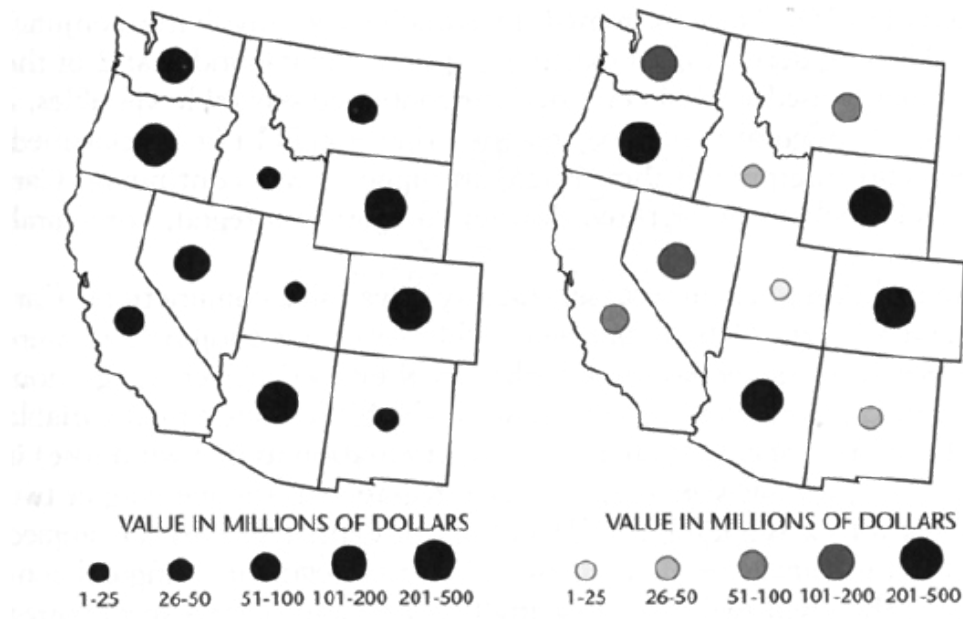
Corresponding feature space diagrams

**3.4.2. Mehrfachkanäle****Zweifachkodierung**

- Zwei unterschiedliche Variable kodieren mit zwei unterschiedlichen visuellen Kodierungen
- Farbe, Größe, Form, Helligkeit
- Redundanzgewinn, falls sich zwei Variable verstärken

- Interferenzverlust: Schwierigkeit eine Dimension zu ignorieren während nur die andere betrachtet wird.

Beispiel 31: Korrelation: Größe und Grauwert



Beispiel 32: Orthogonal: Seitenverhältnis

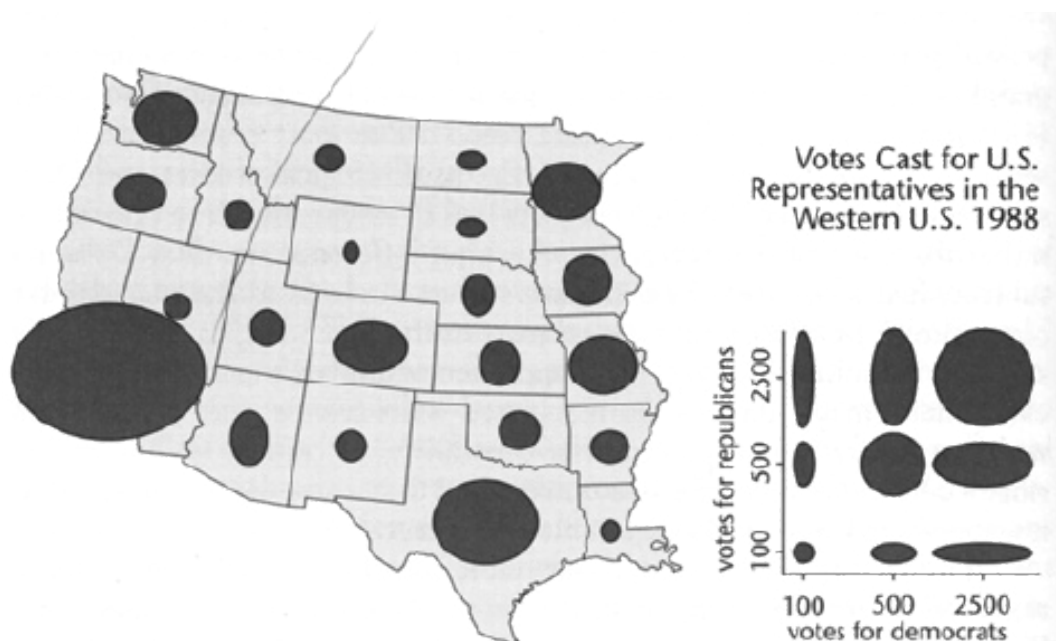
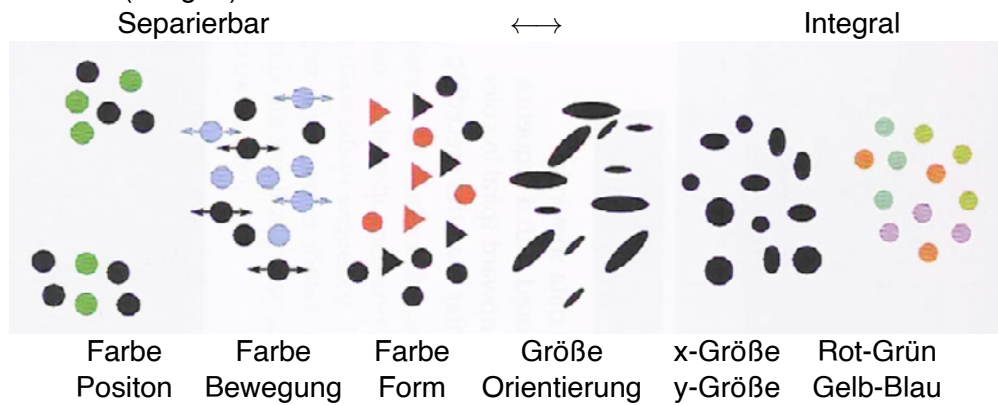


FIGURE 3.38. An example of the use of an ellipse as a map symbol in which the horizontal and vertical axes represent different (but presumably related) variables.

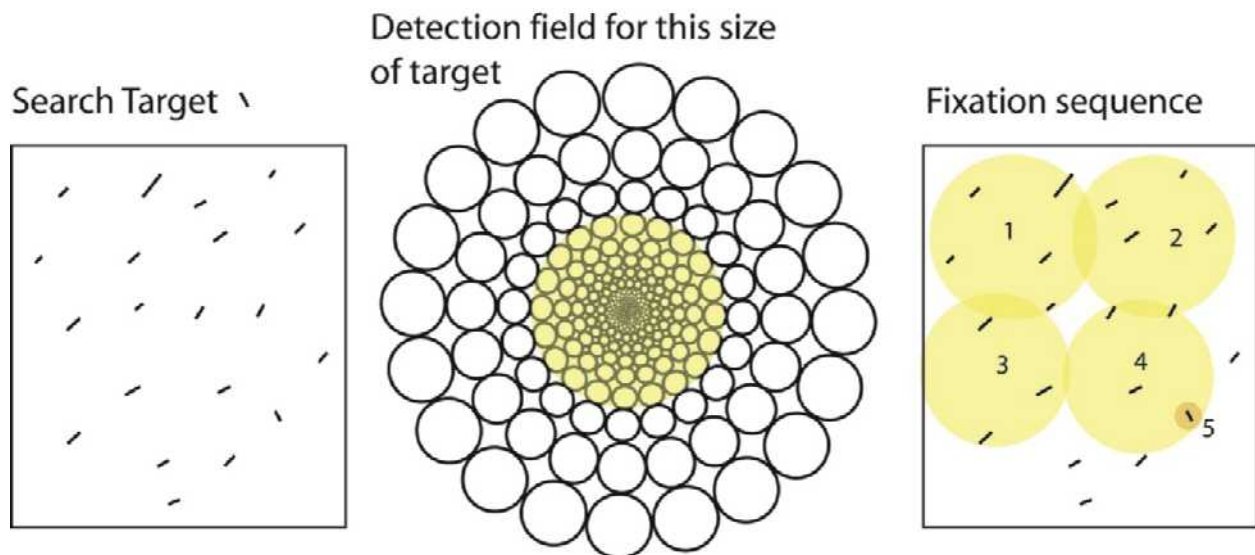
Getrennt – Gemeinsam

- Getrennt (Separierbar): Jeder Merkmalskanal kann unabhängig bewertet werden
- Gemeinsam (Integral): Zwei Kanäle werden holistisch betrachtet.

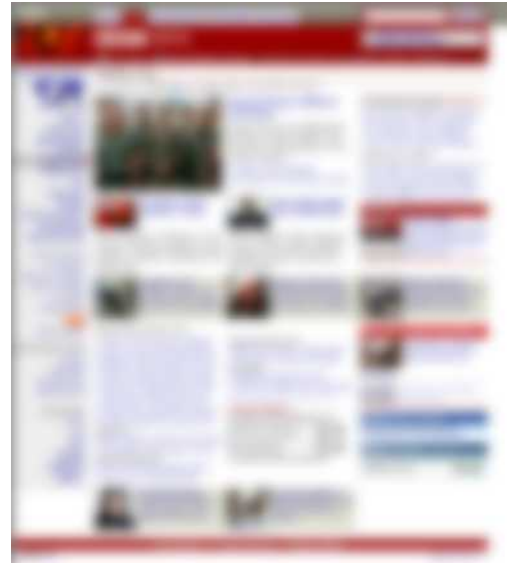


3.5. Multiskaleneffekte

3.5.1. Visuelle Suchstrategien

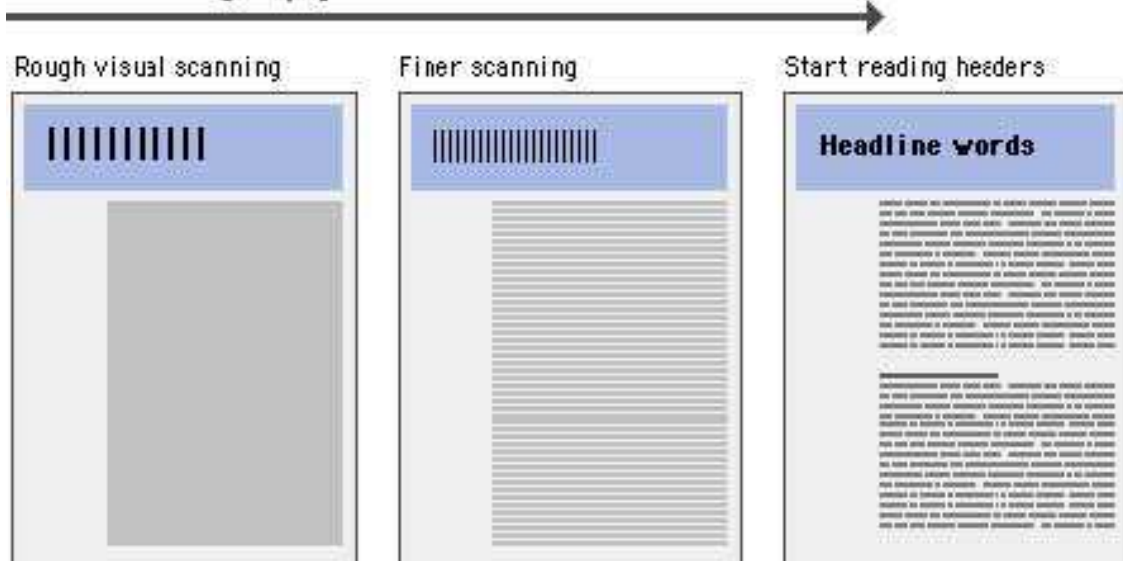


Beispiel 33: BBC News: Geglättet zur Hervorhebung der Struktur



3.5.2. Suchhierarchie

Visual scanning of page structure over time



3.5.3. Merkmalskontrast

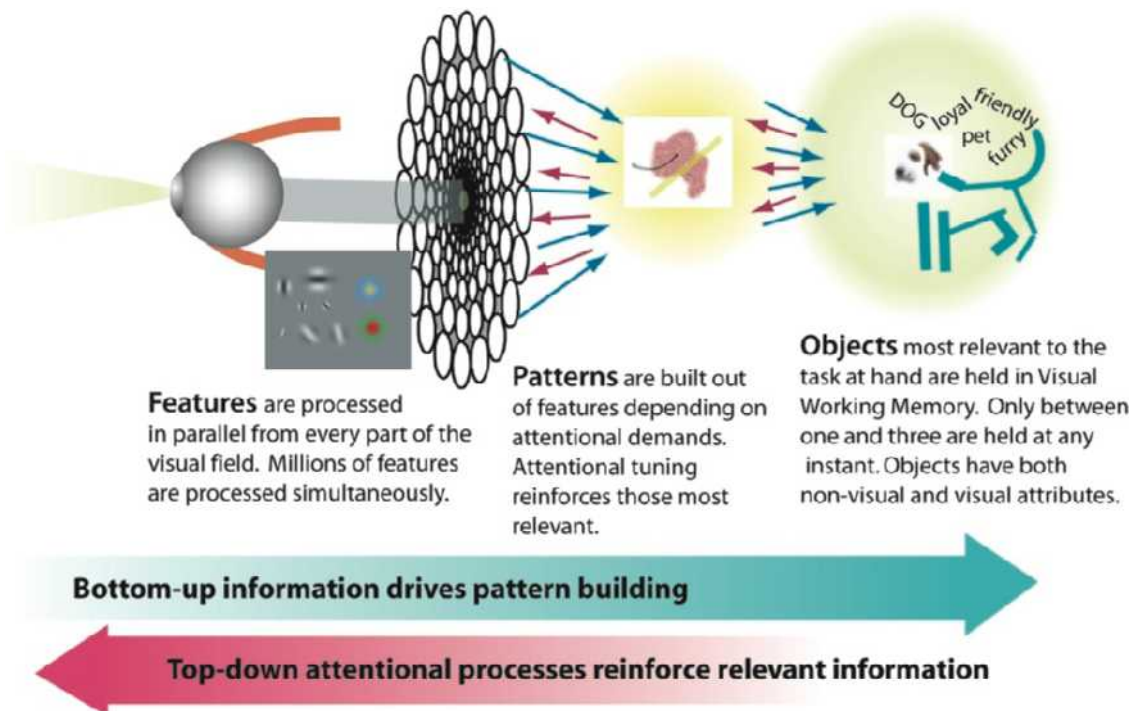
Dull; no focal points,
no graphic structure



Stronger visual structure;
better contrast, visual entry points



3.5.4. Wahrnehmungshierarchie



3.6. Beispielvideos

Beispiel 34: Daniel J. Simons: Selective Attention Test



<https://www.youtube.com/watch?v=vJG698U2Mvo>

Beispiel 35: Richard Wiseman: Colour Changing Card Trick



https://www.youtube.com/watch?annotation_id=annotation_262395&feature=iv&src_vid=voAntzB7EwE&v=v3iPrBrGSJM

Beispiel 36: Chris Chabris: When Intuition Fails



<http://vimeo.com/20110651>

Teil II.

Full-Stack JavaScript

4. Webprogrammierung

4.1. Klassisches Backend

Backend

- Ausführung auf dem Server
- Verarbeitung von Anfragen
- Bereitstellung von Daten

Frontend

- Ausführung im Browser
- Darstellung von Daten
- Interaktion mit Benutzer

In der klassischen Webprogrammierung wird zwischen Frontend und Backend unterschieden. Das Frontend ist der Teil der Webanwendung, der im Browser des Benutzers ausgeführt wird. Das Backend ist der Teil der Webanwendung, der auf dem Server ausgeführt wird. Das Backend ist für die Verarbeitung von Anfragen und die Bereitstellung von Daten verantwortlich. Das Frontend ist für die Darstellung der Daten und die Interaktion mit dem Benutzer verantwortlich.

Backend-Technologien

- PHP (in Kombination mit dem Apache-Webserver)
- Java (in Kombination mit Tomcat)
- Python (mit Django oder Flask)
- Ruby (mit Ruby on Rails)
- Node.js (mit Express.js)
- Bun (Alternative zu Node.js)
- Go (mit Gin)

PHP

PHP wird oft in Kombination mit dem Apache Webserver verwendet. Es gibt viele Frameworks und Bibliotheken für PHP, z.B. Laravel, Symfony, CodeIgniter, Zend, Slim, CakePHP, Yii, Phalcon, FuelPHP, Laminas, Aura, Fat-Free, Flight, Silex, etc.

PHP ist eine serverseitige Skriptsprache, die für die Entwicklung von Webanwendungen verwendet wird. PHP-Code wird auf dem Server ausgeführt und generiert HTML-Code, der an den Browser des Benutzers gesendet wird. PHP ist nach wie vor eine der beliebtesten Programmiersprachen für die Webentwicklung und ist weit verbreitet. Einige der bekanntesten PHP-basierten Webanwendungen sind WordPress, Joomla, Typo3, Drupal.

Diesen Technologien ist gemeinsam, dass die darzustellenden Webseiten dynamisch auf dem Server generiert werden oder statische Webseiten ausgeliefert werden. Die Webseiten werden im Browser des Benutzers angezeigt. Die Webseiten können interaktiv sein und auf Benutzereingaben reagieren. Hierzu wird oft JavaScript als Frontend-Technologie verwendet.

4.2. Frontend-Technologien

JavaScript als Frontend-Technologie

- wird im Browser ausgeführt
- ermöglicht interaktive Webseiten

JavaScript wurde entwickelt, um Webseiten interaktiv zu machen. JavaScript wird im Browser des Benutzers ausgeführt und ermöglicht es, dass Webseiten auf Benutzereingaben reagieren. Hierzu wird JavaScript oft in Kombination mit HTML und CSS verwendet. JavaScript ist eine der drei Kern-Technologien des World Wide Web. Die anderen beiden sind HTML und CSS. Zur Zeit ist JavaScript die dominante Technologie für Frontend-Entwicklung.

Webassembly

Auswahl an unterstützten Programmiersprachen:

- C
- C++
- Rust
- Go

WebAssembly ist ein binärer Befehlssatz für Webanwendungen. WebAssembly ermöglicht es, dass Frontend-Webanwendungen in anderen Programmiersprachen als JavaScript geschrieben werden können. WebAssembly wird oft in Kombination mit JavaScript verwendet. WebAssembly ist eine relativ neue Technologie und wird von allen gängigen Browsern unterstützt. WebAssembly ermöglicht es, dass Webanwendungen schneller und effizienter ausgeführt werden können.

4.3. Single-Page-Applications

- verwenden nur eine einzige HTML-Seite
- Inhalte werden dynamisch nachgeladen
- schneller und reaktionsfähiger
- schlechter von Suchmaschinen indexierbar

Single-Pages-Applications (SPA) sind Webanwendungen, die nur eine einzige HTML-Seite verwenden. Die Inhalte der Seite werden dynamisch nachgeladen, ohne dass die Seite neu geladen werden muss. SPAs verwenden oft JavaScript-Frameworks wie Angular, React oder Vue.js. SPAs sind oft schneller und reaktionsfähiger als traditionelle Webanwendungen, jedoch leiden sie unter dem Problem, dass sie nicht so gut von Suchmaschinen indexiert werden können, da die Seiten überwiegend im Browser durch JavaScript generiert werden. Ebenso können die Ladezeiten bei SPAs länger sein, da die gesamte Anwendung beim ersten Aufruf geladen werden muss.

5. Einführung in Node/Bun

- Node/Bun sind JavaScript-Laufzeitumgebungen
- Node/Bun sind Plattformen zum Betrieb von Webapplikationen
- Node
 - wurde 2009 von Ryan Dahl entwickelt
 - ist weit verbreitet
 - nutzt die V8 JavaScript-Engine von Google
 - ist in C++ geschrieben
- Bun
 - wurde 2021 von Jarred Sumner entwickelt
 - ist noch nicht so weit verbreitet
 - nutzt die JavaScriptCore-Engine von Apple
 - ist in Zig geschrieben

Node/Bun ermöglichen JavaScript außerhalb des Browsers zu nutzen. Damit kann JavaScript auch auf dem Server eingesetzt werden. Dies ermöglicht die Entwicklung von Webapplikationen, die komplett in JavaScript geschrieben sind. Hierfür wird Node/Bun mit einem in JavaScript implementierten Webserver kombiniert und der Browser dient als Client. Der Code für den Server und den Client kann in einer Sprache geschrieben werden, jedoch sollte auf die strikte Trennung von Client- und Servercode geachtet werden.

Vorteile

JavaScript ist

- eine weit verbreitete Sprache.
- eine dynamische Sprache.
- eine objektorientierte Sprache.
- eine funktionale Sprache.
- eine asynchrone Sprache.
- auf allen Plattformen verfügbar.
- eine Skriptsprache.

Nachteile

JavaScript

- ist über die Jahre gewachsen.
- enthält viele historische Konstrukte.
- enthält viele Fallstricke.
- ist eine dynamische Sprache. (keine Typsicherheit)
- ist eine asynchrone Sprache. (komplexe Programmierung)
- enthält inzwischen Syntax für Klassen, ist aber keine klassenbasierte Sprache, sondern nutzt prototypenbasierte Vererbung.
- hat keine Standardbibliothek

5.1. Installation

Node kann von der Webseite <https://nodejs.org> heruntergeladen werden. Falls Node genutzt wird, sollte die aktuelle LTS-Version installiert werden. Node ist für alle gängigen Betriebssysteme verfügbar. Die Installation erfolgt über einen Installer oder über einen Paketmanager. Nach der Installation kann die Version auf der Kommandozeile mit dem Befehl `node -v` überprüft werden.

Hierzu muss der Pfad zum Node-Verzeichnis in der Umgebungsvariable `PATH` eingetragen sein. Die Ausführung von JavaScript-Dateien erfolgt mit dem Befehl `node dateiname.js`.

Node interaktiv

Node kann jedoch auch interaktiv auf der Kommandozeile genutzt werden. Hierzu wird der Befehl `node` ohne Parameter aufgerufen. Die Eingabe von JavaScript-Befehlen erfolgt dann direkt auf der Kommandozeile. Mit der Eingabe von `.exit` wird der interaktive Modus beendet. Diese Art der Nutzung von Node ist sehr hilfreich, um die JavaScript-Befehle auszuprobieren und ist unter dem Akronym REPL (Read-Eval-Print-Loop) bekannt und ist in vielen Programmiersprachen verfügbar.

Node-Alternativen

Neben Node.js gibt es auch andere Laufzeitumgebungen für JavaScript. Die bekanntesten sind Deno und Bun. Diese Laufzeitumgebungen sind jedoch nicht so weit verbreitet wie Node.js. Sie haben jedoch einige Vorteile gegenüber Node.js.

So ist Deno sicherer als Node.js, da es standardmäßig keine Zugriffe auf das Dateisystem und das Netzwerk hat. Diese müssen explizit erlaubt werden.

Bun (<https://bun.com>) benutzt die Safari-JavaScript-Engine von Apple anstatt V8 aus Chrome. Dies ermöglicht schnellere Ladezeiten, führt aber zu etwas langsameren Ausführungszeiten. Zudem kann Bun Typescript-Dateien direkt mit Hilfe von Type-Stripping ausführen.

Wir werden in der Veranstaltung Bun nutzen, da es schneller startet, einen Paketmanager eingebaut hat und TypeScript direkt unterstützt.

5.2. Module

- Node/Bun nutzt Module, um den Code zu organisieren
- Module sind Type-/JavaScript-Dateien.
- Module können Funktionen, Variablen und Objekte exportieren.
- Module können Funktionen, Variablen und Objekte importieren.
- Es gibt drei Arten von Modulen: Core-Module, eigene Module und externe Module.
- Core-Module sind in Node/Bun enthalten.
- Eigene Module werden von Node/Bun geladen.
- Externe Module müssen mit einem Paketmanager installiert werden.
- Für Node ist `pnpm` über `npm` zu empfehlen, da ersteres einen lokalen Cache benutzt.
- `bun` hat einen eigenen Paketmanager eingebaut, der mit `bun add` aufgerufen wird.

CommonJS vs. ECMAScript 6

Ursprünglich hatte JavaScript keine Module. Mit der Einführung von Node.js wurde der CommonJS-Standard für das Laden von Modulen in Node eingeführt. Dieser Standard wurde später durch den allgemeinen `import`-Befehl im ES6-Standard ¹ ergänzt. Die beiden Standards unterscheiden sich in der Art, wie Module geladen werden. Daher gilt es zu beachten, dass Module in Node unterschiedlich zu regulärem JavaScript geladen werden können

- Module werden traditionell mit dem Befehl `require` geladen (CommonJS-Standard).

¹JavaScript ist eine Implementierung des ECMAScript-Standards

- Neuere Module können mit dem Befehl **import** geladen werden (ES6-Standard).
- Vorsicht: CommonJS-Importe sollten nicht mit ES6-Importen gemischt werden.

Dies führte zu einer unterschiedlichen Syntax bei der Verwendung von Modulen. Clienten luden Module mit dem Befehl **import**, während Server **require** benutzten. Inzwischen bevorzugen auch Server ES6-Module. Selbst ältere Module können meist mit **import** geladen werden. Daher ist der Befehl **require** nur noch selten nötig. Bun wandelt CommonJS-Module automatisch in ES6-Module um.

package.json Datei

- Node.js nutzt die Datei **package.json** zur Beschreibung des Projekts.
- Die Datei enthält Informationen zum Projekt, wie Name, Version, Autor, Lizenz, Abhängigkeiten, Skripte, etc.
- Mit dem Befehl `pnpm init` wird ein Node-Projekt aus dem Projektverzeichnis heraus initialisiert. Dabei wird auch die Package-Datei erstellt.
- Mit dem Befehl `bun init` wird ein Bun-Projekt initialisiert.
- Die Datei kann mit einem Texteditor bearbeitet werden.
- Falls in der Datei das Attribut **type** auf **module** gesetzt wird, werden die Dateien im Projekt als mit dem Befehl **import** ladbare Module angesehen. Ist der Wert `commonjs` oder ist das Attribut nicht vorhanden, werden die Dateien als mit dem Befehl **require** ladbare Module angesehen.
- Node.js betrachtet alle Dateien als CommonJS-Module, wenn die Dateiergung `.js` oder `.cjs` ist. Dateien mit der Endung `.mjs` werden als ES6-Module betrachtet.

5.2.1. CommonJS-Module

Core-Module laden

```
1 | import path from 'node:path';
2 | const p = path.join('foo', 'bar'); # p = 'foo/bar'
```

Die exportierten Funktionen, Variablen und Objekte des Moduls `path` werden mit der Variablen `path` referenziert. Der Pfad zum Modul wird als String nach dem Schlüsselwort **from** angegeben. Das Präfix `node:` ist in Node optional und kann weggelassen werden, in Bun muss es genutzt werden. Es muss immer genutzt werden, wenn der Name eines eigenen Moduls mit dem des Core-Module übereinstimmt.

5.2.2. Core-Module

Node.js enthält eine Reihe von Core-Modulen. Diese Module sind in der Binärdatei `node` enthalten und können direkt genutzt werden. Hierzu gehören Module für die Arbeit mit Dateien, dem Dateisystem, dem Betriebssystem, dem Netzwerk, der Datenkommunikation, der Datenverschlüsselung. Eine vollständige Liste der Core-Module kann mit dem Befehl

```
1 | bun -p "require('node:module').builtinModules"
```

erhalten werden.

Beispiel 37: Zugriff auf das Dateisystem

```

1 import fs from 'node:fs';
2 fs.readFile('datei.txt', 'utf8', function(err, data) {
3   if (err) throw err;
4   console.log(data);
5 });

```

Beispiel 38: Zugriff auf das Betriebssystem

Laden als CommonJS-Modul:

```

1 const os = require('os');
2 console.log(os.platform());
3 console.log(os.arch());
4 console.log(os.freemem());
5 console.log(os.totalmem());

```

Die zur Verfügung stehenden Funktionen sind unter <https://nodejs.org/api/os.html> beschrieben.

Beispiel 39: Elementarer Webserver

```

1 import http from 'http';
2 const server = http.createServer(function(req, res) {
3   res.writeHead(200, {'Content-Type': 'text/plain'});
4   res.end('Hello World\n');
5 });
6 server.listen(3000);

```

Dies referenziert alle Objekte des Moduls http über http. Das exportierte Objekt enthält die Funktion `createServer`, die einen Webserver erstellt. Der Webserver wird auf dem Port 3000 gestartet.

Die Fähigkeiten des http-Modules sind unter <https://nodejs.org/api/http.html> beschrieben. Jedoch ist es einfacher, einen Webserver mit dem Express-Modul zu implementieren.

5.2.3. Eigene ES6-Module

Eine Datei ist ein ES6-Modul, wenn die Dateiendung `.mjs` ist oder das Attribut `type` in der Datei `package.json` auf `module` gesetzt ist. Objekte werden exportiert indem sie mit dem Schlüsselwort `export` gekennzeichnet werden. Höchstens ein Objekt kann mit dem Schlüsselwort `export default` gekennzeichnet werden. Dieses Objekt wird als Standardobjekt exportiert.

Meist werden Referenzen auf die zu exportierenden Objekte in einem Objekt gesammelt und dann dieses Objekt exportiert anstatt jedes einzelne Objekt mit `export` zu kennzeichnen.

Beispiel 40: ES6-Modul

Das Module `meinmodul.js`

```

1 const PI = 3.141592653589793;
2
3 function addiere(a, b) {
4   return a + b;
5 }
6
7 function subtrahiere(a, b) {
8   return a - b;
9 }
10
11 export { PI, addiere as default };

```

kann mit

```
1 | import addiere, { PI } from "./meinmodul.js";
2 | console.log(addiere(1, PI));
```

importiert werden.

Nur die angeführten Objekte werden importiert. Soll nur das Standardobjekt importiert werden, kann dies mit

```
1 | import add from "./meinmodul.js";
2 | console.log(add(1, 2));
```

geschehen. Dabei wird das Standardobjekt ohne geschweifte Klammern importiert. Dem Standardobjekt kann dabei ein beliebiger Name zugewiesen werden. Sollen alle Objekte importiert werden, kann dies mit

```
1 | import * as mm from "./meinmodul.js";
2 | console.log(mm.default(1, mm.PI));
```

erfolgen. Importierte Objekte können auch umbenannt werden.

```
1 | import { PI as pi, default as add } from "./meinmodul.js";
2 | console.log(add(1, pi));
```

Vorsicht

ES6-Importe funktionieren nicht im interaktiven Modus von Node.js.

5.2.4. Externe Module

Externe Module

Externe Module werden für Node mit dem Paketmanager `pnpm`² installiert. Der Paketmanager kann mit `npm install -g pnpm` installiert werden. Der Paketmanager wird über die Kommandozeile mit dem Befehl `pnpm` aufgerufen. Er lädt die Module aus dem Paketarchiv <https://www.npmjs.com/>. Bun verfügt über einen eingebauten Paketmanager und Module werden mit `bun add <modulname>` installiert. Die Module werden in ein lokales Verzeichnis installiert und in das Verzeichnis `node_modules` im aktuellen Projektverzeichnis verlinkt. Die Befehle `pnpm init` bzw. `bun init` initialisieren ein neues Projekt und erstellen die Datei `package.json`. Die Befehle `pnpm install` bzw. `bun install` installieren die in der Datei `package.json` aufgeführten Module. Die Befehle `pnpm add <modulname>` bzw. `bun add <modulname>` installieren das Modul `<modulname>`. Der Befehl `pnpm add <modulname> -g` bzw. `bun add <modulname> -g` installieren das Modul `<modulname>` global. Der Befehl `pnpm add <modulname> --save` installiert das Modul `modulname` und trägt es in die Datei `package.json` ein.

Globale Installation

Module, die global installiert werden, können von allen Projekten genutzt werden. Der Befehl `pnpm root -g` gibt den Pfad zu diesem globalen Verzeichnis aus. Der Befehl `pnpm list -g` listet alle global installierten Module auf.

Vorsicht

Die globale Installation von Modulen sollte vermieden werden. Es kann zu Konflikten zwischen verschiedenen Versionen von Modulen kommen. Daher sollten Module immer lokal installiert werden.

Entwicklungsmodule

Module, die nur für die Entwicklung benötigt werden, werden mit dem Befehl `pnpm add <modulname> --save-dev` bzw. `bun add -d <modulname>` installiert. Dieser Befehl wird für Module benutzt, die

²Wir bevorzugen `pnpm` über den mitgelieferten Paketmanager `npm`

nur während der Entwicklung, aber nicht im ausführbaren Code benötigt werden. Diese Module werden in der Datei `package.json` in einen speziellen Bereich eingetragen. Bei diesen Modulen handelt es sich um Module, die für die Entwicklung benötigt werden, wie z.B. Testframeworks, Dokumentationsgeneratoren, etc. Ihr Code wird nicht in die Produktivumgebung übernommen.

Module und Git

- Die Datei `package.json` enthält die Liste der Module, die für das Projekt benötigt werden.
- Die von npm erzeugte Datei `package-lock.json` enthält die genauen Versionen der Module, die für das Projekt installiert wurden.
- Die von pnpm erzeugte Datei `pnpm-lock.yaml` enthält die genauen Versionen der Module, die für das Projekt installiert wurden.
- Die von bun erzeugte Datei `bun.lockb` enthält die genauen Versionen der Module, die für das Projekt installiert wurden, analog zu `package-lock.json` und `pnpm-lock.yaml`.
- Die Datei `node_modules` enthält die Module, die für das Projekt installiert wurden.
- Die Datei `.gitignore` enthält das Verzeichnis `node_modules`, da dieses nicht in das Versionskontrollsystem übernommen werden sollten.

Modulmanager

Der Modulmanager npm wird standardmäßig mit Node.js installiert. Der Modulmanager yarn ist eine von Facebook veröffentlichte Alternative zu npm. In den letzten Jahren hat sich jedoch pnpm als schnellster Modulmanager etabliert. pnpm installiert Module in einem globalen Cache und erstellt symbolische Links auf die Module. Dies spart Speicherplatz und Zeit. Der Befehl pnpm wird wie npm aufgerufen. Wir werden in diesem Skript den in bun eingebauten Modulmanager verwenden.

Module laden

`bun add <modul>` sucht in der NPM-Registry nach dem Modul `<modul>`. Die Registry ist unter <https://www.npmjs.com/> erreichbar. Die Registry enthält über 1,5 Millionen Module. Die Registry enthält auch Module, die nicht mehr gepflegt werden. Daher sollte bei der Auswahl eines Moduls auf die Anzahl der Downloads, die Aktualität und die Anzahl der Issues geachtet werden.

5.2.5. Infrastruktur

Probleme in der Modul-Infrastruktur

- Die Modul-Infrastruktur von Node.js ist komplex.
- Module haben oft weitreichende Abhängigkeiten. Damit können mit der Installation eines Moduls viele abhängige Module mitinstalliert werden.
- Module werden oft nicht mehr gepflegt.
- Während der Installation eines Moduls wird oft vor schwerwiegenden Fehlern in den Abhängigkeiten gewarnt.
- Oft werden selbst einfache Funktionalitäten als externes Modul geladen anstatt diese selbst zu implementieren.

Unnötige Module

Es gibt sehr viele Module, die nur eine Funktionalität implementieren, die auch mit den Standardfunktionen von JavaScript implementiert werden kann.

Beispiel 41: Das Modul isarray

Das Modul `isarray` prüft, ob ein Objekt ein Array ist. Dies kann mit dem Befehl `Array.isArray(obj)` geprüft werden. Das Modul `isarray` ist unter <https://www.npmjs.com/package/isarray> verfügbar und hat zur Zeit (Anfang 2024) über 40 Millionen downloads pro Woche. Über 1500 npm-Module benutzen dieses Modul. Das Modul besteht aus einer einzigen Funktion:

```
1 var toString = {}.toString;
2
3 module.exports = Array.isArray || function (arr) {
4   return toString.call(arr) == '[object Array]';
5 };
```

Das left-pad Disaster

Das Module `left-pad` fügt einer Zeichenkette Leerzeichen hinzu, bis die Zeichenkette eine bestimmte Länge hat. Dies kann mit dem Befehl `str.padStart(len, ' ')` erreicht werden. Das Modul ist als

```
1 module.exports = leftpad;
2 function leftpad (str, len, ch) {
3   str = String(str);
4   var i = -1;
5   if (!ch && ch !== 0) ch = ' ';
6   len = len - str.length;
7   while (++i < len) {
8     str = ch + str;
9   }
10  return str;
11 }
```

implementiert.

Der Autor von `leftpad` löschte 2016 sein Paket aus dem NPM-Repository. Dadurch wurden viele Module, wie etwa Facebook, React, Babel, die `leftpad` benutzten, nicht mehr installierbar. Dieses Ereignis wird als `left-pad Disaster` bezeichnet. Es führte zu einer Diskussion über die Abhängigkeiten von Modulen und die Verwendung von Modulen, die nur eine einzige Funktionalität implementieren.

Näheres kann bei „The Register“ unter https://www.theregister.com/2016/03/23/npm_left_pad_chaos/ und bei „heise.de“ unter <https://www.heise.de/news/JavaScript-Paket-aus-NPM-entfernt-Node-Babel-und-Co-scheiterten-beim-Build-3148796.html> gefunden werden.

Datei-Inflation

Die Modul-Infrastruktur von Node.js führt zu einer Datei-Inflation. Die Installation eines Moduls kann zu einer großen Anzahl von Dateien führen. Dies ist insbesondere bei einer größeren Zahl von Abhängigkeiten der Fall. Inzwischen wird aber aktiv versucht, die Anzahl der Dateien in den Modulen zu reduzieren, wie aus den Unterschieden in den Dateizahlen zwischen Node-Modulen und auf Github ersichtlich ist. Diese Reduktion ist wahrscheinlich mit auf die Nutzung Web-Bundlern, wie etwa Webpack, zurückzuführen.

Modul	Anzahl Dateien	Dateien auf Github
typescript	100	70000
express	1000	2500
webpack	2500	7500
babel	3000	30000

5.3. Bun-Spezifika

Bun hat einige Besonderheiten, die es von Node.js unterscheiden.

- Bun hat einen eingebauten Paketmanager.
- Bun unterstützt TypeScript direkt.
- Bun hat eine eingebaute Testumgebung.
- Bun kann nativ ausführbare Dateien kompilieren.
- Bun enthält einen eingebauten Webserver `Bun.serve`. Dieser kann als Alternative zum Node-HTTP-Modul und `http.createServer` genutzt werden.

Vorsicht

Wir werden in der Veranstaltung die meisten dieser Features nicht benutzen. Wir nutzen später die Nuxt und Vite Umgebungen für das Vue-Framework. Der Bun-Bundler und -Webserver kennt deren Organisationsstruktur nicht und sollte daher in diesem Kontext nicht genutzt werden. Wir nutzen den Paketmanager und die eingebaute Unterstützung für TypeScript.

5.4. Bundler

Ein *Bundler* ist traditionell ein Werkzeug, das verschiedene Module und Abhängigkeiten in einer einzigen Datei zusammenfasst. Dies ist geboten, da JavaScript-Module in der Regel in mehreren Dateien organisiert sind. Der Browser kann jedoch nur eine Datei auf einmal laden. Ein Bundler löst dieses Problem, indem er alle Module in einer Datei zusammenfasst.

Oft wird ein Bundler auch als *Task Runner* bezeichnet, da er auch Aufgaben wie das Minimieren von Dateien oder das Kompilieren von Dateien übernehmen kann.

Vorteile

- Bündelt alle Abhängigkeiten in einer Datei
- Ermöglicht das Verwenden von Modulen
- Ermöglicht das Verwenden von npm-Paketen
- Ermöglicht das Verwenden von CSS-Präprozessoren
- Ermöglicht das Verwenden von TypeScript
- Ermöglicht das Verwenden von Angular, React, Vue oder Svelte

Nachteile

- Erhöht die Komplexität des Projekts
- Erhöht die Ladezeit der Webseite
- Erhöht die Größe der Datei

Verbreitete Bundler

- Browserify
- Webpack (sehr verbreitet)
- Parcel (einfach zu bedienen)
- Rollup (Teil von Vite)
- esbuild (sehr schnell)
- bun (integrierter Bundler, sehr schnell)

Mit Ausnahme von esbuild und bun sind alle Bundler in JavaScript geschrieben. Esbuild ist in Go und bun ist in Zig geschrieben. Beide sind daher sehr schnell. Allerdings nutzt esbuild das ES6-Modulsystem und erzeugt sehr viele Dateien anstatt nur einer Datei. Daher ist esbuild für die Entwicklungsphase besser geeignet als für die Produktionsphase. esbuild erzeugt aus Typescript-Dateien JavaScript-Dateien indem die Typinformationen entfernt werden. Damit findet keine Typüberprüfung statt. Diese sollte vorab in der Entwicklungsumgebung erfolgen.

Tree-Shaking

Ein weiterer Vorteil von Bundlern ist das *Tree-Shaking*. Dabei werden nur die Module in die Datei eingebunden, die auch tatsächlich verwendet werden. Dies reduziert die Größe der Datei und damit die Ladezeit der Webseite.

Der Name Tree-Shaking kommt daher, dass der Bundler den Baum der Abhängigkeiten durchsucht und alle nicht benötigten Module entfernt.

Bun als neuer Bundler

Bun hat bereits einen eigenen Bundler integriert, der mit `bun build` aufgerufen wird. Dieser ist für viele Anwendungsfälle ausreichend. Für Anwendungen die Vite oder Nuxt verwenden ist der eingebaute Bundler von Bun jedoch nicht geeignet, da dieser keine Informationen über die jeweils Framework-spezifische Architektur hat.

5.5. Das Vite-Modul

Vite als Build-Tool für Vue-Anwendungen:

- Vite ist ein Modul für Node.js
- für Front-End-Anwendungen
- Unterschiedliche Bundler für Entwicklung und Produktion

Wir werden Vite in der Regel mit dem Vue und TypeScript verwenden für Frontend-Anwendungen verwenden. Vite ist ein Modul, das die Entwicklung von Webanwendungen mit Node.js erleichtert. Es ist auf reine Front-End Anwendungen ausgelegt und enthält auch die Web Bundler `Rollup` und `esbuild`. In der Entwicklungsphase wird ein lokaler Server gestartet, der die Anwendung ausliefert. Dabei werden die Dateien bei Bedarf mit `esbuild` transpiliert (TypeScript). In der Produktionsphase werden die Dateien kompiliert und mit `Rollup` gebündelt und können dann von einem beliebigen Webserver ausgeliefert werden.

Vue-Router, Vite und Nuxt

In den Anwendungen der Veranstaltung wird Vue Router für Single Page Applications verwendet. Vite kann jedoch auch mit anderen Webservern verwendet werden. Insbesondere stellt Nuxt automatisches Routing und Server-Side-Rendering für Vue-Anwendungen bereit.

Dies ist jedoch für den Einstieg zu komplex und wird daher erst später behandelt.

Vite nutzt also unterschiedliche Bundler für die Entwicklung und die Produktion. Zur Zeit wird der Bundler Rollup in der Sprache Rust neu implementiert um die Ausführungsgeschwindigkeit zu erhöhen. Nach Abschluss dieser Arbeiten werden Rollup und esbuild durch Rolldown ersetzt.

Ein Vite-Projekt mit Vue und TypeScript kann mit dem Befehl

```
1 | bun create vite@latest my-vue-app
```

erzeugt werden. Alternativ kann auch die Kurzform

```
1 | bun create vite
```

verwendet werden. In diesem Fall wird nach dem Template gefragt. Hier sollte dann vue und TypeScript ausgewählt werden. Das Projektverzeichnis wird dann my-vue-app genannt.

In VS-Code sollte das Plugin **Vue Volar extension pack** installiert werden. Dieses Plugin enthält die notwendigen Erweiterungen für die Entwicklung von Vue-Anwendungen mit TypeScript.

Installation der Abhängigkeiten

```
1 | cd my-vue-app
2 | bun install
```

TypeScript und dev vs. build

in `package.json` gibt es die Einträge

```
1 | "scripts": {
2 |   "dev": "vite",
3 |   "build": "vue-tsc && vite build",
4 |   "preview": "vite preview"
5 | },
```

zur Ausführung von Vite.

`bun run dev` startet vite als lokalen Server mit dem Projektverzeichnis als Root-Verzeichnis. Daher sind hier auch die Dateien `index.html` und `vite.config.ts` zu finden. `bun run build` kompiliert und bündelt die Anwendung in ein Auslieferungsverzeichnis. `bun run preview` startet einen lokalen Server, der die kompilierte Anwendung ausliefert. Dieser ist nicht für den Produktiveinsatz gedacht, sondern dient nur der Überprüfung der kompilierten Anwendung.

Vorsicht:

vite transpiliiert TypeScript-Dateien mit esbuild, nicht mit dem regulären TypeScript-Compiler tsc. Daher wird auch keine Überprüfung auf Typsicherheit durchgeführt. Diese ist nur im build-Schritt vorgesehen. Für die Entwicklung wird in der Regel `bun run dev` verwendet. Typfehler werden dann nur IDE-intern (z.B. im Syntax-Check von VS-Code) angezeigt.

5.6. Das ViewModel Framework Vue

Vue

- JavaScript Framework für die Entwicklung von Benutzeroberflächen.
- Konzentriert sich auf das ViewModel (VM) im MVVM- Muster³.
- Erlaubt komponentenbasierte Entwicklung.

³Hier wird der Controller im MVC-Muster durch das ViewModel ersetzt.

- Arbeitet mit einem virtuellen DOM.
- Wird in der Version 3 seit September 2020 entwickelt.
- Ist auf <https://vuejs.org> dokumentiert.

5.6.1. Scaffolding

Vite-Scaffolding für Vue

Die Verzeichnisstruktur eines mit Vite erzeugten Vue-Projekts:

my-vue-app

```
+--- src
|   +--- App.vue
|   +--- main.ts
|   +--- style.css
|   +--- components
|       |   +--- HelloWorld.vue
|   +--- assets
|       |   +--- vite.svg
+--- public
|   +--- vite.svg
|
|   +--- index.html
|   +--- package.json
|   +--- vite.config.ts
|   +--- tsconfig.json
|   +--- node_modules
|       |   +--- ...
+--- dist
```

5.6.2. Die Dateien einer Vue-Anwendung

Die index.html Datei

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>Vite + Vue + TS</title>
8   </head>
9   <body>
10    <div id="app"></div>
11    <script type="module" src="/src/main.ts"></script>
12  </body>
13 </html>
```

lädt die durch `id="app"` identifizierte Anwendung durch Ausführung der Datei `main.ts`. Als Icon wird die Datei `vite.svg` verwendet.

Die Datei main.ts

```
1 import { createApp } from 'vue'
2 import App from './App.vue'
3 import './style.css'
4
5 createApp(App).mount('#app')
```

erzeugt die Anwendung aus der Datei `App.vue` und bindet die Datei `style.css` ein. Die Anwendung wird in das Element mit der Id `app` eingebunden.

Die Datei `App.vue`

ist eine Single-File-Component (SFC) und enthält HTML, CSS und JavaScript bzw. TypeScript und stellt die Wurzelkomponente der Anwendung dar.

```

1 <script setup lang="ts">
2 import HelloWorld from './components/HelloWorld.vue'
3 </script>
4
5 <template>
6   <div>
7     <a href="https://vitejs.dev" target="_blank">
8       
9     </a>
10    <a href="https://vuejs.org/" target="_blank">
11      
12    </a>
13  </div>
14  <HelloWorld msg="Vite + Vue" />
15 </template>
16
17 <style scoped>
18 .logo {
19   height: 6em;
20   padding: 1.5em;
21   will-change: filter;
22   transition: filter 300ms;
23 }
24 .logo:hover {
25   filter: drop-shadow(0 0 2em #646cfff);
26 }
27 .logo.vue:hover {
28   filter: drop-shadow(0 0 2em #42b883aa);
29 }
30 </style>

```

Der `<script setup lang="ts">`-Block enthält den TypeScript-Code, der `<template>`-Block das HTML und der `<style>`-Block das CSS. Hier wird die Komponente `HelloWorld` eingebunden, die in der Datei `components/HelloWorld.vue` definiert ist. Sie wird mit dem Parameter `msg` aufgerufen. Das Attribut `scoped` im `<style>`-Block sorgt dafür, dass die CSS-Definitionen nur für die Komponente gelten.

Die `HelloWorld.vue` Komponente

```

1 <script setup lang="ts">
2 import { ref } from 'vue'
3
4 defineProps<{ msg: string }>()
5
6 const count = ref(0)
7 </script>
8 <template>
9   <h1>{{ msg }}</h1>
10
11   <div class="card">
12     <button type="button" @click="count++>count is {{ count }}</button>
13     <p>
14       Edit

```

```
15 |         <code>components/HelloWorld.vue</code> to test HMR
16 |     </p>
17 | </div>

18 | <p>
19 |   Check out
20 |   <a href="https://vuejs.org/guide/quick-start.html#local" target="_blank">
21 |     create-vue </a>, the official Vue + Vite starter
22 | </p>
23 | <p>
24 |   Install
25 |   <a href="https://github.com/vuejs/language-tools" target="_blank">Volar</a> in
26 |   your IDE for a better DX
27 | </p>
28 | <p class="read-the-docs">Click on the Vite and Vue logos to learn more</p>
29 | </template>
30 |
31 | <style scoped>
32 | .read-the-docs {
33 |   color: #888;
34 | }
35 | </style>
```

Parameterübergabe an Komponenten

Im Script-Block wird die von Vue automatisch zur Verfügung gestellte Funktion `defineProps` aufgerufen, um die Parameter zu definieren. Zwischen den spitzen Klammern werden die Typen der Parameter als JavaScript-Objekt angegeben. Die Parameter werden dann im Template-Block mit `{{ msg }}` verwendet. In der aufrufenden Komponente wird der Parameter mit `msg="Vite + Vue"` übergeben.

Reaktive Daten

Die Funktion `ref` erzeugt ein reaktives Objekt, das mit `count.value` gelesen und mit `count.value = 1` geschrieben werden kann. Die Variable `count` wird im Template-Block mit `{{ count }}` verwendet. Mit `@click="count++"` wird der Wert von `count` inkrementiert. Dabei ist `@click` eine Kurzform des Vue-Event-Handlers `v-on:click` der auf das Klicken des Buttons reagiert. Mit dem „Mustache-Operator“ `{{ count }}` wird der Wert von `count` im Template-Block angezeigt.

Anmerkung

Damit verhalten sich die reaktiven Objekte in Vue ähnlich wie die `state`-Objekte in React oder wie `Properties` mit bidirektionalem Binding in JavaFX.

Anmerkung

Mehr zu Vue wird nachfolgend jeweils kontextbezogen bei Bedarf behandelt.

6. JavaScript und TypeScript Grundlagen

Die wichtigsten Eigenschaften von JavaScript und TypeScript sind:

JavaScript

- Skriptsprache
- Client-seitig
- Dynamisch typisiert
- Objektorientiert
- Wird von allen gängigen Browsern unterstützt

TypeScript

- Superset von JavaScript
- Statisch typisiert
- Objektorientiert
- Wird zu JavaScript transpiliert.
- Dateiendung `.ts`

Aus

einer TypeScript-Datei wird mit dem TypeScript-Compiler `tsc` eine JavaScript-Datei erzeugt. In den nachfolgenden Beispielen wird dieser Schritt nicht immer explizit ausgeführt. Es ist oft einfacher die TypeScript-Datei direkt mit dem Node-Wrapper `tsx` auszuführen. Hierzu müssen `tsx` und TypeScript mit `bun add -D typescript tsx` installiert worden sein..

6.1. Datentypen

Zahlen

JavaScript und TypeScript kennen ursprünglich nur den Datentyp **number**. Dieser ist als IEEE 64-bit Gleitkommazahl implementiert. Es gibt keine Ganzzahlen. Eine Zahl kann mit einem Dezimalpunkt oder in Exponentialdarstellung geschrieben werden.

Beispiel 42: number.ts

```
1 | const x : number = 1;
2 | console.log(typeof(x));
```

In TypeScript kann der Datentyp explizit angegeben werden, jedoch kann TypeScript häufig auch den benötigten Datentyp aus dem Kontext erschliessen.

```
1 | > tsc number.ts // hängt von Konfiguration ab
2 | > node number.js
3 | number
```

```
1 | > tsx number.ts
2 | number
```

IEEE 64-bit Gleitkommazahlen

- IEEE 64-bit Gleitkommazahlen haben eine Genauigkeit von 15-17 signifikanten Dezimalstellen.
- Die größte darstellbare Zahl ist $1.7976931348623157 \times 10^{308}$.
- Die kleinste darstellbare Zahl ist 5.0×10^{-324} .
- 1 ulp (unit in the last place) ist $2.220446049250313 \times 10^{-16}$.
- Es sind die üblichen Rechenoperationen (+, -, *, /, %) möglich, jedoch können Rundungsfehler auftreten.

- 0 kann als „false“ interpretiert werden. Alle anderen Zahlen werden gegebenenfalls als „true“ interpretiert.

Spezielle Werte von IEEE Fließkommazahlen

- **NaN** (Not a Number) ist ein spezieller Wert, der das Ergebnis einer ungültigen Operation repräsentiert. **NaN** ist nie gleich zu einer Zahl, auch nicht zu sich selbst.
- **Infinity** und **-Infinity** sind spezielle Werte, die das Ergebnis einer Division durch Null repräsentieren.

Beispiel 43:

```

1 console.log(1/0); // Infinity
2 console.log(-1/0); // -Infinity
3 console.log(1/Infinity); // 0
4 console.log(0/0); // NaN
5
6 console.log(NaN == NaN); // false
7 console.log(NaN === NaN); // false

```

```

1 console.log(Infinity + 5) //Infinity
2 console.log(Infinity - Infinity) //
  Infinity
3 console.log(Infinity * 0) // NaN
4 // NaN, +/-Infinity are numbers:
5 console.log(typeof(NaN))
6 console.log(typeof(Infinity))
7 console.log(typeof(-Infinity))

```

Strings

Zeichenketten werden in JavaScript und TypeScript mit einfachen oder doppelten Anführungszeichen geschrieben. Der Datentyp ist **string**.

Beispiel 44: string.ts

```

1 const x : string = "Hallo";
2 console.log(typeof(x));

```

Analog zum Zahlen-Beispiel ist die Ausgabe hier **string**.

Vorsicht: automatische Typumwandlungen

```

1 6 + "3" // "63"
2 6 - "3" // 3
3 6 * "3" // 18
4 "6" / "3" // 2

```

Vorsicht

Es gibt einen Unterschied zwischen **String** und **new String**

```

1 const s1 = "Hallo";
2 console.log(typeof(s1)); // string
3
4 const s2 = String("Hallo");
5 console.log(typeof(s2)); // string
6
7 const s3 = new String("Hallo");
8 console.log(typeof(s3)); // object
9 console.log(typeof(s3.toString())); // string
10
11 s2 == s3; // true
12 s2 === s3; // false

```

`new String` erzeugt ein Objekt mit der Eigenschaft `String`, während `String` einen `string` erzeugt.

Booleans

Der Datentyp `boolean` kann die Werte `true` oder `false` annehmen. Jedoch werden auch andere Werte als `true` oder `false` als `boolean` interpretiert. Dies kann manchmal überraschende Folgen haben.

Beispiel 45: boolean.ts

```
1 | const x : boolean = true;
2 | console.log(typeof(x));
```

hat die Ausgabe `boolean`.

Vorsicht

JavaScript zeigt für (leere) Arrays und (leere) Strings ungewöhnliches Verhalten:

<pre>1 [] == false; // true 2 [] == true; // false 3 4 "" == false; // true 5 "" == true; // false 6 7 [] == ""; // true 8 [0] == false; // true 9 "0" == false; // true</pre>	<pre>1 0 == false; // true 2 0 == ""; // true 3 0 == []; // true 4 5 if([]) console.log("true"); // true 6 if([] == true) console.log("true"); // keine Ausgabe 7 if("") console.log("true"); // keine Ausgabe 8 if(0) console.log("true"); // keine Ausgabe</pre>
---	--

Man bezeichnet dieses Verhalten als *truthy* und *falsy*. Es beruht auf optimistischen automatischen Typumwandlungen. Daher sollten Vergleiche mit `==` vermieden werden. Besser ist `===`, welches auch den Typ vergleicht.

null

Der Datentyp `null` impliziert das gewollte Fehlen eines Wertes. Dieser Wert wird verwendet, wenn eine Variable oder ein Objekt bewußt keine gültige Referenz enthält. Überraschenderweise ist der Datentyp von `null` `object`.

Beispiel 46: null.ts

```
1 | const x : null = null;
2 | console.log(typeof(x));
```

hat die Ausgabe `object`.

undefined

Der Datentyp `undefined` impliziert das ungewollte Fehlen eines Wertes. Dieser Wert wird verwendet, wenn eine Variable oder ein Objekt keine gültige Referenz enthält, weil sie nicht initialisiert wurde. Der Datentyp von `undefined` ist `undefined`.

Beispiel 47: undefined.js

```
1 | let x;
2 | console.log(x);
3 | console.log(typeof(x));
```

hat die Ausgabe

```
1 | undefined
2 | undefined
```

BigInt

Der Datentyp **bigint** ist eine neue Ergänzung zu JavaScript (ES2020). Er repräsentiert ganze Zahlen beliebiger Größe. Der Datentyp **bigint** wird durch Anhängen des Buchstabens **n** an eine ganze Zahl erzeugt.

Beispiel 48: bigint.ts

```
1 | const x : bigint = 1234567890123456789012345678901234567890n;
2 | console.log(typeof(x)); // bigint
3 | const b : bigint = BigInt("123456789012345678901234567890123456789");
4 | const h : bigint = 0x1234567890123456789012345678901234567890n;
5 | console.log(x/b); // 10n
```

JavaScript-Objekte

Die einfachste Möglichkeit ein Objekt in JavaScript zu erzeugen ist die geschweifte Klammer `{}`. Dieses Objekt ist leer. Ein Objekt besteht aus einer Menge von Eigenschaften. Eine Eigenschaft besteht aus einem Namen und einem Wert. Der Name ist ein String, der Wert kann ein beliebiger Datentyp sein. Einem Objekt können Eigenschaften hinzugefügt werden, indem der Name der Eigenschaft in eckigen Klammern oder als Attribut durch Punkt vom Objekt getrennt angegeben wird. Der Wert der Eigenschaft wird mit einem Gleichheitszeichen zugewiesen. Damit verhalten sich Objekte ähnlich wie assoziative Arrays bzw. Hash-Tabellen.

Beispiel 49: object.js

```
1 | const x = {};
2 | x["a 1"] = 1;
3 | x.a2 = 2;
4 | console.log(x);
5 | console.log(typeof(x));
```

ergibt die Ausgabe

```
1 | { 'a 1': 1, a2: 2 }
2 | object
```

TypeScript-Objekte

Die erlaubten Eigenschaften eines Objekts können in TypeScript mit einem Interface oder einem Type festgelegt werden. Ein Objekt muss dann alle Eigenschaften des Interfaces enthalten. Es darf aber auch weitere Eigenschaften enthalten. Die Eigenschaften eines Objekts können mit einem Fragezeichen auch als optional gekennzeichnet werden. Dann müssen sie nicht enthalten sein.

Beispiel 50: object.ts

als Interface

```
1 | interface X {
2 |     'a 1': number;
3 |     a2?: number;
4 | };
5 | const x: X = { 'a 1': 1, a2: 2};
```

als Type

```
1 | type X = {
2 |     'a 1': number;
3 |     a2?: number;
4 | };
5 | const x: X = { 'a 1': 1, a2: 2};
```

hier ist das Attribut `a2` als optional gekennzeichnet. Im Gegensatz zu JavaScript können in TypeScript Eigenschaften eines Objekts nicht nachträglich hinzugefügt werden.

Unterschied zwischen Interface und Type

Ein Interface kann erweitert werden. Ein Type kann nicht erweitert werden. Ein Interface kann mehrfach deklariert werden. Ein Type kann nur einmal deklariert werden. Ein Interface kann auch als Klasse verwendet werden. Ein Type kann nicht als Klasse verwendet werden.

Beispiel 51: interface.ts

```

1 interface X {
2     'a 1': number;
3 };
4 interface X {
5     a2?: number;
6 };
7 const x: X = {'a 1': 1, a2: 2};

```

Das Interface x wird hier zweimal deklariert. Dabei wird es erweitert. Dies ist bei einem Type nicht möglich.

Erzeugung von JavaScript-Objekten mit create()

In JavaScript gibt es die Funktion `Object.create()`. Diese Funktion erzeugt ein Objekt, das als Prototyp ein anderes Objekt hat. Das erzeugte Objekt enthält keine Eigenschaften. Die Eigenschaften des Prototyps werden als Eigenschaften des erzeugten Objekts verwendet. Die Eigenschaften des Prototyps können von den mit `create` erzeugten Objekten nicht verändert werden. Es können aber weitere Eigenschaften zum erzeugten Objekt hinzugefügt werden.

Beispiel 52: objectcreate.js

```

1 const parent = { z: 1 };
2 const child = Object.create(parent);
3 console.log(parent.isPrototypeOf(child));
4 console.log(Object.getPrototypeOf(child));
5 console.log(child.z);
6 child.z = 2;
7 console.log(child.z);
8 console.log(parent);

```

ergibt die Ausgabe

```

1 true
2 { z: 1 }
3 1
4 2
5 { z: 1 }

```

Erzeugung von JavaScript-Objekten mit new

In JavaScript gibt es die Möglichkeit, Objekte mit dem Schlüsselwort `new` zu erzeugen. Dazu wird eine Funktion verwendet, die mit dem Schlüsselwort `function` definiert wird. Die Funktion wird mit dem Schlüsselwort `new` aufgerufen. Die Funktion erzeugt ein Objekt und gibt dieses zurück. Das Schlüsselwort `this` bezeichnet das erzeugte Objekt. Die Eigenschaften des Objekts werden mit `this` definiert. Die Funktion wird als Konstruktor bezeichnet. Der Name der Funktion wird mit einem Großbuchstaben begonnen. Die Funktion wird mit dem Schlüsselwort `new` aufgerufen.

Vorsicht

Die Funktion kann auch ohne `new` aufgerufen werden. Falls der Code nicht auf "use strict" gesetzt wurde, wird `this` auf das globale Objekt gesetzt. Dies ist in den meisten Fällen nicht erwünscht. Falls "use strict" gesetzt wurde, wird ein Fehler erzeugt.

Object.create und Konstruktor-Funktionen in TypeScript

Weder `Object.create()` noch Konstruktor-Funktionen sind für die Erzeugung von Objekten in TypeScript vorgesehen.

Beispiel 53: objectnew.js

```

1 function X(x, y) {
2     this.x = x;
3     Object.setPrototypeOf(this, y);
4 }
5 const p = { z: 1 };
6
7 const x = new X(2, p);
8 console.log(x.z);
9 console.log(x);
10 console.log(Object.getPrototypeOf(x));

```

ergibt die Ausgabe

```

1 | 1
2 | { x: 2 }
3 | { z: 1 }

```

Konstruktor-Funktionen beginnen per Konvention mit einem Großbuchstaben.

Objekthierarchie

JavaScript und TypeScript kennen eine Objekthierarchie. Alle Objekte haben einen Prototyp. Der Prototyp eines Objekts ist ein anderes Objekt. Dieses Objekt hat wiederum einen Prototyp. Der Prototyp des Prototyps ist das Objekt `Object.prototype`. Dieses Objekt hat keinen Prototyp. Der Prototyp eines Objekts kann mit der Funktion `Object.getPrototypeOf()` ermittelt werden. Der Prototyp eines Objekts kann mit der Funktion `Object.setPrototypeOf()` gesetzt werden. Die Funktion `Object.create()` erzeugt ein Objekt mit einem bestimmten Prototyp.

Wird eine Eigenschaft in einem Objekt nicht gefunden, so sucht JavaScript oder TypeScript in den Prototypen des Objekts nach der Eigenschaft. Wird die Eigenschaft in keinem Prototyp gefunden, so wird `undefined` zurückgegeben.

Prototypische Vererbung

Damit werden in JavaScript und TypeScript die Eigenschaften eines „Basis-Objekts“ einem „Sub-Objekt“ über den Prototypen zur Verfügung gestellt. Dies wird als prototypische Vererbung bezeichnet.

In neuerem JavaScript und in TypeScript kann die prototypische Vererbung durch die Verwendung von Klassen und Vererbungsschlüsselwörtern wie `extends` und `super` simuliert werden. Dies wird als klassische Vererbung bezeichnet.

Intern benutzt JavaScript aber weiterhin die prototypische Vererbung.

Beispiel 54: Prototypische Vererbung

Zuerst wird eine Konstruktorfunktion `Person` als Basis definiert. Dieses Objekt hat eine Methode `greet()`. Dann wird ein Sub-Objekt `Student` definiert, das von `Person` erbt. Das Sub-Objekt hat eine eigene Methode `study()`.

```

1 function Person(name, age) { // Ctor-Funktion
2     this.name = name;
3     this.age = age;
4 }
5 Person.prototype.greet = function() { // Methode hinzufügen
6     console.log(`Name: ${this.name}, Alter: ${this.age}`);
7     return this; // erlaube Methodenverkettung
8 };
9 const person1 = new Person('Alice', 30); // Erzeuge Instanz

```

Der Konstruktor `Person` wird mit dem Schlüsselwort `new` aufgerufen. Dabei wird ein neues Objekt erzeugt. Die Methode `greet()` wird dem Prototypen von `Person` hinzugefügt. Damit ist die Methode für alle Instanzen von `Person` verfügbar. JavaScript setzt den Wert von `Person.prototype.constructor` automatisch auf `Person`.

`Student` erbt von `Person`. Der Konstruktor von `Student` ruft den Konstruktor von `Person` auf. Die Methode `greet()` wird von `Student` geerbt. Die Methode `study()` ist eine eigene Methode von `Student`.

```

10 function Student(name, age, major) {
11     Person.call(this, name, age);
12     this.major = major;
13 }
14 // Prototyp Student erbt von Person
15 Student.prototype = Object.create(Person.prototype);
16 Student.prototype.constructor = Student; // Setze Konstruktor zurück
17
18 // Füge eine Methode zum Student-Prototyp hinzu
19 Student.prototype.study = function() {
20     console.log(`Studienfach: ${this.major}`);
21     return this; // erlaube Methodenverkettung
22 };
23 const student1 = new Student('Anton', 20, 'Informatik');
24 student1.greet().study();

```

Arrays

In JavaScript sind Arrays mit einer Konstruktor-Funktion erzeugte Objekte. Auf die Elemente des Array-Objekts wird wie gewohnt mit einem Index in eckigen Klammern zugegriffen. Die Länge des Arrays kann mit der Eigenschaft `length` abgefragt werden. Über Zuweisung und die Eigenschaft `length` kann die Länge des Arrays verändert werden. Daneben können dem Array-Objekt auch beliebige Eigenschaften zugewiesen werden. Diese Eigenschaften werden nicht als Elemente des Arrays behandelt. Sie werden nicht bei der Iteration über das Array berücksichtigt. Sie werden auch nicht bei der Abfrage der Länge des Arrays berücksichtigt.

Anstelle der Konstruktor-Funktion kann auch die Array-Literal-Schreibweise verwendet werden. Diese Schreibweise ist kürzer und übersichtlicher.

Beispiel 55: Arrays

```

1 // Array mit 3 Elementen
2 const a1 = new Array(1, 2, 3);
3 // füge 4. Element hinzu
4 a1[3] = "vier";
5
6 // Array mit 4 Elementen, jeweils undefined
7 const a2 = new Array(4);
8
9 // Array mit 3 Elementen
10 const a3 = [1, 2, 3];
11 // füge dem Array-Objekt ein Element hinzu
12 a3.vier = "vier";
13
14 // leeres Array
15 const a4 = [];

```

Array-Methoden

Arrays haben eine Reihe von Methoden analog zu denen des Java List-Interfaces. Ebenso können Arrays direkt als Stacks (`push`, `pop`) und Warteschlangen (`shift`, `unshift`) verwendet werden. Die Methode `at` ist analog zu `get` in Java. Allerdings kann `at` auch negative Indizes verwenden. Dann wird vom Ende des Arrays gezählt. Zudem können mit `map`, `forEach`, `filter`, `reduce` und `find` die Elemente eines Arrays verarbeitet werden.

Arrays und automatische Typumwandlungen

Auch für Arrays gibt es in JavaScript einige fragwürdige automatische Typumwandlungen

Beispiel 56:

```

1 const a = 5;
2 const b = [];
3 a + b; // "5"
4 b.push(4); // [4]
5 b + a; // "45"
6 b * a; // 20 (4 * 5)

```

```

7 | b == 4; // true
8 | b.push(2); // [4, 2]
9 | b == 4; // false
10 | b == 42; // false
11 | b + ""; // "4,2"

```

Typisierung von Arrays

In TypeScript können Arrays mit einem Type oder einem Interface typisiert werden. Die Elemente des Arrays müssen dann dem Type oder dem Interface entsprechen. Die Länge des Arrays wird nicht berücksichtigt. Die Länge des Arrays kann mit einem Type oder einem Interface nicht festgelegt werden.

Beispiel 57: array.ts

als Type

```
1 | const x: number[] = [1,2,3];
```

als generisches Array

```
1 | const x: Array<number> = [1,2,3];
```

oder mit einem komplexeren Type oder Interface

```

1 | interface T { x: number; y: string };
2 |
3 | const x: Array<T> = [ // oder T[]
4 |   { x: 1, y: "zwei" },
5 |   { x: 2, y: "drei" }
6 | ];

```

Indizierbare Objekte in TypeScript

Indizierbare Objekte sind Objekte, die mit eckigen Klammern und einem Index verwendet werden können. Sie müssen ein `index`-Attribut haben, das den Typ des Index angibt. Sie müssen ein `length`-Attribut haben, das den Typ der Länge angibt. Sie können beliebige weitere Attribute haben, die den Typ der Elemente angeben. Die Elemente können dann mit eckigen Klammern und einem Index oder mit einem String-Index verwendet werden. Derartige Objekte verhalten sich wie Hash-Tabellen. Sie haben keine der üblichen Array-Methoden. Sie können aber mit den Methoden `Object.keys` und `Object.values` in Arrays umgewandelt werden.

Beispiel 58: indizierbare Objekte

```

1 | interface X {
2 |   [index: number]: number;
3 |   length: number;
4 | }
5 | const x: X = [1, 2, 3];
6 | x[3] = 4;
7 | for(let k in x) {
8 |   console.log(k); // 0, 1, 2, 3
9 | }
10 |
11 | // x["vier"] = 4; // geht nicht
12 | console.log(x.length); // 4

```

Vorsicht

Intern werden indizierbare Objekte als Objekte mit String-Attributen implementiert. Allerdings ist die Verwendung von Strings als Index weniger effizient als ein klassischer Array. Die Implementierung entspricht einer Hash-Tabelle. Daher sollte sie vermieden werden. Hier entspricht das Array-Objekt dem Interface mit der Einschränkung, dass nur Zahlen im Array enthalten sein können.

Dies funktioniert wie die Einschränkung der möglichen Methodenaufrufe von Java-Instanzen auf die Methoden in einem Interface. Daher stehen auch die üblichen Array-Methoden nicht zur Verfügung. Das Objekt hat jedoch die Eigenschaften `length` und `index`. Ebenso können die Elemente mit eckigen Klammern und einem Index verwendet werden, da das Objekt den Array-Prototyp erbt.

Arrays-Methoden für Stack und Warteschlange

push fügt ein Element am Ende des Arrays ein

pop entfernt das letzte Element des Arrays und gibt es zurück

shift entfernt das erste Element des Arrays und gibt es zurück

unshift fügt ein Element am Anfang des Arrays ein

Beispiel 59:

Anwendung der Array-Methoden

```
1 | const a = [1, 2, 3];
2 | a.push(4);
3 | a.pop();
4 | a.shift();
5 | a.unshift(0);
```

führt zu folgenden Inhalten von a:

```
1 | [1, 2, 3]
2 | [1, 2, 3, 4]
3 | [1, 2, 3]
4 | [2, 3]
5 | [0, 2, 3]
```

Vorsicht

shift und **unshift** sind sehr ineffizient. Sie müssen alle Elemente des Arrays verschieben. Daher sollten sie nur für kleine Arrays verwendet werden.

Array-Methoden für Iteration

forEach führt eine Funktion für jedes Element des Arrays aus

map führt eine Funktion für jedes Element des Arrays aus und gibt ein neues Array mit den Ergebnissen zurück.

filter führt eine Funktion für jedes Element des Arrays aus und gibt ein neues Array mit den Elementen zurück, für die die Funktion **true** zurückgibt.

reduce führt eine Funktion für jedes Element des Arrays aus und kombiniert die Ergebnisse zu einem einzelnen Wert.

find führt eine Funktion für jedes Element des Arrays aus und gibt das erste Element zurück, für das die Funktion **true** zurückgibt.

findIndex führt eine Funktion für jedes Element des Arrays aus und gibt den Index des ersten Elements zurück, für das die Funktion **true** zurückgibt.

Es gibt noch sehr viele weitere Methoden. Für Details wird auf die Dokumentation verwiesen.

Vorsicht mit Array.sort

```
1 | const a = [9, 8, 101, 10, 6, 7];
2 | a.sort(); // [10, 101, 6, 7, 8, 9]
3 | function cmp(a, b) {
4 |     return Math.sign(a - b);
5 | }
6 | a.sort(cmp); // [6, 7, 8, 9, 10, 101]
```

Falls der Vergleichs-Operator nicht definiert ist, wird die **toString**-Methode verwendet und **sort** sortiert lexikografisch. Daher werden die Zahlen als Strings verglichen und die Reihenfolge ist nicht die erwartete. Daher sollte **sort** immer mit einer Vergleichsfunktion verwendet werden.

Symbole

Mit ES6 wurden Symbole eingeführt. Sie sind ein neuer primitiver Datentyp mit eindeutigen Werten. Sie können als Schlüssel in Objekten verwendet werden. Sie werden mit dem „Symbol-Konstruktor“ erzeugt. Die Nutzung des `new` Operators ist nicht erlaubt und führt zu einer Fehlermeldung. Der Funktion `Symbol` kann ein optionaler String übergeben werden, der als Beschreibung dient.

```
1 | const s = Symbol("test");
2 | const s2 = Symbol("test");
3 | console.log(s == s2); // false
4 | console.log(typeof s); // symbol
5 | console.log(s.description); // test
6 | console.log(s.valueOf()); // Symbol(test)
```

Der Wert eines Symbols ist während der Programm-Laufzeit eindeutig. Zwei Symbole mit dem gleichen Namen sind nicht gleich.

Symbole in Objekten

Neben den bekannten, String-basierten Eigenschaften von Objekten, können auch Symbole als Eigenschaften verwendet werden. Diese sind nicht mit den anderen Eigenschaften vergleichbar. Auf sie kann nur über die eckigen Klammern zugegriffen werden. Auf die Eigenschaften kann mit der Methode `Object.getOwnPropertySymbols` zugegriffen werden.

Symbole werden in der Regel in Modulen verwendet, um Eigenschaften zu definieren, die für reguläre Nutzer nicht zugreifbar sein sollen.

Beispiel 60: Umgang mit Symbolen

```
1 | const sym = Symbol("test");
2 | const obj = {};
3 | obj[sym] = "test"; // obj.sym = "test" geht nicht
4 | console.log(obj[sym]); // test
5 | console.log(Object.getOwnPropertySymbols(obj)); // [Symbol(test)]
```

6.2. Klassen

Klassen in JavaScript

Mit ES6 (ES2015) wurden Klassen in JavaScript eingeführt, um die klassische Vererbung zu unterstützen. Klassen werden mit dem Schlüsselwort `class` definiert. JavaScript unterstützt keine Mehrfachvererbung. `#` vor dem Namen einer Eigenschaft oder Methode macht diese Eigenschaft oder Methode privat (seit ES2022).

Beispiel 61: class.js

```
1 | class X {
2 |     #a1; // private Eigenschaft
3 |     static #a2; // private statische Eigenschaft
4 |
5 |     static { // statischer Konstruktor
6 |         X.#a2 = 2;
7 |     }
8 |
9 |     constructor(x) {
10 |         this.a1 = x;
11 |     }
}
```

```

12 |   get a1() { // getter
13 |       return this.#a1;
14 |   }
15 |   set a1(x) { // setter
16 |       this.#a1 = x;
17 |   }
18 |
19 |   static get a2() { // getter
20 |       return X.#a2;
21 |   }
22 |   static set a2(x) { // setter
23 |       X.#a2 = x;
24 |   }
25 |
26 |   #method1() { // private Methode
27 |       return this.#a1;
28 |   }
29 | }

```

In JavaScript kann **this** nicht weggelassen werden. Es muss immer angegeben werden.

Vorsicht: In JavaScript bezieht sich **this** auf das Objekt, in dem die Methode aufgerufen wird. Dies unterscheidet sich von Java, wo **this** immer auf die erzeugte Instanz der Klasse verweist.

Nutzung der Klasse:

```

1 | const x = new X(1);
2 | console.log(x.a1); // getter
3 | x.a2 = 3; // setter

```

Klassen in TypeScript

Klassen in TypeScript sind älter als JavaScript-Klassen. Sie werden mit dem Schlüsselwort **class** definiert. TypeScript unterstützt **public**, **protected** und **private**, jedoch ist in neuerem Code die JavaScript-Syntax mit **#** zu bevorzugen. TypeScript unterstützt keine Mehrfachvererbung, jedoch können mehrere Interfaces implementiert werden. Dies geschieht mit dem Schlüsselwort **implements**.

TypeScript unterstützt das Schlüsselwort **override**, um eine Methode einer Basisklasse zu überschreiben. **override** kann nicht auf Interface-Methoden angewendet werden.

Beispiel 62: Vererbung und Interfaces in TypeScript

```

1 | class X {
2 |     private a1: number;
3 |
4 |     constructor(x: number) {
5 |         this.a1 = x;
6 |     }
7 |     methode(): number {
8 |         return this.a1;
9 |     }
10 | }
11 |
12 | interface I {
13 |     f(): number;
14 | }
15 |
16 | class C extends X implements I {
17 |     constructor(x: number) {
18 |         super(x);
19 |     }
20 |
21 |     f(): number {
22 |         return 2*this.methode();
23 |     }
24 |
25 |     override methode(): number {
26 |         return super.methode()+1;
27 |     }
28 | }
29 |
30 | const c : C = new C(1);
31 | console.log(c.f());
32 | console.log(c.methode());

```

6.3. Ausnahmen

- Ausnahmen sind Fehler, die während der Ausführung eines Programms auftreten.
- In JavaScript gibt es zwei Arten von Ausnahmen:
 - **Error**-Objekte, die von JavaScript selbst erzeugt werden.
 - **Error**-Objekte, die von Entwicklern erzeugt werden.

- **Error**-Objekte haben eine Methode `toString()`, die den Namen und die Nachricht des Fehlers zurückgibt.
- **Error**-Objekte haben eine Eigenschaft
 - `name` zur Identifizierung des Fehlers.
 - `message` zur Beschreibung des Fehlers..
 - `stack`, die den Aufrufstapel des Programms zum Zeitpunkt des Fehlers enthält.
 - `fileName`, die den Namen der Datei enthält, in der der Fehler aufgetreten ist.
 - `lineNumber`, die die Zeilennummer in der Datei enthält, in der der Fehler aufgetreten ist.
 - `columnNumber`, die die Spaltennummer in der Datei enthält, in der der Fehler aufgetreten ist.

Beispiele für Error-Objekte

- **EvalError**, wenn ein Fehler bei der Auswertung von JavaScript-Code auftritt.
- **RangeError**, wenn ein Wert außerhalb des zulässigen Bereichs liegt.
- **ReferenceError**, wenn auf eine nicht existierende Variable zugegriffen wird.
- **SyntaxError**, wenn ein Syntaxfehler in JavaScript-Code auftritt.
- **TypeError**, wenn ein Wert nicht von dem erwarteten Typ ist.
- **URIError**, wenn ein Fehler bei der Verwendung von URI-Funktionen auftritt.

Beispiel für die Verwendung von Error-Objekten

```
1 try {  
2   throw new Error('Ein Fehler ist aufgetreten');  
3 } catch (e) {  
4   console.log(e.name + ': ' + e.message);  
5 }
```

gibt auf der Konsole aus: **Error: Ein Fehler ist aufgetreten.**

Im Unterschied zu Java gibt es in JavaScript keine `throws`-Klausel, die anzeigt, dass eine Methode eine Ausnahme erzeugen kann. Daher ist es in JavaScript nicht möglich Ausnahmen zu erzwingen.

Eigene Error-Objekte

lassen sich am besten durch Vererbung von **Error** realisieren:

```
1 class MyError extends Error {  
2   constructor(message) {  
3     super(message);  
4     this.name = this.constructor.name;  
5   }  
6 }
```

kann dann wie folgt verwendet werden:

```
1 try {  
2   throw new MyError('Ein Fehler ist aufgetreten');  
3 } catch (e) {  
4   console.log(e.name + ': ' + e.message);  
5 }
```

und gibt auf der Konsole aus:

MyError: Ein Fehler ist aufgetreten.

6.4. Funktionen

6.4.1. Reguläre Funktionen

JavaScript-Funktionen

Funktion sind in JavaScript ebenfalls Objekte. Daher können Funktionen auch als Parameter an andere Funktionen übergeben werden. Funktionen können auch als Rückgabewert einer Funktion verwendet werden. Funktionen können auch anonym sein.

Funktionen können mit dem Schlüsselwort **function** definiert werden. JavaScript-Funktionen können beliebig viele Parameter haben. Es ist nicht erforderlich, dass die Anzahl der Parameter mit der Anzahl der Argumente übereinstimmt. Die Parameter können auch weggelassen werden. Dann wird der Wert **undefined** verwendet. Es ist auch möglich, mehr Argumente als Parameter zu übergeben. Diese Argumente können mit dem Schlüsselwort **arguments** in der Funktion abgefragt werden. Die Parameter einer Funktion können mit dem Schlüsselwort **...** als Array definiert werden. Dann können beliebig viele Argumente übergeben werden. Diese Argumente werden in einem Array gespeichert.

In Klassen und Objekten können Funktionen einfach als Methoden ohne das Schlüsselwort **function** definiert werden.

Beispiel 63: Funktionsparameter

```

1 function f1(x) {
2     console.log(x);
3     for (let i = 0; i < arguments.length; i++) {
4         console.log(arguments[i]);
5     }
6 }

```

ergibt die Ausgabe

```

1 f1(1, 2, 3);
2 1
3 1
4 2
5 3

```

Zusätzliche unbenannte Parameter können in JavaScript, aber nicht in TypeScript übergeben werden. Diese Argumente können mit **arguments** abgefragt werden. **arguments** enthält alle Argumente, die übergeben wurden.

Vorsicht

arguments ist ein Array-ähnliches Objekt. Es ist kein Array. Es kann nicht mit **map** oder **forEach** verwendet werden.

Beispiel 64: Array-Parameter

```

1 function f2(...x) {
2     for (const elem of x) {
3         console.log(elem);
4     }
5 }
6 f2(1, 2, 3);

```

ergibt die Ausgabe

```

1 1
2 2
3 3

```

analog in Typescript:

```

1 function f2(...x: number[]) : void {
2     for (const elem of x) {
3         console.log(elem);
4     }
5 }

```

aber der Datentyp der Parameter ist hier beschränkt. Es können nur Zahlen übergeben werden. Dies kann einfach mit dem Schlüsselwort **any** anstatt **number** umgangen werden, ist dann aber nicht mehr typsicher.

6.4.2. Closures

Beispiel 65: Anonyme Funktion und fehlende Parameter

```

1 function closure() {
2     let para = { x: 1};
3     return function (x) {
4         if (x === undefined) {
5             return para.x;
6         } else {
7             para.x = x;
8         }
9     };
10 }

11 const getset = closure();
12 console.log(getset());
13 getset(2);
14 console.log(getset());

```

ergibt die Ausgabe

```

1 | 1
2 | 2

```

Mit diesem Vorgehen wurden in älterem JavaScript Getter, Setter und private Eigenschaften implementiert. Das Objekt `para` ist eine Closure. Es wird von der anonymen Funktion referenziert. Es wird nicht freigegeben, wenn die Funktion beendet ist.

6.4.3. Anonyme Funktionen

Unterschiede zwischen regulären und anonymen Funktionen

Bei regulären Funktionen folgt auf das Schlüsselwort `function` der Name der Funktion und dann die Parameterliste. Bei anonymen Funktionen folgt auf das Schlüsselwort `function` sofort die Parameterliste. Der Name der Funktion fehlt. Der Scope von regulären Funktionen ist die Datei in der sie definiert sind, außer sie werden explizit exportiert. Anonyme Funktionen sind nur im Scope in dem sie definiert sind sichtbar. Reguläre Funktionen können in einer Datei benutzt werden bevor sie definiert sind. Anonyme Funktionen können erst benutzt werden, nachdem sie definiert sind. Anonyme Funktionen werden meist an eine Variable zugewiesen, als Rückgabe-Objekt einer Funktion genutzt oder als Parameter an eine Funktion übergeben. Sie können jedoch auch Immediately Invoked Function Expressions (IIFE) sein. Diese Funktionen werden sofort aufgerufen.

Beispiel 66: IIFE

```

1 (function (text) {
2     console.log(text);
3 })("IIFE");

```

Um IIFE von regulären Funktionen zu unterscheiden, wird die Funktion in Klammern eingeschlossen. Dann wird ein Argument übergeben. Die Funktion wird sofort aufgerufen. Sie gibt den Text `IIFE` auf der Konsole aus.

Beispiel 67: Als Methoden in Objekten

```

1 const obj = {
2     f(text) {
3         console.log(text);
4     }
5 };
6 obj.f("Methode");

```

Die Methode `f` des Objekts `obj` gibt den Text `Methode` auf der Konsole aus.

Lambda-Kalkül

Das λ -Kalkül ist eine formale Sprache zur Definition von Funktionen. Es wurde von Alonzo Church entwickelt. Es ist die Grundlage der funktionalen Programmierung. Die Java-Streams sind eine Implementierung des λ -Kalküls in Java. Die `map`-, `filter`- und `reduce`-Methoden sind die wichtigsten Methoden der funktionalen Programmierung und auch in JavaScript und TypeScript verfügbar.

Lambda-Ausdrücke sind anonyme Funktionen, die in der funktionalen Programmierung häufig als Funktionsargumente verwendet werden. Die JavaScript-Syntax ähnelt der Java-Syntax. Auf die Parameterliste folgt das Schlüsselwort `=>`, gefolgt vom Funktionsrumpf. Der Rumpf kann auch in geschweifte Klammern eingeschlossen werden. Dann muss `return` verwendet werden, um einen Wert zurückzugeben.

Beispiel 68: Lambda-Ausdrücke und `this` in JavaScript

```

1 function recv(f) {
2   console.log(f("Umgebung: "));
3 }
4
5 class C {
6   constructor() {
7     this.x = "instance";
8   }
9   m(y) { // reguläre Funktion
10    return y + this?.x;
11  }
12  p() { // Lambda-Ausdruck
13    recv(y => y + this.x);
14  }
15  q() { // reguläre Funktion
16    recv(this.m);
17  }
18 }

```

```

19 const c = new C();
20 console.log(
21   c.m("Umgebung: ")
22 ); // Umgebung: instance
23 c.p(); // Umgebung: instance
24 c.q(); // Umgebung: undefined
25 recv(c.m); // Umgebung: undefined

```

Das Fragezeichen vor dem Punkt in der Methode `m` ist der optionale Zugriffsoperator. Er verhindert, dass ein Fehler auftritt, wenn `this` nicht definiert ist.

Vorsicht

Das Schlüsselwort `this` verhält sich in JavaScript anders als in Java. In Java ist `this` eine Referenz auf die Instanz einer Klasse. In JavaScript bezieht sich `this` in einer regulären Methode auf das umgebende Objekt während des Aufrufs der Methode. In einem Lambda-Ausdruck bezieht sich `this` auf das umgebende Objekt während der Definition des Lambda-Ausdrucks. Daher funktionieren im obigen Beispiel die Aufrufe von `recv` in `q` und `recv(c.m)` nicht, da in der Funktion `recv` `this` nicht definiert ist. In JavaScript kann ein Objekt mit dem Schlüsselwort `bind` an eine Funktion gebunden werden. Damit wird `this` in der Funktion auf das Objekt gebunden, das als Parameter an `bind` übergeben wird.

Beispiel 69: Binden von `this`

```

26 const mBound = c.m.bind(c);
27 recv(mBound); // Umgebung: instance

```

Das Binden von Funktionen an umgebende Objekte kann benutzt werden, um Methoden aus einem Objekt an ein anderes Objekt zu binden. Dies ist in Java nicht möglich.

6.4.4. Dynamische Erzeugung von Funktionen

In JavaScript können Funktionen dynamisch erzeugt werden. Dazu wird die Funktion `Function` verwendet. Sie erwartet beliebig viele Strings als Parameter. Der letzte String ist der Funktionsrumpf. Die vorherigen Strings sind die Namen der Parameter. Die Parameter werden durch Kommata getrennt. Die Parameterliste wird in runde Klammern eingeschlossen. Der Funktionsrumpf wird in geschweifte Klammern eingeschlossen. Der Funktionsrumpf kann auch in einem String stehen. Dann muss der String in Anführungszeichen stehen. Die Parameterliste kann auch leer sein. Dann müssen die runden Klammern trotzdem angegeben werden. Die Funktion `Function` erzeugt eine anonyme Funktion. Diese Funktion kann einer Variablen zugewiesen werden. Die Funktion kann auch direkt aufgerufen werden.

Beispiel 70:

```

1 | const f = new Function("x", "y", `{ // Funktionsrumpf als String
2 |   for (const elem of arguments) {
3 |     console.log(elem);
4 |   }
5 |   console.log();
6 |   return x + y;
7 | }`);
8 | console.log(f(1, 2)); // 3
9 |
10 | const g = new Function("return 1 + 2;");
11 | console.log(g()); // 3

```

6.4.5. Iteratoren**Iteratoren**

Iteratoren sind Objekte, die eine Methode `next` haben. Diese Methode gibt ein Objekt zurück, das die Attribute `value` und `done` besitzt. Das Attribut `value` enthält den nächsten Wert. Das Attribut `done` ist ein Boolean. Dieser ist `true`, wenn es keine weiteren Werte gibt. Iteratoren werden in JavaScript und TypeScript verwendet, um über Arrays und Maps zu iterieren.

Beispiel 71: Array-Iteratoren

Auf den Array-eigenen Iterator kann über das Symbol `Symbol.iterator` zugegriffen werden. Dieser Iterator kann mit der Methode `next` iteriert werden.

```

1 | const arr = [1, 2, 3];
2 | const iter = arr[Symbol.iterator]();
3 | console.log(iter.next()); // { value: 1, done: false }
4 | console.log(iter.next()); // { value: 2, done: false }
5 | console.log(iter.next()); // { value: 3, done: false }
6 | console.log(iter.next()); // { value: undefined, done: true }

```

Der Iterator wird auch von der `for`-Schleife verwendet.

```

1 | const arr = [1, 2, 3];
2 | for (const elem of arr) {
3 |   console.log(elem);
4 | }

```

Eigene Iteratoren

Eigene Iteratoren müssen also die Funktion `next` mit einem Rückgabe-Objekt `{ value: ..., done: ... }` implementieren. Das zu iterierende Objekt muss das Symbol `Symbol.iterator` enthalten. Dieses Symbol muss eine Funktion referenzieren, die den Iterator zurückgibt.

Beispiel 72: Iterator für ein Objekt mit mehreren Feldern

Es soll über alle Arrays im Objekt iteriert werden.

```

1 | const names = {
2 |   minf: [ "Anton", "Berta"],
3 |   other: 5; // kein Array
4 |   winf: [ "Cäsar", "Dora"]
5 | }
6 | names[Symbol.iterator] = function () {
7 |   const finish = { value: undefined, done: true };
8 |   const value = Object.values(this);
9 |   let i = 0;
10 |   let j = 0;

```

```

11     return {
12         next: function () {
13             if (i >= value.length) return finish;
14
15             let a = value[i];
16             while(!Array.isArray(a)) {
17                 if (++i >= value.length) return finish;
18                 a = value[i];
19             }
20             const v = a[j++];
21             if (j >= a.length) {
22                 i++;
23                 j = 0;
24             }
25             return { value: v, done: false };
26         },
27     };
28 };

```

Die dem Iterator zugeordnete Funktion dient als Closure für die next Funktion.

Beispiel 73: Iterator für eine Klasse mit mehreren Feldern

Für Klassen mit privaten Feldern funktioniert der Aufruf von `Object.values(this)` nicht, da die privaten Felder nicht sichtbar sind. Es muss entweder im Iterator direkt auf die Felder zugegriffen werden oder eine (private) Methode implementiert werden, welche die privaten Felder zurückgibt. Die Klasse wird in einem Modul (`klasse.mts`) definiert.

```

1  export default class Klasse<T> implements Iterable<T> {
2      #a1: T[];
3      #a2: T[];
4      constructor(a1: T[], a2: T[]) {
5          this.#a1 = Array.from(a1);
6          this.#a2 = Array.from(a2);
7      }
8
9      get iterator() : Iterator<T, IteratorResult<T>> {
10         return this[Symbol.iterator]();
11     }
12
13     [Symbol.iterator](): Iterator<T, IteratorResult<T>> {
14         const finish : IteratorResult<T> =
15             { value: undefined, done: true };
16         const value = [this.#a1, this.#a2];
17         let i = 0;
18         let j = 0;
19         return {
20             next(): IteratorResult<T> {
21                 if (i >= value.length) return finish;
22                 const a = value[i];
23                 const v = a[j++] as T;
24                 if (j >= a.length) {
25                     i++;
26                     j = 0;
27                 }
28                 return { value: v, done: false };
29             },
30         };
31     }

```

Die Klasse wird in einem anderen Modul (`main.ts`) benutzt.

```

1 import Klasse from "./klasse.mts";
2
3 const s1 = ["eins", "zwei"];
4 const s2 = ["alpha", "beta"];
5 const c = new Klasse<string>(s1, s2);
6
7 const iter = c.iterator; // getter oder c[Symbol.iterator]();
8 console.log(iter); // { next: [Function: next] }
9 let i = iter.next();
10 while(!i.done) {
11     console.log(i.value);
12     i = iter.next();
13 }
14
15 for (let k of c) {
16     console.log(k);
17 }

```

6.4.6. Generatoren

Während Iteratoren in JavaScript und TypeScript Objekte mit der Methode `next` sind und über bestehende Ansammlungen von Objekten iterieren, sind Generatoren Funktionen die mit jedem Aufruf von `next` einen Wert zurückgeben. Dieser Wert muss nicht bereits vorhanden sein, sondern kann nach Bedarf generiert werden. Damit eignen sich Generatoren jedoch auch zur Implementierung von Iteratoren. Generatoren werden mit dem Schlüsselwort **function*** definiert. Generatoren geben beim Aufruf von `next` einen Wert zurück, wenn sie während der Ausführung des Funktionskörpers

- auf das Schlüsselwort **yield** treffen. Dann wird der Wert von **yield** zurückgegeben. Die Ausführung des Generators wird bis zum nächsten Aufruf von `next` unterbrochen. Dann wird die Ausführung des Generators nach dem **yield** fortgesetzt.
- auf ein **return** treffen. Dann wird der Wert von **return** zurückgegeben. Die Ausführung des Generators wird beendet.
- auf ein **throw** treffen. Dann wird der Wert von **throw** zurückgegeben. Die Ausführung des Generators wird beendet.

Der Generator wird mit dem ersten Aufruf von `next` gestartet. Generatoren können auch rekursiv sein. Generatoren können auch als oder in Kombination mit Iteratoren verwendet werden.

Beispiel 74: Generator für eine Fibonacci-Folge

<pre> 1 function* fibonacci() { 2 let a = 0n; // BigInt 3 let b = 1n; // BigInt 4 while (true) { 5 const restart = yield a; 6 [a, b] = [b, a + b]; 7 if (restart) { 8 a = 0n; 9 b = 1n; 10 } 11 } 12 } </pre>	<pre> 1 const fib = fibonacci(); 2 3 for(let i = 0; i < 10; i++) { 4 console.log(fib.next().value); 5 } 6 7 console.log(fib.next(true).value); 8 9 for(let i = 1; i < 10; i++) { 10 console.log(fib.next().value); 11 } </pre>
---	---

Wird

ein Wert an `next` übergeben, wird dieser Wert von **yield** in `restart` gespeichert. Der Generator wird dann neu gestartet. Der Generator enthält eine Endlosschleife und wird daher nie beendet.

Beispiel 75: Range-Generator

```
1 function* range(start, end, step) {  
2   if (end === undefined) {  
3     end = start;  
4     start = 0;  
5   }  
6   if (step === undefined) {  
7     step = 1;  
8   }  
9   for (let i = start; i < end; i += step) {  
10    yield i;  
11  }  
12 }  
13  
14 for (let i of range(5)) {  
15   console.log(i);  
16 }
```

6.5. Asynchrones JavaScript**6.5.1. Callbacks****Problem**

In JavaScript gibt es keine Möglichkeit, eine Funktion zu unterbrechen und auf ein Ergebnis zu warten. Es gibt aber viele Situationen, in denen bei einem normalen, in der Ausführung länger dauerndem Funktionsaufruf das Programm nicht mehr auf Nutzereingaben reagiert. Daher benutzt man in solchen Fällen *asynchrone* Funktionen. Dabei wird eine Funktion aufgerufen, die dann im Hintergrund arbeitet und das Ergebnis an eine andere Funktion übergibt, die dann das Ergebnis verarbeitet. Diese andere Funktion wird in der Regel als Parameter an die asynchrone Funktion übergeben und wird *Callback* genannt.

Vorsicht

JavaScript ist eine *single-threaded* Sprache. Das bedeutet, dass es nur einen Ausführungsstrang gibt. Wenn ein asynchroner Aufruf gestartet wird, wird dieser in eine Warteschlange eingereiht und erst ausgeführt, wenn der aktuelle Codeblock abgearbeitet ist. Die asynchrone Funktion kehrt sofort zurück und der Code nach der Funktion wird weiter ausgeführt.

Die Situation ist unterschiedlich, wenn es um die Ausführung von asynchronem Code in einem Browser oder um Aufrufe des unterliegenden Betriebssystems geht. Dann kann der Browser oder das Betriebssystem die asynchrone Funktion in einem eigenen Thread ausführen und das Ergebnis an JavaScript zurückgeben. Erst dann wird die Callback-Funktion in die Warteschlange eingereiht und ausgeführt.

Beispiel 76:

```

1 function callback(result) {
2   console.log(result);
3 }
4
5 function f(cb) {
6   setTimeout(() => {
7     cb("Ergebnis");
8   }, 1000);
9   console.log("Nachher");
10 }
11
12 f(callback);

```

Es wird die Funktion `f` definiert. Diese enthält zwei Funktionsaufrufe: Die asynchrone Funktion `setTimeout` hat zwei Parameter: Eine Callback-Funktion und eine Zeitspanne, nach der die Callback-Funktion ausgeführt werden soll. Die zweite Funktion ist `console.log`, die den Text `Nachher` ausgibt. Die Funktion `f` wird mit der Callback-Funktion aufgerufen. Die Callback-Funktion gibt das Ergebnis auf der Konsole aus. Die Reihenfolge der Ausgaben ist: `Nachher` und eine Sekunde später `Ergebnis`.

Callback-Hell

Die Callback-Funktionen können auch wieder asynchrone Funktionen aufrufen. Dann entsteht eine Verschachtelung von Callback-Funktionen, die *Callback-Hell* genannt wird. Das erschwert die Lesbarkeit des Codes und die Wartung. Insbesondere, wenn Fehlerbehandlung notwendig ist, wird der Code unübersichtlich.

Da Callbacks meist nur innerhalb einer Funktion benötigt werden, werden sie dann als anonyme Funktionen oder Lambda-Ausdrücke übergeben. Diese sind jedoch schwer zu warten und zu testen.

6.5.2. Promises

Promises sind ein – mit JavaScript ES6 eingeführtes – Konzept, um die Callback-Hell zu vermeiden. Ein Promise ist ein Objekt, das ein Ergebnis einer asynchronen Operation repräsentiert. Ein Promise kann in drei Zuständen sein:

- *pending*: Das Versprechen ist noch nicht erfüllt oder abgelehnt.
- *fulfilled*: Das Versprechen wurde erfüllt.
- *rejected*: Das Versprechen wurde abgelehnt.

Ein Promise hat zwei Methoden, die aufgerufen werden können. Die dort übergebenen Callbacks werden ausgeführt, wenn das Versprechen erfüllt (`then`) oder abgelehnt (`catch`) wurde.

Beispiel 77:

```

1 function f(arg) {
2   return new Promise(
3     (resolve, reject) => {
4       setTimeout(() => {
5         if (arg === undefined) {
6           resolve("Erfolg");
7         } else {
8           reject("Fehler");
9         }
10      }, 1000);
11    });
12 }
13
14 f().then(console.log)
15   .catch(console.log);

```

Die Funktion `f` gibt ein Promise-Objekt zurück. Dieses Objekt wird mit einer Funktion initialisiert, die zwei Parameter hat: `resolve` und `reject`. Die Funktion `setTimeout` ruft die Funktion `resolve` auf, wenn das Argument `arg` `undefined` ist, andernfalls wird die Funktion `reject` aufgerufen. Das Promise-Objekt wird mit `then` und `catch` aufgerufen. Die an `then` übergebene Funktion wird aufgerufen, wenn das Promise erfüllt wurde, die an `catch` übergebene Funktion wird aufgerufen, wenn das Promise abgelehnt wurde.

Mit diesem Vorgehen kann die Callback-Hell vermieden werden, da die Callbacks in einer Kette von `then`- oder `catch`-Funktionen aufgerufen werden.

6.5.3. `async/await`

Mit ES2017 wurde das Konzept von `async` und `await` eingeführt. Damit kann asynchroner Code wie synchroner Code aussehen. Die Schlüsselwörter `async` und `await` können nur in Funktionen verwendet werden. Eine Funktion, die mit `async` deklariert wird, gibt immer ein Promise-Objekt zurück. In einer Funktion, die mit `async` deklariert wurde, kann mit `await` auf das Ergebnis eines Promise gewartet werden.

`await` darf nur innerhalb einer mit `async` deklarierten Funktion verwendet werden. Es wartet auf das Ergebnis eines Promise-Objekts. Das Ergebnis wird in einer Variablen gespeichert. Der Code nach dem `await`-Ausdruck wird erst ausgeführt, wenn das Promise der `await`-Funktion erfüllt oder abgelehnt wurde.

Unterschied zwischen `async/await` und synchronem Code

Der Code wird nicht synchron ausgeführt. Die Funktion, die mit `async` deklariert wurde, gibt intern ein Promise-Objekt zurück und die aufrufende Funktion wird weiter ausgeführt. Die `async`-Funktion läuft nachdem Sie aus der Warteschlange übernommen wurde wird bis zur Funktion mit der `await`-Anweisung. Dann wird gewartet bis diese Funktion Ihr Promise beendet hat. Das bedeutet, dass der Code nach dem `await`-Ausdruck erst ausgeführt wird, wenn das Promise der `await`-Funktion erfüllt wurde. In der Zwischenzeit kann der Ausführungsstrang (also die die asynchrone Funktion aufrufende Funktion) andere Aufgaben erledigen.

Beachte

`async` und `await` sind reservierte Wörter und können daher nicht als Variablennamen verwendet werden. Mit `async` deklarierte Funktionen und Funktionen, die `await` verwenden, geben immer ein Promise-Objekt zurück. Dabei bleibt das Promise der `async`-Funktion in der Regel für den Programmierer unsichtbar, da das Schlüsselwort `async` JavaScript veranlasst, das Promise-Objekt zu erzeugen, zurückzugeben und zu behandeln. Dieses Promise ist erst erfüllt, wenn die `async`-Funktion erfolgreich beendet wurde. Falls in der `async`-Funktion ein Fehler auftritt, wird das Promise abgelehnt. Damit kann es in der `async`-Funktion mehrere `await`-Anweisungen geben, die auf das Ergebnis von verschiedenen Promises warten.

Beispiel 78:

Aufruf von `f` mit `await`. Fehlerbehandlung

mit `catch`-Methode des Promise-Objekts

```

1 | async function g() {
2 |   let result =
3 |     await f().catch(console.log);
4 |   console.log(result);
5 |   result =
6 |     await f(1).catch(console.log);
7 | }
8 |
9 | g();
10 | console.log("Nachher");

```

Wenn ein Fehler auftritt, wird die Funktion `console.log` aufgerufen. Dieser wird von `catch` der Fehler übergeben.

oder mit `try/catch`

```

1 | async function g() {
2 |   try {
3 |     let result = await f();
4 |     console.log(result);
5 |     result = await f(1);
6 |   } catch (e) {
7 |     // Fehlerbehandlung
8 |     console.log(e);
9 |   }
10 | }
11 |
12 | g();
13 | console.log("Nachher");

```

als dritte Möglichkeit kann die Fehlerbehandlung in der aufrufenden Funktion erfolgen:

```

1 | async function g() {
2 |   let result = await f();
3 |   console.log(result);

```

```

4 |   result = await f(1);
5 |   console.log(result);
6 | }
7 |
8 | g().catch( err => {
9 |   // Fehlerbehandlung
10 |   console.log(err);
11 | });
12 | console.log("Nachher");

```

Rückgabewerte von `async`-Funktionen

`async`-Funktionen geben immer ein Promise-Objekt zurück. Wenn die `async`-Funktion keinen `return`-Ausdruck enthält, wird das Promise-Objekt mit dem Wert `undefined` erfüllt. Auf das Ergebnis der `async`-Funktion kann mit `then` und `catch` zugegriffen werden. Die `async`-Funktion kann auch mit `await` aufgerufen werden, z.B. über eine IIFE mit einem asynchronen Aufruf.

Beispiel 79:

```

1 | async function g() {
2 |   let result = await f(1)
3 |   .catch(console.log);
4 |   console.log(result);
5 |   result = await f();
6 |   return result;
7 | }

```

```

g().then(console.log)
   .catch(console.log);

```

oder mit einer IIFE:

```

1 | (async () => {
2 |   try {
3 |     let result = await g();
4 |     console.log(result);
5 |   } catch (e) {
6 |     console.log(e);
7 |   }
8 | })();

```

Ein direkter Aufruf von `g` mit `await` ist nicht möglich, da `await` nur in einer `async`-Funktion verwendet werden kann. Daher wird die `async`-Funktion in einer IIFE aufgerufen.

6.6. Globale Umgebungen für JavaScript

Im Frontend und im Backend wird JavaScript im Kontext unterschiedlicher globaler Umgebungen ausgeführt.

NodeJS-Backend

Unter Node.js ist die globale Umgebung `global`. Dort sind alle globalen Variablen und Funktionen definiert. Die globalen Variablen und Funktionen einer Browser-Umgebung sind nicht verfügbar.

Browser-Frontend

Im Browser ist die globale Umgebung `window`. Dort sind alle globalen Variablen und Funktionen definiert. Die globalen Variablen und Funktionen einer Node.js-Umgebung sind nicht verfügbar. Das `window`-Objekt enthält insbesondere das `document`-Objekt, welches das DOM repräsentiert.

`globalThis`

In beiden Umgebungen ist das `globalThis`-Objekt (ES2019) verfügbar. Es enthält alle globalen Variablen und Funktionen, die in der jeweiligen Umgebung definiert sind. Also entspricht `globalThis` in Node.js dem `global`-Objekt und im Browser dem `window`-Objekt.

Vorsicht

Der Wert von `this` in einer Funktion hängt von der Art der Funktionsausführung und der Funktionsdefinition ab. In einer Funktion, die als Methode eines Objekts aufgerufen wird, ist `this` das Objekt

selbst. In einer global definierten Funktion, ist `this` `undefined` falls die Funktion mit “use strict” implementiert wurde, andernfalls `globalThis`.

Beispiel 80:

```

1 function f() {
2   console.log(this === globalThis);
3   console.log(this);
4 }
5
6 function g() {
7   "use strict";
8   console.log(this === globalThis);
9   console.log(this);
10 }
11
12 const o = {
13   f: f,
14   g: g
15 };
16 f(); // true, globalThis
17 g(); // false, undefined
18 o.f(); // false, o
19 o.g(); // false, o

```

Beispiel 81: Einfügen von Objekten in die globale Umgebung

```

1 function z(x) {
2   this.value = x;
3 }
4 console.log(globalThis.value); // undefined
5 z("Hallo");
6 console.log(globalThis.value); // Hallo
7 console.log(value); // Hallo
8 console.log(globalThis.z) // [Function: z]
9 const o = { z: z };
10 o.z("Welt"); // o = { z: [Function: z], value: 'Welt' }

```

Vermeidung von globalen Variablen

Es ist eine gute Praxis, globale Variablen zu vermeiden, da sie leicht zu unerwarteten Seiteneffekten führen können. In modernen JavaScript-Anwendungen werden globale Variablen und Funktionen vermieden, indem sie in Module eingebettet werden. Daher sollten Funktionen in der Regel mit “use strict” implementiert werden.

Übliche globale Variablen und Funktionen

In beiden Umgebungen sind die folgenden globalen Objekte und Funktionen verfügbar:

- `console` erlaubt die Ausgabe von Nachrichten in die Konsole
- `setTimeout` führt eine Funktion nach einer bestimmten Zeit aus
- `setInterval` führt eine Funktion in regelmäßigen Abständen aus. Sie wird für Animationen verwendet.
- `clearTimeout` löscht einen Timer
- `clearInterval` löscht einen Intervall-Timer
- `fetch` sendet eine Anfrage asynchron an einen Server und empfängt die Antwort. Sie ist in Browsern standardmäßig verfügbar und wird für AJAX-Anfragen verwendet. Dort ersetzt sie weitgehend die veralteten Funktionen `XMLHttpRequest` und `ActiveXObject`. In Node.js ist Sie seit Version 22 verfügbar.

6.6.1. Browser und das Document Object Model

Browser stellen zusätzliche APIs zu Verfügung, die zum Beispiel für die Interaktion mit dem Dokument und dem Browserfenster verwendet werden. Dazu gehören:

- `document` repräsentiert das Document Object Model (DOM)
- `window` repräsentiert das Browserfenster
- `navigator` enthält Informationen über den Browser
- `location` enthält Informationen über die URL der Seite
- `history` enthält Informationen über die Browser-History
- `localStorage` und `sessionStorage` ermöglichen das Speichern von Daten im Browser
- ...

Im nachfolgenden Abschnitt wird das Document Object Model genauer betrachtet.

Document Object Model

Das Document Object Model (DOM) ist eine Programmierschnittstelle, die es ermöglicht, auf HTML- und XML-Dokumente zuzugreifen und sie zu verändern. Das DOM repräsentiert das Dokument als Baumstruktur, in der jedes Element, jedes Attribut und jeder Textknoten als Objekt dargestellt wird. Das DOM ist unabhängig von der Programmiersprache und dem Betriebssystem und kann von jedem Programmierer in jeder Programmiersprache verwendet werden, die die DOM-Schnittstelle unterstützt.

DOM-Struktur

Das DOM ist eine Baumstruktur, die aus Knoten besteht. Die wichtigsten Knoten sind:

- **Document-Knoten** repräsentiert das gesamte Dokument
- **Element-Knoten** repräsentieren HTML-Elemente
- **Text-Knoten** repräsentieren Text innerhalb von Elementen
- **Attribute-Knoten** repräsentieren Attribute von Elementen
- **Kommentar-Knoten** repräsentieren Kommentare

Zugriff auf DOM-Elemente

DOM-Elemente können mit JavaScript über die `document`-API abgerufen und verändert werden. Dazu gehören:

- `document.getElementById(id)` gibt das Element mit der angegebenen ID zurück
- `document.getElementsByTagName(name)` gibt eine Liste aller Elemente mit dem angegebenen Tag-Namen zurück
- `document.getElementsByClassName(name)` gibt eine Liste aller Elemente mit dem angegebenen Klassennamen zurück
- `document.querySelector(selector)` gibt das erste Element zurück, das mit dem angegebenen CSS-Selektor übereinstimmt

- `document.querySelectorAll(selector)` gibt eine Liste aller Elemente zurück, die mit dem angegebenen CSS-Selektor übereinstimmen

DOM-Elemente verändern

DOM-Elemente können mit JavaScript verändert werden. Dazu gehören:

- `element.innerHTML` gibt den HTML-Inhalt eines Elements zurück oder setzt ihn
- `element.textContent` gibt den Text-Inhalt eines Elements zurück oder setzt ihn
- `element.setAttribute(name, value)` setzt das Attribut eines Elements
- `element.getAttribute(name)` gibt den Wert des Attributs eines Elements zurück
- `element.style.property` gibt den Wert einer CSS-Eigenschaft eines Elements zurück oder setzt ihn
- `element.classList` gibt die Klassenliste eines Elements zurück
- `element.classList.add(className)` fügt eine Klasse zu einem Element hinzu
- `element.classList.remove(className)` entfernt eine Klasse von einem Element
- `element.classList.toggle(className)` fügt eine Klasse zu einem Element hinzu, wenn sie nicht vorhanden ist, andernfalls wird sie entfernt
- ...

DOM-Elemente erstellen und löschen

DOM-Elemente können mit JavaScript erstellt und gelöscht werden. Dazu gehören:

- `document.createElement(tagName)` erstellt ein neues Element mit dem angegebenen Tag-Namen.
- `element.appendChild(child)` fügt ein neues Kind-Element zu einem Element hinzu.
- `element.removeChild(child)` entfernt ein Kind-Element von einem Element
- `element.replaceChild(newChild, oldChild)` ersetzt ein Kind-Element durch ein anderes
- `element.cloneNode(deep)` erstellt eine Kopie eines Elements
- `element.remove()` entfernt ein Element aus dem DOM
- `element.parentElement` gibt das Elternelement eines Elements zurück
- `element.childNodes` gibt eine Liste aller Kind-Elemente eines Elements zurück
- `element.firstChild` gibt das erste Kind-Element eines Elements zurück
- `element.lastChild` gibt das letzte Kind-Element eines Elements zurück
- ...

Vorsicht

`createElement` erzeugt ein neues Element. Dieses muss aber noch in das DOM eingefügt werden, um sichtbar zu werden. Dazu wird es mit `appendChild` oder `insertBefore` in ein anderes Element eingefügt. Das Element kann auch mit `innerHTML` oder `outerHTML` in den HTML-Code eines anderen Elements eingefügt werden. `innerHTML` und `outerHTML` sollten aber mit Vorsicht verwendet werden, da sie XSS⁴-Angriffe ermöglichen können.

`remove` entfernt ein Element aus dem DOM. Es ist nicht mehr sichtbar, aber es existiert weiterhin im Speicher. Es kann mit `appendChild` oder `insertBefore` wieder in das DOM eingefügt werden.

DOM-Elemente positionieren

DOM-Elemente können mit JavaScript positioniert werden. Dazu gehören:

- `element.getBoundingClientRect()` gibt die Größe und Position eines Elements zurück
- `element.offsetTop` gibt die Entfernung eines Elements zum oberen Rand des `offsetParent`-Elements zurück
- `element.offsetLeft` gibt die Entfernung eines Elements zum linken Rand des `offsetParent`-Elements zurück
- `element.offsetWidth` gibt die Breite eines Elements zurück
- `element.offsetHeight` gibt die Höhe eines Elements zurück
- `element.clientTop` gibt die Breite des oberen Randes eines Elements zurück
- `element.clientLeft` gibt die Breite des linken Randes eines Elements zurück
- `element.clientWidth` gibt die Breite des Inhaltsbereichs eines Elements zurück
- `element.clientHeight` gibt die Höhe des Inhaltsbereichs eines Elements zurück
- ...

DOM-Events

DOM-Elemente können mit JavaScript auf Ereignisse reagieren. Dazu gehören:

- `element.addEventListener(event, function)` fügt einen Event-Listener zu einem Element hinzu
- `element.removeEventListener(event, function)` entfernt einen Event-Listener von einem Element
- `event.preventDefault()` verhindert das Standardverhalten eines Ereignisses
- `event.stopPropagation()` stoppt die weitere Ausbreitung eines Ereignisses
- `event.target` gibt das Element zurück, auf das das Ereignis ausgelöst wurde
- `event.currentTarget` gibt das Element zurück, auf das der Event-Listener angewendet wurde
- `event.type` gibt den Typ des Ereignisses zurück
- `event.key` gibt den Schlüssel zurück, der bei einem Tastaturereignis gedrückt wurde

⁴Cross-Site-Scripting

- ...

Beispiel 82: DOM-Events und Event-Listener

JavaScript-Code:

```
1 | const button = document.getElementById("button");
2 | button.addEventListener("click", function() {
3 |     console.log("Button clicked");
4 | });
```

HTML-Code:

```
1 | <body>
2 | ...
3 | <button id="button">Click me</button>
4 | ...
5 | </body>
```

6.6.2. Virtuelles DOM

Viele Frontend Frameworks stellen ein Virtuelles DOM zur Verfügung. Das Virtuelle DOM ist eine JavaScript-Repräsentation des DOM. Es wird verwendet, um die Änderungen am DOM zu berechnen, bevor sie auf den Bildschirm gerendert werden. Das Virtuelle DOM ist in der Regel eine Baumstruktur, die aus JavaScript-Objekten besteht. Es ist effizienter als das reale DOM, da es nicht direkt auf den Bildschirm gerendert wird. Stattdessen wird es mit dem realen DOM verglichen und nur die Änderungen werden auf den Bildschirm gerendert. Das Virtuelle DOM wird von React, Vue.js und anderen Frontend-Frameworks verwendet.

Die Nutzung des Virtuellen DOMs ist eine der Hauptgründe, warum Frontend-Frameworks schneller sind als reines JavaScript. Für den Entwickler ist das Virtuelle DOM transparent, da es von den Frameworks verwaltet wird.

7. JavaScript-Backend

In der Full-Stack-Entwicklung wird oft JavaScript sowohl im Frontend als auch im Backend verwendet. Im Backend können verschiedene Frameworks und Bibliotheken verwendet werden. Die typische Kombination ist Node mit `Express.js`. Anstelle von Node können auch andere Laufzeitumgebungen wie Deno oder Bun verwendet werden. Da `Express.js` das am weitesten verbreitete Framework ist, wird es hier kurz vorgestellt, jedoch nicht weiter vertieft.

7.1. Das Express-Modul

Express

`Express.js` ist das am häufigsten genutzte Backend in `Node.js`.

- Middleware
- Routing
- Template-Engine

Express ist sogenannte Middleware. Es stellt eine Reihe von Funktionen zur Verfügung, die die Entwicklung von Webapplikationen vereinfachen. Es ist jedoch kein vollständiges Framework, sondern nur eine Sammlung von Funktionen. Es ist sehr schlank und kann durch weitere Module, wie etwa der Template-engine `pug` erweitert werden. Express liefert sowohl serverseitig generierte HTML-Seiten als auch Daten im JSON-Format aus. Ebenso können Dateien ausgeliefert werden.

7.1.1. Erste Express Installation

Scaffolding mit dem Express-Generator

Eine Express-Anwendung kann mit dem Express-Generator erzeugt werden. Allerdings arbeitet dieser mit alter Syntax, inklusive CommonJS-Modulen. Daher wird diese erste Installation nur zur Erklärung der Arbeitsweise von Express durchgeführt.

Hierbei wird die HTML-Template-Engine `pug` verwendet. Diese wird ähnlich wie PHP genutzt um HTML-Dateien für die Auslieferung durch den Server zu erzeugen. Pug ist jedoch weniger mächtig als PHP und wird daher nur für die Erzeugung von HTML-Dateien verwendet.

Ausführung von `bun x express-generator --view=pug simplebackend` erzeugt ein Express-Projekt mit dem Namen `simplebackend`, der HTML-Template-Engine `pug` und einer vorgegebenen Verzeichnis-Struktur:

```
simplebackend/
+--- [bin]
|   +--- www
+--- [public] (static files)
|   +--- [images]
|   +--- [javascripts]
|   +--- [stylesheets]
+--- [routes] (JavaScript files)
+--- [view]   (HTML - pug files)
+--- app.js
+--- package.json
```

Installation

`bun install` im Verzeichnis des Express-Projekts liefert unter anderem:

```
WARN 1 deprecated subdependencies found: mkdirp@0.5.1
Packages: +10
+++++++
Progress: resolved 10, reused 3, downloaded 7, added 10, done
  create : simplebackend/
  .....
  change directory:
    $ cd simplebackend
  install dependencies:
    $ npm install
  run the app:
    $ DEBUG=simplebackend:* npm start
```

Die `npm`-Anweisungen sollten durch `bun` ersetzt werden. Die Installation von `express` und `pug` erfolgt durch `bun install`. Die Anweisung `DEBUG=simplebackend:* bun start` startet den Server. Die Anwendung ist unter `http://localhost:3000` erreichbar.

7.1.2. Starten des Servers

Starten des Servers

`bun start` startet den Server. Die Konsolen-Ausgabe ist:

```
> simplebackend@0.0.0 start simplebackend
> bun ./bin/www
```

entsprechend der Definition in `package.json`:

```
"scripts": {
  "start": "bun ./bin/www"
},
```

Die Startup-Datei `www`

assoziiert die Konfigurations-Datei `app.js` mit dem Server und startet diesen:

```
1 |#!/usr/bin/env bun
2 |
3 |var app = require('./app');
4 |var debug = require('debug')('myapp:server');
5 |var http = require('http');
6 |
7 |var port = normalizePort(process.env.PORT || '3000');
8 |app.set('port', port);
9 |
10 |var server = http.createServer(app);
11 |
12 |server.listen(port);
13 |server.on('error', onError);
14 |server.on('listening', onListening);
15 |
16 |... // weitere Funktionen
```

7.1.3. Server-Konfiguration

Die Konfigurations-Datei app.js

```

1 | var createError = require('http-errors');
2 | var express = require('express');
3 | var path = require('path');
4 | var cookieParser = require('cookie-parser');
5 | var logger = require('morgan');
6 |
7 | var indexRouter = require('./routes/index');
8 | var usersRouter = require('./routes/users');
9 |
10 | var app = express();

```

Die Dateien `index.js` und `user.js` im Verzeichnis `routes` bestimmen die Reaktion des Servers auf die Anfragen des Clients mit vorgegebenen Resource-URLs.

```

11 | // view engine setup
12 | app.set('views', path.join(__dirname, 'views'));
13 | app.set('view engine', 'pug');

```

HTML-Ausgaben werden über die Pug-Template-Engine aus den entsprechenden pug-Dateien im `views`-Verzeichnis erzeugt.

```

14 | app.use(logger('dev'));
15 | app.use(express.json());
16 | app.use(express.urlencoded({ extended: false }));
17 | app.use(cookieParser());
18 | app.use(express.static(path.join(__dirname, 'public')));

```

Statische Inhalte werden aus dem `public`-Verzeichnis ausgeliefert. `app.use` ist eine Funktion, die eine Middleware-Funktion als Parameter erwartet. Die Middleware-Funktion wird bei jeder Anfrage ausgeführt. Sie kann die Anfrage verändern und leitet die Anfrage bei Bedarf an die nächste Middleware-Funktion weiter. Damit werden die Middleware-Funktionen auf der Server-Seite in einer Kette hintereinander geschaltet.

```

19 | app.use('/', indexRouter);
20 | app.use('/users', usersRouter);

```

Für `app.use` mit zwei Parametern bezeichnet der erste Parameter die URL, für die der zweite Parameter zuständig ist. Hier werden die beiden Routen `/` und `/users` an die beiden Router `indexRouter` und `usersRouter` delegiert.

```

21 | // catch 404 and forward to error handler
22 | app.use(function(req, res, next) {
23 |   next(createError(404));
24 | });
25 |
26 | // error handler
27 | app.use(function(err, req, res, next) {
28 |   // set locals, only providing error in development
29 |   res.locals.message = err.message;
30 |   res.locals.error = req.app.get('env') === 'development' ? err : {};
31 |
32 |   // render the error page
33 |   res.status(err.status || 500);
34 |   res.render('error');
35 | });
36 |
37 | module.exports = app;

```

Der Users-Router

wird für die URL `http://localhost:3000/users` aufgerufen und in der Datei `routes/users.js` definiert:

```
1 | var express = require('express');
2 | var router = express.Router();
3 |
4 | /* GET users listing. */
5 | router.get('/', function(req, res, next) {
6 |   res.send('respond with a resource');
7 | });
8 |
9 | module.exports = router;
```

Hierbei enthält `req` Informationen zur Anfrage und `res` generiert die Antwort. Die Funktion `next` leitet die Anfrage an die nächste Middleware-Funktion weiter. Sie wird hier nicht benötigt.

Der Index-Router

wird für die URL `http://localhost:3000` aufgerufen und in der Datei `routes/index.js` definiert:

```
1 | var express = require('express');
2 | var router = express.Router();
3 |
4 | /* GET home page. */
5 | router.get('/', function(req, res, next) {
6 |   res.render('index', { title: 'Express' });
7 | });
8 |
9 | module.exports = router;
```

Die Funktion `res.render` rendert die pug-Datei `index.pug` im Verzeichnis `views` mit dem Parameter `title`.

Die Index-Pug-Dateien

Pug ist eine Template-Engine, die HTML-Dateien aus einer Pug-Datei generiert. Die Syntax ist sehr einfach und wird in der Datei `views/index.pug` verwendet. Die Datei `views/layout.pug` dient hier als Layout-Datei für alle pug-Dateien:

```
1 | doctype html
2 | html
3 |   head
4 |     title= title
5 |     link(rel='stylesheet', href='/stylesheets/style.css')
6 |   body
7 |     block content
```

Die Datei `views/index.pug` erweitert die Datei `views/layout.pug` und enthält:

```
1 | extends layout
2 |
3 | block content
4 |   h1= title
5 |   p Welcome to #{title}
```

7.2. Nuxt.js

`Nuxt.js` ist ein Build-Tool für ein Full-Stack-Framework mit `Vue.js`, das die Entwicklung von Single Page Applications (SPA) und Server-Side-Rendering (SSR) vereinfacht. Es ist eine Erweiterung von `Vue.js` und bietet viele Funktionen, die die Entwicklung von Webanwendungen erleichtern.

Wir werden `Nuxt.js` erst verwenden nachdem wir einfachere Beispielprojekte umgesetzt haben, da es für den Einstieg in die Webentwicklung zu komplex ist. Es ist jedoch ein sehr mächtiges Werkzeug, das die Entwicklung von Webanwendungen erheblich erleichtert. Wir beschränken uns zunächst auf die Entwicklung von Frontend-Anwendungen mit `vue.js` und `vite`. Erst nachdem die wesentlichen Grundlagen von Vue verstanden sind, werden wir `Nuxt.js` verwenden.

8. Frontend-Entwicklung mit Vue

8.1. Entwicklungsumgebung

8.1.1. Vue CLI

Das Vue CLI ist ein Befehlszeilenwerkzeug, das zur Entwicklung von Vue-Anwendungen genutzt wird. Es bietet eine Vielzahl von Befehlen, die die Entwicklung von Vue-Anwendungen erleichtern. Dazu gehören Befehle zum Erstellen von Projekten, zum Hinzufügen von Plugins und zum Bereitstellen von Anwendungen. Wir werden das Vue CLI nicht verwenden.

8.1.2. Vite

Vite ist ein Build-Tool für moderne Web-Entwicklung. Es ist schnell, einfach zu konfigurieren und unterstützt die Entwicklung von Vue-Anwendungen. Es bietet eine Vielzahl von Befehlen zur Unterstützung der Entwicklung von Vue-Anwendungen. Dazu gehören Befehle zum Erstellen von Projekten, zum Hinzufügen von Plugins und zum Bereitstellen von Anwendungen.

8.1.3. Visual Studio Code

Wir werden VS Code als Entwicklungsumgebung verwenden. Hierzu sollten die folgenden Erweiterungen installiert werden:

- Vite
- Vue-Official
- Prettier - Code Formatter
- Project Manager (optional)

Im Browser sollten die `Vue.js devtools` installiert werden.

Vorsicht

Die Installation von Erweiterungen erfolgt auf den Hochschulrechnern nur für den aktuellen Benutzer. Bei der Verwendung eines anderen Rechners müssen die Erweiterungen erneut installiert werden. Daher ist es sinnvoll möglichst immer denselben Rechner zu verwenden.

8.2. Aufsetzen eines Vue-Projekts

Erstellen eines einfachen Vue-Projekts

```
1 bun create vite@latest
2 Project name: einfach
3 Select a framework: Vue
4 Select a variant: Official Vue Starter
5 ...
6 Select features to include in your project:
7 TypeScript? Yes
8 JSX Support? No
9 Router? No
10 Pinia for state management? No
11 Vitest for unit testing? No
12 End-to-End testing solution? No
13 ESLint for code quality? Yes
14 Prettier for code formatting? Yes
```

Laden und Installation der Abhängigkeiten

```
1 | cd einfach
2 | bun install
3 | bun add -D dotenv
4 | bun run dev
```

Damit wird das Projekt erstellt, die Abhängigkeiten installiert und die Default-Anwendung gestartet. Zusätzlich wird noch das Paket `dotenv` installiert, um Umgebungsvariablen zu verwenden.

Einfache Anwendung

Wir werden die Default-Anwendung modifizieren um uns mit dem Komponentensystem und den Direktiven von Vue vertraut zu machen. Dazu werden neue Komponenten erstellt und `App.vue` angepasst.

Anpassung der Konfiguration

Das Modul `dotenv` erlaubt es, Umgebungsvariablen zu verwenden. Diese werden in der Datei `.env` definiert. Diese Variablen können dann in der Anwendung verwendet werden.

```
1 | VITE_PORT=3000
2 | VITE_HOST=localhost
```

In `.env` können Umgebungsvariable nur als Schlüssel-Wert-Paare definiert werden. Eine Nutzung dieser Variablen in `.env` ist nicht möglich. Die Variablen müssen in der Anwendung mit `process.env.VITE_PORT` verwendet werden.

Dazu wird das Modul `dotenv` in der Datei `vite.config.ts` eingebunden.

vite.config.ts

```
1 | import { fileURLToPath, URL } from 'node:url';
2 | import { defineConfig } from 'vite';
3 | import vue from '@vitejs/plugin-vue';
4 | import vueDevTools from 'vite-plugin-vue-devtools';
5 | import dotenv from 'dotenv';
6 |
7 | dotenv.config(); // neu eingefügt
8 | export default defineConfig({
9 |   plugins: [ vue(), vueDevTools() ],
10 |   server: { // neu eingefügt
11 |     port: parseInt(process.env.VITE_PORT || '3000'),
12 |     host: process.env.VITE_HOST || 'localhost',
13 |     open: true,
14 |   },
15 |   resolve: { alias: {
16 |     '@': fileURLToPath(new URL('./src', import.meta.url))
17 |   }, },
18 | })
```

Struktur einer einfachen Vue-Anwendung

In der Regel erzeugt Vite eine Standardanwendung, die dann modifiziert werden kann. Die Struktur der Anwendung ist in der Regel wie folgt:

```

projekt
+-- .env
+-- vite.config.ts
+-- index.html
+-- package.json
+-- node_modules
+-- public
+-- src
    +-- App.vue
    +-- assets
    +-- components
    |   +-- HelloWorld.vue
    +-- main.ts

```

Lediglich die Umgebung `.env` ist neu und die Konfiguration `vite.config.ts` wurde bereits angepasst. Die Datei `index.html` ist die Standard-HTML-Datei, die von Vite erstellt und beim Starten der Anwendung geladen wird.

Das Verzeichnis `public` enthält die statischen Dateien, die von Vite nicht verarbeitet werden. Das Verzeichnis `src` enthält den Quellcode der Anwendung. Die Datei `main.ts` ist der Einstiegspunkt der Anwendung. Die Datei `App.vue` ist die Hauptkomponente der Anwendung.

Das Verzeichnis `assets` enthält die statischen Dateien, die von Vite verarbeitet werden. Das Verzeichnis `components` enthält die Komponenten der Anwendung. Diese Komponenten werden in der Datei `App.vue` verwendet.

8.3. Vue-Komponenten

- Vue-Komponenten sind die Bausteine einer Vue-Anwendung
- Sie bestehen aus HTML, CSS und JavaScript
- Sie können in anderen Komponenten eingebunden werden
- Sie können Daten und Methoden enthalten
- Falls ein Vue-Router vorhanden ist, werden sie typischerweise in den Views eingebunden.

8.3.1. Erstellung

Vue-Komponenten erlauben das UI in wiederverwendbare Teile zu unterteilen. Eine typische Webseite besteht aus Header, Footer, Sidebar und Content. Jeder dieser Teile kann als eigene Komponente erstellt werden. Sidebar und Content können wiederum in weitere Komponenten unterteilt werden. Die Erstellung einer Komponente erfolgt in einer Datei mit der Endung `.vue`.

Vue-Direktiven

Vue-Komponenten können mit Vue-Direktiven und Hooks erweitert werden. Vue-Direktiven sind spezielle Attribute, die das Verhalten von HTML-Elementen ändern. Vue-Direktiven werden durch das Präfix `v-` gekennzeichnet. Oft existieren jedoch Kurzformen.

Vue-Hooks

Vue-Hooks sind Funktionen, die in den Lebenszyklus einer Komponente eingreifen. Vue-Hooks werden durch das Präfix `on` gekennzeichnet.

Beispiel einer Vue-Komponente

Die Datei `Liste.vue` enthält eine Liste von Komponenten. Die Listenelemente werden wiederum von der Komponente `Eintrag.vue` dargestellt. Hierbei werden die HTML-Inhalte von `h2` und `p` in `Eintrag.vue` durch die Slots-Inhalte ersetzt. Slots erlauben den HTML-Code an eine Kindkomponente zu übergeben.

```

1 <script setup lang="ts">
2 import Eintrag from "../Eintrag.vue";
3 </script>
4 <template>
5 <Eintrag>
6   <template v-slot:heading>Eintrag 1</template>

```

```

7 |     <template v-slot:content>Der Inhalt des Eintrags</template>
8 | </Eintrag>
9 | <Eintrag>
10 |     <template v-slot:heading>Eintrag 2</template>
11 |     <template v-slot:content>Der Inhalt von Eintrag 2</template>
12 | </Eintrag>
13 | </template>

```

Differenzierung durch Slots

Die Datei `Eintrag.vue` enthält die Darstellung eines Listeneintrags.

```

1 | <template>
2 |   <div class="item">
3 |     <h2><slot name="heading"></slot></h2>
4 |     <p><slot name="content"></slot></p>
5 |   </div>
6 | </template>
7 | <style scoped>
8 |   <!-- Style-Definition -->
9 | </style>

```

Der Slot `content` enthält den Inhalt des Eintrags. Der Slot `heading` enthält die Überschrift des Eintrags. Auf diese Weise können die Einträge in der Liste individuell gestaltet werden. Die Einträge werden in der Regel noch mit CSS gestaltet.

Slots

Slots sind Platzhalter für HTML-Elemente, die von einem Parent-Element in ein Kind-Element eingefügt werden. Slots erlauben es, den Inhalt eines Kind-Elements von einem Parent-Element bestimmen zu lassen. Slots können benannt werden, um mehrere Slots in einem Element zu verwenden.

Slots können auch als Default-Slots definiert werden. Diese werden verwendet, wenn im Parent-Element kein Slot angegeben wird. Ein Default-Slot wird ohne Namen definiert: `<slot></slot>`.

Vorbesetzte Slot-Inhalte

Slot-Inhalte können auch vorbesetzt werden. Der vorbesetzte Inhalt wird verwendet, wenn im Parent-Element kein Slot-Inhalt angegeben wird.

Referenzierung von Slots

In Parent-Elementen wird der gewünschte Slot durch das `template`-Attribut `v-slot:<slotname>` durch die Angabe des Slot-Namens referenziert. Der Inhalt des Templates wird dann in den Slot eingefügt. Da `v-slot` ein häufig genutztes Direktiv-Attribut ist, kann es auch als Kurzform `#<slotname>` geschrieben werden.

Scoped Slots

Slots können auch als Scoped Slots definiert werden. Scoped Slots erlauben es, Daten aus dem Kind-Element in die, im Parent-Element definierten, Slot-Inhalte zu integrieren. Dazu wird der Slot mit einem Parameter versehen, der die Daten enthält. Der Parameter wird in der Kind-Komponente benutzt und referenziert die entsprechenden JavaScript-Objekte.

Beispiel 83: components/Eintrag.vue mit Scoped Slot

```

1 | <!-- <Eintrag>-template -->
2 | <script setup lang="ts">
3 |   // Define the data that will be used in slots
4 |   const message = "Überschrift";
5 |   const slotData = {

```

```

6   eintrag1: "Hier sind einige Beispieldaten zum Anzeigen",
7   eintrag2: "Hier sind andere Daten zu zeigen"
8 };
9 </script>

10 <template>
11   <div class="eintrag">
12     <h3 class="heading">
13       <slot name="heading" :msg="message">Default Heading</slot>
14     </h3>
15     <div class="content">
16       <!-- Pass the slotData object to the content slot -->
17       <slot name="content" v-bind="slotData">Default Content</slot>
18     </div>
19   </div>
20 </template>

```

In Vue ist `v-bind` eine Kurzform für `v-bind`. Der Slot `content` wird mit dem `slotData`-Objekt befüllt. Dieses Objekt enthält die Daten, die im Parent-Element referenziert werden können. Der Slot `heading` wird mit dem Inhalt der Variable `message` befüllt. Diese Variablen sind in der Kind-Komponente definiert und werden im Parent-Element referenziert.

component/Liste.vue

```

1 <script setup lang="ts">
2 import Eintrag from "./Eintrag.vue";
3 </script>
4
5 <template>
6   <div class="liste-container">
7     <h2>Liste von Einträgen</h2>
8
9     <Eintrag>
10       <template v-slot:heading="{ msg }">{{ msg }}: Eintrag 1</template>
11       <template v-slot:content="slotProps">
12         <div class="content-container">
13           <p><strong>Daten:</strong> {{ slotProps.eintrag1 }}</p>
14         </div>
15       </template>
16     </Eintrag>
17
18     <Eintrag>
19       <template #heading="{ msg }">{{ msg }}: Eintrag 2</template>
20       <template #content="{ eintrag2 }">
21         <div class="content-container">
22           <!-- Demonstrating destructured slot props -->
23           <p><strong>Daten:</strong> {{ eintrag2 }}</p>
24         </div>
25       </template>
26     </Eintrag>
27   </div>
28 </template>

```

Der Slot `heading` in der Kind-Komponente erwartet einen Parameter `msg`, dieser wird im Parent-Element angegeben. Der Slot `content` erwartet ein Objekt `slotProps`, das die Daten enthält. Diese Daten können im Slot-Inhalt referenziert werden. Im zweiten `<Eintrag>` wird anstatt `slotProps` im Slot `content` JavaScript-Destructuring benutzt und das Attribut `eintrag2` referenziert. Dieses wird dann im Slot-Inhalt referenziert.

Default-Slot

Falls Daten über den Default-Slot übergeben werden sollen, wird der Slot ohne Namen definiert: `<slot :data="data"></slot>`. Im Parent-Element wird der Default-Slot dann mit dem Namen `#default` referenziert: `<template #default="slotProps">`.

Aus der Datei `App.vue` werden die bestehenden Komponenten entfernt. Stattdessen wird die Komponente `Liste` importiert und in der Template-Section eingebunden. Die CSS Definitionen bleiben unverändert.

```

1  <script setup lang="ts">
2  import Liste from './components/Liste.vue'
3  </script>
4
5  <template>
6    <header>
7      <h1>Vue Example</h1>
8    </header>
9
10   <main>
11     <Liste />
12   </main>
13 </template>
14
15 <style scoped>
16 header {
17   line-height: 1.5;
18 }
19 .logo {
20   display: block;
21   margin: 0 auto 2rem;
22 }
23 @media (min-width: 1024px) {
24   header {
25     display: flex;
26     place-items: center;
27     padding-right: calc(var(--section-gap) / 2);
28   }
29
30   .logo {
31     margin: 0 2rem 0 0;
32   }
33   header .wrapper {
34     display: flex;
35     place-items: flex-start;
36     flex-wrap: wrap;
37   }
38 }
39 </style>

```

Resultierende Darstellung

Die App enthält eine Liste von Einträgen, die durch die Komponente `Liste` dargestellt werden. Die Einträge werden durch die Komponente `Eintrag` dargestellt. Diese enthält die Slots für den Titel und den Inhalt des Eintrags.

Vue Example

Liste von Einträgen

Überschrift: Eintrag 1

Daten: Hier sind einige Beispieldaten zum Anzeigen

Überschrift: Eintrag 2

Daten: Hier sind andere Daten zu zeigen

Die Darstellung der App *einfach* im Browser

8.3.2. Vue-Hooks und weitere Direktiven

Im folgenden Beispiel werden der `onMounted`-Hook und weitere Vue-Direktiven vorgestellt. Hierzu wird ein neues Projekt erstellt und um eine Komponente erweitert. Die Komponente enthält eine Liste von Einträgen. Diese werden als Array in einer JSON-Datei gespeichert und in der Liste dargestellt.

```

1 | bun create vite@latest
2 | > project-name: listbsp
3 | > vue
4 | > customize with create-vue
5 | TypeScript? ... Yes
6 | JSX Support? ... No
7 | Router (SPA development) ... No
8 | Pinia (state management) ... No
9 | Vitest (Unit Testing) ... No
10 | End-to-End Testing No
11 | ESLint (error protection) ... Yes
12 | Prettier (code formatting) ... Yes
13 | cd listbsp
14 | bun install
15 | bun add -D dotenv
16 | bun dev

```

Es werden wieder die `.env`-Datei mit

```

1 | VITE_HOST=localhost
2 | VITE_PORT=3000

```

erstellt und die entsprechenden Änderungen an `vite.config.ts` vorgenommen. Im Verzeichnis `src/components` wird eine neue Datei `ListBsp.vue` erstellt.

```

1 | <script setup lang="ts">
2 | import { ref, onMounted } from "vue";
3 |
4 | interface Tweet {
5 |   inhalt: String,
6 |   user: String,
7 |   likes: Number
8 | }
9 | const props = defineProps<{ apiUrl: String }>();
10 | const items = ref([] as Tweet[]);
11 |
12 | onMounted(async () => {
13 |   const response = await fetch(props.apiUrl as string)
14 |   if (!response.ok) {
15 |     throw new Error('Network response was not ok')
16 |   }
17 |   const data = await response.json()
18 |   items.value = data.tweets as Tweet[]
19 | });
20 | </script>

```

Der `items`-Array wird mit den Daten aus der JSON-Datei befüllt. Die JSON-Datei enthält ein Array von Tweets. Die URL der JSON-Datei wird als Prop an die Komponente übergeben. Die Funktion `onMounted` ist einer der zur Verfügung stehenden Hooks um Aktionen mit einem Lebenszyklus zu verbinden. Sie wird aufgerufen, sobald die Komponente erstellt wurde. In dieser Funktion wird die JSON-Datei geladen und die Daten in den `items`-Array geschrieben. Dies muss verzögert geschehen, da sonst bei der Ausführung des Skripts die URL noch nicht gesetzt wäre.

JSON-Datei

Die JSON-Datei `tweets.json` enthält ein Array von Tweets und wird im Verzeichnis `public` abgelegt.

```

1 {
2   "tweets": [
3     {"inhalt": "Hunger?", "user": "Max", "likes": 20},
4     {"inhalt": "Hühnchen?", "user": "Moritz", "likes": 10},
5     {"inhalt": "Sie sind weg!", "user": "Witwe Bolte", "likes": 0}
6   ]
7 }

```

In der Template-Section der Komponente wird die Liste der Tweets dargestellt. Solange die Daten noch nicht geladen sind, wird eine Ladeanzeige angezeigt. Diese wird durch die Direktive `v-if` gesteuert. Die Direktive `v-for` wird verwendet, um die Liste der Tweets zu durchlaufen und die einzelnen Tweets darzustellen indem die Daten in die Slots eingefügt werden. Hierbei wird die `v-bind`-Direktive verwendet, um die Daten an die Slots zu übergeben. Diese Direktive kann auch als Kurzform `:` geschrieben werden.

```

21 <template>
22   <ul>
23     <li v-if="!items.length">Loading...</li>
24     <li v-for="(item, index) in items" :key="index">
25       <slot name="item" :="item" />
26     </li>
27   </ul>
28 </template>

```

v-if

Die Direktive `v-if` wird verwendet, um den Inhalt eines Elements nur anzuzeigen, wenn eine Bedingung erfüllt ist. Diese Direktive kann auch mit `v-else` und `v-else-if` kombiniert werden, um mehrere Bedingungen zu überprüfen.

v-for

Die Direktive `v-for` wird verwendet, um eine Liste von Elementen zu durchlaufen und für jedes Element eine Instanz der Komponente zu erstellen. Diese Direktive kann auch mit `v-if` kombiniert werden, um nur bestimmte Elemente anzuzeigen.

Die Schleife wird mit `(item, index) in items` definiert. Hierbei wird `item` als Element und `index` als Index des Elements in der Liste verwendet. Der Index ist optional und kann weggelassen werden. `items` ist der Name des Arrays, das die Daten enthält.

Die Zuweisung `:key` ist erforderlich, um Vue zu helfen, die Elemente in der Liste zu identifizieren. In `<slot name="item" :="item"/>` wird der Slot `item` mit den Daten des Tweets befüllt. Dabei ist `:="item"` eine Kurzform für `v-bind:item`.

Die Listenelemente werden nun noch mit CSS-Anweisungen gestaltet. Dazu wird die `scoped`-Direktive verwendet, um die CSS-Anweisungen nur auf die Komponente anzuwenden.

```

29 <style scoped>
30   ul {
31     list-style-type: none;
32     padding: 5px;
33     background: linear-gradient(315deg, #42d392 25%, #647eff);
34   }
35
36   li {
37     padding: 5px 20px;
38     margin: 10px;

```

```

39 |   background: #ccc
40 | }
41 | </style>

```

Die Komponente wird in der `App.vue`-Datei eingebunden. Die URL der JSON-Datei wird als Prop übergeben. Die App-Datei wird hierzu entsprechend angepasst.

```

1 | <script setup lang="ts">
2 | import { ref } from 'vue'
3 | import ListBsp from '@components/ListBsp.vue'
4 |
5 | const apiUrl = ref(`http://${import.meta.env.VITE_HOST}:${import.meta.env.
   |   VITE_PORT}/tweets.json`)
6 | </script>

```

Die Umgebungsvariablen `VITE_XXXX` wurden in der `.env`-Datei definiert. Diese Datei ist im Wurzelverzeichnis des Projekts abgelegt. Die App-Template wird wie folgt geändert:

```

7 | <template>
8 |   <header>
9 |     <div class="wrapper">
10 |       <h1>List Beispiel</h1>
11 |     </div>
12 |   </header>
13 |
14 |   <main>
15 |     <ListBsp :apiUrl="apiUrl">
16 |       <template #item="item">
17 |         <div class="item">
18 |           <p>{{ item.inhalt }}</p>
19 |           <p class="meta">von {{ item.user }} | {{ item.likes }} likes</p>
20 |         </div>
21 |       </template>
22 |     </ListBsp>
23 |   </main>
24 | </template>

```

Mit `:` wird die Prop `apiUrl` an die `ListBsp`-Komponente übergeben. Jeder `item`-Slot wird mit den Daten eines Tweets befüllt. Die Daten sind in der `item`-Variable verfügbar. Diese wird in `ListBsp` mit der `v-for`-Direktive erzeugt und kann von der App-Komponente referenziert werden. Die Styles der App-Komponente werden noch angepasst, indem Einträge für `main` und `.meta` hinzugefügt werden.

```

25 | <style scoped>
26 | main {
27 |   flex: 1;
28 |   font-size: 1.2em;
29 |   color: #000;
30 | }
31 |
32 | .meta {
33 |   font-size: 1em;
34 |   color: #070;
35 | }
36 | ...
37 | </style>

```

8.4. Vue Router

8.4.1. Installation

- Vue Router ist ein offizielles Plugin für Vue.js

- Es ermöglicht das Definieren von Routen in einer Vue-Anwendung
- Es ermöglicht das Navigieren zwischen den Routen
- Es ermöglicht das Laden von Komponenten für die Routen

Installation

Falls der Router nicht bei der Erstellung des Projekts hinzugefügt wurde, kann er nachträglich installiert werden:

```
1 | bun add vue-router
```

Fall anstelle von vite das Build-Tool Nuxt verwendet wird, nutzt Nuxt den vue-router intern um die Routen automatisch über das pages-Verzeichnis zu definieren.

Wir erzeugen eine neue Vue-Anwendung mit dem Namen barchart.

```
1 | bun create vite barchart
```

Die Vue-Anwendung wird mit dem Vue-Starter-Template erzeugt. Als Optionen werden TypeScript, der Router, ESLint und Prettier ausgewählt. Danach wird in das Verzeichnis barchart gewechselt und die Abhängigkeiten installiert. Zusätzlich wird wieder dotenv verwendet und vite.config.ts angepasst.

Wir werden eine Vue-Anwendung erstellen, die ein CSS-basiertes Balkendiagramm darstellt. Das Balkendiagramm wird auf einer eigenen Seite dargestellt. Diese Seite wird über den Router geladen und kann über einen Link aufgerufen werden.

8.4.2. Grundlagen

- Vue Router ist ein Plugin, das in die Vue-Instanz eingebunden wird
- Es wird in der Datei src/main.ts eingebunden
- Es wird in der Datei src/router/index.ts konfiguriert

src/main.ts

```
1 | import { createApp } from 'vue'
2 |
3 | import App from './App.vue'
4 | import router from './router'
5 |
6 | const app = createApp(App)
7 | app.use(router)
8 | app.mount('#app')
```

src/router/index.ts

Die Routen werden in einer eigenen Datei konfiguriert. Die Datei views/BarView.vue existiert noch nicht und muss erstellt werden.

```
1 | import { createRouter, createWebHistory } from 'vue-router'
2 | import HomeView from '../views/HomeView.vue'
3 |
4 | const router = createRouter({
5 |   history: createWebHistory(import.meta.env.BASE_URL),
6 |   routes: [ {
7 |     path: '/',
8 |     name: 'home',
```

```

 9     component: HomeView
10   },
11   { // neu hinzugefügt
12     path: '/chart',
13     name: 'chart',
14     component: () => import('../views/BarView.vue')
15   },
16   {
17     path: '/about',
18     name: 'about',
19     // route level code-splitting
20     // this generates a separate chunk (About.[hash].js) for this route
21     // which is lazy-loaded when the route is visited.
22     component: () => import('../views/AboutView.vue')
23   }
24 ];
25
26 export default router;

```

Die verfügbaren Routen werden in einem Array definiert. Die Vue-Komponenten für die einzelnen Seiten werden per Konvention im Verzeichnis `views` abgelegt. Die einzelnen Views stellen dann die jeweiligen Seiten mit Hilfe weiterer Vue-Komponenten dar.

src/views/BarView.vue

Um die BarView zu erstellen werden Standard-Teile einer Vue-Komponente benötigt: Skript, Template und Style. Letzteres ist hier optional.

```

1 <script setup lang="ts">
2 import BarChart from "../components/BarChart.vue";
3 </script>
4 <template>
5   <BarChart />
6 </template>

```

Die Komponente BarChart wird aus dem Verzeichnis `components` eingebunden. Diese ist für die Darstellung des Bar-Charts zuständig.

Die Komponente BarChart wird in der Datei `src/components/BarChart.vue` erstellt.

components/BarChart.vue

```

7 <template>
8   <h1>Bar Chart</h1>
9   <div class="chart-container">
10     <div
11       v-for="(data, index) in chartData"
12       :key="index"
13       class="bar"
14       :style="{ height: data.value + 'pt' }"
15     >
16       <span class="label">{{ data.label }}</span>
17     </div>
18   </div>
19 </template>

```

Der Template-Teil enthält den Titel und die Darstellung des Balkendiagramms. Die Daten für den Bar-Chart werden in der Variable `chartData` erwartet.

```

20 <style scoped>
21 .chart-container {
22   display: flex;
23   align-items: flex-end;
24   height: 20em;
25   border: 1px solid #ccc;
26   padding: 0.5em;
27   position: relative;
28 }
29
30 .h1 {
31   width: 100%;
32   font-color: #40a040;
33 }
34 .bar {
35   width: 2em;
36   margin: 0 0.25em;
37   background-color: #3498db;
38   position: relative;
39   display: flex;
40   align-items: flex-end;
41   justify-content: center;
42   margin-bottom: 2em;
43 }
44
45 .label {
46   position: absolute;
47   bottom: -2em;
48   font-size: 1em;
49   text-align: center;
50   width: 100%;
51 }
52 </style>
53 </template>

```

Der Style-Teil enthält die Definition des Styles für die Komponente. Die Darstellung des Bar-Charts erfolgt in einem Container, der die Balken enthält. Jeder Balken wird durch ein `div`-Element dargestellt, das die Höhe des Balkens definiert. Die Beschriftung des Balkens wird unterhalb des Balkens angezeigt. Der Skript-Teil enthält die Definition der Daten für die Balkendiagramme.

```

1 <script setup lang="ts">
2 import { ref } from 'vue';
3
4 const chartData = ref([
5   { label: 'A', value: 28 },
6   { label: 'B', value: 55 },
7   { label: 'C', value: 43 },
8   { label: 'D', value: 91 },
9   { label: 'E', value: 81 },
10  { label: 'F', value: 53 },
11  { label: 'G', value: 19 },
12  { label: 'H', value: 87 },
13 ])
14 </script>

```

Die `ref`-Funktion wird verwendet, um die Daten für den Bar-Chart zu definieren. Die Daten bestehen aus einem Label und einem Wert. Die Daten werden in der Variable `chartData` gespeichert. Durch `ref` wird die Reaktivität der Daten sichergestellt. Zuletzt wird der `BarView` noch in der Datei `src/App.vue` eingebunden. Dazu wird der Router-Link in die Navigation zwischen `Home` und `About` eingebaut. Der Rest der Datei bleibt unverändert.

src/App.vue

```

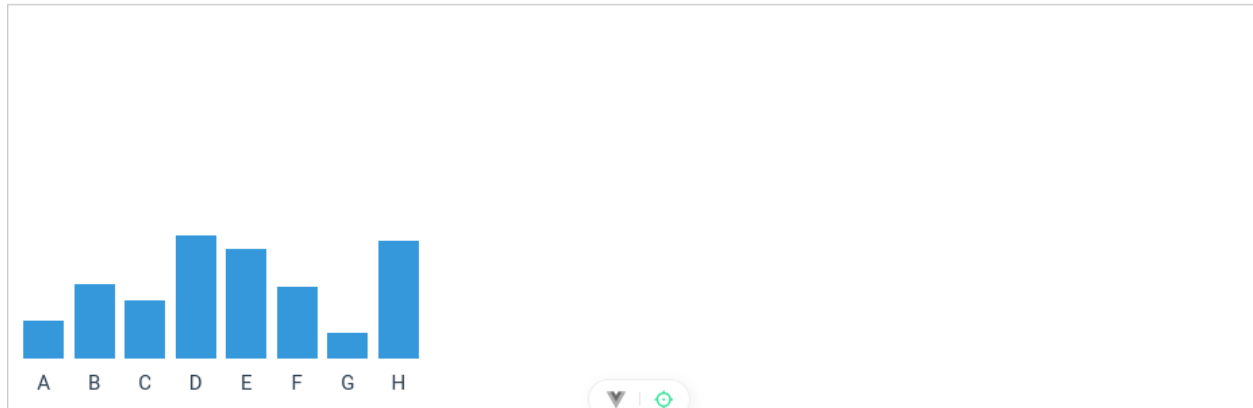
1 <nav>
2   <RouterLink to="/">Home</RouterLink>
3   <!-- neu hinzugefügt -->
4   <RouterLink to="/chart">Bar Chart</RouterLink>
5   <RouterLink to="/about">About</RouterLink>
6 </nav>

```

Das `<RouterLink>`-Element wird verwendet, um zwischen den Routen zu navigieren. Der `to`-Parameter gibt die Route an, zu der navigiert werden soll. Diese wurde in `src/route/index.ts` definiert. Der Router-Link wird vom Router-Plugin bereitgestellt und ist eine Vue-Komponente, die den Link für die Navigation zwischen den Routen darstellt.

Darstellung im Browser

Beispiel für Balkendiagramm mit CSS



8.5. D3 in Vue-Komponenten

D3 kann in einer Vue-Komponente eingebunden werden. Dazu muss jedoch beachtet werden, dass D3 sein eigenes System für den Zugriff auf Elements implementiert. Dieses ist nicht mit dem virtuellen Vue-DOM kompatibel. Daher muss hier extra darauf geachtet werden, dass D3 direkt auf das DOM zugreift.

Nachdem das Projekt mit Vite erstellt wurde kann D3 installiert werden:

```
1 | cd <projektdir>
2 | bun install
3 | bun add d3
4 | bun add -D @types/d3
```

Die TypeScript-Datentypen für D3 müssen separat installiert werden. Diese sind notwendig, um TypeScript mit D3 zu verwenden. Anschließend kann D3 in einer Vue-Komponente verwendet werden. Die Typen sind nur für die Entwicklung notwendig und werden daher als `-D` installiert.

Wir fügen die neue Komponente `D3Circle.vue` in das Verzeichnis `src/components` ein. Ebenso wird ein neuer View `view/CircleView.vue` erzeugt und dann in `src/router/index.vue` eingetragen.

Beispiel 84: Kreis mit D3 in Vue-Komponente

<pre> 1 <template> 2 <div ref="divElem"></div> 3 </template> 4 <script setup lang="ts"> 5 import {ref,onMounted} from "vue"; 6 import * as d3 from "d3"; 7 const divElem = ref<HTMLElement null>(null); 8 9 onMounted(() => { 10 const svg = drawSVG(); 11 if(svg && divElem.value) { 12 divElem.value.appendChild(svg); 13 } 14 }); </pre>	<pre> 15 function drawSVG() : SVGSVGElement null { 16 const svg = d3.create("svg"); 17 if(svg === null) return null; 18 svg.attr("width", 200).attr("height", 200); 19 svg.append("circle") .attr("cx", 100) .attr("cy", 100) .attr("r", 50); 22 return svg.node(); 23 } 24 25 </script> 26 <style> /* Globale Styles */ 27 svg { border: 2px solid black; } 28 svg > circle { fill: red; } 29 </style> </pre>
---	--

d3 benutzt Selektoren. Diese unterscheiden sich von den zugehörigen HTML-Elementen. Daher wird hier `d3.create("svg")` verwendet, um ein D3-SVG-Element zu erstellen. Mit `svg.node()` wird

das eigentliche SVG-Element zurückgegeben. Dieses kann dann in das DOM eingefügt werden.

Daher sollte darauf geachtet werden, dass D3-Elemente nicht direkt in das Vue-DOM eingefügt werden, sondern erst als node zurückgegeben und dann eingefügt werden.

Alle D3-Anweisungen müssen im onMounted-Hook ausgeführt werden, da hier sichergestellt ist, dass das DOM vollständig geladen ist. D3 kann nur auf Elemente zugreifen, die bereits im DOM vorhanden sind oder von D3 selbst erstellt wurden.



Kreis mit D3 in Vue-Komponente

9. Full-Stack Vue mit Nuxt

9.1. Installation

- Nuxt ist ein Framework für Vue.js
- Es ist ein Meta-Framework, das auf Vue.js aufbaut
- Es ist ein Full-Stack-Framework, das sowohl Frontend als auch Backend abdeckt
- Es ist ein statisches Site-Generator-Framework
- Es ist ein Server-Side-Rendering-Framework
- Es ist ein Single-Page-Application-Framework

Wir werden Nuxt nur in einfachen Anwendungen verwenden. Daher werden wir uns auf die Grundlagen beschränken. Eine ausführliche Besprechung von Nuxt ist im Rahmen dieser Vorlesung nicht möglich.

Aufsetzen eines Nuxt-Projektes

```
1 | bun x nuxi init my-nuxt-app
2 | cd my-nuxt-app
3 | bun install
4 | bun run dev -o
```

Es wird ein neues Nuxt-Projekt erstellt. Der Befehl `bun x` lädt das Nuxt-CLI `nuxi` herunter und führt es aus. Anschließend wird das Projekt in das Verzeichnis `my-nuxt-app` erstellt. Mit `bun run dev -o` wird die erzeugte default Single-Page-Application im Entwicklermodus gestartet. Die `-o`-Option öffnet den Browser automatisch auf Port 3000. Der Port kann mit der Option `-p` geändert werden.

9.2. Nuxt-Ordnerstruktur

Die Ordnerstruktur eines Nuxt-Projekts unterscheidet sich von der eines Vite-Projekts, obwohl Nuxt auf Vite basiert. Die Ordnerstruktur ist jedoch nicht festgelegt. Einige der wichtigsten Verzeichnisse in einem Nuxt-Projekt sind:

- `assets`: Statische Dateien wie Bilder, Schriftarten, CSS, etc.
- `components`: Wiederverwendbare Vue-Komponenten
- `layouts`: Layouts für die Anwendung
- `pages`: Seiten der Anwendung
- `public`: Statische Dateien, die nicht kompiliert werden
- `utils`: Hilfsfunktionen

public

Lediglich das `public`-Verzeichnis ist in der Standardkonfiguration vorhanden. Die anderen Verzeichnisse müssen erstellt werden. Nuxt erkennt viele Verzeichnisse automatisch und bindet sie in die Anwendung ein.

components

Hier werden die `vue`-Dateien für die Komponenten abgelegt. Nuxt erkennt automatisch die Komponenten und bindet sie in die Anwendung ein. Das Verzeichnis muss bei Bedarf erstellt werden.

app.vue

Layouts sind die Vorlagen für die Seiten der Anwendung. Sie können in der Datei `app.vue` mit

```
1 <template>
2   <NuxtLayout>
3     <NuxtPage />
4   </NuxtLayout>
5 </template>
```

aktiviert werden.

Wird eine Multi-Page-Application erstellt, so wird die Datei `app.vue` nicht mehr benötigt.

layouts-Verzeichnis

Hier werden die `vue`-Dateien für die Layouts abgelegt. Auch die Layouts werden automatisch erkannt. Falls kein Layout definiert ist, wird das Standardlayout verwendet.

default.vue

```
1 <template>
2   <div>
3     <p> Dies ist das Standardlayout für alle Seiten</p>
4     <slot />
5   </div>
6 </template>
```

Der Seiteninhalt wird mit dem `slot`-Element eingefügt.

pages Verzeichnis

Hier werden die `vue`-Dateien für die Seiten abgelegt. Die Seite `index.vue` wird als Startseite verwendet. Die Namen der Dateien entsprechen den Routen der Anwendung.

9.3. Zustandsverwaltung mit Pinia

9.3.1. Vorteile eines Store-Plugin

- **Zentralisierte Zustandsverwaltung:** Ein Store-Plugin ermöglicht die Verwaltung des Zustands an einem zentralen Ort.
- **Verbesserte Code-Organisation:** Die Trennung der Zustandslogik von den Komponenten führt zu besserer Code-Organisation.
- **Vorhersehbare Zustandsänderungen:** Store-Plugins erzwingen eine vorhersehbare Art der Zustandsänderung.
- **Erweitertes Debugging:** Mit einem zentralisierten Store können Tools wie Vue DevTools zur Inspektion und zum Debuggen des Zustands verwendet werden.
- **Wiederverwendbarkeit:** Stores können in verschiedenen Komponenten wiederverwendet werden, was die Code-Wiederverwendung fördert.
- **Typensicherheit:** Pinia bietet hervorragende Unterstützung für TypeScript, was die Typensicherheit und die Autovervollständigung verbessert.

9.3.2. Installation in einem Vue-Projekt

```
1 | bun add pinia
```

Pinia wird in der Regel in der Datei

- `src/main.ts` eingebunden
- `src/store/index.ts` konfiguriert

`src/main.ts`

```
1 | import { createApp } from 'vue';
2 | import App from './App.vue';
3 | import store from './store';
4 |
5 | const app = createApp(App);
6 | app.use(store);
7 | app.mount('#app');
```

`src/store/index.ts`

```
1 | import { createPinia }
2 |   from 'pinia';
3 |
4 | const store = createPinia();
5 |
6 | export default store;
```

9.3.3. Konfiguration in einem Nuxt-Projekt

```
1 | bun add pinia @pinia/nuxt
```

- `@pinia/nuxt` ist ein Plugin für Nuxt
- Pinia wird als Modul in der Datei `nuxt.config.ts` konfiguriert.

`nuxt.config.ts`

Der `modules`-Eintrag in `nuxt.config.ts` muss um `@pinia/nuxt` erweitert werden. Falls die `modules`-Eigenschaft noch nicht vorhanden ist, muss sie hinzugefügt werden.

```
1 | modules: [
2 |   '@pinia/nuxt',
3 | ],
```

9.3.4. Erstellen eines Stores

- Ein Pinia-Store wird in der Regel im `store`-Verzeichnis erstellt.
- Ein Store wird mit der Funktion `defineStore` erstellt.
- Ein Store besteht aus einem Zustand und optionalen Aktionen.

`store/counter.ts`

```
1 | import { defineStore } from 'pinia';
2 |
3 | export const useCounterStore = defineStore('counter', {
4 |   state: () => ({ count: 0, }),
5 |   actions: {
6 |     increment() {
7 |       this.count++
8 |     },
9 |   },
10 | });
```

9.3.5. Verwendung des Stores in Komponenten

```

1 | <template>
2 |   <div>
3 |     <p>Count: {{ counter.count }}</p>
4 |     <button @click="counter.increment">Increment</button>
5 |   </div>
6 | </template>
7 |
8 | <script setup lang="ts">
9 | import { useCounterStore } from '~/store/counter';
10 |
11 | const counter = useCounterStore();
12 | </script>

```

9.4. HTTP-Anfragen mit Axios

9.4.1. Axios vs. fetch

Axios ist ein HTTP-Client für JavaScript, der eine einfachere API als `fetch` bietet. Er wird häufig in Nuxt-Projekten verwendet, um API-Aufrufe zu handhaben. Eine typische Anwendung von Axios ist das Abrufen von Daten von einer API und das Anzeigen dieser Daten in einer Vue-Komponente. Die Vorteile von Axios gegenüber `fetch` sind:

- **Einfachheit und Benutzerfreundlichkeit:** Axios bietet eine einfache und intuitive API für HTTP-Anfragen.
- **Promise-basiert:** Axios ist promise-basiert und funktioniert gut mit modernen JavaScript-Features wie `async` und `await`.
- **Automatische JSON-Datenumwandlung:** Axios wandelt JSON-Daten automatisch um, was die Arbeit mit APIs erleichtert.
- **Interceptors:** Axios ermöglicht das Einrichten von Request- und Response-Interceptors für benutzerdefinierte Header, Authentifizierung, Logging und Fehlerbehandlung.
- **Abbrechbare Anfragen:** Axios unterstützt abbrechbare Anfragen mit dem `CancelToken`-API.
- **Breite Browser-Unterstützung:** Axios funktioniert in allen modernen Browsern und unterstützt ältere Browser wie Internet Explorer 11.
- **Eingebauter CSRF-Schutz:** Axios bietet eingebauten Schutz vor Cross-Site Request Forgery (CSRF).

9.4.2. Installation in einem Vue-Projekt

```
1 | bun add axios
```

src/axios.js

```

1 | import axios from 'axios'
2 |
3 | const instance = axios.create({
4 |   baseURL: 'https://api.example.com',
5 |   timeout: 1000,
6 |   headers: { 'X-Custom-Header': 'foobar' }

```

```

7 | })
8 |
9 | export default instance

```

9.4.3. Verwendung in Komponenten

Beispielkomponente

```

1 | <template>
2 |   <div>
3 |     <p>Data: {{ data }}</p>
4 |   </div>
5 | </template>
6 | <script setup lang="ts">
7 | import { ref, onMounted } from 'vue'
8 | import axios from './axios'
9 | const data = ref(null)
10 | onMounted(async () => {
11 |   try {
12 |     const response = await axios.get('/endpoint')
13 |     data.value = response.data
14 |   } catch (error) {
15 |     console.error('Error fetching data:', error)
16 |   }
17 | })
18 | </script>

```

9.4.4. Installation in einem Nuxt-Projekt

```

1 | bun add @nuxtjs/axios

```

nuxt.config.ts

Zur Konfiguration muss das axios-Modul in der modules-Eigenschaft des nuxt.config.ts-Files hinzugefügt werden.

```

1 | modules: [
2 |   '@nuxtjs/axios',
3 | ],
4 | axios: {
5 |   baseURL: 'https://api.example.com', // Basis-URL für API-Anfragen
6 | },

```

9.4.5. Verwendung in Komponenten

Beispielkomponente

```

1 | <template>
2 |   <div>
3 |     <p>Data: {{ data }}</p>
4 |   </div>
5 | </template>

```

In Nuxt besteht die Konvention Eigenschaften, die in den Kontext injiziert werden mit einem \$-Zeichen zu beginnen. Das axios-Objekt wird in Nuxt als \$axios injiziert und ist damit in allen Komponenten verfügbar. Mit dem \$-Präfix wird in Nuxt per Konvention signalisiert, dass es sich um eine injizierte Eigenschaft handelt.

```

6 | <script setup lang="ts">
7 | import { ref, onMounted } from 'vue'
8 | import { useNuxtApp } from '#app'
9 |
10 | const data = ref(null)
11 | const { $axios } = useNuxtApp()
12 |
13 | onMounted(async () => {
14 |   try {
15 |     const response = await $axios.get('/endpoint')
16 |     data.value = response.data
17 |   } catch (error) {
18 |     console.error('Error fetching data:', error)
19 |   }
20 | })
21 | </script>

```

9.4.6. Verwendung von Interceptors

Interceptors in Axios sind Funktionen, die es ermöglichen, Anfragen und Antworten zu bearbeiten, bevor sie gesendet oder empfangen werden. Sie bieten eine leistungsstarke Möglichkeit, wiederverwendbare Logik für alle HTTP-Anfragen und -Antworten in Anwendungen zu implementieren.

Hauptanwendungsfälle für Interceptors

- Hinzufügen von Authentifizierungs-Token

Interceptors können verwendet werden, um Authentifizierungs-Token automatisch zu jeder Anfrage hinzuzufügen.

- Logging

Interceptors ermöglichen es, Anfragen und Antworten zu protokollieren. Dies kann hilfreich sein, um Debugging-Informationen zu sammeln oder die Leistung einer Anwendung zu überwachen.

- Fehlerbehandlung

Mit Interceptors können Fehler global behandelt werden. So können z.B. bestimmte Aktionen ausgeführt werden, wenn eine Anfrage fehlschlägt.

- Modifikation von Anfragen und Antworten

Interceptors können verwendet werden, um Anfragen und Antworten zu modifizieren. Sie können z.B. Header hinzufügen, Daten transformieren oder die URL ändern, bevor die Anfrage gesendet wird.

Nachfolgend wird gezeigt, wie Interceptors in einem Nuxt 3-Projekt verwendet werden können, einschließlich der Installation, Konfiguration und Erstellung eines Plugins für die Interceptors. Interceptors bieten eine leistungsstarke Möglichkeit, wiederverwendbare Logik für alle HTTP-Anfragen und -Antworten in einer Anwendung zu implementieren.

plugins/axios.ts

```

1 | export default defineNuxtPlugin(({ $axios, redirect }) => {
2 |   $axios.onRequest(config => {
3 |     console.log('Making request to ' + config.url)
4 |     // Beispiel: Authentifizierungs-Token hinzufügen

```

```

5     config.headers.common['Authorization'] = `Bearer ${localStorage.getItem('token
      ')}'
6   })
7
8   $axios.onResponse(response => {
9     console.log('Response received from ' + response.config.url)
10    return response
11  })
12
13  $axios.onError(error => {
14    const code = parseInt(error.response && error.response.status)
15    if (code === 400) {
16      redirect('/400')
17    }
18    return Promise.reject(error)
19  })
20 })

```

nuxt.config.ts

```

1 export default defineNuxtConfig({
2   modules: [
3     '@nuxtjs/axios',
4   ],
5   axios: {
6     baseURL: 'https://api.example.com',
7   },
8   plugins: [
9     '~/plugins/axios.ts',
10  ],
11 })

```

Teil III.

2d-Visualisierung im Frontend mit D3

10. Einführung in D3.js

10.1. Kreis mit D3 in einer Nuxt-Anwendung

Zu Beginn wird ein neues Nuxt-Projekt erstellt:

```
1 | bun x nuxi init nuxtapp
```

Hierbei ist nuxtapp der Name des Projekts. Anschließend kann D3 installiert und das Projekt gestartet werden:

```

1 | cd nuxtapp
2 | bun install
3 | bun add d3
4 | bun add -D @types/d3
5 | bun dev run -o

```

Beispiel 85: index.vue

In nuxt ist bereits ein Router integriert. Es müssen lediglich die entsprechenden Seiten als Dateien in das pages-Verzeichnis eingestellt werden. Die Hauptseite enthält hier nur Verweise auf die anderen Seiten.

```

1 <template>
2 <H1>Welcome to this Homepage</H1>
3 <ul>
4   <li><NuxtLink to="/d3circle">Beispielkreis in D3</NuxtLink></li>
5   <li><NuxtLink to="/twocircles">Zwei Kreise in D3</NuxtLink></li>
6   <li><NuxtLink to="/about">Über diese Seite</NuxtLink></li>
7 </ul>
8 </template>

```

Die weiteren Seiten werden in eigenen Dateien im pages-Verzeichnis abgelegt. Die Links auf diese Seiten werden mit der Nuxt-Komponente `NuxtLink` erstellt.

onecircle.vue

Dies wiederholt das vorhergehende D3-Vue-Beispiel.

```

1 <template>
2   <div ref="divElement"></div>
3   <p><NuxtLink to="/">Home</NuxtLink></p>
4 </template>
5
6 <style>
7 svg {
8   border: 2px solid black;
9   background-color: lightgray;
10 }
11
12 div > svg > circle {
13   fill: red;
14 }
15 </style>j
16
17 <script lang="ts" setup>
18 import { ref, onMounted } from "vue";
19 import * as d3 from "d3";
20
21 const divElem = ref<HTMLElement | null>(null);
22
23 function drawSVG() : SVGSVGElement | null {
24   const svg = d3.create("svg");
25   if (svg === null) return null;
26
27   svg.attr("width", 200).attr("height", 200);
28   svg.append("circle")
29     .attr("cx", 100)
30     .attr("cy", 100)
31     .attr("r", 100)
32     .attr("class", "circleClass");
33   return svg.node();
34 }
35 onMounted(() => {
36   const svg = drawSVG();
37   if (svg && divElem.value) {
38     divElem.value.appendChild(svg);
39   }
40 });
41 </script>

```

Auch hier gilt: d3-Befehle können erst ausgeführt werden, nachdem die Seite vollständig geladen ist. Daher wird die Funktion `drawSVG` in der `onMounted`-Funktion aufgerufen. Der Kreis wird in ein SVG-Element eingefügt, das wiederum in das `div`-Element eingefügt wird. Die Kreisfarbe wird über die CSS-Klasse `circleClass` festgelegt.

about.vue

Die Seite `about.vue` enthält Informationen über die Seite. Hier ist der Inhalt minimal gehalten.

```

1 <template>
2   <div>
3     <h1>About this Page</h1>
4     <p>This is a simple page created with Nuxt.js.</p>
5     <p><NuxtLink to="/">Home</NuxtLink></p>
6   </div>
7 </template>

```

Im letzten Teil des Beispiels wird ein zweiter Kreis hinzugefügt. Dieser wird in einer eigenen Datei `twocircles.vue` erstellt. Der zweite Kreis wird in der Funktion `drawSVG` hinzugefügt. Die Kreise haben unterschiedliche Farben und Größen. Hierbei werden jedoch die Daten der Kreise in einem Array gespeichert und die Kreise automatisch mit D3 erstellt. Die `select`-Methode wird durch D3 auf jeden Eintrag des Arrays angewendet. Die Kreise werden in einem `g`-Element gruppiert, um sie gemeinsam zu transformieren.

twocircles.vue

```

1 <template>
2   <div id="d3pane"></div>
3   <p><NuxtLink to="/">Home</NuxtLink></p>
4 </template>
5 <style>
6   svg {
7     background-color: lightgray;
8   }
9   #d3pane {
10    border: 1px solid black;
11  }
12  #d3pane circle {
13    fill: green;
14    stroke: red;
15    stroke-width: 2;
16  }
17 </style>
18
19 <script setup lang="ts">
20   import { onMounted } from "vue";
21   import * as d3 from "d3";
22
23   type CircleData = { r: number, x: number, y: number };
24
25   function doD3Stuff(){
26     const svg = d3.create("svg")
27       .attr("width", 200)
28       .attr("height", 200);
29     const data : CircleData[] =
30       [{ r: 50, x: 50, y: 100 }, { r: 25, x: 150, y: 100} ];

```

Die Kreiseigenschaften werden in einem Array gespeichert. Die Kreise haben unterschiedliche Radien und Positionen. Die Kreise werden in einem `svg`-Element erstellt.

```

31     const circleData = svg.selectAll<
32       SVGCircleElement, CircleData> ("circle")
33       .data(data);

```

Mit `selectAll` wird eine Auswahl von Kreisen im SVG-Element erstellt. Die Methode `data` bindet die Daten an die Kreise. Es können sowohl weniger (inklusive keine), aber auch mehr Kreise vorhanden sein, als Daten vorhanden sind. `circleData` enthält die Auswahl, nicht die Kreise selbst.

```

34 |     const circles = circleData.enter()
35 |       .append("circle");

```

Mit `enter` werden die Kreise erstellt, falls sie noch nicht existieren. Die `append`-Methode fügt die neu erzeugten Kreise zu den bereits bestehenden Kreisen hinzu und gibt ein D3-Selection mit den Kreisen als Inhalt zurück. Es ist erlaubt, dass noch keine Kreise bestehen. In diesem Fall werden alle Kreise neu erstellt.

```

36 |     circles.attr("r", (d: CircleData) => d.r)
37 |       .attr("cx", (d: CircleData) => d.x)
38 |       .attr("cy", (d: CircleData) => d.y);
39 |
40 |     const d3pane = d3.select("#d3pane").node() as HTMLElement;
41 |     d3pane.appendChild(svg.node()!);
42 | }
43 |
44 | onMounted(doD3Stuff);
45 | </script>

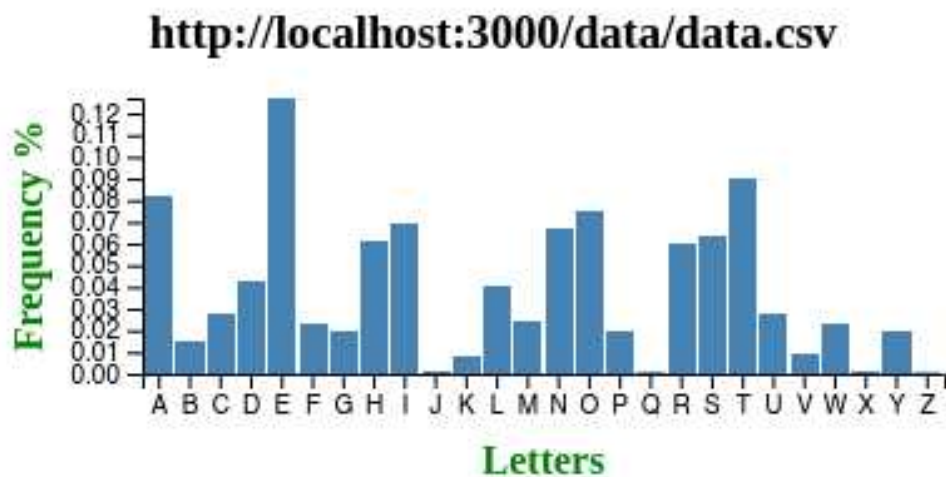
```

Mit `attr` werden die Attribute der Kreise gesetzt. Die Kreise werden in das `svg`-Element eingefügt. `circles` enthält die Kreise, die in das `svg`-Element eingefügt werden. Die `attr`-Methoden wirken durch D3 auf alle ausgewählten Kreise. Schleifen sind in D3 meist nicht notwendig.

10.2. Balkendiagramm mit D3

Es soll ein Balkendiagramm erstellt werden, das die Häufigkeit von Buchstaben in einem englischsprachigen Text darstellt. Hierzu werden die Häufigkeiten der Buchstaben aus einer CSV-Datei eingelesen und dann an eine Komponente übergeben, die das Balkendiagramm erstellt. Das Balkendiagramm soll mit Achsbeschriftungen versehen werden.

Darstellung des Balkendiagramms



Erstellung als Nuxt-Projekt

1. Erzeuge Projekt mit `bun x nuxi init balkendiagramm`.
2. Wechsle in das Projektverzeichnis `cd balkendiagramm`.
3. Führe `bun install` aus.
4. Elemente installieren: `bun add d3` und `bun add -D @types/d3`.

5. Erstelle die Verzeichnisse `components`, `pages` und `public/data`.
6. Lösche die Datei `app.vue`.
7. erstelle die Datei `.env` mit dem Inhalt `NEXT_API_URL=http://localhost:3000`.

nuxt.config.ts

Die Datei `nuxt.config.ts` enthält die Konfiguration des Projekts. Per Voreinstellung wird mit `@` das Quellverzeichnis `src` angenommen. Wir möchten dagegen unser Projektverzeichnis `balkendiagramm` als Quellverzeichnis verwenden.

```

1 import { defineNuxtConfig } from 'nuxt/config';
2 import { resolve } from 'path';
3
4 export default defineNuxtConfig({
5   compatibilityDate: '2024-11-01',
6   devtools: { enabled: true },
7   alias: {
8     '@': resolve(__dirname, '.'),
9   },
10  runtimeConfig: {
11    public: { // Variable aus .env lesen.
12      apiUrl: process.env.NEXT_API_URL || 'http://localhost:3000'
13    }
14  }
15 });

```

Datei `public/data/data.csv`

Die Datei enthält die Häufigkeit der Buchstaben. Die erste Zeile gibt die Namen der Spalten an. Die asynchrone Funktion `d3.csv` liest die Spalten in der Datei als Strings ein.

Vorsicht: Leerzeichen werden als Strings erkannt und müssen daher in der Datei entfernt werden.

1	letter,frequency	15	N,.06749
2	A,.08167	16	O,.07507
3	B,.01492	17	P,.01929
4	C,.02782	18	Q,.00095
5	D,.04253	19	R,.05987
6	E,.12702	20	S,.06327
7	F,.02288	21	T,.09056
8	G,.02015	22	U,.02758
9	H,.06094	23	V,.00978
10	I,.06966	24	W,.02360
11	J,.00153	25	X,.00150
12	K,.00772	26	Y,.01974
13	L,.04025	27	Z,.00074
14	M,.02406		

pages/index.vue

```

1 <template>
2 <div>
3   <div v-if="loading" class="loading">Loading data...</div>
4   <Balken v-else-if="data.length > 0"
5     :width="400"
6     :height="200"
7     :config="chartConfig"
8     :data="data" />
9   <div v-else class="error">No data available</div>
10 </div>
11 </template>

```

Im Template wird die Komponente `Balken` eingebunden. dabei ist `:` ein Alias für die `v-bind`-Anweisung von Vue. Diese sorgt für eine passende Datenkonversion aus den übergebenen Strings. Die Daten werden als Prop übergeben. Die if-Bedingungen sind nötig, um sicherzustellen, dass die Komponente erst dann gerendert wird, wenn die Daten verfügbar sind.

```

12 | <script setup lang="ts">
13 | import { ref, onMounted, computed } from 'vue';
14 | import * as d3 from 'd3';
15 | import Balken, { type BarData, type ChartConfig }
16 |   from '@/components/Balken.vue';
17 | import { useRuntimeConfig } from 'nuxt/app';
18 |
19 | const config = useRuntimeConfig(); // aus nuxt.config.js lesen
20 | const dataUrl = '/data/data.csv';
21 | const apiUrl = config.public.apiUrl; // aus nuxt.config.js
22 | const data = ref<BarData[]>([]);
23 | const loading = ref(true);
24 |
25 | // Erzeuge ein reaktives ChartConfig-Objekt
26 | const chartConfig = computed<ChartConfig>(() => ({
27 |   title: apiUrl + dataUrl;
28 |   xlabel: 'Letter',
29 |   ylabel: 'Frequency %'
30 | })))

```

Die Nuxt-Funktion `useRuntimeConfig` liest die Konfigurationsdaten aus der Datei `nuxt.config.js`. Die Konfigurationsdaten werden in der Variable `config` gespeichert. Die Konfigurationsdaten enthalten die URL der API, die in der Datei `.env` definiert ist.

Die Vue-Funktion `computed` erzeugt ein reaktives Objekt, das sich automatisch aktualisiert, wenn sich die abhängigen Werte ändern. In diesem Fall wird das Objekt `chartConfig` aktualisiert, wenn sich die Werte von `apiUrl` und `dataUrl` ändern. Das Objekt enthält die Achsenbeschriftungen und den Titel des Diagramms.

```

31 | async function readData() {
32 |   loading.value = true;
33 |   try {
34 |     const csvData : BarData[] = await d3.csv(dataUrl, d => ({
35 |       name: d.letter, // letter wird zu name im Diagramm
36 |       value: +d.frequency // frequency wird zu Zahl value
37 |     }));
38 |     data.value = csvData;
39 |   } catch (error) {
40 |     console.error('Problem with your fetch operation:', error);
41 |   } finally {
42 |     loading.value = false;
43 |   }
44 | }
45 |
46 | onMounted(readData);
47 | </script>

```

`d3.csv` ist eine asynchrone Funktion, die eine URL und eine Funktion als Parameter erhält. Die Funktion wird für jede Zeile der CSV-Datei aufgerufen. Die Funktion gibt ein Objekt zurück, das die Werte für die Balken enthält.

Daher muss die Funktion `readData` als `async` deklariert werden. Die Daten werden in der Variable `csvData` gespeichert und an die Komponente `Balken` übergeben. Im Fehlerfall wird eine Fehlermeldung ausgegeben.

Die Funktion `readData` benutzt zum Lesen der Daten eine Funktion aus D3. Diese kann erst aufgerufen werden, nachdem die Komponente vollständig geladen ist. Daher wird die Funktion

`readData` in der Funktion `onMounted` aufgerufen. Die Formatierung der Seite erfolgt über das `scoped` Stylesheet. Wir führen in dieser Veranstaltung in der Regel nur die unbedingt nötigen Stylesheets an. In der Praxis sollte das Stylesheet um weitere Styles ergänzt werden.

```

48 | <style scoped>
49 | .loading {
50 |   padding: 20px;
51 |   text-align: center;
52 |   font-style: italic;
53 | }
54 | .error {
55 |   padding: 20px;
56 |   color: red;
57 |   text-align: center;
58 | }
59 | </style>

```

Die Komponente `Balken` wird in der Datei `components/Balken.vue` definiert. Die Komponente erhält die Props `width`, `height`, `chartConfig` und `data`. Die Daten werden in der Methode `draw` in ein Balkendiagramm umgewandelt.

`components/Balken.vue`

```

1 | <template>
2 | <div ref="divElem"></div>
3 | </template>

```

Die `Balken`-Komponente enthält initial nur ein leeres `div`-Element.

```

4 | <style>
5 | .bar {
6 |   fill: steelblue;
7 | }
8 | .x-axis-label, .y-axis-label {
9 |   font-size: 12pt;
10 |  font-weight: bold;
11 | }
12 | </style>

```

Der Style für D3-Elemente wird in einem `style`-Tag definiert. Dieser darf nicht `scoped` sein, da die Styles für die D3-Elemente nicht nur in der Komponente, sondern auch in den eingefügten SVG-Elementen gelten sollen.

```

13 | <script lang="ts">
14 | type BarData = {
15 |   name: string;
16 |   value: number;
17 | };
18 |
19 | type ChartConfig = {
20 |   title: string;
21 |   xlabel: string;
22 |   ylabel: string;
23 | };
24 |
25 | export { type BarData, type ChartConfig };

```

Die Typen `BarData` und `ChartConfig` werden exportiert, damit sie in anderen Dateien verwendet werden können. Ein ES-Modul Export kann nicht in einem Setup-Block erfolgen, daher werden die Typen in einem separaten `script`-Element definiert.

```

26 | <script lang="ts">
27 | import { ref, onMounted, watch } from 'vue';

```

```

28 | import * as d3 from 'd3';
29 |
30 | const props = defineProps<{
31 |   width: number;
32 |   height: number;
33 |   config: ChartConfig;
34 |   data: BarData[];
35 | }>();
36 |
37 | const divElem = ref<HTMLDivElement | null>(null);

```

Die Attribute des Balken-Tags werden über `defineProps` definiert. Das `div`-Element wird über `ref` referenziert.

Methode: `drawSVG`

Die Methode `drawSVG` erstellt ein SVG-Element und berechnet die Größe des Diagramms. Die Größe des Diagramms wird durch die Werte `width` und `height` bestimmt. Die Daten für das Balkendiagramm sind in der `data`-Prop enthalten. Die Labels sind in der `config`-Prop enthalten.

```

38 | function drawSVG() : SVGSVGElement | null {
39 |   const svg = d3.create("svg");
40 |   if (svg === null) return null;
41 |
42 |   const w = +props.width;
43 |   const h = +props.height;
44 |   svg.attr("width", w)
45 |     .attr("height", h);
46 |
47 |   const barData = props.data;
48 |   if (barData.length === 0) return null;
49 |
50 |   // Add margins to make room for axes
51 |   const margin = { top: 20, right: 20, bottom: 50, left: 60 };
52 |   const innerWidth = w - margin.left - margin.right;
53 |   const innerHeight = h - margin.top - margin.bottom;
54 |   if (innerWidth <= 10 || innerHeight <= 10) return null;
55 |
56 |   const g = svg.append("g")
57 |     .attr("transform", `translate(${margin.left},${margin.top})`);
58 |
59 |   const x = d3.scaleBand()
60 |     .domain(barData.map(d => d.name))
61 |     .range([margin.left, w - margin.right])
62 |     .padding(0.1);
63 |
64 |   const y = d3.scaleLinear()
65 |     .domain([0, d3.max(barData, d => d.value) ?? 1])
66 |     .range([h - margin.bottom, margin.top]);
67 |
68 |   const g = svg.append("g")
69 |     .attr("transform", `translate(0,${margin.top})`);
70 |
71 |   g.selectAll("rect")
72 |     .data(barData)
73 |     .join("rect")
74 |     .attr("class", "bar")
75 |     .attr("x", d => x(d.name) ?? 0)
76 |     .attr("y", d => y(d.value))
77 |     .attr("width", x.bandwidth())
78 |     .attr("height", d => y(0) - y(d.value));

```

Nach dem Erstellen der Balken werden die Achsenbeschriftungen hinzugefügt. Hierzu werden die Achsen in jeweils eigene Gruppen eingebettet. Diese Gruppen werden dann entsprechend der Position der Achsen transformiert.

```

78 // Add x-axis (without label)
79 g.append("g")
80   .attr("class", "x-axis")
81   .attr("transform", `translate(0,${h - margin.bottom})`)
82   .call(d3.axisBottom(x));
83
84 // Add y-axis (without label)
85 g.append("g")
86   .attr("class", "y-axis")
87   .attr("transform", `translate(${margin.left},0)`)
88   .call(d3.axisLeft(y));

```

Zuletzt werden die Achsenbeschriftungen hinzugefügt. Die Beschriftungen werden ebenfalls in eigenen Gruppen eingebettet und entsprechend transformiert.

```

89 // Add x-axis label separately
90 g.append("text")
91   .attr("class", "x-axis-label")
92   .attr("x", innerWidth / 2)
93   .attr("y", innerHeight + 40)
94   .attr("text-anchor", "middle")
95   .text(props.config.xlabel);
96 // Add y-axis label separately
97 g.append("text")
98   .attr("class", "y-axis-label")
99   .attr("transform", "rotate(-90)")
100  .attr("x", -innerHeight / 2)
101  .attr("y", -40)
102  .attr("text-anchor", "middle")
103  .text(props.config.ylabel);

```

Der Titel des Diagramms wird in der Mitte des Diagramms hinzugefügt. Der Titel wird im SVG-Element als Text-Element eingefügt. Daher wird für die Zentrierung `w` und nicht `innerWidth` verwendet.

```

104 svg.append("text")
105   .attr("class", "chart-title")
106   .attr("x", w / 2)
107   .attr("y", margin.top / 2)
108   .attr("text-anchor", "middle")
109   .text(props.config.title);
110
111 return svg.node();
112 }

```

Die Methode `drawSVG` gibt das SVG-Element zurück. Wenn das Element nicht erstellt werden konnte, wird `null` zurückgegeben.

Es ist zu Beachten, dass `svg` ein Selection-Objekt ist. Um das SVG-Element zu erhalten, muss die Methode `node` aufgerufen werden.

Methode: `updateChart`

Die Methode `updateChart` wird aufgerufen, um das erzeugte SVG-Element in die Komponente einzuhängen. Sie entfernt zunächst alle Elemente aus dem `div`-Element. Anschließend wird das Balkendiagramm neu gezeichnet und in das `div`-Element eingefügt.

```

113 function updateChart() {
114   if (divElem.value === null) return;
115   d3.select(divElem.value).selectAll("*").remove();

```

```
116 |   const svg = drawSVG();
117 |   const p = document.createElement("p");
118 |   p.textContent = "No data available";
119 |   if (svg === null) {
120 |     divElem.value.appendChild(p);
121 |   } else {
122 |     divElem.value.appendChild(svg);
123 |   }
124 | }
125 | onMounted(updateChart);
```

Die auskommentierte `watch`-Anweisung sorgt dafür, dass das Balkendiagramm neu gezeichnet wird, wenn sich die Daten ändern. Da die Daten nur einmal beim Laden der Seite eingelesen werden, ist diese Anweisung hier nicht nötig.

```
126 | // Add a watcher for changes in the data prop
127 | // watch(() => props.data, () => {
128 | //   updateChart();
129 | // }, { deep: true });
130 | </script>
```

11. Umgang mit Daten in D3

11.1. D3-Daten und das DOM

D3 enthält keine eigenen Datenstrukturen, sondern arbeitet mit den Datenstrukturen des DOM. Allerdings bietet D3 eine Reihe von Hilfsfunktionen, um den Umgang mit den Daten zu erleichtern. Jedoch enthält D3 keine Schleifenkonstrukte, sondern arbeitet mit den Methoden `enter()`, `exit()` und `update()`. Diese Methoden sind auf Selektionen anwendbar und erzeugen neue Selektionen, die die Daten repräsentieren.

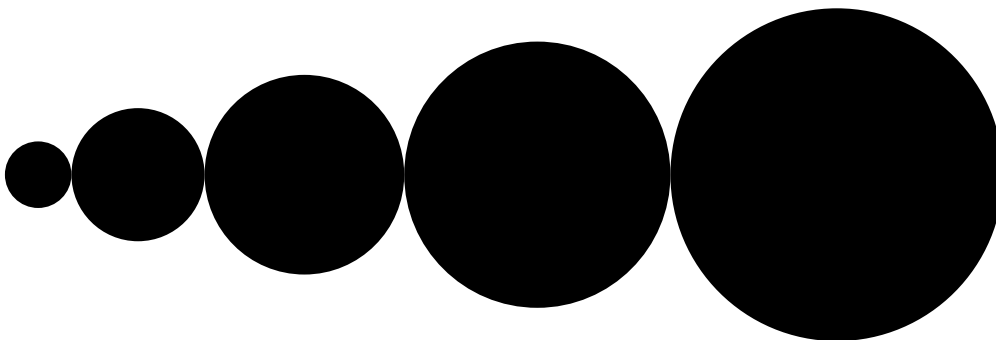
Beispiel 86: Kreise mit Daten verbinden

```

1  const svg =
2    d3.select("body")
3      .append("svg")
4      .attr("width", 320)
5      .attr("height", 120);
6  const data =
7    [10, 20, 30, 40, 50];
9  const circles = svg.selectAll("circle")
10     .data(data)
11     .enter()
12     .append("circle")
13     .attr("cx", (d, i) => (i+1)*d + 10)
14     .attr("cy", 60)
15     .attr("r", (d, i) => d);

```

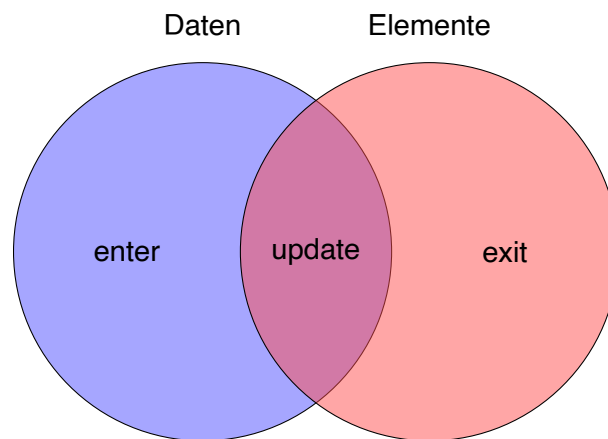
Dies erzeugt einen Kreis für jedes Element in `data` und setzt den Radius entsprechend des Werts im Array. Die `selectAll("circle")` Methode erzeugt eine leere Selektion. Die `data(data)`-Methode stellt die Daten den Selektoren zur Verfügung. Die `enter()` Methode erzeugt eine neue Selektion, die die Daten repräsentiert, die noch nicht mit einem Element verknüpft sind. Die `append("circle")` Methode erzeugt für jedes Element in der Selektion ein Kreiselement. Die `attr()` Methode nutzt die übergebene Funktion um die Attribute der Kreiselemente zu setzen..



Durch obigen Code erzeugte Kreise

Die Funktion `attr("cx", (d,i)=>(i+1)*d + 10)` setzt den Wert für das Attribut `cx` (x-Koordinate des Mittelpunkts) des Kreises. Der erste Parameter der Funktion ist der Datenwert, der zweite Parameter ist der Index des Datenwertes im Array. Der Index wird genutzt, um die Kreise nebeneinander anzuordnen. D3 wird also nicht mitgeteilt, wie etwas durchzuführen ist, sondern was erwünscht ist. Die Kreise werden mit Daten assoziiert. Pro Datenwert soll ein Kreis erzeugt werden.

`selectAll("circle")` teilt D3 mit, dass Kreise mit Daten assoziiert werden sollen. Das grundlegende Konzept von D3 ist ein "data join":

**enter**

Für rein statische Visualisierungen ist der `enter`-Teil des `data join` der einzige relevante Teil. Für dynamische Visualisierungen sind auch der `update`- und `exit`-Teil relevant. Der `update`-Teil wird genutzt, um die Visualisierung zu aktualisieren, wenn sich die Daten ändern. Der `exit`-Teil wird genutzt, um Elemente zu entfernen, wenn sie nicht mehr benötigt werden.

merge

Die `update`-Selektion enthält die Elemente, die bereits vorhanden sind und mit Daten assoziiert sind. Diese wird implizit durch die `selectAll`-Methode erstellt. Sollen die Elemente aktualisiert werden, so wird die `update`-Selektion genutzt. Mit der `merge`-Methode können die `enter`- und `update`-Selektionen zusammengeführt werden. Dort werden also neue Elemente erzeugt und bestehende Elemente aktualisiert.

- `enter`: Nur in den Daten, nicht im DOM
- `update`: In den Daten und im DOM
- `exit`: Nur im DOM, nicht in den Daten

11.2. Dynamische Visualisierungen**11.2.1. Dynamik mit `enter` und `exit`**

Dynamische Visualisierungen sind solche, die sich ändern, wenn sich die Daten ändern. D3 bietet die Methoden `enter`, `update` und `exit`, um dynamische Visualisierungen zu erstellen. Die Methode `enter` wird genutzt, um Elemente zu erzeugen, die in den Daten vorhanden sind, aber noch nicht im DOM. Die Methode `update` wird genutzt, um Elemente zu aktualisieren, die in den Daten und im DOM vorhanden sind. Die Methode `exit` wird genutzt, um Elemente zu entfernen, die im DOM vorhanden sind, aber nicht in den Daten.

11.2.2. Dynamik mit `join`

Seit Version 5 besitzt D3 eine neue Methode `join`, die die Methoden `enter`, `update` und `exit` zusammenfasst. Die Methode `join` wird auf einer Selektion aufgerufen und erhält als Parameter die Daten. Die Methode gibt eine Selektion zurück, die die Daten repräsentiert. Die Methode `join` erzeugt die `enter`-, `update`- und `exit`-Selektionen.

11.3. Ausführliches komponentenbasiertes Beispielprojekt

Mit dem Projekt Animated wird ein Beispiel für eine dynamische Visualisierung gezeigt. In diesem wird mit Hilfe von Komponenten eine Webseite erstellt, die eine Reihe von Umsetzungen für dynamische D3-Visualisierungen enthält. Wir werden die Webseite schrittweise erstellen. Zunächst wird das Projekt mit nuxi erstellt:

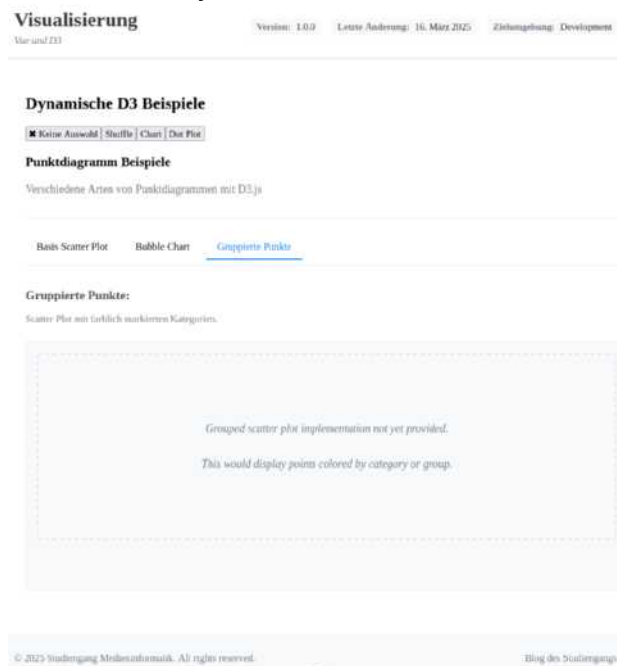
```

1 | bun x nuxi init Animated
2 | cd Animated
3 | bun install
4 | bun add d3
5 | bun add -D @types/d3
6 | rm app.vue
7 | mkdir pages
8 | mkdir components
9 | mkdir utils/chart

```

Die

initiale Webseite wird in der Datei `index.vue` erstellt. Sie besteht aus Komponenten für Header, Footer und Inhalt. Zuvor werden die Layouts für Header und Footer erstellt.



Die initiale Webseite für Animated

11.3.1. Erstellung der Layouts

layouts/default.vue

```

1 | <template>
2 |   <div class="app-layout">
3 |     <AppHeader />
4 |     <main class="main-content">
5 |       <slot />
6 |     </main>
7 |     <AppFooter />
8 |   </div>
9 | </template>
10 | <script setup>
11 | import AppHeader from '~/layouts/components/AppHeader.vue';
12 | import AppFooter from '~/layouts/components/AppFooter.vue';
13 | </script>
14 | <style scoped>
15 | .app-layout {

```

```

16   min-height: 100vh;
17   display: flex;
18   flex-direction: column;
19 }
20
21 .main-content {
22   flex: 1;
23   padding: 1rem;
24   max-width: 1200px;
25   width: 100%;
26   margin: 0 auto;
27 }
28 </style>

```

layouts/components/AppFooter.vue

```

1  <template>
2    <footer class="app-footer"
3      <div class="footer-content">
4        <p class="copyright">
5          &copy; {{ currentYear }} Studiengang Medieninformatik.
6        </p>
7        <div class="footer-links">
8          <a href="https://www.medieninformatik.de"
9            target="_blank">Blog des Studiengangs</a>
10        </div>
11      </div>
12    </footer>
13  </template>
14
15  <script setup lang="ts">
16    import { ref, onMounted } from 'vue';
17    // Reactive current year for copyright
18    const currentYear = ref(new Date().getFullYear());
19  </script>
20
21  <style scoped>
22    .app-footer {
23      background-color: #f8f9fa;
24      border-top: 1px solid #e9ecef;
25      padding: 1.5rem 0;
26      margin-top: 2rem;
27      width: 100%;
28    }
29
30    .footer-content {
31      max-width: 1200px;
32      margin: 0 auto;
33      padding: 0 1rem;
34      display: flex;
35      flex-direction: column;
36      align-items: center;
37      gap: 1rem;
38    }
39
40    .copyright {
41      margin: 0;
42      color: #6c757d;
43      font-size: 0.9rem;
44    }
45
46    .footer-links {
47      display: flex;
48      gap: 1.5rem;
49    }
50
51    .footer-links a {
52      color: #6c757d;
53      text-decoration: none;
54      font-size: 0.9rem;
55      transition: color 0.2s ease;
56    }
57
58    .footer-links a:hover {
59      color: #495057;
60    }
61
62    @media screen and (min-width: 768px) {
63      .footer-content {
64        flex-direction: row;
65        justify-content: space-between;
66      }
67    }

```

```

61 | }
62 | </style>

```

Der Footer enthält ein Copyright und einen Link zum Blog des Studiengangs Medieninformatik. Er wird in der Datei `layouts/default.vue` eingebunden.

layouts/components/AppHeader.vue

Der Header enthält den Titel der Webseite, eine Tagline, die Versionsnummer, das Datum der letzten Änderung und die Zielumgebung. Die Daten für die Versionsnummer und die Zielumgebung werden aus der Konfiguration der Anwendung gelesen. Das Datum der letzten Änderung wird dynamisch erzeugt.

```

1 | <template>
2 |   <header class="app-header">
3 |     <div class="header-content">
4 |       <div class="logo-container">
5 |         <h1 class="site-title">Visualisierung</h1>
6 |         <p class="site-tagline">Vue und D3</p>
7 |       </div>
8 |       <div class="meta-info">
9 |         <div class="meta-item">
10 |           <span class="meta-label">Version:</span>
11 |           <span class="meta-value">1.0.0</span>
12 |         </div>
13 |         <div class="meta-item">
14 |           <span class="meta-label">Letzte Änderung:</span>
15 |           <span class="meta-value">{{ formattedDate }}</span>
16 |         </div>
17 |         <div class="meta-item">
18 |           <span class="meta-label">Zielumgebung:</span>
19 |           <span class="meta-value">{{ environment }}</span>
20 |         </div>
21 |       </div>
22 |     </div>
23 |   </header>
24 | </template>
25 | <script setup lang="ts">
26 | import { ref, onMounted, computed } from 'vue';
27 | import { useRuntimeConfig } from '#app'; // Updated Nuxt 3 import
28 |
29 | // Access runtime config for environment info
30 | const config = useRuntimeConfig();
31 | console.log('Runtime config:', config); // Debug log runtime config
32 |
33 | // Safely get API URL with default value
34 | const apiUrl = computed(() : string => {
35 |   const url = config?.public?.apiUrl;
36 |   return typeof url === 'string' ? url : 'http://localhost:3000';
37 | });
38 | // Determine environment based on API URL
39 | const environment = computed() : string => {
40 |   const url = apiUrl.value;
41 |   if (url.includes('localhost')) return 'Development';
42 |   if (url.includes('staging')) return 'Staging';
43 |   return 'Production';
44 | };
45 |
46 | // Format current date
47 | const formattedDate = computed() : string => {
48 |   const options: Intl.DateTimeFormatOptions = {
49 |     year: 'numeric',

```

```

50     month: 'long',
51     day: 'numeric'
52   };
53   return new Date().toLocaleDateString(undefined, options);
54 });
55 </script>

56 <style scoped>
57 .app-header {
58   background-color: #ffffff;
59   box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
60   padding: 1rem 0;
61   transition: transform 0.3s ease;
62 }
63
64 .header-content {
65   max-width: 1200px;
66   margin: 0 auto;
67   padding: 0 1rem;
68   display: flex;
69   flex-direction: column;
70   gap: 1rem;
71 }

72 .logo-container {
73   display: flex;
74   flex-direction: column;
75   align-items: center;
76 }
77
78 .site-title {
79   margin: 0;
80   font-size: 1.8rem;
81   color: #343a40;
82   font-weight: 700;
83 }
84
85 .site-tagline {
86   margin: 0.25rem 0 0;
87   font-size: 0.9rem;
88   color: #6c757d;
89   font-style: italic;
90 }

91 .meta-info {
92   display: flex;
93   flex-wrap: wrap;
94   justify-content: center;
95   gap: 1rem;
96   font-size: 0.85rem;
97 }
98 .meta-item {
99   padding: 0.25rem 0.5rem;
100  background-color: #f8f9fa;
101  border-radius: 4px;
102  display: flex;
103  gap: 0.5rem;
104 }
105 .meta-label {
106   color: #6c757d;
107   font-weight: 600;
108 }

109 .meta-value {
110   color: #212529;
111 }
112
113 @media screen and (min-width: 768px) {
114   .header-content {
115     flex-direction: row;
116     justify-content: space-between;
117     align-items: center;
118   }
119
120   .logo-container {
121     align-items: flex-start;
122   }
123
124   .meta-info {
125     justify-content: flex-end;
126   }
127 }
128 </style>

```

11.3.2. Erstellung der Indexseite

pages/index.vue

Die Datei `pages/index.vue` enthält die initiale Webseite. Sie besteht aus einem Header, einem Footer und einem Abschnitt für den Inhalt. Die Inhalte können über Buttons als Komponenten geladen werden.

```

9 <template>
10   <div class="content-wrapper">
11     <h2 class="content-title">Dynamische D3 Beispiele</h2>

```

```

12
13 <div class="visualization-tabs">
14   <!-- Simple unselect button -->
15   <button
16     class="tab-button unselect-button"
17     :class="{ active: !activeTab }"
18     @click="unselectTab()">
19     #x2716; Keine Auswahl
20   </button>
21   <!-- Existing visualization tabs -->
22   <button
23     v-for="tab in visualizations"
24     :key="tab.id"
25     :class="['tab-button', { active: activeTab === tab.id }]"
26     @click="setActiveTab(tab.id)">
27     {{ tab.name }}
28   </button>
29 </div>
30
31 <div class="visualization-container">
32   <!-- Loading state -->
33   <div v-if="loading" class="loading-state">
34     <p>Lade Visualisierung...</p>
35   </div>
36   <!-- Shuffle -->
37   <div v-else-if="activeTab === 'shuffle'" class="visualization">
38     <ShuffleSelector />
39   </div>
40
41   <!-- Charts visualization -->
42   <div v-else-if="activeTab === 'chart'" class="visualization">
43     <ChartSelector />
44   </div>
45
46   <!-- Scatter plot visualization -->
47   <div v-else-if="activeTab === 'dot-plot'" class="visualization">
48     <ScatterSelector />
49   </div>
50   <!-- Default state when nothing is selected -->
51   <div v-else class="visualization empty-state">
52     <div class="placeholder">
53       <h3>Keine Visualisierung ausgewählt</h3>
54       <p>Bitte wählen Sie eine Visualisierung aus den Tabs oben.</p>
55     </div>
56   </div>
57 </div>
58 </div>
59 </template>
60
61 <script setup lang="ts">
62 import { ref, onMounted } from 'vue';
63 import ChartSelector from '~/components/ChartSelector.vue';
64 import ShuffleSelector from '~/components/ShuffleSelector.vue';
65 import ScatterSelector from '~/components/ScatterSelector.vue';
66
67 // Define layout - redundant for default layout
68 definePageMeta({
69   layout: 'default'
70 });
71
72 // Available visualizations
73 const visualizations = [

```

```

73   { id: 'shuffle', name: 'Shuffle' },
74   { id: 'chart', name: 'Chart' },
75   { id: 'dot-plot', name: 'Dot Plot' }
76 ];
77 // UI state - no tab selected by default
78 const activeTab = ref('');
79 const loading = ref(false);
80
81 // Simulate loading when changing tabs
82 function setActiveTab(tabId: string) {
83   loading.value = true;
84   activeTab.value = tabId;
85
86   // Simulate loading delay
87   setTimeout(() => {
88     loading.value = false;
89   }, 300);
90 }
91
92 // Unselect the current tab
93 function unselectTab() {
94   loading.value = true;
95   activeTab.value = '';
96   setTimeout(() => {
97     loading.value = false;
98   }, 300);
99 }
100
101 onMounted(() => {
102   // Initially no loading needed
103   loading.value = false;
104 });
105 </script>

```

```

106 <style scoped>
107 .content-wrapper {
108   max-width: 1200px;
109   margin: 0 auto;
110   padding: 1rem;
111 }
112 </style>

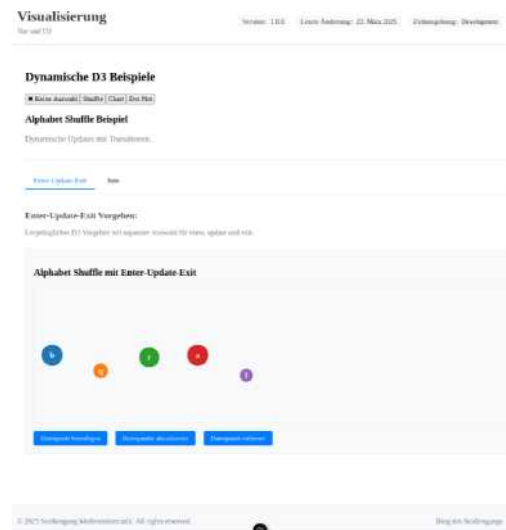
```

11.3.3. Die Shuffle-Komponenten

components/ShuffleSelector.vue

Die Shuffle-Komponente enthält eine Liste von Elementen, die durch Klicken auf einen Button zufällig angeordnet werden. Die Elemente werden durch Kreise dargestellt. Die Anordnung wird durch einen Buttons zufällig geändert. Ebenso existieren Buttons um Elemente hinzuzufügen und zu entfernen.

Das Shuffle-Beispiel enthält zwei Implementierungen: enter-update-exit benutzt das alte D3-Vorgehen um Änderungen in den Daten darzustellen. join nutzt die mit Version 5 in D3 eingeführte vorgehensweise. Die Implementierung wird durch Klicken auf die Buttons ausgewählt. Die Wahl wird durch die Variable activeImpl gesteuert. Die Implementierung wird durch die Komponenten ShuffleEnter und ShuffleJoin dargestellt



Die Shuffle-Komponente

```

1 <template>
2   <div class="shuffle-selector">
3     <h3>Alphabet Shuffle Beispiel</h3>
4     <div class="viz-description">
5       <p>Dynamische Updates.</p>
6     </div>
7     <div class="implementation-tabs">
8       <button :class="['impl-button', { active: activeImpl === 'enter-update-exit'
9         }]"
10        @click="activeImpl = 'enter-update-exit'">
11        Enter-Update-Exit
12      </button>
13      <button
14        :class="['impl-button', { active: activeImpl === 'join' }]"
15        @click="activeImpl = 'join'">
16        Join
17      </button>
18    </div>
19    <div class="implementation-container">
20      <template v-if="activeImpl === 'enter-update-exit'">
21        <div class="implementation-view">
22          <h4>Enter-Update-Exit Vorgehen:</h4>
23          <p class="impl-description">Ursprüngliches D3 Vorgehen mit separater
24            Auswahl für enter, update und exit.</p>
25          <div class="alphabet-visualization">
26            <ShuffleEnter />
27          </div>
28        </div>
29      </template>
30      <template v-else="activeImpl === 'join'">
31        <div class="implementation-view">
32          <h4>Join Vorgehen:</h4>
33          <p class="impl-description">Modernes D3 Vorgehen mit join.</p>
34          <div class="alphabet-visualization">
35            <ShuffleJoin />
36          </div>
37        </div>
38      </template>
39    </div>
40  </div>
41 </template>

```

```

40 <script setup lang="ts">
41 import { ref } from 'vue';
42 import ShuffleEnter from './ShuffleEnter.vue';
43 import ShuffleJoin from './ShuffleJoin.vue';
44 // Default to enter-update-exit
45 const activeImpl = ref('enter-update-exit');
46 </script>

47 <style scoped>
48 .shuffle-selector {
49   width: 100%;
50   max-width: 100%;
51   overflow-x: hidden;
52 }
53 .viz-description {
54   margin-bottom: 1.5rem;
55   padding-bottom: 1rem;
56   border-bottom: 1px solid #e9ecef;
57   color: #6c757d;
58 }

59 .implementation-tabs {
60   display: flex;
61   gap: 0.5rem;
62   margin: 1rem 0;
63   border-bottom: 1px solid #dee2e6;
64   padding-bottom: 0.5rem;
65 }
66
67 .impl-button {
68   background-color: transparent;
69   border: none;
70   padding: 0.5rem 1rem;
71   cursor: pointer;
72   font-size: 0.9rem;
73   border-radius: 4px 4px 0 0;
74   transition: all 0.2s ease;
75   border-bottom: 2px solid transparent
76   ;
77
78   .impl-button:hover {
79     background-color: #f8f9fa;
80   }
81
82   .impl-button.active {
83     color: #007bff;
84     border-bottom-color: #007bff;
85     font-weight: 500;
86   }
87
88   .implementation-container {
89     padding-top: 1rem;
90     width: 100%;
91     overflow-x: hidden;
92
93     .placeholder {
94       display: flex;
95       justify-content: center;
96       align-items: center;
97       background-color: #f8f9fa;
98       border: 2px dashed #dee2e6;
99       border-radius: 4px;
100      padding: 2rem;
101      text-align: center;
102      color: #6c757d;
103      font-style: italic;
104      height: 300px;
105      max-width: 100%;
106      overflow: hidden;
107      box-sizing: border-box;
108
109      .alphabet-visualization {
110        width: 100%;
111        min-height: 200px;
112      }
113    }
114  }
115
116   .impl-description {
117     color: #6c757d;
118     font-size: 0.9rem;
119     margin-bottom: 1.5rem;
120   }
121
122   h4 {
123     margin-top: 0;
124     margin-bottom: 0.5rem;
125     font-size: 1.1rem;
126     color: #495057;
127   }

```

```

123   border: 1px solid #e9ecef;
124   border-radius: 4px;
125   padding: 1rem;
126   background-color: #f8f9fa;
127   max-width: 100%;
128   overflow: hidden;
129   box-sizing: border-box;
130 }
131
132 /* Add style to ensure both views take full width but only one is visible */
133 .implementation-view {
134   width: 100%;
135 }
136 </style>

```

utils/shuffle/ShuffleBase.ts

Die Klasse `ShuffleBase` enthält die Basisfunktionalität für das Shuffle-Beispiel. Sie enthält die Datenpunkte, die verfügbaren Buchstaben, die genutzten Buchstaben, die nächste ID, das SVG-Element, den Chartinhalt, den Container, die Breite, die Höhe und den Rand. Die Klasse enthält Getter und Setter für die Attribute. Jeder Datenpunkt besteht aus einer eindeutigen ID, einem Buchstaben und einem Radius.

```

1  import * as d3 from 'd3';
2
3  export type DataPoint = {
4    id: number;
5    letter: string;
6    radius: number
7  };
8
9  type SVGElement = d3.Selection<SVGSVGElement, unknown, null, undefined> | null;
10 type SVGGroup = d3.Selection<SVGGEElement, unknown, null, undefined> | null;
11
12 export abstract class ShuffleBase {
13   #data: DataPoint[] = [];
14   #availableLetters: string[] = [];
15   #usedLetters: string[] = [];
16   #nextId: number = 0;
17   #svg: SVGElement = null;
18   #chartContent: SVGGroup = null;
19   #container: HTMLElement | null;
20   #width: number = 600;
21   #height: number = 300;
22   #margin = { top: 20, right: 20, bottom: 30, left: 40 };
23
24   constructor(container: HTMLElement | null) {
25     this.#container = container;
26   }

```

```

26 // Getters and setters
27 get data(): DataPoint[] {
28     return this.#data;
29 }
30 set data(data: DataPoint[]) {
31     this.#data = data;
32 }
33
34 get availableLetters(): string[] {
35     return this.#availableLetters;
36 }
37
38 set availableLetters(
39     letters: string[]) {
40     this.#availableLetters = letters;
41 }

58 get svg(): SVGElement {
59     return this.#svg;
60 }
61 set svg(svg: SVGElement) {
62     this.#svg = svg;
63 }
64
65 get chartContent(): SVGGroup {
66     return this.#chartContent;
67 }
68 set chartContent(content: SVGGroup) {
69     this.#chartContent = content;
70 }
71
72 get container(): HTMLElement | null {
73     return this.#container;
74 }

75 set container(container: HTMLElement
76     | null) {
77     this.#container = container;
78 }
79
80 get width(): number {
81     return this.#width;
82 }
83
84 set width(width: number) {
85     this.#width = width;
86 }
87
88 get height(): number {
89     return this.#height;
90 }
91
92 set height(height: number) {
93     this.#height = height;
94 }

94 get margin(): { top: number; right: number; bottom: number; left: number } {
95     return this.#margin;
96 }
97 set margin(margin: { top: number; right: number; bottom: number; left: number })
98     {
99     this.#margin = margin;
100 }
101
102 get innerWidth(): number {
103     return this.#width - this.#margin.left - this.#margin.right;
104 }
105 get innerHeight(): number {
106     return this.#height - this.#margin.top - this.#margin.bottom;
107 }

42 get usedLetters(): string[] {
43     return this.#usedLetters;
44 }
45
46 set usedLetters(
47     letters: string[]) {
48     this.#usedLetters = letters;
49 }
50
51 get nextId(): number {
52     return this.#nextId;
53 }
54
55 set nextId(id: number) {
56     this.#nextId = id;
57 }

75 set container(container: HTMLElement
76     | null) {
77     this.#container = container;
78 }
79
80 get width(): number {
81     return this.#width;
82 }
83
84 set width(width: number) {
85     this.#width = width;
86 }
87
88 get height(): number {
89     return this.#height;
90 }
91
92 set height(height: number) {
93     this.#height = height;
94 }

94 get margin(): { top: number; right: number; bottom: number; left: number } {
95     return this.#margin;
96 }
97 set margin(margin: { top: number; right: number; bottom: number; left: number })
98     {
99     this.#margin = margin;
100 }
101
102 get innerWidth(): number {
103     return this.#width - this.#margin.left - this.#margin.right;
104 }
105 get innerHeight(): number {
106     return this.#height - this.#margin.top - this.#margin.bottom;
107 }

```

Die Klasse enthält ebenso Methoden zum Hinzufügen und Entfernen von Datenpunkten, zum Inkrementieren der nächsten ID, zum Initialisieren, Hinzufügen und Entfernen von Buchstaben. Die Methode zum Update des Charts ist jedoch abstrakt. Die Klasse enthält keine Methoden zur Darstellung der Datenpunkte. Diese werden in den Subklassen implementiert.

```

107 // Initialize random data points
108 initializeData(): void {
109     this.availableLetters = 'abcdefghijklmnopqrstuvwxyz'.split('');

```

```

110     this.usedLetters = [];
111
112     // Shuffle available letters and take 5 to start
113     const shuffled = [...this.availableLetters].sort(
114         () => Math.random() - 0.5);
115     const initialLetters = shuffled.slice(0, 5);
116     // Generate initial data points
117     this.data = initialLetters.map(letter => {
118         const id = this.nextId++;
119         this.usedLetters.push(letter);
120         return {
121             id,
122             letter,
123             radius: Math.floor(Math.random() * 20) + 10 // radius in [10,30)
124         } as DataPoint;
125     });
126
127     initialLetters.forEach(letter => { // Remove used letters
128         const index = this.availableLetters.indexOf(letter);
129         if (index !== -1) { // remove selected from available letters
130             this.availableLetters.splice(index, 1);
131         }
132     });
133 }
134 // create SVG-element and a group to hold the margin.
135 createChart(): void {
136     if (!this.container) return;
137
138     this.svg = d3.select(this.container)
139         .append('svg')
140         .attr('width', this.width)
141         .attr('height', this.height);
142     this.chartContent = this.svg.append('g')
143         .attr('transform', `translate(${this.margin.left}, ${this.margin.top})`);
144 }
145 // create a new data point and add it to the data array
146 // then update the chart.
147 addDataPoint(): void {
148     if (this.availableLetters.length === 0) return;
149
150     // Select a random unused letter
151     const randomIndex = Math.floor(
152         Math.random() * this.availableLetters.length);
153     const letter = this.availableLetters[randomIndex];
154     this.availableLetters.splice(randomIndex, 1);
155     this.usedLetters.push(letter);
156
157     const newPoint = {
158         id: this.nextId++,
159         letter,
160         radius: Math.floor(Math.random() * 20) + 10
161     };
162     this.data.push(newPoint);
163     this.updateChart();
164 }
165 // Update data points (change radii but keep letters)
166 updateDataPoints(): void {
167     // Update the radius of each data point
168     this.data = this.data.map(d => ({
169         ...d, // keep existing properties
170         radius: Math.floor(Math.random() * 20) + 10
171     }));

```

```

172
173     // Update the chart
174     this.updateChart();
175   }
176   removeDataPoint(): void { // Remove the leftmost point
177     if (this.data.length === 0) return;
178
179     const leftmost = this.data.shift()!; //! = never null
180
181     // Add the letter back to available letters
182     this.availableLetters.push(leftmost.letter);
183
184     // Remove from used letters
185     const letterIndex = this.usedLetters.indexOf(leftmost.letter);
186     if (letterIndex !== -1) {
187       this.usedLetters.splice(letterIndex, 1);
188     }
189     // Update the chart
190     this.updateChart();
191   }
192
193   abstract updateChart(): void; // implement in subclass.
194 }

```

components/ShuffleEnter.vue

Die Komponente `ShuffleEnter` implementiert das Shuffle-Beispiel mit dem alten D3-Vorgehen. Hier wird die Methode `enter` für die noch nicht vorhandenen Daten aufgerufen, die Methode `exit` entfernt mit nicht mehr vorhandenen Daten assoziierte Elemente. Die Datenpunkte werden in den Daten und im DOM aktualisiert. Die in der Komponente implementierte Klasse `ShuffleEnterImplem` sorgt für die Darstellung der Daten als Visualisierung. Die Klasse erbt von der Basisklasse `ShuffleBase`, welche die Datenpunkte enthält und aktualisiert.

Zuerst werden in der Template die drei Buttons zum Hinzufügen, Aktualisieren und Entfernen von Datenpunkten dargestellt. Danach wird das SVG-Element für die Visualisierung erstellt. Die Visualisierung wird in der Methode `updateChart` gezeichnet.

```

1 <template>
2   <div class="animated-shuffle-enter">
3     <h3>Alphabet Shuffle mit Enter-Update-Exit</h3>
4     <div ref="chartContainer" class="chart-container"></div>
5     <div class="controls">
6       <button @click="addDataPoint" class="control-button" :disabled="
7         usedLettersCount >= 26">
8         Datenpunkt hinzufügen
9       </button>
10      <button @click="updateData" class="control-button" :disabled="
11        dataPointsCount === 0">
12        Datenpunkte aktualisieren
13      </button>
14      <button @click="removeDataPoint" class="control-button" :disabled="
15        dataPointsCount === 0">
16        Datenpunkt entfernen
17      </button>
18    </div>
19  </div>
20 </template>

```

Implementierung der Klasse `ShuffleEnterImplem`

Die Klasse `ShuffleEnterImplem` implementiert die Visualisierung der Datenpunkte. Die Basisklasse übernimmt die Verwaltung der Daten. Die Implementierung ist für die Visualisierung verantwort-

lich. Die Variablen `dataPointsCount` und `usedLettersCount` geben die Anzahl der Datenpunkte und der genutzten Buchstaben an. Sie erlauben die Aktivierung und Deaktivierung der Buttons zum Hinzufügen, Aktualisieren und Entfernen von Datenpunkten zu steuern.

```

18 <script setup lang="ts">
19 import { ref, onMounted } from 'vue';
20 import * as d3 from 'd3';
21 import { ShuffleBase, type DataPoint } from '../utils/shuffle/ShuffleBase';
22
23 // Reference to chart container
24 const chartContainer = ref<HTMLElement | null>(null);
25
26 // Reactive variables for button disabling
27 const usedLettersCount = ref(0);
28 const dataPointsCount = ref(0);
29
30 // Create ShuffleEnter implementation using the base class
31 class ShuffleEnterImpl extends ShuffleBase {
32   constructor(container: HTMLElement | null) {
33     // Call the parent constructor to properly initialize
34     super(container);
35   }
36   override updateChart(): void {
37     if (!this.chartContent) return;
38
39     const getXPosition = (_, i: number) => i * (this.innerWidth / (this.data.
40       length || 1));
41     const getYPosition = (d: DataPoint) => this.innerHeight - d.radius * 5;
42     const getRadius = (d: DataPoint) => d.radius;
43     // Update the reactive counters
44     usedLettersCount.value = this.usedLetters.length;
45     dataPointsCount.value = this.data.length;
46     // Create a color scale based on letters
47     const colorScale = d3.scaleOrdinal<string>()
48       .domain(this.usedLetters)
49       .range(d3.schemeCategory10);
50
51     // Update: Select all existing circles and bind data
52     const circles = this.chartContent.selectAll<SVGCircleElement, DataPoint>('
    circle.point')
    .data(this.data, d => d.id);

```

Der Aufruf von `data(this.dat, d => d.id)` bindet die Daten an die vorhandenen Kreis-Elemente. Das zweite Argument `d => d.id` sorgt dafür, dass nicht der Array-Index, sondern die eindeutige id der `DataPoints` als Schlüssel für die Elemente benutzt wird.

Die Methode `enter` fügt neue Elemente hinzu. Es werden also mit `append` neue Elemente für die noch nicht gebundenen Daten erzeugt.

```

53 // ENTER: Add new circles
54 const enterCircles = circles.enter()
55   .append('circle')
56   .attr('class', 'point')
57   .attr('cx', getXPosition)
58   .attr('cy', getYPosition)
59   .attr('r', getRadius)
60   .attr('fill', d => colorScale(d.letter))
61   .attr('stroke', 'white')
62   .attr('stroke-width', 1);
63 // Add text labels to the new circles
64 enterCircles.each(function(d) { // no arrow function, since we need 'this'
65   const circle = d3.select(this); // "this" is the circle element

```

```

66     d3.select(circle.node()?.parentNode as Element)
67       .append('text')
68       .attr('class', `text-${d.id}`)
69       .attr('x', parseFloat(circle.attr('cx')))
70       .attr('y', parseFloat(circle.attr('cy')))
71       .attr('text-anchor', 'middle')
72       .attr('dy', '0.35em')
73       .attr('fill', 'white')
74       .style('font-weight', 'bold')
75       .text(d.letter);
76   });
77   // UPDATE: Reposition all existing circles based on their index
78   circles
79     .attr('cx', getXPosition)
80     .attr('cy', getYPosition);
81   // Update text positions
82   this.data.forEach((d, i) => {
83     const textElement = this.chartContent!.select(`.text-${d.id}`);
84     if (textElement.empty()) {
85       this.chartContent!.append('text')
86         .attr('class', `text-${d.id}`)
87         .attr('x', i * (this.innerWidth / (this.data.length || 1)))
88         .attr('y', this.innerHeight - d.radius * 5)
89         .attr('text-anchor', 'middle')
90         .attr('dy', '0.35em')
91         .attr('fill', 'white')
92         .style('font-weight', 'bold')
93         .text(d.letter);
94     } else {
95       textElement
96         .attr('x', i * (this.innerWidth / (this.data.length || 1)))
97         .attr('y', this.innerHeight - d.radius * 5);
98     }
99   });
100   // EXIT: Remove any circles that are no longer in the data
101   circles.exit().remove();
102
103   // Remove associated text elements - reference this.data directly
104   this.chartContent.selectAll('text')
105     .each(function() {
106       const el = d3.select(this);
107       const className = el.attr('class');
108       if (className) {
109         const id = className.split('-')[1];
110         if (id && !shuffleEnter?.data.some(d => d.id.toString() === id)) {
111           el.remove();
112         }
113       }
114     });
115   }
116 }
117 // Create the chart instance as a shallow ref so the template can access its
   properties
118 let shuffleEnter : ShuffleEnterImpl | null = null;
119
120 // Methods exposed to template
121 const addDataPoint = () => {
122   shuffleEnter!.addDataPoint();
123   // Update reactive state after action
124   usedLettersCount.value = shuffleEnter!.usedLetters.length;
125   dataPointsCount.value = shuffleEnter!.data.length;
126 };

```

```

127
128 const updateData = () => {
129   shuffleEnter!.updateDataPoints();
130 };
131 const removeDataPoint = () => {
132   shuffleEnter!.removeDataPoint();
133
134   usedLettersCount.value = shuffleEnter!.usedLetters.length;
135   dataPointsCount.value = shuffleEnter!.data.length;
136 };
137
138 onMounted(() => {
139   if (chartContainer.value) {
140     shuffleEnter = new ShuffleEnterImpl(chartContainer.value);
141     if (shuffleEnter !== null) {
142       shuffleEnter.initializeData();
143       shuffleEnter.createChart();
144       shuffleEnter.updateChart();
145       usedLettersCount.value = shuffleEnter.usedLetters.length;
146       dataPointsCount.value = shuffleEnter.data.length;
147     }
148   }
149 });
150 </script>

```

```

151 <style scoped>
152 .animated-shuffle-enter {
153   width: 100%;
154   max-width: 100%;
155   overflow-x: hidden;
156 }
157 .chart-container {
158   width: 100%;
159   height: 300px;
160   background-color: #f8f9fa;
161   border: 1px solid #e9ecef;
162   border-radius: 4px;
163   margin: 1rem 0;
164 }
165 .controls {
166   display: flex;
167   gap: 1rem;
168   margin-top: 1rem;
169   flex-wrap: wrap;
170 }

```

```

171 .control-button {
172   background-color: #007bff;
173   color: white;
174   border: none;
175   padding: 8px 16px;
176   border-radius: 4px;
177   cursor: pointer;
178   font-size: 14px;
179   transition: background-color 0.3s;
180 }
181 .control-button:hover:not(:disabled){
182   background-color: #0056b3;
183 }
184 .control-button:disabled {
185   background-color: #6c757d;
186   cursor: not-allowed;
187   opacity: 0.6;
188 }
189 </style>

```

components/ShuffleJoin.vue

Die Komponente `ShuffleJoin` implementiert das Shuffle-Beispiel mit dem neuen D3-Vorgehen. Hier wird die Methode `join` für die betroffenen Elemente aufgerufen. `join` hat drei Funktionen als Parameter: Die erste wird auf alle neuen Elemente angewendet (`enter`), die zweite kümmert sich um die Aktualisierungen (`update`), die dritte entfernt nicht mehr benötigte Elemente (`exit`). Die Datenpunkte werden in den Daten und im DOM aktualisiert. Die in der Komponente implementierte Klasse `ShuffleJoinImpl` sorgt für die Darstellung der Daten als Visualisierung. Die Klasse erbt von der Basisklasse `ShuffleBase`, welche die Datenpunkte enthält und aktualisiert. [2em]

Zuerst werden in der Template die drei Buttons zum Hinzufügen, Aktualisieren und Entfernen von Datenpunkten dargestellt. Danach wird das SVG-Element für die Visualisierung erstellt. Die Visualisierung wird in der Methode `updateChart` gezeichnet.

```
1 <template>
```

```

2   <div class="animated-shuffle-join">
3     <h3>Alphabet Shuffle mit Join</h3>
4     <div ref="chartContainer" class="chart-container"></div>
5     <div class="controls">
6       <button @click="addDataPoint" class="control-button" :disabled="
          usedLettersCount >= 26">
7         Datenpunkt hinzufügen
8       </button>
9       <button @click="updateData" class="control-button" :disabled="
          dataPointsCount === 0">
10        Datenpunkte aktualisieren
11      </button>
12      <button @click="removeDataPoint" class="control-button" :disabled="
          dataPointsCount === 0">
13        Datenpunkt entfernen
14      </button>
15    </div>
16  </div>
17 </template>

```

Implementierung der Klasse ShuffleJoinImplem

Die Klasse `ShuffleJoinImplem` implementiert die Visualisierung der Datenpunkte. Die Basisklasse übernimmt die Verwaltung der Daten. Die Implementierung ist für die Visualisierung verantwortlich. Die Variablen `dataPointsCount` und `usedLettersCount` geben die Anzahl der Datenpunkte und der genutzten Buchstaben an. Sie erlauben die Aktivierung und Deaktivierung der Buttons zum Hinzufügen, Aktualisieren und Entfernen von Datenpunkten zu steuern.

```

18 <script setup lang="ts">
19 import { ref, onMounted } from 'vue';
20 import * as d3 from 'd3';
21 import { ShuffleBase, type DataPoint } from '../utils/shuffle/ShuffleBase';
22
23 // Reference to chart container
24 const chartContainer = ref<HTMLElement | null>(null);
25
26 // Reactive variables for button disabling
27 const usedLettersCount = ref(0);
28 const dataPointsCount = ref(0);
29
30 // Create ShuffleJoin implementation using the base class
31 class ShuffleJoinImpl extends ShuffleBase {
32   constructor(container: HTMLElement | null) {
33     // Call the parent constructor to properly initialize
34     super(container);
35   }
36   override updateChart(): void {
37     if (!this.chartContent) return;
38
39     // Update the reactive counters
40     usedLettersCount.value = this.usedLetters.length;
41     dataPointsCount.value = this.data.length;
42
43     // Create a color scale based on letters
44     const colorScale = d3.scaleOrdinal<string>()
45       .domain(this.usedLetters)
46       .range(d3.schemeCategory10);
47
48     // Calculate positions - SAME as ShuffleEnter
49     const getXPosition = (_, i: number) => i * (this.innerWidth / (this.data.length || 1));

```

```

50  const getYPosition = (d: DataPoint) => this.innerHeight - d.radius * 5;
51  const getRadius = (d: DataPoint) => d.radius;
52  // Use JOIN pattern - key difference from ShuffleEnter
53  this.chartContent.selectAll<SVGCircleElement, DataPoint>('circle.point')
54    .data(this.data, d => d.id)
55    .join(
56      enter => enter.append('circle')
57        .attr('class', 'point')
58        .attr('cx', getXPosition)
59        .attr('cy', getYPosition)
60        .attr('r', getRadius)
61        .attr('fill', d => colorScale(d.letter))
62        .attr('stroke', 'white')
63        .attr('stroke-width', 1),
64      update => update
65        .attr('cx', getXPosition)
66        .attr('cy', getYPosition)
67        .attr('r', getRadius),
68      exit => exit.remove()
69    );
70  // Use JOIN pattern for text labels
71  this.chartContent
72    .selectAll<SVGTextElement, DataPoint>('text.label')
73    .data(this.data, d => d.id)
74    .join(
75      enter => enter.append('text')
76        .attr('class', 'label')
77        .attr('x', getXPosition)
78        .attr('y', getYPosition)
79        .attr('text-anchor', 'middle')
80        .attr('dy', '0.35em')
81        .attr('fill', 'white')
82        .style('font-weight', 'bold')
83        .text(d => d.letter),
84      update => update
85        .attr('x', getXPosition)
86        .attr('y', getYPosition)
87      exit => exit.remove()
88    );
89  }
90 }
91 // Create the chart instance
92 let shuffleJoin : ShuffleJoinImpl | null = null;
93
94 // Methods exposed to template - keep same as ShuffleEnter
95 const addDataPoint = () => {
96   if (shuffleJoin == null) return;
97
98   shuffleJoin!.addDataPoint();
99   usedLettersCount.value = shuffleJoin.usedLetters.length;
100  dataPointsCount.value = shuffleJoin.data.length;
101 };
102
103 const updateData = () => {
104   if (shuffleJoin === null) return;
105
106   shuffleJoin!.updateDataPoints();
107   usedLettersCount.value = shuffleJoin.usedLetters.length;
108   dataPointsCount.value = shuffleJoin.data.length;
109 };
110 const removeDataPoint = () => {
111   if (shuffleJoin === null) return;

```

```

112 shuffleJoin.removeDataPoint();
113 usedLettersCount.value = shuffleJoin.usedLetters.length;
114 dataPointsCount.value = shuffleJoin.data.length;
115 };
116
117 onMounted(() => { // Lifecycle hooks
118   if (chartContainer.value) {
119     // Simple initialization, just like ShuffleEnter
120     shuffleJoin = new ShuffleJoinImpl(chartContainer.value);
121     if (shuffleJoin == null) return;
122     shuffleJoin.initializeData();
123     shuffleJoin.createChart();
124     shuffleJoin.updateChart();
125     usedLettersCount.value = shuffleJoin.usedLetters.length;
126     dataPointsCount.value = shuffleJoin.data.length;
127   }
128 });
129 </script>
130 <style scoped>
131   .animated-shuffle-join {
132     width: 100%;
133     max-width: 100%;
134     overflow-x: hidden;
135   }
136   .chart-container {
137     width: 100%;
138     height: 300px;
139     background-color: #f8f9fa;
140     border: 1px solid #e9ecef;
141     border-radius: 4px;
142     margin: 1rem 0;
143   }
144   .controls {
145     display: flex;
146     gap: 1rem;
147     margin-top: 1rem;
148     flex-wrap: wrap;
149   }
150   .control-button {
151     background-color: #28a745;
152     color: white;
153     border: none;
154     padding: 8px 16px;
155     border-radius: 4px;
156     cursor: pointer;
157     font-size: 14px;
158     transition: background-color 0.3s;
159   }
160   .control-button:hover:not(:disabled){
161     background-color: #218838;
162   }
163   .control-button:disabled {
164     background-color: #6c757d;
165     cursor: not-allowed;
166     opacity: 0.6;
167   }
168 </style>

```

11.4. Animation mit Transitions

11.4.1. Kontinuierliche Übergänge

Bisher haben wir die Positionen von Elementen im SVG-Dokument verändert. Diese Änderungen wurden unmittelbar ausgeführt. D3 bietet die Möglichkeit, diese Änderungen über einen Zeitraum zu animieren. Dies geschieht mit der Methode `transition()`. Werden Elemente nach dem Aufruf von `transition()` verändert, geschieht dies nicht sofort, sondern über einen Zeitraum. Der Standardwert beträgt 250ms. Die Animation kann mit der Methode `duration()` angepasst werden. Ebenso kann mit der Methode `delay()` eine Verzögerung angegeben werden. Diese wird in Millisekunden angegeben. Die Animation kann mit der Methode `ease()` angepasst werden.

Die Änderungen selbst werden durch die D3-Methode `timer` in regelmäßigen Abständen angestoßen.

11.4.2. Chart-Auswahl

Die unterschiedlichen animierten Diagramme in der Beispielanwendung werden mit der `ChartSelector`

-Komponente ausgewählt. Dort werden ein Balkendiagramm, ein Liniendiagramm, ein Tortendiagramm und ein Donut-Diagramm in einer Auswahl angezeigt. Diese können jeweils über Buttons ausgewählt werden. Die Auswahl wird in der `activeChartType`-Variable gespeichert. Diese wird dann in den Slots der Diagrammkomponenten verwendet. Die Diagramme sind in separaten Komponenten implementiert. Diese werden dann in der `chartSelector`-Komponente eingebunden.

```

1 <template>
2   <div class="chart-selector">
3     <h3>Diagramm-Beispiele</h3>
4     <div class="viz-description">
5       <p>Verschiedene Diagrammtypen mit D3.js</p>
6     </div>
7
8     <!-- Chart type selection tabs -->
9     <div class="implementation-tabs">
10      <button
11        :class="['impl-button', { active: activeChartType === 'bar-chart' }]"
12        @click="activeChartType = 'bar-chart'">
13        Balkendiagramm
14      </button>
15      <button
16        :class="['impl-button', { active: activeChartType === 'line-chart' }]"
17        @click="activeChartType = 'line-chart'">
18        Liniendiagramm
19      </button>
20      <button
21        :class="['impl-button', { active: activeChartType === 'pie-chart' }]"
22        @click="activeChartType = 'pie-chart'">
23        Kreisdiagramm
24      </button>
25      <button
26        :class="['impl-button', { active: activeChartType === 'circle-segment' }]"
27        @click="activeChartType = 'circle-segment'">
28        Ringdiagramm
29      </button>
30    </div>
31    <!-- Chart type content -->
32    <div class="implementation-container">
33      <div v-if="activeChartType === 'bar-chart'">
34        <h4>Balkendiagramm:</h4>
35        <p class="impl-description">Zeigt kategorische Daten durch die Höhe der
36          Balken an.</p>
37        <slot name="bar-chart">
38          <AnimatedBarChart />
39        </slot>
40      </div>
41      <div v-else-if="activeChartType === 'line-chart'">
42        <h4>Liniendiagramm:</h4>
43        <p class="impl-description">Zeigt Trends und Verläufe über die Zeit.</p>
44        <slot name="line-chart">
45          <AnimatedLineChart />
46        </slot>
47      </div>
48      <div v-else-if="activeChartType === 'pie-chart'">
49        <h4>Kreisdiagramm:</h4>
50        <p class="impl-description">Zeigt proportionale Anteile einer Gesamtmenge.
51          </p>
52        <slot name="pie-chart">
53          <AnimatedPieChart />
54        </slot>
55      </div>

```

```

55 |
56 |     <div v-else-if="activeChartType === 'circle-segment'">
57 |       <h4>Ringdiagramm:</h4>
58 |       <p class="impl-description">Zeigt Daten als Segmente eines Kreises.</p>
59 |       <slot name="circle-segment-chart">
60 |         <AnimatedDonutChart />
61 |       </slot>
62 |     </div>
63 |   </div>
64 | </div>
65 | </template>

```

Im Script-Teil der ChartSelector-Komponente wird die activeChartType-Variable definiert. Diese wird dann in den Slots der Diagrammkomponenten verwendet. Die Diagramme sind in separaten Komponenten implementiert und müssen hier importiert werden.

```

66 | <script setup lang="ts">
67 | import { ref } from 'vue';
68 | import AnimatedBarChart from './AnimatedBarChart.vue';
69 | import AnimatedLineChart from './AnimatedLineChart.vue';
70 | import AnimatedDonutChart from './AnimatedDonutChart.vue';
71 | import AnimatedPieChart from './AnimatedPieChart.vue';
72 |
73 | // Default to bar chart
74 | const activeChartType = ref('bar-chart');
75 | </script>

```

Im nachfolgenden <style>-Block werden die Styles für die ChartSelector-Komponente definiert. Die Styles sind in einem scoped-Tag eingebettet, damit sie nur für diese Komponente gelten.

<pre> 76 <style scoped> 77 .chart-selector { 78 width: 100%; 79 overflow-x: hidden; 80 } 81 82 .viz-description { 83 margin-bottom: 1.5rem; 84 padding-bottom: 1rem; 85 border-bottom: 1px solid #e9ecef; 86 color: #6c757d; 87 } 88 89 .implementation-tabs { 90 display: flex; 91 gap: 0.5rem; 92 margin: 1rem 0; 93 border-bottom: 1px solid #dee2e6; 94 padding-bottom: 0.5rem; 95 } </pre>	<pre> 96 .impl-button { 97 background-color: transparent; 98 border: none; 99 padding: 0.5rem 1rem; 100 cursor: pointer; 101 font-size: 0.9rem; 102 border-radius: 4px 4px 0 0; 103 transition: all 0.2s ease; 104 border-bottom: 2px solid 105 transparent; 106 } 107 .impl-button:hover { 108 background-color: #f8f9fa; 109 } </pre>
---	---

```

110 .impl-button.active {
111   color: #007bff;
112   border-bottom-color: #007bff;
113   font-weight: 500;
114 }
115
116 .implementation-container {
117   padding-top: 1rem;
118   width: 100%;
119   overflow-x: hidden;
120 }
121
122 .impl-description {
123   color: #6c757d;
124   font-size: 0.9rem;
125   margin-bottom: 1.5rem;
126 }
127
128 h4 {
129   margin-top: 0;
130   margin-bottom: 0.5rem;
131   font-size: 1.1rem;
132   color: #495057;
133 }
134
135 .placeholder {
136   display: flex;
137   justify-content: center;
138   align-items: center;
139   background-color: #f8f9fa;
140   border: 2px dashed #dee2e6;
141   border-radius: 4px;
142   padding: 2rem;
143   text-align: center;
144   color: #6c757d;
145   font-style: italic;
146   height: 300px;
147 }
148 </style>

```

11.4.3. Template für Chart-Komponenten

- Die Komponenten für die geplanten Diagramme werden separat implementiert. Jedoch weisen sie eine Reihe von Gemeinsamkeiten auf. Daher wird ein Template für die Diagrammkomponenten erstellt. Dieses Template wird dann in den einzelnen Diagrammkomponenten genutzt. Diese werden dann oben in der ChartSelector-Komponente eingebunden.
- Gemeinsame Styles werden in CSS-Dateien definiert:
 - Eine CSS-Datei für die Diagrammkomponenten zur Verwendung in den scoped-Tags.
 - Eine CSS-Datei für die D3-Elemente, da diese nicht in einem scoped-Tag eingebettet werden können.
- Eine Basisklasse BaseChart zur Animation der Diagramme. Diese wird in den einzelnen Diagrammkomponenten verwendet. Diese Klasse ist in der Datei utils/charts/BaseChart.ts implementiert.
- Hinzu kommt noch eine TypeScript-Datei utils/charts/useChart.ts zur parametrisierten Erstellung der Diagramme. Diese wird in den einzelnen Diagrammkomponenten verwendet.

./components/ChartTemplate.vue

Die ChartTemplate.vue Komponente dient als wiederverwendbare Basis für die Diagrammtypen in der Anwendung. Sie implementiert eine einheitliche Struktur und nutzt Slots, um flexible Anpassungen zu ermöglichen:

```

1 <template>
2   <div :class="['chart-wrapper', wrapperClass]">
3     <h3>{{ title }}</h3>
4     <div ref="chartContainer" class="chart-container"></div>
5     <div class="controls">
6       <button @click="onToggleAnimation" :class="['control-button', buttonClass]">
7         {{ isAnimating ? 'Animation stoppen' : 'Animation starten' }}
8       </button>
9       <slot name="extra-controls"></slot>
10    </div>
11    <slot></slot>

```

```

12 | </div>
13 | </template>

```

Die verwendeten Slots

1. Benannter Slot: `extra-controls`

- Dieser Slot befindet sich innerhalb des Controls-Bereichs, direkt nach dem Standard-Animations-Button
- Er ermöglicht es übergeordneten Komponenten, zusätzliche Steuerelemente (wie Buttons, Slider, etc.) einzufügen
- Typisches Anwendungsbeispiel: Diagramm-spezifische Konfigurationsoptionen neben der Animation

2. Default-Slot (unbenannt):

- Dieser Slot befindet sich am Ende der Komponente
- Er ermöglicht es, beliebige zusätzliche Inhalte am unteren Ende des Diagramms einzufügen
- Typisches Anwendungsbeispiel: Legenden, Erklärungen oder zusätzliche Informationen zum Diagramm

```

14 | <script setup lang="ts">
15 | import { ref, defineProps, defineEmits } from 'vue';
16 |
17 | // Create ref for chart container
18 | const chartContainer = ref<HTMLElement | null>(null);
19 |
20 | const props = defineProps({ // Parameters from parent
21 |   title: { type: String, required: true },
22 |   wrapperClass: { type: String, default: '' },
23 |   buttonClass: { type: String, default: '' },
24 |   isAnimating: { type: Boolean, required: true }
25 | });
26 |
27 | const emit = defineEmits(['toggle-animation']); // Define events
28 |
29 | function onToggleAnimation() { // Handle animation toggle
30 |   emit('toggle-animation');
31 | }
32 | defineExpose( { chartContainer } ); // Expose chartContainer to parent
33 | </script>

```

11.4.4. Styles für die Chart-Komponenten

Das gemeinsame Stylesheet für die Diagrammkomponenten wird in der `chart-styles.css` definiert. Diese Datei wird dann in den einzelnen Diagrammkomponenten eingebunden. Die Styles sind in einem `scoped`-Tag eingebettet, damit sie nur für diese Komponente gelten.

```

34 | <style scoped>
35 | @import '../assets/styles/chart-styles.css';
36 | </style>

```

./assets/styles/chart-styles.css

Die Styles für die Diagrammkomponenten sind in der Datei `chart-styles.css` definiert. Diese Datei wird dann in den einzelnen Diagrammkomponenten eingebunden. Die Styles sind in einem `scoped`-Tag eingebettet, damit sie nur für diese Komponente gelten.

```

1  /* scoped Vue Component Styles */
2  .chart-wrapper {
3    width: 100%;
4    max-width: 100%;
5    background-color: white;
6    border-radius: 8px;
7    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
8    padding: 1rem;
9    overflow-x: hidden;
10 }
11
12 .chart-container {
13   width: 100%;
14   height: 300px;
15   margin: 1rem 0;
16   overflow: visible; /* Allow chart axes to
17                      be visible */
18 }
19
20 .control-button:hover:not(:disabled) {
21   opacity: 0.9;
22 }
23
24 .control-button:disabled {
25   background-color: #6c757d;
26   cursor: not-allowed;
27   opacity: 0.6;
28 }
29
30 /* Chart specific button colors */
31 .bar-chart-button {
32   background-color: #4CAF50;
33 }
34
35 .bar-chart-button:hover {
36   background-color: #45a049;
37 }
38
39 .controls {
40   display: flex;
41   justify-content: flex-end;
42   margin-top: 1rem;
43 }
44
45 .control-button {
46   color: white;
47   border: none;
48   padding: 8px 16px;
49   border-radius: 4px;
50   cursor: pointer;
51   font-size: 14px;
52   transition: background-color
53     0.3s;
54 }
55
56 .line-chart-button {
57   background-color: #4285F4;
58 }
59
60 .line-chart-button:hover {
61   background-color: #3b78e7;
62 }
63
64 .pie-chart-button {
65   background-color: #E91E63;
66 }
67
68 .pie-chart-button:hover {
69   background-color: #C2185B;
70 }
71
72 .donut-chart-button {
73   background-color: #FF9800;
74 }
75
76 .donut-chart-button:hover {
77   background-color: #F57C00;
78 }

```

./assets/styles/d3-element-styles.css

Für D3-Elemente dürfen keine `scoped`-Styles verwendet werden, da sie nur im DOM, aber nicht im virtuellen DOM vorhanden sind. Daher sind diese CSS-Styles in einer separaten Datei `d3-element-styles.css` definiert. Diese Datei wird dann in den einzelnen Diagrammkomponenten eingebunden.

```

1  /* unscoped D3 styles */
2  .x-axis path,
3  .y-axis path,
4  .x-axis line,
5  .y-axis line {
6    stroke: #ccc;
7  }
8  .x-axis text,
9  .y-axis text {
10   fill: #666;
11   font-size: 11px;
12 }

26 .line-path:hover {
27   stroke-width: 3;
28 }
29
30 .data-point {
31   transition: r 0.3s;
32   cursor: pointer;
33 }
34
35 .data-point:hover {
36   r: 6;
37 }
38
39 /* Pie/Donut elements */
40 .segment {
41   transition: opacity 0.3s;
42   cursor: pointer;
43 }

13 /* Bar chart elements */
14 .bar {
15   transition: fill 0.3s;
16 }
17 .bar:hover {
18   opacity: 0.8;
19 }
20
21 /* Line chart elements */
22 .line-path {
23   transition: stroke-width 0.3s;
24   fill: none;
25 }

44 .segment:hover {
45   opacity: 0.8;
46 }
47
48 /* Text labels */
49 .bar-label,
50 .data-label,
51 .label-text {
52   font-size: 12px;
53   font-weight: bold;
54   fill: #333;
55   pointer-events: none;
56 }
57
58 /* Helper class for SVG labels */
59 .pointer-events-none {
60   pointer-events: none;
61 }

```

11.4.5. Basisklasse zur Chart-Animation

util/chart/BaseChart.ts

Die Basisklasse BaseChart wird in den konkreten Diagrammklassen erweitert. Diese Basisklasse enthält die Grunddaten und Methoden der Animation.

```

1  import * as d3 from 'd3';
2
3  export type Margin = { top: number; right: number; bottom: number; left: number };
4
5  export abstract class BaseChart {
6    #width: number;
7    #height: number;
8    #margin: Margin;
9    #innerWidth: number;
10   #innerHeight: number;
11   #animationSpeed: number;
12   #isAnimating: boolean = false;
13   #timer: d3.Timer | null = null;

```

Der Konstruktor der Basisklasse BaseChart initialisiert die Breite, Höhe, Ränder und die Animationsgeschwindigkeit. Die Ränder werden in einem Objekt übergeben. Die Breite und Höhe des Diagramms werden aus den Werten der Ränder abgeleitet. Die Animationsgeschwindigkeit wird in Millisekunden angegeben. Falls keine Werte übergeben werden, werden Standardwerte verwendet.

```

14  constructor(
15      width: number = 600,
16      height: number = 300,
17      margin: Margin = { top: 20, right: 20, bottom: 30, left: 40 },
18      animationSpeed: number = 1000
19  ) {
20      this.#width = width;
21      this.#height = height;
22      this.#margin = margin;
23      this.#innerWidth = width - margin.left - margin.right;
24      this.#innerHeight = height - margin.top - margin.bottom;
25      this.#animationSpeed = animationSpeed;
26  }
27  // Getters
28  get width(): number {
29      return this.#width;
30  }
31
32  get height(): number {
33      return this.#height;
34  }
35
36  get margin(): Margin {
37      return this.#margin;
38  }
39
40  get innerWidth(): number {
41      return this.#innerWidth;
42  }
43
44  get innerHeight(): number {
45      return this.#innerHeight;
46  }
47
48  get animationSpeed(): number {
49      return this.#animationSpeed;
50  }
51
52  get isAnimating(): boolean {
53      return this.#isAnimating;
54  }
55
56  // abstract methods
57  abstract renderChart(): void;
58  abstract initializeData(): void;
59  abstract updateData(): void;
60
61  protected abstract updateChart(): void;
62
63  startAnimation(): void {
64      if (this.#isAnimating || this.#animationSpeed <= 0) return;
65      this.#isAnimating = true;
66      const startTime = Date.now();
67      let lastUpdateTime = startTime;
68      // Use d3.timer for better performance and animation control
69      this.#timer = d3.timer((elapsed) => {
70          const now = Date.now();
71          if (now - lastUpdateTime >= this.#animationSpeed) {
72              this.updateData();
73              this.updateChart();
74              lastUpdateTime = now;
75          }
76          return false; // Continue the timer
77      });

```

Die `timer`-Methode sorgt für ein regelmäßiges Update der Animation. Die Methode `startAnimation()` startet die Animation. Diese wird mit der Methode `stopAnimation()` gestoppt. Die Methode `toggleAnimation()` wechselt zwischen diesen Zuständen.

d3.timer Parameter

Die Funktion `d3.timer(callback[, delay[, time]])` akzeptiert folgende Parameter:

- **callback:** Eine Funktion, die bei jedem Frame aufgerufen wird. Sie erhält einen Parameter `elapsed`, der die verstrichene Zeit in Millisekunden seit dem Timer-Start angibt. Wenn die Callback-Funktion `true` zurückgibt, wird der Timer gestoppt.

- **delay** (optional): Eine Verzögerung in Millisekunden, bevor der Timer startet. Standardwert ist 0.
- **time** (optional): Ein Zeitpunkt, ab dem der Timer starten soll (in der Form von `Date.now()`).

Im obigen Beispiel gibt die Callback-Funktion `false` zurück, damit der Timer kontinuierlich weiterläuft.

```

78 | stopAnimation(): void {
79 |     if (this.#timer) {
80 |         this.#timer.stop();
81 |         this.#timer = null;
82 |     }
83 |     this.#isAnimating = false;
84 | }
85 | toggleAnimation(isAnimating: boolean): boolean {
86 |     if (isAnimating) {
87 |         this.stopAnimation();
88 |         return false;
89 |     } else {
90 |         this.startAnimation();
91 |         return true;
92 |     }
93 | }
94 | destroy(): void {
95 |     this.stopAnimation();
96 | }
97 | }
```

11.4.6. Funktion zur Erzeugung der Chart-Komponenten

./utils/charts/useChart.ts

Es fehlt noch eine Funktion um die Diagrammkomponenten mit Hilfe der Templates zu erzeugen. Die Funktion managed die Diagramminstanz und sorgt dafür, dass D3-Funktionen nur nach dem Mounten der Komponente aufgerufen werden. Sie wird in den einzelnen Diagrammkomponenten verwendet. Ihre Parameter sind:

- **chartRef**: Referenz auf die Komponente, die ein `chartContainer` bereitstellt.
- **chartClass**: Der Konstruktor der Diagrammklasse. Sein erstes Element ist der Container, in dem das Diagramm gezeichnet wird. Die weiteren Parameter sind die Optionen für die Diagrammklasse.
- **options**: optionale Konfiguration des Diagramms.

Die Funktion gibt ein Objekt mit den folgenden Eigenschaften zurück:

- **isAnimating**: Boolean, der angibt, ob die Animation aktiv ist.
- **toggleAnimation**: Funktion, die die Animation startet oder stoppt.
- **chart**: Referenz auf das Diagramm, wird nur in komplexeren Anwendungen benötigt.

```

1 import { BaseChart } from './BaseChart';
2 import { ref, onMounted, onUnmounted, shallowRef } from 'vue';
3
4 export function useChart<T extends BaseChart>({
5   chartRef: any,
6   ChartClass: new(container: HTMLElement|null, ...args: any[]) => T,
7   options: any = {}
8 }) {
9   const isAnimating = ref(false); // Animation state
10  // Chart instance - using shallowRef for performance
11  const chart = shallowRef<T | null>(null);
12
13  const toggleAnimation = () => {
14    if (chart.value) {
15      isAnimating.value = chart.value.toggleAnimation(isAnimating.value);
16    }
17  };
18  onMounted(() => {
19    if (chartRef.value?.chartContainer) {
20      // Create chart instance with container and options
21      chart.value = new ChartClass(
22        chartRef.value.chartContainer,
23        ...Object.values(options)
24      );
25
26      // Initialize and render
27      chart.value.initializeData();
28      chart.value.renderChart();
29
30      // Start animation by default
31      chart.value.startAnimation();
32      isAnimating.value = true;
33    } else {
34      console.warn('Chart container not available in onMounted hook. Check
35        component structure.');
```

11.5. Diagramm-Implementierung

11.5.1. Animiertes Balkendiagramm

Zuerst soll ein Balkendiagramm erstellt werden. Dieses wird in der `AnimatedBarChart`-Komponente implementiert. Diese erbt von der Basisklasse `BaseChart`.

Die Animation wird mit der Methode `toggleAnimation()` gestartet und auch angehalten. Diese wird mit dem Button aufgerufen. Der Button zeigt den aktuellen Zustand der Animation an. Die Animation

wird in der `AnimatedBarChart`-Klasse implementiert. Die Werte für `isAnimated` und `toggleAnimation()` werden durch den Aufruf von `useCharts` generiert.

```

1 <template>
2   <ChartTemplate
3     ref="templateRef"
4     title="Animiertes Balkendiagramm"
5     wrapperClass="animated-bar-chart"
6     buttonClass="bar-chart-button"
7     :isAnimating="isAnimating"
8     @toggle-animation="toggleAnimation">
9   </ChartTemplate>
10 </template>

```

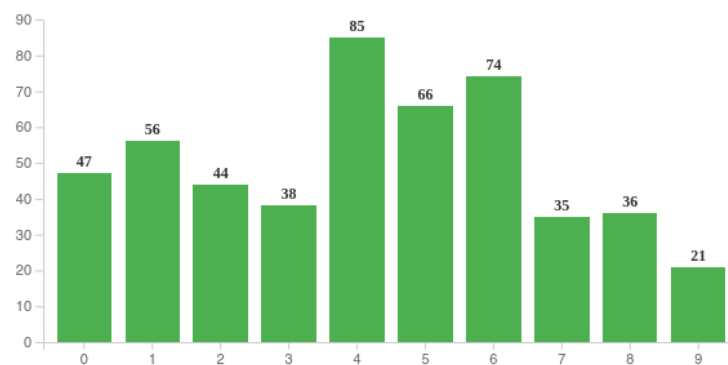


Zustand der Komponenten nach Auswahl des Balkendiagramms

Balkendiagramm:

Zeigt kategorische Daten durch die Höhe der Balken an.

Animiertes Balkendiagramm



Animation starten

Darstellung des Balkendiagramms

Das Interface `DataPoint` definiert die Datenpunkte, welche aus einer ID und einem Wert bestehen. Die Klasse `AnimatedBarChart` dient zur Darstellung der Daten als Balkendiagramm.

```

11 <script setup lang="ts">
12 import { ref } from 'vue';
13 import * as d3 from 'd3';
14 import { BaseChart, type Margin } from '../utils/charts/BaseChart';

```

```

15 import { useChart } from '../utils/charts/useChart';
16 import ChartTemplate from './ChartTemplate.vue';
17
18 interface DataPoint {
19   id: number;
20   value: number;
21 }
22 // Concrete implementation for animated bar chart
23 class AnimatedBarChart extends BaseChart {
24   #data: DataPoint[] = [];
25   #nextId: number = 0;
26   #svg: d3.Selection<SVGSVGElement, unknown, null, undefined> | null = null;
27   #container: HTMLElement | null = null;
28   #barPadding: number;
29
30   constructor(
31     container: HTMLElement | null = null,
32     width: number = 600,
33     height: number = 300,
34     margin: Margin = { top: 20, right: 20, bottom: 30, left: 40 },
35     barPadding: number = 0.2
36   ) {
37     super(width, height, margin, 1000); // Animation speed of 1000ms
38     this.#container = container;
39     this.#barPadding = barPadding;
40   }

```

In der Methode `initializeData()` wird ein Array mit 10 Datenpunkten erstellt. Diese werden mit zufälligen Werten zwischen 1 und 100 initialisiert. Die ID wird dabei um eins erhöht. Die abstrakte Methode aus der Basisklasse wird hier implementiert.

```

41   override initializeData(): void {
42     this.#data = Array.from({ length: 10 }, () => ({
43       id: this.#nextId++,
44       value: Math.floor(Math.random() * 100) + 1
45     } as DataPoint));
46   }

```

Bei jedem Update wird der älteste Datenpunkt entfernt und ein neuer Datenpunkt hinzugefügt. Die ID wird dabei um eins erhöht. Die Methode `updateData()` implementiert die abstrakte Methode aus `BaseChart`.

```

47   override updateData(): void {
48     const newData = [...this.#data];
49     newData.shift(); // Remove the oldest data point
50     newData.push({ // Add a new data point at the end
51       id: this.#nextId++,
52       value: Math.floor(Math.random() * 100) + 1
53     });
54     this.#data = newData;
55   }

```

Die Methode `renderChart()` hat zwei Aufgaben: Falls das SVG-Element noch nicht existiert, wird es erstellt und Gruppenelemente für die Inhalte und die Achsen erzeugt. Die Breite des Containers wird dabei berücksichtigt, um das Diagramm responsiv zu gestalten. Falls das SVG-Element bereits existiert wird lediglich die Größe der `ViewBox` responsiv angepasst. In jedem Fall aktualisiert die Methode `updateChart()` die Achsen und die Balken.

```

56   override renderChart(): void {
57     if (!this.#container) return;
58
59     // Get container width to make the chart responsive

```

```

60     const containerWidth = this.#container.clientWidth;
61     const adjustedWidth = Math.max(containerWidth - 40, 300); // 40px for padding,
        with minimum width
62     // Create or select SVG element
63     let svg : d3.Selection<SVGSVGElement, unknown, null, undefined> = d3.select(
        this.#container).select('svg');
64     if (svg.empty()) {
65         svg = d3.select(this.#container)
66             .append('svg')
67             .attr('width', '100%') // Use percentage width instead of fixed width
68             .attr('height', this.height)
69             .attr('viewBox', `0 0 ${adjustedWidth} ${this.height}`) // Add viewBox for
                responsiveness
70             .attr('preserveAspectRatio', 'xMidYMid meet');
71
72         svg.append('g') // Main g element for transformation
73             .attr('transform', `translate(${this.margin.left},${this.margin.top}})`
74             .attr('class', 'chart-content');
75         svg.append('g') // X axis
76             .attr('class', 'x-axis')
77             .attr('transform', `translate(${this.margin.left},${this.height - this.
                margin.bottom}})`);
78         svg.append('g') // Y axis
79             .attr('class', 'y-axis')
80             .attr('transform', `translate(${this.margin.left},${this.margin.top}})`);
81     } else { // Update viewBox on existing SVG
82         svg.attr('viewBox', `0 0 ${adjustedWidth} ${this.height}`);
83     }
84     this.#svg = svg;
85     this.updateChart();
86 }

```

Die Methode `updateChart` aktualisiert die Achsen und die Balken. Die X-Achse wird mit `d3.scaleBand` erzeugt, da jeder Balken eine Bandbreite hat. Die Y-Achse wird mit `d3.scaleLinear` erzeugt, da die Werte linear sind. Die Achsen werden mit `d3.axisBottom` und `d3.axisLeft` erzeugt und entsprechend platziert.

```

87     protected override updateChart(): void {
88         if (!this.#svg) return;
89
90         // Update scales
91         const xScale = d3.scaleBand<string>()
92             .domain(this.#data.map((_, i) => i.toString()))
93             .range([0, this.innerWidth])
94             .padding(this.#barPadding);
95
96         const yScale = d3.scaleLinear()
97             .domain([0, d3.max(this.#data, d => d.value) || 100])
98             .nice()
99             .range([this.innerHeight, 0]);
100
101         this.#svg.select<SVGElement>('.x-axis') // Update X axis
102             .call(d3.axisBottom(xScale) as any);
103
104         this.#svg.select<SVGElement>('.y-axis') // Update Y axis
105             .call(d3.axisLeft(yScale) as any);
106         // Update bars with join pattern
107         this.#svg.select('.chart-content')
108             .selectAll<SVGRectElement, DataPoint>('.bar')
109             .data(this.#data, d => d.id)
110             .join(
111                 enter => enter.append('rect')

```

```

112     .attr('class', 'bar')
113     .attr('x', (_, i) => xScale(i.toString()) || 0)
114     .attr('y', this.innerHeight)
115     .attr('width', xScale.bandwidth())
116     .attr('height', 0)
117     .attr('fill', '#4CAF50')
118     .call(enter => enter.transition()
119           .duration(500)
120           .attr('y', d => yScale(d.value))
121           .attr('height', d => this.innerHeight - yScale(d.value))
122     ),

```

- Die neuen Balken werden mit der Methode `enter` erstellt und hinten angefügt. Ihre Höhe wird zunächst auf 0 gesetzt. Dann wird mit `transition().duration(500)` durch eine Animation der Balken auf seine durch die Daten gegebene Höhe gezogen.
- Mit `update` werden die Balken aktualisiert. Da das linke Element entfernt wurde, werden die Balken auf die neue Position verschoben. Auch hier wird eine Animation verwendet.
- Die Balken, die nicht mehr in den Daten vorhanden sind, werden mit `exit` entfernt. Die Animation bringt die Balken auf die Höhe 0 bevor sie entfernt werden.

```

123     update => update.transition()
124       .duration(500)
125       .attr('x', (_, i) => xScale(i.toString()) || 0)
126       .attr('y', d => yScale(d.value))
127       .attr('width', xScale.bandwidth())
128       .attr('height', d => this.innerHeight - yScale(d.value)),
129     exit => exit.transition()
130       .duration(500)
131       .attr('y', this.height)
132       .attr('height', 0)
133       .remove()
134   );

```

Neben den Balken müssen auch die Labels an den Balken auf den neuen Stand gebracht werden. Hierzu werden die neuen Labels von „durchsichtig“ auf „opaque“ animiert. Die bereits bestehenden Labels werden entsprechend der zugehörigen Balken nach links verschoben. Die nicht mehr benötigten Labels werden von „opaque“ nach „durchsichtig“ animiert und dann entfernt.

```

135     // Display values as text with join pattern
136     this.svg.select('.chart-content')
137       .selectAll<SVGTextElement, DataPoint>('.bar-label')
138       .data(this.#data, d => d.id)
139       .join(
140         enter => enter.append('text')
141           .attr('class', 'bar-label')
142           .attr('x', (_, i) => (xScale(i.toString()) || 0) + xScale.bandwidth() /
143             2)
143           .attr('y', d => yScale(d.value) - 5)
144           .attr('text-anchor', 'middle')
145           .style('opacity', 0)
146           .text(d => d.value)
147           .call(enter => enter.transition()
148                 .duration(500)
149                 .style('opacity', 1)
150           ),
151         update => update.transition()
152           .duration(500)

```

```

153         .attr('x', (_, i) => (xScale(i.toString()) || 0) + xScale.bandwidth() /
154             2)
155         .attr('y', d => yScale(d.value) - 5)
156         .text(d => d.value),
157     exit => exit.transition()
158         .duration(500)
159         .style('opacity', 0)
160         .remove()
161     };
162 }

```

Der Aufruf von `useChart` erzeugt die Instanz der Klasse `AnimatedBarChart`. Diese wird in der Komponente `templateRef` dargestellt. Die aktuelle Komponente wird in die Komponente `ChartTemplate` eingebunden. Hierzu wird die Referenz auf diese Komponente in der `templateRef`-Variablen gespeichert und dann an die Funktion `useChart` übergeben.

```

163 const templateRef = ref<InstanceType<typeof ChartTemplate> | null>(null);
164
165 const { isAnimating, toggleAnimation } = useChart(
166     templateRef,
167     AnimatedBarChart,
168     {
169         width: 600,
170         height: 300,
171         margin: { top: 20, right: 20, bottom: 30, left: 40 },
172         barPadding: 0.2
173     }
174 );
175 </script>

```

Das übergebene Option-Element enthält die Parameter für den Aufruf des `AnimatedBarChart`-Konstruktors. Diese Funktion gibt die Variablen `isAnimating` und `toggleAnimation` zurück. Diese werden dann in der Komponente verwendet um die Animation zu starten und zu stoppen.

Die `<style>`-Elemente für die `AnimatedBarChart`-Komponente werden aus einer Datei eingelesen. Die Styles für die D3-Elemente sind in einer separaten Datei `d3-element-styles.css` definiert, da diese nicht in einem `scoped`-Tag eingebettet werden können. Diese Datei wird dann in den einzelnen Diagrammkomponenten eingebunden.

```

176 <style scoped>
177 /* Component-specific styles using shared chart styles */
178 @import '../assets/styles/chart-styles.css';
179 </style>
180
181 <style>
182 /* D3 elements use unscoped styles */
183 @import '../assets/styles/d3-element-styles.css';
184 </style>

```

11.5.2. Animiertes Liniendiagramm

Es soll ein Liniendiagramm mit animierten Übergängen erstellt werden. Als Basis sollen wieder das `ChartTemplate` und die Basisklasse `BaseChart` benutzt werden. Die Daten sind zufällig generiert und enthalten neben einem Wert auch eine ID und einen Zeitstempel. Auf der x-Achse wird die Zeit abgebildet, auf der y-Achse der Wert. Die Zeit der Daten ist monoton ansteigend, d.h. es neue Daten erhalten einen aktuellen Zeitstempel. Damit ist das zugehörige Template der Komponente `AnimatedLineChart` wie folgt:

```

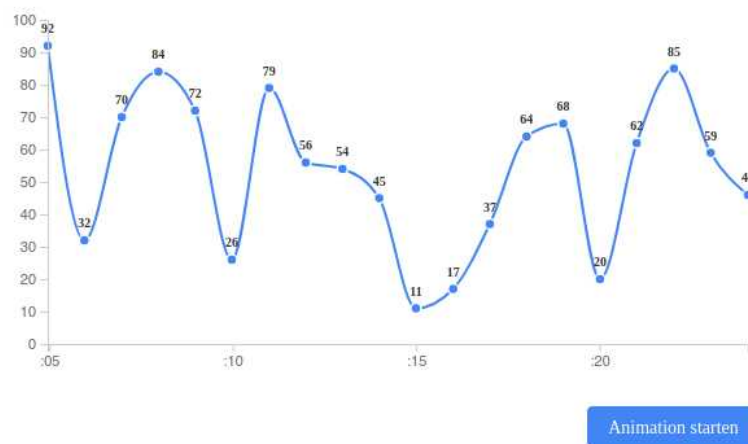
1 <template>
2   <ChartTemplate
3     ref="templateRef"
4     title="Animiertes Liniendiagramm"
5     wrapperClass="animated-line-chart"
6     buttonClass="line-chart-button"
7     :isAnimating="isAnimating"
8     @toggle-animation="toggleAnimation">
9   </ChartTemplate>
10 </template>

```

Liniendiagramm:

Zeigt Trends und Verläufe über die Zeit.

Animiertes Liniendiagramm



Liniendiagramm mit Animation

```

11 <script setup lang="ts">
12 import { ref } from 'vue';
13 import * as d3 from 'd3';
14 import { BaseChart } from '../utils/charts/BaseChart';
15 import { useChart } from '../utils/charts/useChart';
16 import ChartTemplate from './ChartTemplate.vue';
17
18 interface DataPoint {
19   id: number;
20   value: number;
21   timestamp: number;
22 }
23
24 // Reference to the template component
25 const templateRef = ref<InstanceType<typeof ChartTemplate> | null>(null);

```

Die Klasse `Animated` erbt von `BaseChart` und implementiert die Logik für das Liniendiagramm. Damit werden in der Klasse Methoden zur Initialisierung und zum Update der Daten implementiert. Die Klasse erlaubt eine maximale Zahl von Datenpunkten gleichzeitig darzustellen. Wird diese Anzahl überschritten, so werden nach und nach die ältesten Datenpunkte entfernt.

```

26 class AnimatedLineChart extends BaseChart {
27   #data: DataPoint[] = [];
28   #nextId: number = 0;

```

```

29 | #svg: d3.Selection<SVGSVGElement, unknown, null, undefined> | null = null;
30 | #container: HTMLElement | null = null;
31 | #lineColor: string;
32 | #maxDataPoints: number;

```

Der Konstruktor bestimmte die Größe der Zeichenfläche, die Ränder, die Farbe der Linie und die maximale Anzahl an Datenpunkten. Die Animation wird mit einer Dauer von 1000ms initialisiert.

```

33 | constructor(
34 |   container: HTMLElement | null = null,
35 |   width: number = 600,
36 |   height: number = 300,
37 |   margin: { top: number; right: number; bottom: number; left: number } = { top:
    |     20, right: 20, bottom: 30, left: 40 },
38 |   lineColor: string = '#4285F4',
39 |   maxDataPoints: number = 20
40 | ) {
41 |   super(width, height, margin, 1000); // Animation speed of 1000ms
42 |   this.#container = container;
43 |   this.#lineColor = lineColor;
44 |   this.#maxDataPoints = maxDataPoints;
45 | }

```

Die Anfangsdaten werden in der Methode `initializeData` generiert. Dabei wird eine Liste aus 10 Datenpunkten erstellt. Die Zeitstempel werden dabei mit einer Sekunde Abstand generiert. Der erste Datensatz erhält einen Zeitstempel der 9 Sekunden in der Vergangenheit liegt. Der 10te Datensatz erhält den aktuellen Zeitstempel. Die ID wird fortlaufend vergeben.

```

46 | override initializeData(): void {
47 |   const now = Date.now();
48 |   this.#data = Array.from({ length: 10 }, (_, i) => ({
49 |     id: this.#nextId++,
50 |     value: Math.floor(Math.random() * 100) + 1,
51 |     timestamp: now - (9 - i) * 1000 // Add timestamps going backward from now
52 |   }));
53 | }

```

Die Daten werden aktualisiert indem der älteste Datenpunkt entfernt wird, wenn die maximale Anzahl an Datenpunkten erreicht ist. Ein zufälliger Datenpunkt wird mit dem aktuellen Zeitstempel hinzugefügt.

```

55 | override updateData(): void {
56 |   const now = Date.now();
57 |   const newData = [...this.#data];
58 |
59 |   // Remove the oldest data point if we've reached the maximum
60 |   if (newData.length >= this.#maxDataPoints) {
61 |     newData.shift();
62 |   }
63 |   // Add a new data point at the end
64 |   newData.push({
65 |     id: this.#nextId++,
66 |     value: Math.floor(Math.random() * 100) + 1,
67 |     timestamp: now
68 |   });
69 |   this.#data = newData;
70 | }

```

Nun wird wieder das SVG-Element erzeugt bzw. aktualisiert. Dabei wird die Breite des Containers ermittelt und die SVG-Größe entsprechend angepasst. Die Achsen werden ebenfalls aktualisiert. Die X-Achse wird mit Zeitstempeln und die Y-Achse mit den Werten der Datenpunkte gefüllt. Die Datenpunkte werden als Kreise dargestellt, die Linie wird mit der D3-Funktion `line` erzeugt.

```

71 | override renderChart(): void {
72 |   if (!this.#container) return;
73 |
74 |   // Get container width to make the chart responsive
75 |   const containerWidth = this.#container.clientWidth;
76 |   const adjustedWidth = Math.max(containerWidth - 40, 300); // 40px for padding,
       |     with minimum width
77 |
78 |   // Create or select SVG element
79 |   let svg = d3.select(this.#container).select<SVGSVGElement>('svg');

```

Falls das SVG-Element noch nicht existiert, wird es jetzt erzeugt. Dabei wird sicher gestellt, dass die Zeichenfläche responsiv ist.

```

80 |   if (svg.empty()) {
81 |     svg = d3.select(this.#container)
82 |       .append('svg')
83 |       .attr('width', '100%') // Use percentage width instead of fixed width
84 |       .attr('height', this.height)
85 |       .attr('viewBox', `0 0 ${adjustedWidth} ${this.height}`) // Add viewBox for
       |         responsiveness
86 |       .attr('preserveAspectRatio', 'xMidYMid meet');

```

Nun werden drei Gruppen in das SVG-Element eingefügt: Eine Gruppe für den eigentlichen Diagramminhalt, eine für die X-Achse und eine für die Y-Achse.

```

87 |     // Main g element for transformation
88 |     svg.append('g')
89 |       .attr('class', 'chart-content')
90 |       .attr('transform', `translate(${this.margin.left},${this.margin.top})`);
91 |
92 |     // X axis
93 |     svg.append('g')
94 |       .attr('class', 'x-axis')
95 |       .attr('transform', `translate(${this.margin.left},${this.height - this.
       |         margin.bottom})`);
96 |
97 |     // Y axis
98 |     svg.append('g')
99 |       .attr('class', 'y-axis')
100 |       .attr('transform', `translate(${this.margin.left},${this.margin.top})`);

```

Zuletzt wird noch ein Clip-Pfad hinzugefügt, damit die Linie nicht über den Rand des Diagramms hinausgeht. Dies ist besonders wichtig, wenn das Diagramm animiert wird. Hierzu wird das <defs> Element aus SVG verwendet. In diesem können Elemente gespeichert werden, die erst später verwendet werden.

```

101 |     svg.append('defs').append('clipPath')
102 |       .attr('id', 'clip')
103 |       .append('rect')
104 |       .attr('width', this.innerWidth)
105 |       .attr('height', this.innerHeight)
106 |       .attr('x', 0)
107 |       .attr('y', 0);
108 |   } else { // Update viewBox on existing SVG
109 |     svg.attr('viewBox', `0 0 ${adjustedWidth} ${this.height}`);
110 |   }
111 |
112 |   this.#svg = svg;
113 |   this.updateChart();
114 | }

```

Die Werte der x-Achse bestehen aus den Zeitstempeln der sichtbaren Datenpunkte. Hierzu werden die Zeitstempel in ein `Date` Objekt umgewandelt. Die Funktion `d3.extent` bestimmt hierzu das benötigte Zeitintervall aus den vorhandenen Daten. Die Y-Achse wird mit den Werten der Datenpunkte gefüllt.

```

115 |   protected override updateChart(): void {
116 |       if (!this.#svg || this.#data.length === 0) return;
117 |       // Use a consistent transition duration for all animated elements
118 |       const transitionDuration = 500;
119 |       // Update scales
120 |       const xScale = d3.scaleTime()
121 |         .domain(d3.extent(this.#data, d => new Date(d.timestamp)) as [Date, Date])
122 |         .range([0, this.innerWidth]);
123 |       const yScale = d3.scaleLinear()
124 |         .domain([0, d3.max(this.#data, d => d.value) || 100])
125 |         .nice()
126 |         .range([this.innerHeight, 0]);

```

Bei jedem Update müssen die Achsen aktualisiert werden. Dies geschieht mit der D3-Funktion `axisBottom` für die X-Achse und `axisLeft` für die Y-Achse. Die Achsen werden mit einer Transition animiert, damit die Änderungen flüssig wirken. Die Funktion `d3.axisBottom` zeichnet die x-Achse mit den Beschriftungen und den Ticks unterhalb der Achse. Ebenso zeichnet die Funktion `d3.axisLeft` die y-Achse mit den Beschriftungen und den Ticks links von der Achse. Die Ticks sind die kleinen Striche, die die Achse in gleichmäßige Abschnitte unterteilen. Die Anzahl der Ticks wird mit der Funktion `ticks` bestimmt. In diesem Fall werden 5 Ticks für die X-Achse und 10 Ticks für die Y-Achse verwendet.

```

127 |       // Update X axis
128 |       this.#svg.select<SVGElement>('.x-axis')
129 |         .transition().duration(transitionDuration)
130 |         .call(d3.axisBottom(xScale).ticks(5));
131 |       // Update Y axis
132 |       this.#svg.select<SVGElement>('.y-axis')
133 |         .transition().duration(transitionDuration)
134 |         .call(d3.axisLeft(yScale));

```

Die Datenpunkte werden mit Linien verbunden. Einige der zur Verfügung stehenden Kurven sind:

- `d3.curveLinear` - gerade Linien
- `d3.curveStep` - Treppenlinien
- `d3.curveBasis` - glatte Kurven (kubische Splines)
- `d3.curveMonotoneX` - glatte kubische Splines, die monoton sind
- `d3.curveCatmullRom` - Catmull-Rom-Splines

Die von Funktion `d3.line<DataPoint>` erzeugte Funktion wird in der Variable `line` gespeichert. Diese Funktion wird dann verwendet, um die Linie zu zeichnen.

```

135 |       // Define the line function
136 |       const line = d3.line<DataPoint>()
137 |         .x(d => xScale(new Date(d.timestamp)))
138 |         .y(d => yScale(d.value))
139 |         .curve(d3.curveMonotoneX); // Smooth curve
140 |
141 |       // Create or update the line path
142 |       const chartContent = this.#svg.select('.chart-content');
143 |       let path = chartContent.select<SVGPathElement>('.line-path');

```

Falls das Pfad-Element noch nicht existiert, wird es jetzt erzeugt. Dabei wird der Clip-Pfad verwendet, damit die Linie nicht über den Rand des Diagramms hinausgeht. Danach werden die Datenpunkte an die Linien gebunden. Falls schon ein Pfad dargestellt ist, wird dieser mit einem Übergang aktualisiert.

```

144     if (path.empty()) {
145         path = chartContent.append('path')
146             .attr('class', 'line-path')
147             .attr('clip-path', 'url(#clip)')
148             .attr('fill', 'none')
149             .attr('stroke', this.#lineColor)
150             .attr('stroke-width', 2);
151     }
152
153     // Update the path
154     path.datum(this.#data)
155         .transition()
156         .duration(transitionDuration)
157         .attr('d', line as any);

```

Die Datenpunkte selbst werden mit Hilfe von Kreisen dargestellt. Diese werden mit der D3-Funktion `join` aktualisiert.

```

158     // Update data points using join pattern
159     const dataPoints = chartContent.selectAll<SVGCircleElement,
160         DataPoint>('.data-point').data(this.#data, d => d.id);
161
162     dataPoints.join(
163         enter => enter.append('circle')
164             .attr('class', 'data-point')
165             .attr('cx', d => xScale(new Date(d.timestamp)))
166             .attr('cy', d => yScale(d.value))
167             .attr('r', 0)
168             .attr('fill', this.#lineColor)
169             .attr('stroke', '#fff')
170             .attr('stroke-width', 1.5)
171             .call(enter => enter.transition()
172                 .duration(transitionDuration)
173                 .attr('r', 4)
174             ),

```

Vorsicht

Im obigen `enter`-Block wird für den Übergang `transition` nicht direkt, sondern über die Methode `call` aufgerufen. Diese erlaubt eine Methode für die aktuelle Selektion aufzurufen, ohne die Methoden-Pipeline zu unterbrechen. Dies ist notwendig, da der Übergang sonst nicht korrekt funktioniert; Würde `transition` sofort aufgerufen, dann würde nicht auf den Abschluss des vorherigen Übergangs wartet und damit würden die `update`- und `exit`-Blöcke nicht korrekt ausgeführt werden.

```

175         update => update.transition()
176             .duration(transitionDuration)
177             .attr('cx', d => xScale(new Date(d.timestamp))),
178             .attr('cy', d => yScale(d.value)),
179         exit => exit.transition()
180             .duration(transitionDuration)
181             .attr('r', 0)
182             .remove()
183     );

```

Mit D3 kann auch auf Ereignisse reagiert werden. Hierzu wird die Methode `on` verwendet. Diese wird auf die Selektion angewendet, die das Ereignis empfangen soll. In diesem Fall wird auf das

mouseover-Ereignis reagiert. Wenn der Mauszeiger über einen Datenpunkt bewegt wird, ändert sich die Farbe des Kreises. Dies geschieht mit einer Transition, damit die Änderung flüssig wirkt.

```

184 // Update line color on hover
185 chartContent.selectAll<SVGCircleElement, DataPoint>('.data-point')
186   .on('mouseover', function () {
187     d3.select(this).transition()
188       .duration(transitionDuration)
189       .attr('fill', '#FF0000'); // Change color on hover
190   })
191   .on('mouseout', function () {
192     d3.select(this).transition()
193       .duration(transitionDuration)
194       .attr('fill', this.#lineColor); // Reset color
195   });

```

Nun werden die Label der Datenpunkte aktualisiert.

```

196 // Update value labels using join pattern
197 const labelVerticalOffset = 10;
198 chartContent.selectAll<SVGTextElement, DataPoint>('.data-label')
199   .data(this.#data, d => d.id)
200   .join(
201     enter => enter.append('text')
202       .attr('class', 'data-label')
203       .attr('x', d => xScale(new Date(d.timestamp)))
204       .attr('y', d => yScale(d.value) - labelVerticalOffset)
205       .attr('text-anchor', 'middle')
206       .style('font-size', '10px')
207       .style('opacity', 0)
208       .text(d => d.value)
209       .call(enter => enter.transition()
210         .duration(transitionDuration)
211         .style('opacity', 1)
212       ),

```

Auch hier musste wieder die Methode call verwendet werden, damit die Animation korrekt funktioniert.

```

213     update => update.transition()
214       .duration(transitionDuration)
215       .attr('x', d => xScale(new Date(d.timestamp))),
216       .attr('y', d => yScale(d.value) - labelVerticalOffset)
217     exit => exit.transition()
218       .duration(transitionDuration)
219       .style('opacity', 0)
220       .remove()
221   );
222 }
223 }

```

Analog zum Balkendiagramm wird auch hier mit Hilfe der Funktion useChart die Instanz von AnimatedLineChart erzeugt und in ChartTemplate eingebettet.

```

224 const { isAnimating, toggleAnimation } = useChart(
225   templateRef,
226   AnimatedLineChart,
227   {
228     width: 600,
229     height: 300,
230     margin: { top: 20, right: 20, bottom: 30, left: 40 },
231     lineColor: '#4285F4',
232     maxDataPoints: 20

```

```

233 |   }
234 | );
235 | </script>

```

Die beiden Style-Abschnitte sind identisch mit den vorherigen Beispielen.

```

236 | <style scoped>
237 | /* Component-specific styles using shared chart styles */
238 | @import '../assets/styles/chart-styles.css';
239 | </style>
240 |
241 | <style>
242 | /* D3 elements use unscoped styles */
243 | @import '../assets/styles/d3-element-styles.css';
244 | </style>

```

11.5.3. Animiertes Tortendiagramm

Es soll ein Tortendiagramm mit animierten Übergängen erstellt werden. Als Basis sollen wieder das ChartTemplate und die Basisklasse BaseChart benutzt werden. Die Daten sind zufällig generiert und enthalten neben einem Wert auch eine ID, ein Label und eine Farbe. Die Summe aller Werte entspricht 100%. Die Segmentflächen des Kreises entsprechen den Anteilen der Werte. Damit ist das zugehörige Template der Komponente AnimatedPieChart wie folgt:

```

1 | <template>
2 |   <ChartTemplate
3 |     ref="templateRef"
4 |     title="Animiertes Kreisdiagramm"
5 |     wrapperClass="animated-pie-chart"
6 |     buttonClass="pie-chart-button"
7 |     :isAnimating="isAnimating"
8 |     @toggle-animation="toggleAnimation">
9 |   </ChartTemplate>
10 | </template>

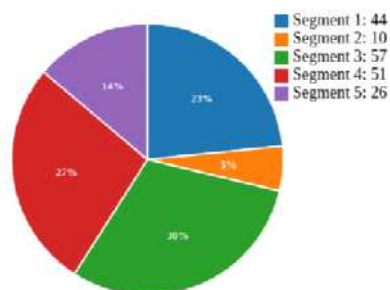
```

Balkendiagramm Liniendiagramm Kreisdiagramm Ringdiagramm

Kreisdiagramm:

Zeigt proportionale Anteile einer Gesamtmenge.

Animiertes Kreisdiagramm



Animation starten

Animiertes Tortendiagramm

```

11 | <script setup lang="ts">
12 | import { ref } from 'vue';
13 | import * as d3 from 'd3';

```

```

14 import { BaseChart } from '../utils/charts/BaseChart';
15 import { useChart } from '../utils/charts/useChart';
16 import ChartTemplate from './ChartTemplate.vue';
17
18 interface DataPoint {
19   id: number;
20   value: number;
21   label: string;
22   color: string;
23 }

```

Das Interface PieSegment wird genutzt um den aktuellen Zustand der Segmente zu speichern. Dies ist notwendig, um in der d3-Funktion attrTween den aktuellen Zustand des Segments zu interpolieren. Die Funktion interpolate interpoliert zwischen dem aktuellen und dem neuen Zustand des Segments. Der aktuelle Zustand wird in der Eigenschaft _current gespeichert.

```

24 // Improved interface for SVG elements with _current property
25 interface PieSegment extends SVGPathElement {
26   _current?: d3.PieArcDatum<DataPoint>;
27 }
28
29 // Reference to the template component
30 const templateRef = ref<InstanceType<typeof ChartTemplate> | null>(null);
31 // Concrete implementation for animated pie chart
32 class AnimatedPieChart extends BaseChart {
33   #data: DataPoint[] = [];
34   #nextId: number = 0;
35   #svg: d3.Selection<SVGSVGElement, unknown, null, undefined> | null = null;
36   #container: HTMLElement | null = null;
37   #radius: number;
38   #colors: readonly string[];
39   constructor(
40     container: HTMLElement | null = null,
41     width: number = 600,
42     height: number = 400,
43     margin: { top: number; right: number; bottom: number; left: number } = { top:
44       40, right: 40, bottom: 40, left: 40 },
45     radius: number = 150,
46     colors: readonly string[] = d3.schemeCategory10
47   ) {
48     super(width, height, margin, 1500); // Animation speed of 1500ms
49     this.#container = container;
50     this.#radius = radius;
51     this.#colors = colors;
52   }
53   initializeData(): void {
54     // Create 5 random segments for the pie
55     this.#data = Array.from({ length: 5 }, (_, i) => ({
56       id: this.#nextId++,
57       value: Math.floor(Math.random() * 50) + 10, // Random value between 10-60
58       label: `Segment ${i + 1}`,
59       color: this.#colors[i % this.#colors.length]
60     }));
61   }
62   updateData(): void {
63     // Update existing segment values and occasionally add/remove segments
64     const newData = [...this.#data];
65
66     // Randomly update values
67     newData.forEach(item => {
68       item.value = Math.max(5, item.value + (Math.random() > 0.5 ? 1 : -1) * (Math
69         .floor(Math.random() * 10)));

```

```

69 // Occasionally add a new segment (20% chance if less than 8 segments)
70 if (Math.random() > 0.8 && newData.length < 8) {
71   newData.push({
72     id: this.#nextId++,
73     value: Math.floor(Math.random() * 40) + 10,
74     label: `Segment ${newData.length + 1}`,
75     color: this.#colors[newData.length % this.#colors.length]
76   });
77 }
78 // Occasionally remove a segment (20% chance if more than 3 segments)
79 if (Math.random() > 0.8 && newData.length > 3) {
80   const indexToRemove = Math.floor(Math.random() * newData.length);
81   newData.splice(indexToRemove, 1);
82 }
83 this.#data = newData;
84 }
85 renderChart(): void {
86   if (!this.#container) return;
87
88   // Create or select SVG element
89   let svg = d3.select(this.#container).select<SVGSVGElement>('svg');
90   if (svg.empty()) {
91     svg = d3.select(this.#container)
92       .append('svg')
93       .attr('width', this.width)
94       .attr('height', this.height);
95
96     // Add group for pie - center the chart
97     svg.append('g')
98       .attr('class', 'pie-container')
99       .attr('transform', `translate(${this.width / 2}, ${this.height / 2})`);
100     // Add group for legends
101     svg.append('g')
102       .attr('class', 'legend-container')
103       .attr('transform', `translate(${this.width - this.margin.right - 120}, ${
104         this.margin.top})`);
105   }
106
107   this.#svg = svg;
108   this.updateChart();
109 }
110 protected updateChart(): void {
111   if (!this.#svg || this.#data.length === 0) return;
112
113   // Calculate total for percentage
114   const total = d3.sum(this.#data, d => d.value);
115
116   // Use a consistent transition duration for all animated elements
117   const transitionDuration = 750;
118
119   // Create pie layout
120   const pie = d3.pie<DataPoint>()
121     .value(d => d.value)
122     .sort(null); // Don't sort, maintain the original order
123   // Create arc generator
124   const arc = d3.arc<d3.PieArcDatum<DataPoint>>()
125     .innerRadius(0) // Set to 0 for a pie chart (not a donut)
126     .outerRadius(this.#radius);
127
128   // Generate pie chart data - transforms original data into pie arc data
129   // Each element becomes: { data: originalDataPoint, value, startAngle,
130     endAngle, etc. }

```

```

131 const arcs = pie(this.#data);
132
133 // Update pie segments using join pattern
134 const pieContainer = this.#svg.select('.pie-container'):
135 pieContainer.selectAll<PieSegment>
136   d3.PieArcDatum<DataPoint>>('.segment')
137   .data(arcs, d => d.data.id.toString())
138   .join(
139     enter => enter.append('path')
140       .attr('class', 'segment')
141       .attr('fill', d => d.data.color) // d.data contains our original
142         DataPoint
143       .attr('stroke', 'white')
144       .attr('stroke-width', 2)
145       .style('opacity', 0)
146       .attr('d', arc)
147       .each(function(d) {
148         // Store initial state for transitions
149         (this as PieSegment)._current = d;
150       })
151       .call(enter => enter.transition()
152         .duration(transitionDuration)
153         .style('opacity', 1)
154       ),
155     update => update.transition()
156       .duration(transitionDuration)
157       .attrTween('d', function(d) {
158         const segment = this as PieSegment;
159         const interpolate = d3.interpolate(segment._current || d, d);
160         segment._current = interpolate(0);
161         return (t: number) => arc(interpolate(t)) || '';
162       }),
163     exit => exit.transition()
164       .duration(transitionDuration)
165       .style('opacity', 0)
166       .remove()
167   );
168 // Add labels to the segments using join pattern
169 const labelArc = d3.arc<d3.PieArcDatum<DataPoint>>()
170   .innerRadius(this.#radius * 0.6)
171   .outerRadius(this.#radius * 0.6);
172
173 // Labels with join pattern
174 const textElem = pieContainer.selectAll<SVGTextElement>
175   d3.PieArcDatum<DataPoint>>('.label')
176   .data(arcs, d => d.data.id.toString())
177   .join(
178     enter => enter.append('text')
179       .attr('class', 'label')
180       .attr('transform', d => `translate(${labelArc.centroid(d)})`)
181       .attr('dy', '.35em')
182       .attr('text-anchor', 'middle')
183       .style('font-size', '12px')
184       .style('font-weight', 'bold')
185       .style('fill', 'white')
186       .style('opacity', 0)
187       .text(d => Math.round((d.data.value / total) * 100) + '%')
188       .call(enter => enter.transition()
189         .duration(transitionDuration)
190         .style('opacity', 1)
191       ),
192     update => update.transition()
193       .duration(transitionDuration)

```

```

193     .attr('transform', d => `translate(${labelArc.centroid(d)})`)
194     .text(d => Math.round((d.data.value / total) * 100) + '%'),
195     exit => exit.transition()
196     .duration(transitionDuration)
197     .style('opacity', 0)
198     .remove()
199   );
200   // Update legend using join pattern
201   const legendContainer = this.svg.select('.legend-container');
202
203   // Create legend groups with join pattern
204   const legendGroups = legendContainer.selectAll<SVGElement, DataPoint>('.
    legend')
205     .data(this.#data, d => d.id.toString())
206     .join(
207       enter => {
208         const group = enter.append('g')
209         .attr('class', 'legend')
210         .attr('transform', (_, i) => `translate(0, ${i * 20})`)
211         .style('opacity', 0);
212
213         // Add rectangle to legend group
214         group.append('rect')
215         .attr('width', 15)
216         .attr('height', 15)
217         .attr('fill', d => d.color);
218         // Add text to legend group
219         group.append('text')
220         .attr('x', 20)
221         .attr('y', 12)
222         .text(d => `${d.label}: ${d.value}`);
223
224         return group.call(g => g.transition()
225           .duration(transitionDuration)
226           .style('opacity', 1));
227       },
228       update => update.transition()
229         .duration(750)
230         .attr('transform', (_, i) => `translate(0, ${i * 20})`)
231         .call(update => update.select('text')
232           .text(d => `${d.label}: ${d.value}`)
233         ),
234       exit => exit.transition()
235         .duration(transitionDuration)
236         .style('opacity', 0)
237         .remove()
238     );
239   }
240 }
241 // Use our chart composable with the template reference
242 const { isAnimating, toggleAnimation } = useChart(
243   templateRef,
244   AnimatedPieChart,
245   {
246     width: 600,
247     height: 400,
248     margin: { top: 40, right: 40, bottom: 40, left: 40 },
249     radius: 150,
250     colors: d3.schemeCategory10
251   }
252 );
253 </script>

```

```

254 <style scoped>
255 /* Component-specific styles using shared chart styles */
256 @import '../assets/styles/chart-styles.css';
257 </style>
258
259 <style>
260 /* D3 elements use unscoped styles */
261 @import '../assets/styles/d3-element-styles.css';
262 </style>

```

11.5.4. Animierter Donut Chart

Es soll ein Ringdiagramm mit animierten Übergängen erstellt werden. Als Basis sollen wieder das ChartTemplate und die Basisklasse BaseChart benutzt werden. Die Daten sind zufällig generiert und enthalten neben einem Wert auch eine ID, ein Label und eine Farbe. Die Summe aller Werte entspricht 100%. Die Teilflächen des Ringes entsprechen den Anteilen der Werte. Damit ist das zugehörige Template der Komponente AnimatedDonutChart wie folgt:



components/AnimatedDonutChart.vue

```

2 <template>
3   <ChartTemplate
4     ref="templateRef"
5     title="Animiertes Ringdiagramm"
6     wrapperClass="animated-donut-chart"
7     buttonClass="donut-chart-button"
8     :isAnimating="isAnimating"
9     @toggle-animation="toggleAnimation">
10   </ChartTemplate>
11 </template>
12
13 <script setup lang="ts">
14 import { ref } from 'vue';
15 import * as d3 from 'd3';
16 import { BaseChart } from '../utils/charts/BaseChart';
17 import { useChart } from '../utils/charts/useChart';
18 import ChartTemplate from './ChartTemplate.vue';

```

```

18
19 interface DataPoint {
20   id: number;
21   value: number;
22   label: string;
23   color: string;
24 }
25 // Add interface for SVG elements with _current property
26 interface DonutSegment extends SVGPathElement {
27   _current?: d3.PieArcDatum<DataPoint>;
28 }
29 // Reference to the template component
30 const templateRef = ref<InstanceType<typeof ChartTemplate> | null>(null);

```

Das Interface DonutSegment wird genutzt um den aktuellen Zustand der Segmente zu speichern. Dies ist notwendig, um in der d3-Funktion attrTween den aktuellen Zustand des Segments zu interpolieren. Die Funktion interpolate interpoliert zwischen dem aktuellen und dem neuen Zustand des Segments. Der aktuelle Zustand wird in der Eigenschaft _current gespeichert.

```

31 // Concrete implementation for animated donut chart
32 class AnimatedDonutChart extends BaseChart {
33   #data: DataPoint[] = [];
34   #nextId: number = 0;
35   #svg: d3.Selection<SVGSVGElement, unknown, null, undefined> | null = null;
36   #container: HTMLElement | null = null;
37   #innerRadius: number;
38   #outerRadius: number;
39   #colors: readonly string[];
40   constructor(
41     container: HTMLElement | null = null,
42     width: number = 600,
43     height: number = 400,
44     margin: { top: number; right: number; bottom: number; left: number } = { top:
45       40, right: 40, bottom: 40, left: 40 },
46     innerRadius: number = 80,
47     outerRadius: number = 150,
48     colors: readonly string[] = d3.schemeCategory10
49   ) {
50     super(width, height, margin, 2000); // Slower animation speed of 2000ms
51     this.#container = container;
52     this.#innerRadius = innerRadius;
53     this.#outerRadius = outerRadius;
54     this.#colors = colors;
55   }
56   initializeData(): void {
57     // Create 5 random segments for the donut
58     this.#data = Array.from({ length: 5 }, (_, i) => ({
59       id: this.#nextId++,
60       value: Math.floor(Math.random() * 50) + 10, // Random value between 10-60
61       label: `Segment ${i + 1}`,
62       color: this.#colors[i % this.#colors.length]
63     }));
64   }
65   updateData(): void {
66     // Update existing segment values and occasionally add/remove segments
67     const newData = [...this.#data];
68     // Randomly update values
69     newData.forEach(item => {
70       item.value = Math.max(5, item.value + (Math.random() > 0.5 ? 1 : -1) * (Math
71         .floor(Math.random() * 10)));
72     });
73   }
74 }

```

```

72 // Occasionally add a new segment (20% chance if less than 8 segments)
73 if (Math.random() > 0.8 && newData.length < 8) {
74   newData.push({
75     id: this.#nextId++,
76     value: Math.floor(Math.random() * 40) + 10,
77     label: `Segment ${newData.length + 1}`,
78     color: this.#colors[newData.length % this.#colors.length]
79   });
80 }
81 // Occasionally remove a segment (20% chance if more than 3 segments)
82 if (Math.random() > 0.8 && newData.length > 3) {
83   const indexToRemove = Math.floor(Math.random() * newData.length);
84   newData.splice(indexToRemove, 1);
85 }
86 this.#data = newData;
87 }
88 renderChart(): void {
89   if (!this.#container) return;
90
91   // Create or select SVG element
92   let svg = d3.select(this.#container).select<SVGSVGElement>('svg');
93   if (svg.empty()) {
94     svg = d3.select(this.#container)
95       .append('svg')
96       .attr('width', this.width)
97       .attr('height', this.height);
98
99     // Add group for donut segments
100    svg.append('g')
101      .attr('class', 'donut-segments-container')
102      .attr('transform', `translate(${this.width / 2}, ${this.height - this.
        margin.bottom})`); // Position at bottom center
103    // Add group for donut
104    svg.append('g')
105      .attr('class', 'donut-container')
106      .attr('transform', `translate(${this.width / 2}, ${this.height - this.
        margin.bottom})`); // Position at bottom center
107
108    // Add group for labels
109    svg.append('g')
110      .attr('class', 'labels-container')
111      .attr('transform', `translate(${this.width / 2}, ${this.height - this.
        margin.bottom})`);
112  }
113
114  this.#svg = svg;
115  this.updateChart();
116 }
117 protected updateChart(): void {
118   if (!this.#svg || this.#data.length === 0) return;
119
120   const transitionDuration = 500; // Duration for transitions
121
122   // Calculate total for percentage
123   const total = d3.sum(this.#data, d => d.value);
124
125   // Create pie layout that only covers half a circle (0 to 180 degrees)
126   const pie = d3.pie<DataPoint>()
127     .value(d => d.value)
128     .startAngle(-90 * (Math.PI / 180)) // -90 degrees in radians
129     .endAngle(90 * (Math.PI / 180)) // 90 degrees in radians
130     .sort(null);

```

```

131 // Create arc generator
132 const arc = d3.arc<d3.PieArcDatum<DataPoint>>()
133   .innerRadius(this.#innerRadius)
134   .outerRadius(this.#outerRadius)
135   .cornerRadius(4)
136   .padAngle(0.02);
137
138 // Create outer arc for label positioning
139 const outerArc = d3.arc<d3.PieArcDatum<DataPoint>>()
140   .innerRadius(this.#outerRadius * 1.1)
141   .outerRadius(this.#outerRadius * 1.1);
142
143 // Generate pie chart data
144 const arcs = pie(this.#data);
145 const donutContainer = this.#svg.select('.donut-container');
146 // Update donut segments using join pattern
147 donutContainer.selectAll<DonutSegment, d3.PieArcDatum<DataPoint>>('.segment')
148   .data(arcs, d => d.data.id.toString())
149   .join(
150     enter => enter.append('path')
151       .attr('class', 'segment')
152       .attr('fill', d => d.data.color)
153       .attr('stroke', 'white')
154       .attr('stroke-width', 2)
155       .style('opacity', 0)
156       .attr('d', arc)
157       .each(function(d) {
158         // Store initial state for transitions
159         (this as DonutSegment)._current = d;
160       })
161       .call(enter => enter.transition()
162         .duration(transitionDuration)
163         .style('opacity', 1)
164       ),
165     update => update.transition()
166       .duration(transitionDuration)
167       .attrTween('d', function(d) {
168         const segment = this as DonutSegment;
169         const interpolate = d3.interpolate(segment._current || d, d);
170         segment._current = interpolate(0);
171         return (t: number) => arc(interpolate(t)) || '';
172       }),
173     exit => exit.transition()
174       .duration(transitionDuration)
175       .style('opacity', 0)
176       .remove()
177   );
178 // Update text labels using join pattern
179 const labelContainer = this.#svg.select('.labels-container');
180 labelContainer.selectAll<SVGTextElement, d3.PieArcDatum<DataPoint>>('.label-
181   text')
182   .data(arcs, d => d.data.id.toString())
183   .join(
184     enter => enter.append('text')
185       .attr('class', 'label-text')
186       .style('text-anchor', d => {
187         const centroid = outerArc.centroid(d);
188         return centroid[0] < 0 ? 'end' : 'start';
189       })
190       .style('opacity', 0)
191       .attr('transform', d => {
192         const pos = outerArc.centroid(d);

```

```

192     pos[0] = pos[0] * 1.1; // Move labels further out
193     return `translate(${pos})`;
194   })
195   .text(d => `${d.data.label}: ${Math.round((d.data.value / total) * 100)}%`)
196   .call(enter => enter.transition()
197     .duration(transitionDuration)
198     .style('opacity', 1)
199   ),
200   update => update.transition()
201     .duration(transitionDuration)
202     .attr('transform', d => {
203       const pos = outerArc.centroid(d);
204       pos[0] = pos[0] * 1.1; // Move labels further out
205       return `translate(${pos})`;
206     })
207     .style('text-anchor', d => {
208       const centroid = outerArc.centroid(d);
209       return centroid[0] < 0 ? 'end' : 'start';
210     })
211     .attr('transform', d => {
212       const pos = outerArc.centroid(d);
213       pos[0] = pos[0] * 1.1; // Move labels further out
214       return `translate(${pos})`;
215     })
216     .text(d => `${d.data.label}: ${Math.round((d.data.value / total) * 100)}%`),
217   exit => exit.transition()
218     .duration(transitionDuration)
219     .style('opacity', 0)
220     .remove()
221 );
222
223 // Update polylines using join pattern
224 labelContainer.selectAll<SVGPolylineElement, d3.PieArcDatum<DataPoint>>>('label-line')
225   .data(arcs, d => d.data.id.toString())
226   .join(
227     enter => enter.append('polyline')
228       .attr('class', 'label-line')
229       .style('opacity', 0)
230       .attr('points', d => {
231         const arcPos = arc.centroid(d);
232         const outerPos = outerArc.centroid(d);
233         const posX = outerPos[0] * 1.1;
234         outerPos[0] = posX;
235         return `${arcPos[0]},${arcPos[1]} ${outerPos[0]},${outerPos[1]} ${posX},
236           ${outerPos[1]}`;
237       })
238       .attr('fill', 'none')
239       .attr('stroke', '#999')
240       .attr('stroke-width', 1)
241       .call(enter => enter.transition()
242         .duration(transitionDuration)
243         .style('opacity', 0.5)
244       ),
245     update => update.transition()
246       .duration(transitionDuration)
247       .attr('points', d => {
248         const arcPos = arc.centroid(d);
249         const outerPos = outerArc.centroid(d);
250         const posX = outerPos[0] * 1.1;

```

```

250         outerPos[0] = posX;
251         return `${arcPos[0]},${arcPos[1]} ${outerPos[0]},${outerPos[1]} ${posX}
           },${outerPos[1]}`;
252     }},
253     exit => exit.transition()
254         .duration(transitionDuration)
255         .style('opacity', 0)
256         .remove()
257     );
258 }
259 }
260 // Use our chart composable with the template reference
261 const { isAnimating, toggleAnimation } = useChart(
262     templateRef,
263     AnimatedDonutChart, {
264         width: 600,
265         height: 400,
266         margin: { top: 40, right: 40, bottom: 40, left: 40 },
267         innerRadius: 80,
268         outerRadius: 150,
269         colors: d3.schemeCategory10
270     }
271 );
272 </script>
273 <style scoped>
274 /* Component-specific styles using shared chart styles */
275 @import '../assets/styles/chart-styles.css';
276 </style>
277
278 <style>
279 /* D3 elements use unscoped styles */
280 @import '../assets/styles/d3-element-styles.css';
281 </style>

```

11.6. Scatterplots

11.6.1. Einbindung

- Scatterplots sind eine Art von Diagramm, das verwendet wird, um die Beziehung zwischen zwei Variablen darzustellen
- Sie bestehen aus Punkten, die auf einem Koordinatensystem angeordnet sind
- Jeder Punkt repräsentiert einen Datenpunkt mit zwei Werten
- Scatterplots können verwendet werden, um Trends, Muster und Ausreißer in den Daten zu identifizieren

components/ScatterSelector.vue

```

1 <template>
2   <div class="scatter-selector">
3     <h3>Punktdiagramm Beispiele</h3>
4     <div class="viz-description">
5       <p>Verschiedene Arten von Punktdiagrammen mit D3.js</p>
6     </div>
7
8     <!-- Implementation selection tabs -->
9     <div class="implementation-tabs">

```

```

10     <button
11       :class=["impl-button", { active: activeImpl === 'basic-scatter' }]"
12       @click="activeImpl = 'basic-scatter'">
13       Basis Scatter Plot
14     </button>
15     <button
16       :class=["impl-button", { active: activeImpl === 'bubble-chart' }]"
17       @click="activeImpl = 'bubble-chart'">
18       Bubble Chart
19     </button>
20     <button
21       :class=["impl-button", { active: activeImpl === 'grouped-scatter' }]"
22       @click="activeImpl = 'grouped-scatter'">
23       Scatter Matrix
24     </button>
25   </div>

```

Die Komponente enthält drei Buttons zur Auswahl der Implementierung. Wird ein Button gedrückt, wird die Variable `activeImpl` gesetzt. Diese Variable wird in der Darstellung der Implementierung verwendet. Zugleich wird der Button als ausgewählt hervorgehoben.

```

26   <!-- Implementation content -->
27   <div class="implementation-container">
28     <div v-if="activeImpl === 'basic-scatter'">
29       <h4>Basis Scatter Plot:</h4>
30       <p class="impl-description">Einfache Darstellung von Datenpunkten in einem
31         X-Y-Koordinatensystem.</p>
32       <div class="scatter-visualization">
33         <ScatterPlot />
34       </div>
35     </div>
36     <div v-else-if="activeImpl === 'bubble-chart'">
37       <h4>Bubble Chart:</h4>
38       <p class="impl-description">Scatter Plot mit unterschiedlich großen
39         Punkten, die eine dritte Dimension darstellen.</p>
40       <div class="scatter-visualization">
41         <BubbleChart />
42       </div>
43     </div>
44     <div v-else>
45       <h4>Scatter Matrix:</h4>
46       <p class="impl-description">Scatter Matrix für paarweise Korrelation von
47         Attributen
48         Gezeigt werden die Daten von Kristen Gorman für Pinguine nahe der Palmer
49         Station in der Antarktis.</p>
50       <div class="scatter-visualization">
51         <ScatterMatrix />
52       </div>
53     </div>
54   </div>
55 </template>

```

Je nach Auswahl wird die entsprechende Komponente dargestellt. Es stehen Die Komponenten `ScatterPlot`, `BubbleChart` und `ScatterMatrix` zur Verfügung.

```

54 <setup lang="ts">
55 import { ref } from 'vue';
56
57 // Default implementation

```

```
58 | const activeImpl = ref('basic-scatter');
59 | </script>
```

Im Script-Teil wird die Variable `activeImpl` initialisiert. Ihr Wert kann durch drücken der Buttons geändert werden.

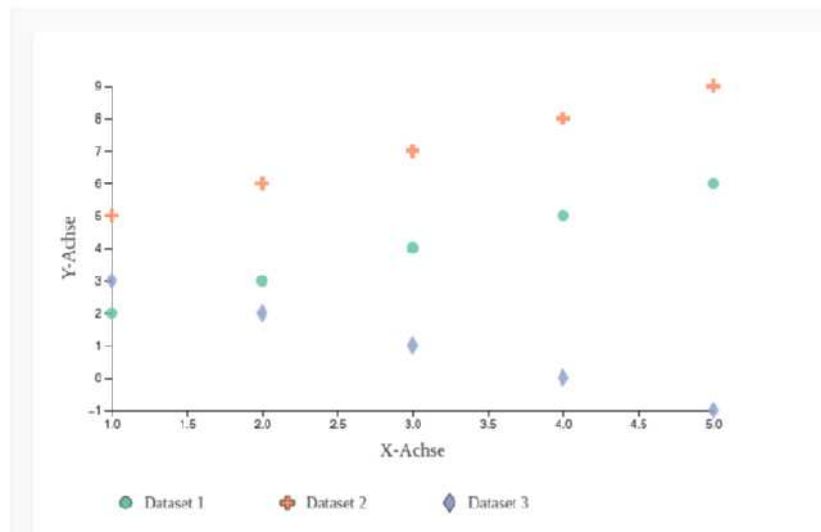
```
60 | <style scoped>
61 | .scatter-selector {
62 |   width: 100%;
63 |   max-width: 100%;
64 |   overflow-x: hidden;
65 | }
66 |
67 | .viz-description {
68 |   margin-bottom: 1.5rem;
69 |   padding-bottom: 1rem;
70 |   border-bottom: 1px solid #e9ecef;
71 |   color: #6c757d;
72 | }
73 | .implementation-tabs {
74 |   display: flex;
75 |   gap: 0.5rem;
76 |   margin: 1rem 0;
77 |   border-bottom: 1px solid #dee2e6;
78 |   padding-bottom: 0.5rem;
79 |   flex-wrap: wrap;
80 | }
81 |
82 | .impl-button {
83 |   background-color: transparent;
84 |   border: none;
85 |   padding: 0.5rem 1rem;
86 |   cursor: pointer;
87 |   font-size: 0.9rem;
88 |   border-radius: 4px 4px 0 0;
89 |   transition: all 0.2s ease;
90 |   border-bottom: 2px solid transparent;
91 | }
92 |
93 | .impl-button:hover {
94 |   background-color: #f8f9fa;
95 | }
96 |
97 | .impl-button.active {
98 |   color: #007bff;
99 |   border-bottom-color: #007bff;
100 |   font-weight: 500;
101 | }
102 |
103 | .implementation-container {
104 |   padding-top: 1rem;
105 |   width: 100%;
106 |   overflow-x: hidden;
107 | }
108 |
109 |
110 |
111 |
112 |
113 | h4 {
114 |   margin-top: 0;
115 |   margin-bottom: 0.5rem;
116 |   font-size: 1.1rem;
117 |   color: #495057;
118 | }
119 |
120 | .placeholder {
121 |   display: flex;
122 |   flex-direction: column;
123 |   justify-content: center;
124 |   align-items: center;
125 |   background-color: #f8f9fa;
126 |   border: 2px dashed #dee2e6;
127 |   border-radius: 4px;
128 |   padding: 2rem;
129 |   text-align: center;
130 |   color: #6c757d;
131 |   font-style: italic;
132 |   height: 300px;
133 |   max-width: 100%;
134 |   overflow: hidden;
135 |   box-sizing: border-box;
136 |
137 | .scatter-visualization {
138 |   width: 100%;
139 |   min-height: 400px;
140 |   border: 1px solid #e9ecef;
141 |   border-radius: 4px;
142 |   padding: 1rem;
143 |   background-color: #f8f9fa;
144 |   max-width: 100%;
145 |   overflow: hidden;
146 |   box-sizing: border-box;
147 | }
148 | </style>
```

11.6.2. Einfacher Scatterplot

Basis Scatter Plot:

Einfache Darstellung von Datenpunkten in einem X-Y-Koordinatensystem.

Wir erstellen einen einfachen Scatterplot mit D3.js. Der Scatterplot wird in einer Vue-Komponente dargestellt. Die Darstellung benutzt wieder die Basisklasse BaseChart und wird in der Datei components/Scatterplot.vue implementiert. Die zugehörigen Daten werden aus der Datei `public/data/scatter_data.csv` geladen.



Scatterplot mit D3.js

public/data/scatter_data.csv

Die darzustellenden Werte sind in dieser Datei als CSV-Datei gespeichert. Die Datei wird mit dem `d3.csv`-Befehl geladen. Die Daten werden in der Methode `loadData` geladen und in der Methode `draw` dargestellt. Jede Zeile der CSV-Datei wird als Punkt dargestellt. Der Wert `dataset` wird genutzt, um die Punkte zu färben und damit zu gruppieren.

1	x,y,dataset	9	3,7,Dataset 2
2	1,2,Dataset 1	10	4,8,Dataset 2
3	2,3,Dataset 1	11	5,9,Dataset 2
4	3,4,Dataset 1	12	1,3,Dataset 3
5	4,5,Dataset 1	13	2,2,Dataset 3
6	5,6,Dataset 1	14	3,1,Dataset 3
7	1,5,Dataset 2	15	4,0,Dataset 3
8	2,6,Dataset 2	16	5,-1,Dataset 3

components/Scatterplot.vue

```

1 <template>
2   <div class="scatterplot-wrapper">
3     <div ref="containerRef"></div>
4   </div>
5 </template>

```

Die Darstellung erfolgt durch einfaches Einfügen des Containers `containerRef`. Da der Inhalt des Containers durch D3.js erzeugt wird, erfolgt das Einfügen erst nachdem das virtuelle DOM komplett in das tatsächliche DOM eingefügt wurde.

```

6 <script setup lang="ts">
7 import { ref, onMounted } from 'vue';
8 import * as d3 from 'd3';
9 import { BaseChart } from '../utils/charts/BaseChart';

```

```

10 interface DataPoint {
11   x: number;
12   y: number;
13   dataset: string;
14 }
15 // D3 symbol type enum
16 enum D3SymbolType {
17   Circle = 'circle',
18   Cross = 'cross',
19   Diamond = 'diamond',
20   Square = 'square',
21   Star = 'star',
22   Triangle = 'triangle',
23   Wye = 'wye'
24 }

```

Die Datenpunkte enthalten eine (x,y)-Koordinate und den Namen des Datensatzes. Anhand des Namens wird ein Symbol und eine Farbe zur Darstellung des Punktes ausgewählt. Eine Reihe verfügbarer D3-Symbole sind in der Enum D3SymbolType definiert.

```

25 const d3SymbolMap = {
26   [D3SymbolType.Circle]: d3.symbolCircle,
27   [D3SymbolType.Cross]: d3.symbolCross,
28   [D3SymbolType.Diamond]: d3.symbolDiamond,
29   [D3SymbolType.Square]: d3.symbolSquare,
30   [D3SymbolType.Star]: d3.symbolStar,
31   [D3SymbolType.Triangle]: d3.symbolTriangle,
32   [D3SymbolType.Wye]: d3.symbolWye
33 };
34
35 interface DatasetInfo {
36   label: string;
37   color: string;
38   symbol: D3SymbolType;
39 }

```

In d3SymbolMap werden die Enum-Werte den d3-Symbolen zugeordnet. Damit die Eigenschaften in diesem Objekt über die Enum-Werte als Index angesprochen werden können, sind die Enum-Werte als Strings definiert. Damit werden die Strings als Index in die d3SymbolMap verwendet. Ohne diese Zuordnung führt dies sonst zu einem TypeScript-Fehler.

```

40 // Color palette for datasets
41 const colorPalette = d3.schemeSet2;
42
43 const width = 600;
44 const height = 400;
45 const margin = { top: 30, right: 30, bottom: 60, left: 50 };
46 const legendHeight = 30;
47
48 class ScatterPlot extends BaseChart {
49   #svg: d3.Selection<SVGSVGElement, unknown, null, undefined> | null = null;
50   #data: DataPoint[][] = [];
51   #datasetInfos: DatasetInfo[] = [];
52   #visible: boolean[] = [];
53   #container: HTMLElement | null = null;
54   #dataSource: string;
55
56   constructor(container: HTMLElement | null, dataSource: string) {
57     super(width, height, margin, 0);
58     this.#container = container;
59     this.#dataSource = dataSource;
60   }
61
62   async initializeData() {
63     await this.loadData();
64     this.renderChart();
65   }
66
67   async updateData() {
68     await this.loadData();
69   }

```

```

68     this.updateChart();
69 }

70 async loadData() {
71     const rows = await d3.csv(this.#dataSource, (d: any) => ({
72         x: +d['x'],
73         y: +d['y'],
74         dataset: d['dataset'] || '',
75     }));
76     // Get unique dataset labels
77     const uniqueLabels = Array.from(new Set((rows as DataPoint[]).map(d => d.
78         dataset)));
79     // Assign symbols and colors in round-robin fashion
80     this.#datasetInfos = uniqueLabels.map((label, i) => ({
81         label,
82         color: colorPalette[i % colorPalette.length],
83         symbol: Object.values(D3SymbolType)[i % Object.values(D3SymbolType).length]
84     }));

85     // Group data by dataset label
86     this.#data = this.#datasetInfos.map(dsInfo =>
87         (rows as DataPoint[]).filter(d => d.dataset === dsInfo.label)
88     );
89     // Set all visible by default (or preserve previous state if possible)
90     if (this.#visible.length !== this.#datasetInfos.length) {
91         this.#visible = Array(this.#datasetInfos.length).fill(true);
92     }

93     setDataSource(newSource: string) {
94         this.#dataSource = newSource;
95     }

96

97     renderChart() {
98         if (!this.#container) return;
99         let svg = d3.select(this.#container).select<SVGSVGElement>('svg');
100         if (svg.empty()) {
101             svg = d3.select(this.#container)
102                 .append('svg')
103                 .attr('width', this.width)
104                 .attr('height', this.height)
105                 .attr('viewBox', `0 0 ${this.width} ${this.height}`)
106                 .attr('style', 'background: #fff;');
107         } else {
108             svg.attr('width', this.width)
109                 .attr('height', this.height)
110                 .attr('viewBox', `0 0 ${this.width} ${this.height}`)
111                 .attr('style', 'background: #fff;');
112         }
113         this.#svg = svg;
114         this.updateChart();
115     }

116     updateChart() {
117         if (!this.#svg) return;
118         const svg = this.#svg;
119         svg.selectAll('*').remove();
120         svg
121             .attr('width', this.width)
122             .attr('height', this.height)
123             .attr('viewBox', `0 0 ${this.width} ${this.height}`)

```

```

124     .attr('style', 'background: #fff;');
125
126     const allVisible = this.#data
127       .map((ds, i) => (this.#visible[i] ? ds : []))
128       .flat();
129
130     if (allVisible.length > 0) {
131       const xScale = d3
132         .scaleLinear()
133         .domain(d3.extent(allVisible, d => d.x) as [number, number])
134         .nice()
135         .range([this.margin.left, this.width - this.margin.right]);
136
137       const yExtent = d3.extent(allVisible, d => d.y) as [number, number];
138       const yMin = Math.min(0, yExtent[0]);
139       const yMax = Math.max(0, yExtent[1]);
140       const yScale = d3
141         .scaleLinear()
142         .domain([yMin, yMax])
143         .nice()
144         .range([this.height - this.margin.bottom - legendHeight, this.margin.top])
145         ;
146
147       svg
148         .append('g')
149         .attr('transform', `translate(0,${this.height - this.margin.bottom -
150           legendHeight})`)
151         .call(d3.axisBottom(xScale));
152
153       svg
154         .append('g')
155         .attr('transform', `translate(${this.margin.left},0)`)
156         .call(d3.axisLeft(yScale));
157
158       svg.append('text')
159         .attr('class', 'x axis-label')
160         .attr('text-anchor', 'middle')
161         .attr('x', this.margin.left + (this.width - this.margin.left - this.margin
162           .right) / 2)
163         .attr('y', this.height - this.margin.bottom - legendHeight + 40)
164         .attr('font-size', 16)
165         .attr('fill', '#333')
166         .text('X-Achse');
167
168       svg.append('text')
169         .attr('class', 'y axis-label')
170         .attr('text-anchor', 'middle')
171         .attr('transform', `rotate(-90)`)
172         .attr('x', -this.margin.top - (this.height - this.margin.top - this.margin
173           .bottom - legendHeight) / 2)
174         .attr('y', 18)
175         .attr('font-size', 16)
176         .attr('fill', '#333')
177         .text('Y-Achse');
178
179       this.#datasetInfos.forEach((ds, i) => {
180         if (!this.#visible[i]) return;
181         const symbolGen = d3.symbol().type(d3.symbolMap[ds.symbol]).size(80);
182         svg
183           .append('g')
184           .selectAll('path')
185           .data(this.#data[i])
186           .join('path')

```

```

178         .attr('d', symbolGen)
179         .attr('transform', d =>
180             `translate(${xScale(d.x)},${yScale(d.y)})`)
181         .attr('fill', ds.color)
182         .attr('opacity', 0.85);
183     });
184 }

185 const legendG = svg.append('g')
186     .attr('class', 'd3-legend')
187     .attr('transform',
188         `translate(${this.margin.left},${this.height - legendHeight + 10})`);
189
190 const legendItemWidth = 140;
191 const legendItemHeight = 20;

192 const legend = legendG.selectAll('g.legend-item')
193     .data(this.#datasetInfos)
194     .join('g')
195     .attr('class', 'legend-item')
196     .attr('cursor', 'pointer')
197     .attr('opacity', (d, i) => this.#visible[i] ? 1 : 0.4)
198     .attr('transform', (d, i) =>
199         `translate(${i * legendItemWidth},0)`)
200     .on('click', (event, d) => {
201         const idx = this.#datasetInfos
202             .findIndex(ds => ds.label === d.label);
203         if (idx !== -1) {
204             this.#visible[idx] = !this.#visible[idx];
205             this.updateChart();
206         }
207     });

208 legend.each(function(ds, i) {
209     const g = d3.select(this);
210     let symbolGen = d3.symbol().type(d3SymbolMap[ds.symbol]).size(80);
211     g.append('path')
212         .attr('d', symbolGen)
213         .attr('fill', ds.color)
214         .attr('stroke', '#333')
215         .attr('stroke-width', 1)
216         .attr('transform', `translate(12,${legendItemHeight / 2})`);
217     g.append('text')
218         .text(ds.label)
219         .attr('x', 28)
220         .attr('y', legendItemHeight / 2 + 5)
221         .attr('font-size', 14)
222         .attr('fill', '#333');
223     });
224 }
225 }

226 const containerRef = ref<HTMLElement | null>(null);
227 let scatterPlot: ScatterPlot | null = null;
228
229 // Example: dynamic data source
230 const dataSource = ref('/data/scatter_data.csv');
231
232 onMounted(() => {
233     scatterPlot = new ScatterPlot(containerRef.value, dataSource.value);
234     scatterPlot.initializeData();

```

```

235 | });
236 | </script>

237 | <style scoped>
238 | .scatterplot-wrapper {
239 |   width: 100%;
240 |   max-width: 100%;
241 |   background: #fff;
242 |   border-radius: 8px;
243 |   box-shadow: 0 2px 4px rgba(0,0,0,0.07);
244 |   padding: 1rem;
245 |   margin-bottom: 1rem;
246 | }
247 | svg {
248 |   width: 100%;
249 |   height: 400px;
250 |   display: block;
251 | }
252 | </style>

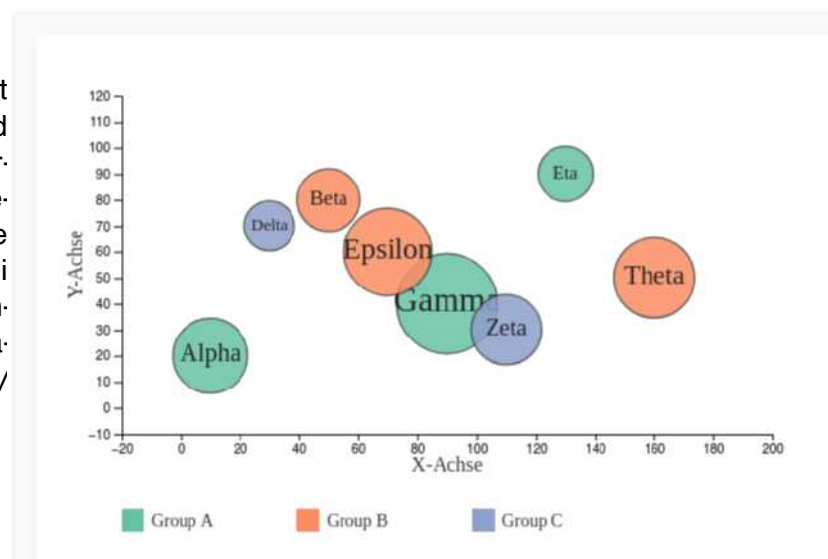
```

11.6.3. Bubblechart

Bubble Chart:

Scatter Plot mit unterschiedlich großen Punkten, die eine dritte Dimension darstellen.

Wir erstellen einen Bubblechart mit D3.js. Der Bubblechart wird in einer Vue-Komponente dargestellt. Die Darstellung benutzt wieder die Basisklasse BaseChart und wird in der Datei components/Bubblechart.vue implementiert. Die zugehörigen Daten werden aus der Datei src/data/bubble_data.csv geladen.



Bubblechart mit D3.js

src/components/Bubblechart.vue

```

1 | <template>
2 |   <div class="bubblechart-wrapper">
3 |     <div ref="containerRef"></div>
4 |   </div>
5 | </template>

6 | <script setup lang="ts">
7 | import { ref, onMounted } from 'vue';
8 | import * as d3 from 'd3';
9 | import { BaseChart } from '../utils/charts/BaseChart';
10 |

```

```

11 interface BubbleData {
12   x: number;
13   y: number;
14   r: number;
15   label: string;
16   category: string;
17 }
18
19 type AugmentedBubbleData = BubbleData & { visible: boolean };
20
21 const width = 600;
22 const height = 400;
23 const margin = { top: 30, right: 30, bottom: 60, left: 50 };
24
25 class BubbleChartClass extends BaseChart {
26   #svg: d3.Selection<SVGSVGElement, unknown, null, undefined> | null = null;
27   #data: BubbleData[] = [];
28   #container: HTMLElement | null = null;
29   #dataSource: string;
30   #colorScale: d3.ScaleOrdinal<string, string, never>;
31   #visibleCategories: Set<string> = new Set();
32   #tooltip: d3.Selection<HTMLDivElement, unknown, null, undefined> | null = null;
33
34   constructor(container: HTMLElement | null, dataSource: string) {
35     super(width, height, margin, 0);
36     this.#container = container;
37     this.#dataSource = dataSource;
38     this.#colorScale = d3.scaleOrdinal(d3.schemeSet2);
39   }
40
41   async initializeData() {
42     await this.loadData();
43     this.renderChart();
44   }
45
46   async updateData() {
47     await this.loadData();
48     this.updateChart();
49   }
50
51   setDataSource(newSource: string) {
52     this.#dataSource = newSource;
53   }
54
55   async loadData() {
56     const rows = await d3.csv(this.#dataSource, (d: any) => ({
57       x: +d['x'],
58       y: +d['y'],
59       r: +d['r'],
60       label: d['label'] || '',
61       category: d['category'] || '',
62     }));
63     this.#data = rows as BubbleData[];
64     // Update color scale domain
65     const categories = Array.from(new Set(this.#data.map(d => d.category)));
66     this.#colorScale.domain(categories);
67     if (this.#visibleCategories.size === 0) {
68       categories.forEach(c => this.#visibleCategories.add(c));
69     }
70   }

```

```

68 renderChart() {
69   if (!this.#container) return;
70   let svg = d3.select(this.#container).select<SVGSVGElement>('svg');
71   if (svg.empty()) {
72     svg = d3.select(this.#container)
73       .append('svg')
74       .attr('width', this.width)
75       .attr('height', this.height)
76       .attr('viewBox', `0 0 ${this.width} ${this.height}`)
77       .attr('style', 'background: #fff;');
78   } else {
79     svg.attr('width', this.width)
80       .attr('height', this.height)
81       .attr('viewBox', `0 0 ${this.width} ${this.height}`)
82       .attr('style', 'background: #fff;');
83   }
84   this.#svg = svg;

85   // Tooltip div
86   if (!this.#tooltip) {
87     this.#tooltip = d3.select(this.#container)
88       .append('div')
89       .attr('class', 'bubble-tooltip')
90       .style('position', 'absolute')
91       .style('pointer-events', 'none')
92       .style('background', 'rgba(255,255,255,0.95)')
93       .style('border', '1px solid #888')
94       .style('border-radius', '6px')
95       .style('padding', '8px 12px')
96       .style('font-size', '14px')
97       .style('color', '#222')
98       .style('box-shadow', '0 2px 8px rgba(0,0,0,0.15)')
99       .style('display', 'none');
100   }
101   this.updateChart();
102 }

103 updateChart() {
104   if (!this.#svg) return;
105   const svg = this.#svg;
106   // Remove only axes and legend, not bubbles group
107   svg.selectAll('g.axis').remove();
108   svg.selectAll('g.bubble-legend').remove();
109
110   if (this.#data.length === 0) return;
111
112   const transitionTime = 500;
113   const allCategories = this.#colorScale.domain();
114   const bubblesData: AugmentedBubbleData[] = this.#data.map(d => ({...d, visible
    : this.#visibleCategories.has(d.category)}));
115   const filteredData = bubblesData.filter(d => d.visible);
116   const xMin = d3.min(filteredData, d => d.x - d.r) ?? 0;
117   const xMax = d3.max(filteredData, d => d.x + d.r) ?? 1;
118   const yMin = d3.min(filteredData, d => d.y - d.r) ?? 0;
119   const yMax = d3.max(filteredData, d => d.y + d.r) ?? 1;

120   const xScale = d3.scaleLinear()
121     .domain([xMin, xMax])
122     .nice()
123     .range([this.margin.left, this.width - this.margin.right]);
124   const yScale = d3.scaleLinear()

```

```

125     .domain([yMin, yMax])
126     .nice()
127     .range([this.height - this.margin.bottom - 40, this.margin.top]);
128     const yBottom = this.height - this.margin.bottom - 10;
129     // Axes
130     svg.append('g')
131         .attr('class', 'axis')
132         .attr('transform', `translate(0,${this.height - this.margin.bottom - 40})`)
133         .call(d3.axisBottom(xScale));
134     svg.append('g')
135         .attr('class', 'axis')
136         .attr('transform', `translate(${this.margin.left},0)`)
137         .call(d3.axisLeft(yScale));

138     svg.append('text') // X Axis Title
139         .attr('class', 'x axis-label')
140         .attr('text-anchor', 'middle')
141         .attr('x', this.margin.left + (this.width - this.margin.left - this.margin.
            right) / 2)
142         .attr('y', this.height - this.margin.bottom - 10)
143         .attr('font-size', 16)
144         .attr('fill', '#333')
145         .text('X-Achse');

146
147     svg.append('text') // Y Axis Title
148         .attr('class', 'y axis-label')
149         .attr('text-anchor', 'middle')
150         .attr('transform', `rotate(-90)`)
151         .attr('x', -this.margin.top - (this.height - this.margin.top - this.margin.
            bottom - 40) / 2)
152         .attr('y', 18)
153         .attr('font-size', 16)
154         .attr('fill', '#333')
155         .text('Y-Achse');

156     // Bubbles group (persistent)
157     let bubblesGroup: d3.Selection<SVGGElement, unknown, null, undefined> = svg.
        select('g.bubbles');
158     if (bubblesGroup.empty()) {
159         bubblesGroup = svg.append('g').attr('class', 'bubbles');
160     }
161     const bubbles = bubblesGroup.selectAll('g.bubble')
162         .data(bubblesData, (d, i) => (d as AugmentedBubbleData).label + (d as
            AugmentedBubbleData).category)
163     .join(
164         (enter) => {
165             const g = enter.append('g')
166                 .attr('class', 'bubble')
167                 .attr('transform', (d: AugmentedBubbleData) => `translate(${xScale(d.x
                    )},${yBottom})`)
168                 .style('opacity', 0);

169             g.append('circle')
170                 .attr('r', (d: AugmentedBubbleData) => d.r)
171                 .attr('fill', (d: AugmentedBubbleData) =>
                    this.#colorScale(d.category))
172                 .attr('stroke', '#333')
173                 .attr('stroke-width', 1.2)
174                 .attr('opacity', 0.85);
175             g.append('text')
176                 .text((d: AugmentedBubbleData) => d.label)
177

```

```

178     .attr('text-anchor', 'middle')
179     .attr('dominant-baseline', 'middle')
180     .attr('font-size',
181         (d: AugmentedBubbleData) => Math.max(10, d.r / 1.5))
182     .attr('fill', '#222');
183     g.filter((d: AugmentedBubbleData) => d.visible)
184     .transition().duration(transitionTime)
185     .attr('transform', (d: AugmentedBubbleData) =>
186         `translate(${xScale(d.x)},${yScale(d.y)})`)
187     .style('opacity', 1);
188     return g;
189 },

190 (update) => update
191     .transition().duration(transitionTime)
192     .attr('transform', (d: AugmentedBubbleData) => d.visible
193         ? `translate(${xScale(d.x)},${yScale(d.y)})`
194         : `translate(${xScale(d.x)},${yBottom})`)
195     .style('opacity', (d: AugmentedBubbleData) => d.visible ? 1 : 0),
196 (exit) => exit
197     .transition().duration(transitionTime)
198     .attr('transform', (d: AugmentedBubbleData) => `translate(${xScale(d.x)}
199         ,${yBottom})`)
200     .style('opacity', 0)
201     .remove()
202 );

202 // Tooltip logic: show on group hover, hide on mouseleave
203 bubblesGroup.selectAll('g.bubble')
204     .on('mousemove', (event, d) => {
205         const datum = d as BubbleData;
206         if (!this.#tooltip) return;
207         this.#tooltip
208             .style('display', 'block')
209             .style('left', (event.offsetX + 20) + 'px')
210             .style('top', (event.offsetY - 10) + 'px')
211             .html(
212                 `<b>${datum.label}</b><br>x: ${datum.x}<br>y: ${datum.y}<br>r: ${datum
213                     .r}<br>category: ${datum.category}`
214             );
215     })
216     .on('mouseleave', () => {
217         if (this.#tooltip) this.#tooltip.style('display', 'none');
218     });

218 // Legend below the graph
219 const legendG = svg.append('g')
220     .attr('class', 'bubble-legend')
221     .attr('transform', `translate(${this.margin.left},${this.height - this.
222         margin.bottom + 20})`);
223 const legendItemWidth = 140;
224 const legend = legendG.selectAll('g.legend-item')
225     .data(allCategories)
226     .join('g')
227     .attr('class', 'legend-item')
228     .attr('cursor', 'pointer')
229     .attr('opacity', d => this.#visibleCategories.has(d) ? 1 : 0.4)
230     .attr('transform', (d, i) => `translate(${i * legendItemWidth},0)`

230     .on('click', (event, d) => {
231         if (this.#visibleCategories.has(d)) {

```

```

232         this.#visibleCategories.delete(d);
233     } else {
234         this.#visibleCategories.add(d);
235     }
236     this.updateChart();
237 });
238 legend.append('rect')
239     .attr('width', 18)
240     .attr('height', 18)
241     .attr('fill', d => this.#colorScale(d));
242 legend.append('text')
243     .text(d => d)
244     .attr('x', 24)
245     .attr('y', 14)
246     .attr('font-size', 14)
247     .attr('fill', '#333');
248 }
249 }

250 const containerRef = ref<HTMLElement | null>(null);
251 let bubbleChart: BubbleChartClass | null = null;
252
253 onMounted(() => {
254     bubbleChart = new BubbleChartClass(containerRef.value, '/data/bubble_data.csv');
255     bubbleChart.initializeData();
256 });
257 </script>

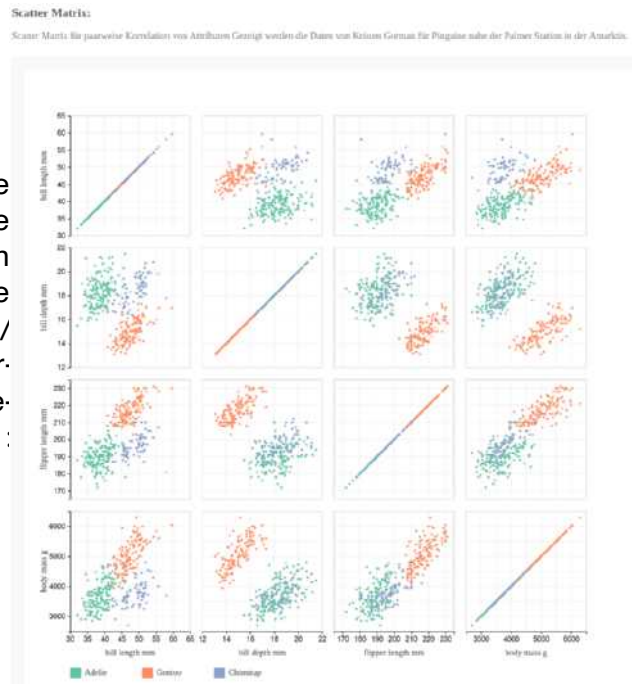
258 <style scoped>
259 .bubblechart-wrapper {
260     width: 100%;
261     max-width: 100%;
262     background: #fff;
263     border-radius: 8px;
264     box-shadow: 0 2px 4px rgba(0,0,0,0.07);
265     padding: 1rem;
266     margin-bottom: 1rem;
267     position: relative;
268 }

269 svg {
270     width: 100%;
271     height: 400px;
272     display: block;
273 }
274 .bubble-tooltip {
275     z-index: 10;
276     pointer-events: none;
277 }
278 </style>

```

11.6.4. Scattermatrix

Wir erstellen eine Scattermatrix mit D3.js. Die Scattermatrix wird in einer Vue-Komponente dargestellt. Die Darstellung ist ohne Klassen mit reinem D3 effizienter durchzuführen. Die Implementierung erfolgt in der Datei `components/Scattermatrix.vue`. Die zugehörigen Daten werden aus der Datei `public/data/penguins.csv` geladen. Diese Datei kann auf Github unter https://github.com/dataprofessor/data/blob/master/penguins_cleaned.csv heruntergeladen werden.



Scattermatrix mit D3.js

`src/components/Scattermatrix.vue`

```

1  <template>
2    <div class="scatter-matrix-wrapper">
3      <div ref="containerRef"></div>
4    </div>
5  </template>

6  <script setup lang="ts">
7    import { ref, onMounted } from 'vue';
8    import * as d3 from 'd3';
9
10   interface PenguinDatum {
11     species: string;
12     bill_length_mm: number;
13     bill_depth_mm: number;
14     flipper_length_mm: number;
15     body_mass_g: number;
16   }

17   const attributes: (keyof PenguinDatum)[] = [
18     'bill_length_mm',
19     'bill_depth_mm',
20     'flipper_length_mm',
21     'body_mass_g',
22   ];
23   const n = attributes.length;
24   const size = 200; // slightly smaller cell size
25   const gap = 20; // gap between subplots
26   const padding = 60; // reduced top and left margin
27   const axisLabelPad = 40; // move y-label closer to plot area
28   const width = size * n + gap * (n - 1) + padding + 40;
29   const height = size * n + gap * (n - 1) + padding + 80;
30

```

```

31 | const color = d3.scaleOrdinal(d3.schemeSet2);

32 | const containerRef = ref<HTMLElement | null>(null);
33
34 | onMounted(async () => {
35 |   const data : PenguinDatum[] = (await d3.csv('/data/penguins.csv', d => {
36 |     if ( // Only keep rows with all values present
37 |       d['bill_length_mm'] === 'NA' ||
38 |       d['bill_depth_mm'] === 'NA' ||
39 |       d['flipper_length_mm'] === 'NA' ||
40 |       d['body_mass_g'] === 'NA' ||
41 |       !d['species']
42 |     ) return null;
43 |     return {
44 |       species: d['species'],
45 |       bill_length_mm: +d['bill_length_mm'],
46 |       bill_depth_mm: +d['bill_depth_mm'],
47 |       flipper_length_mm: +d['flipper_length_mm'],
48 |       body_mass_g: +d['body_mass_g'],
49 |     } as PenguinDatum;
50 |   })).filter(Boolean) as PenguinDatum[];

51 |   // Set color domain
52 |   color.domain(Array.from(new Set(data.map((d: PenguinDatum) => d.species))));
53
54 |   // Compute scales for each attribute
55 |   const scales: Record<string, d3.ScaleLinear<number, number>> = {};
56 |   for (const attr of attributes) {
57 |     const extent = d3.extent(data, (d: PenguinDatum) => +(d[attr])) as [number,
58 |       number];
59 |     // Add 5% padding to each side
60 |     const range = extent[1] - extent[0];
61 |     const pad = range * 0.05;
62 |     // Use full cell area for each plot
63 |     scales[attr + '_x'] = d3.scaleLinear()
64 |       .domain([extent[0] - pad, extent[1] + pad])
65 |       .nice()
66 |       .range([0, size]);

67 |     scales[attr + '_y'] = d3.scaleLinear()
68 |       .domain([extent[0] - pad, extent[1] + pad])
69 |       .nice()
70 |       .range([size, 0]);
71 |   }
72 |   // Create SVG
73 |   const svg = d3.select(containerRef.value)
74 |     .append('svg')
75 |     .attr('width', width)
76 |     .attr('height', height)
77 |     .attr('viewBox', `0 0 ${width} ${height}`)
78 |     .attr('style', 'background: #fff;');
79
80 |   // Draw scatter matrix
81 |   for (let i = 0; i < n; ++i) {
82 |     for (let j = 0; j < n; ++j) {
83 |       const x0 = padding + j * (size + gap);
84 |       const y0 = padding + i * (size + gap);
85 |       const cell = svg.append('g')
86 |         .attr('transform', `translate(${x0},${y0})`);

87 |   // Draw frame

```

```

87     cell.append('rect')
88         .attr('width', size)
89         .attr('height', size)
90         .attr('fill', 'none')
91         .attr('stroke', '#bbb');
92
93     // Draw grid lines (both x and y for every subplot)
94     const xTicks = scales[attributes[j] + '_x'].ticks(6);
95     const yTicks = scales[attributes[i] + '_y'].ticks(6);
96
97     cell.selectAll('.grid-x')
98         .data(xTicks)
99         .join('line')
100        .attr('class', 'grid-x')
101        .attr('x1', d => scales[attributes[j] + '_x'](d))
102        .attr('x2', d => scales[attributes[j] + '_x'](d))
103        .attr('y1', 0)
104        .attr('y2', size)
105        .attr('stroke', '#eee')
106        .attr('stroke-width', 1);
107    cell.selectAll('.grid-y')
108        .data(yTicks)
109        .join('line')
110        .attr('class', 'grid-y')
111        .attr('y1', d => scales[attributes[i] + '_y'](d))
112        .attr('y2', d => scales[attributes[i] + '_y'](d))
113        .attr('x1', 0)
114        .attr('x2', size)
115        .attr('stroke', '#eee')
116        .attr('stroke-width', 1);
117
118    // Draw points (no clamping, use full cell)
119    cell.selectAll('circle')
120        .data(data)
121        .join('circle')
122        .attr('cx', d => scales[attributes[j] + '_x'](+d[attributes[j]]))
123        .attr('cy', d => scales[attributes[i] + '_y'](+d[attributes[i]]))
124        .attr('r', 2)
125        .attr('fill', d => color(d.species));
126
127    // X axis (bottom row)
128    if (i === n - 1) {
129        const xAxis = d3.axisBottom(scales[attributes[j] + '_x'])
130            .ticks(5)
131            .tickSize(4)
132            .tickPadding(4);
133        if (attributes[j] === 'body_mass_g') {
134            xAxis.tickFormat(d3.format('d'));
135        }
136        cell.append('g')
137            .attr('transform', `translate(0,${size})`)
138            .call(xAxis)
139            .call(g => g.select('.domain').attr('stroke-width', 1))
140            .call(g => g.selectAll('text').attr('font-size', 12));
141
142        // X axis label
143        cell.append('text')
144            .attr('x', size / 2)
145            .attr('y', size + axisLabelPad)
146            .attr('text-anchor', 'middle')
147            .attr('font-size', 14)

```

```

144     .attr('fill', '#333')
145     .text(attributes[j].replace(/_/g, ' '));
146 }

147 // Y axis (first column)
148 if (j === 0) {
149     const yAxis = d3.axisLeft(scales[attributes[i] + '_y'])
150     .ticks(5)
151     .tickSize(4)
152     .tickPadding(4);
153     if (attributes[i] === 'body_mass_g') {
154         yAxis.tickFormat(d3.format('d'));
155     }
156     cell.append('g')
157     .attr('transform', `translate(0,0)`)
158     .call(yAxis)
159     .call(g => g.select('.domain').attr('stroke-width', 1))
160     .call(g => g.selectAll('text').attr('font-size', 12));

161 // Y axis label
162 cell.append('text')
163     .attr('x', -axisLabelPad)
164     .attr('y', size / 2)
165     .attr('text-anchor', 'middle')
166     .attr('font-size', 14)
167     .attr('fill', '#333')
168     .attr('transform', `rotate(-90,${-axisLabelPad},${size / 2})`)
169     .text(attributes[i].replace(/_/g, ' '));
170 }
171 }
172 }

173 const legend = svg.append('g') // Draw legend
174 .attr('class', 'scatter-matrix-legend')
175 .attr('transform', `translate(${padding},${height - 20})`);
176 const species = color.domain();
177 legend.selectAll('rect')
178     .data(species)
179     .join('rect')
180     .attr('x', (d, i) => i * 120)
181     .attr('width', 18)
182     .attr('height', 18)
183     .attr('fill', d => color(d));
184 legend.selectAll('text')
185     .data(species)
186     .join('text')
187     .attr('x', (d, i) => i * 120 + 24)
188     .attr('y', 13)
189     .attr('font-size', 14)
190     .attr('fill', '#333')
191     .text(d => d);
192 });
193 </script>

194 <style scoped>
195 .scatter-matrix-wrapper {
196     width: 100%;
197     max-width: 100%;
198     background: #fff;
199     border-radius: 8px;
200     box-shadow: 0 2px 4px rgba(0,0,0,0.07);

```

```

201 padding: 1rem;
202 margin-bottom: 1rem;
203 overflow-x: auto;
204 }
205 svg {
206 width: 100%;
207 height: 600px;
208 display: block;
209 }
210 </style>

```

11.7. Darstellung hierarchischer Daten

Hierarchische Daten sind Daten, die in einer Baumstruktur organisiert sind. Diese Struktur ermöglicht es, Beziehungen zwischen den Daten darzustellen. Ein Beispiel für hierarchische Daten ist ein Dateisystem, in dem Ordner und Dateien in einer Baumstruktur angeordnet sind.

Daten: public/daten/flare.json

In den nachfolgend vorgestellten Beispielen werden die Daten aus der Datei `flare.json` gelesen. Diese Datei enthält eine Sammlung von Knoten, die jeweils einen Namen enthalten. Innere Knoten haben einen `children`-Eintrag, der eine Liste von Unterknoten enthält. Die Blattknoten enthalten statt der Kinder einen `size`-Eintrag. Diese Struktur ermöglicht es, komplexe hierarchische Beziehungen darzustellen.

Die Datei ist auch auf Github unter <https://github.com/d3/d3-hierarchy/blob/main/test/data/flare.json> zu finden.

11.7.1. Einbindung

Die nachfolgenden Beispiele werden in die bestehende Vue-Komponentenstruktur eingebunden. Hierzu wird eine neue Komponente `HierarchySelector.vue` erstellt. Diese stellt die Auswahl-Buttons für die verschiedenen Hierarchie-Darstellungen bereit. Die einzelnen Hierarchie-Darstellungen werden in eigenen Komponenten implementiert, die in der `HierarchySelector.vue` eingebunden werden.

Die Komponente `HierarchySelector` muss dann noch in die Komponente `pages/index.vue` eingebunden werden, damit sie im Hauptfenster der Anwendung angezeigt wird.

components/HierarchySelector.vue

```

1 <template>
2   <div>
3     <div class="button-group">
4       <button :class="{selected: selected === 'flare'}"
5         @click="selected = 'flare'">FlareTree</button>
6       <button :class="{selected: selected === 'radial'}"
7         @click="selected = 'radial'">Radial Cluster Tree</button>
8       <button :class="{selected: selected === 'treemap'}"
9         @click="selected = 'treemap'">TreeMap</button>
10      <button :class="{selected: selected === 'circlepacking'}"
11        @click="selected = 'circlepacking'">Circle Packing</button>
12      <button :class="{selected: selected === 'forcedirected'}"
13        @click="selected = 'forcedirected'">Force Directed</button>
14      <button :class="{selected: selected === 'sunburst'}"
15        @click="selected = 'sunburst'">Sunburst</button>
16    </div>

```

Mit den Buttons wird die Komponente für die jeweilige Hierarchie-Darstellung ausgewählt.

```

17     <div v-if="selected === 'flare'">
18       <FlareTree :width="1200" :height="2400" :node-radius="6" :vertical-spacing="
19         150" />
20     </div>
21     <div v-else-if="selected === 'radial'">
22       <RadialClusterTree :width="1200" :height="1200" :node-radius="6" />
23     </div>
24     <div v-else-if="selected === 'treemap'">
25       <TreeMap :width="1200" :height="800" />
26     </div>
27     <div v-else-if="selected === 'circlepacking'">
28       <CirclePacking :width="1200" :height="1200" />
29     </div>
30     <div v-else-if="selected === 'forcedirected'">
31       <ForceDirected :width="1200" :height="800" />
32     </div>
33     <div v-else-if="selected === 'sunburst'">
34       <SunBurst :width="900" :height="900" />
35     </div>
36   </div>
</template>

```

Im TypeScript-Setup werden lediglich die Komponenten importiert und eine Variable `selected` definiert, die den aktuell ausgewählten Hierarchie-Typ speichert. Diese Variable wird in der Template-Datei verwendet, um die entsprechende Komponente anzuzeigen.

```

37 <script setup lang="ts">
38 import { ref } from 'vue';
39 import FlareTree from './FlareTree.vue';
40 import RadialClusterTree from './RadialClusterTree.vue';
41 import TreeMap from './TreeMap.vue';
42 import CirclePacking from './CirclePacking.vue';
43 import ForceDirected from './ForceDirected.vue';
44 import SunBurst from './SunBurst.vue';
45
46 const selected = ref('flare');
47 </script>

```

Die Buttons sind so gestaltet, dass sie bei Auswahl eine andere Farbe erhalten.

```

48 <style scoped>
49 .button-group {
50   margin-bottom: 1rem;
51 }
52
53 .button-group button {
54   margin-right: 0.5rem;
55   padding: 0.5rem 1rem;
56   font-size: 1rem;
57   cursor: pointer;
58 }
59
60 .button-group button.selected {
61   background: #1976d2;
62   color: #fff;
63   border: 1px solid #1976d2;
64 }
65
66 .placeholder {
67   padding: 2rem;
68   color: #888;
69   background: #f8f8f8;
70   border: 1px dashed #ccc;
71   border-radius: 6px;
72   text-align: center;
73 }
</style>

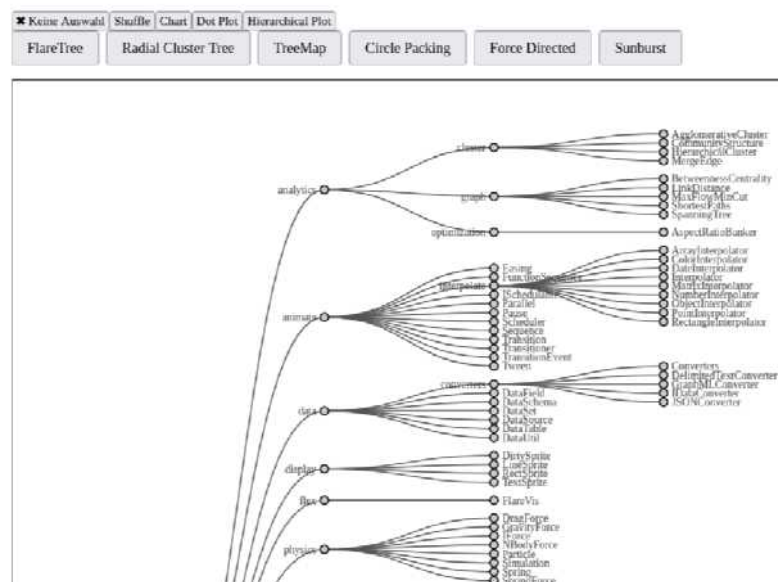
```

11.7.2. Einfache Baumstruktur mit Zoom und Panning

Es wird eine einfache Baumstruktur dargestellt. Die Daten werden aus `flaree.json` gelesen. Es sind folgende Interaktionen möglich:

- Ein-/Ausblenden von Subtrees durch Klicken auf die Knoten.
- Hinein- und Heraus-Zoomen der Baumstruktur mit der Maus.
- Panning der Baumstruktur durch Ziehen mit der Maus.

Dynamische D3 Beispiele



Ein einfacher Baum der Daten aus flare.json.

components/FlareTree.vue

Die Komponente `FlareTree.vue` zeichnet die Baumstruktur in einer einfachen Darstellung. Sie wird ohne Verwendung von Klassen implementiert. Es ist möglich Subtrees durch Klicken auf den Parent-Node ein- und auszublenden. Ebenso kann die gesamte Darstellung mit Hilfe der Maus gezoomt und verschoben werden. Befindet sich die Maus über einem Knoten, so wird sein Name in einem Tooltip angezeigt.

```

1 <template>
2   <svg ref="svg" :width="width" :height="height"></svg>
3 </template>
4 <style scoped>
5   svg {
6     background: #fff;
7   }
8   .flare-tooltip {
9     box-shadow: 0 2px 8px rgba(0,0,0,0.12);
10    transition: opacity 0.1s;
11  }
12 </style>

```

Zuerst werden die Eigenschaften der Komponente definiert. Diese steuern die Größe der SVG-Zeichenfläche, den Radius der Knoten und den vertikalen Abstand zwischen den Knoten. Dann wird noch eine Referenz auf das SVG-Element erstellt, um später darauf zugreifen zu können.

```

13 <script setup lang="ts">
14 import { ref, onMounted, reactive } from 'vue';
15 import * as d3 from 'd3';
16
17 const props = defineProps({
18   width: { type: Number, default: 900 },
19   height: { type: Number, default: 1600 },

```

```

20 |   nodeRadius: { type: Number, default: 8 },
21 |   verticalSpacing: { type: Number, default: 100 }
22 | });
23 |
24 | const svg = ref<SVGSVGElement | null>(null);

```

Nun wird die Sichtbarkeit der Knoten in einer Map gespeichert. Diese Map wird reaktiv gemacht, damit Änderungen automatisch in der Darstellung reflektiert werden. Die Funktionen `isVisible` und `toggleVisibility` werden verwendet, um die Sichtbarkeit der Knoten zu überprüfen und zu ändern. Die Funktion `renderTree` wird aufgerufen, um die Baumstruktur neu zu zeichnen, wenn sich die Sichtbarkeit ändert.

```

25 | const visibility = reactive(new Map<string, boolean>());
26 |
27 | function isVisible(name: string) {
28 |   return visibility.get(name) !== false;
29 | }
30 |
31 | function toggleVisibility(name: string) {
32 |   visibility.set(name, !isVisible(name));
33 |   renderTree();
34 | }

```

Tooltip

Ein Tooltip wird verwendet, um den Namen des Knotens anzuzeigen, wenn die Maus über einen Knoten bewegt wird. Der Tooltip wird als `div`-Element erstellt und in der `body`-Sektion der Seite platziert. Die Position des Tooltips wird dynamisch angepasst, wenn die Maus über einen Knoten bewegt wird. [1em] Es werden Funktionen `showTooltip` und `hideTooltip` definiert, um den Tooltip anzuzeigen und auszublenden. Die Funktion `showTooltip` wird mit den Events `mouseover` und `mousemove` assoziiert. Wenn die Maus den Knoten verlässt (Event: `mouseout`), wird der Tooltip mit der Funktion `hideTooltip` ausgeblendet. [1em]

Das Style-Attribut `z-index` wird verwendet, um sicherzustellen, dass der Tooltip über anderen Elementen angezeigt wird. Der Tooltip wird mit einem weißen Hintergrund, einem grauen Rand und abgerundeten Ecken gestaltet.

```

35 | let tooltip: d3.Selection<HTMLDivElement, unknown, HTMLElement, any> | null = null
36 | ;
37 | function showTooltip(name: string, x: number, y: number) {
38 |   if (!tooltip) {
39 |     tooltip = d3.select('body')
40 |       .append('div')
41 |       .attr('class', 'flare-tooltip')
42 |       .style('position', 'absolute')
43 |       .style('pointer-events', 'none')
44 |       .style('background', '#fff')
45 |       .style('border', '1px solid #888')
46 |       .style('border-radius', '4px')
47 |       .style('padding', '4px 10px')
48 |       .style('font-size', '15px')
49 |       .style('color', '#222')
50 |       .style('z-index', '1000');
51 |     tooltip.style(
52 |       'display', 'block'
53 |     ).style('left', `${x}px`)
54 |     .style('top',
55 |       `${y - 36}px`)
56 |     .text(name);
57 |   }
58 |
59 |   function hideTooltip() {
60 |     if (tooltip) {
61 |       tooltip.style(
62 |         'display', 'none');
63 |     }
64 |   }

```

Tidy Tree

Für die Darstellung der Baumstruktur wird die Funktion `d3.hierarchy` auf die Daten angewendet. Diese Funktion erstellt eine Hierarchie aus den Knoten und ermöglicht es, die Knoten und deren Beziehungen zu traversieren. Die Funktion `d3.tree` erzeugt einen kompakten Baum mit dem

Reingold-Tilford-Algorithmus. Für die Darstellung in kartesischen Koordinaten werden typischerweise die Rollen von x und y vertauscht, so dass die Knoten in vertikaler Richtung angeordnet werden. Für jeden Knoten wird eine Map unterhalten. Diese enthält einen boolschen Wert zur Sichtbarkeit des Subtrees dieses Knotens. Die Funktion `renderTree` wird aufgerufen, um die Baumstruktur zu zeichnen. [1em] In dieser Implementation wird weniger Rücksicht auf TypeScript-Datentypen genommen. Daher wird die Typisierung der Knoten in der Hierarchie nicht vollständig umgesetzt. Die Knoten werden als **any** behandelt, um die Flexibilität zu erhöhen. Dies ist in diesem Beispiel ausreichend, da die Knoten nur Namen und Kinder enthalten.

```

65 function renderTree() {
66   if (!rootData) return;
67   const root = d3.hierarchy(rootData);
68   root.each(node => {
69     if (!isVisible(node.data.name) && node !== root) {
70       node.children = undefined;
71     }
72   });
73   // Increase vertical spacing by scaling the x (vertical) position
74   const treeLayout = d3.tree()
75     .size([props.height, props.width - 200]);
76   treeLayout(root);

```

Nun wird die Ausdehnung des Baumes bestimmt und auf den SVG-Bereich skaliert. Die x-Positionen der Knoten werden so skaliert, dass sie mehr vertikalen Abstand zwischen den Knoten bieten. Die y-Positionen werden so skaliert, dass alle Knoten in den sichtbaren Bereich passen. Der Baum wird dann so verschoben, dass der Wurzelknoten immer sichtbar ist.

```

77   // Scale the x positions to add more vertical space between nodes
78   const allNodes = root.descendants();
79   const minX = d3.min(allNodes, d => d.x ?? 0) ?? 0;
80   const maxX = d3.max(allNodes, d => d.x ?? 0) ?? props.height;
81   // Map x so all nodes fit vertically with margin
82   const verticalScale = d3.scaleLinear()
83     .domain([minX, maxX])
84     .range([props.nodeRadius*2, props.height - props.nodeRadius*2]);
85
86   // Calculate minimum y to shift the tree right if needed
87   const rootLabelWidth =
88     root.data.name.length * 10 + props.nodeRadius + 16;
89   const minY = d3.min(allNodes, d => d.y ?? 0) ?? 0;
90   const yShift = minY < rootLabelWidth ? (rootLabelWidth - minY) : 0;

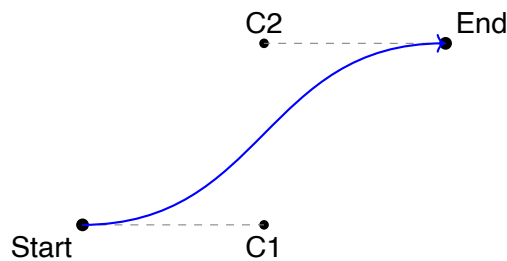
```

Das SVG-Element wird mit einem Rahmen versehen. Zusätzlich wird ein Gruppe erstellt, die den kompletten Baum erhält. Diese Gruppe wird später für das Zoom- und Pan-Verhalten verwendet.

```

91   const d3svg = d3.select(svg.value);
92   d3svg.selectAll('*').remove();
93
94   // Draw a black border around the visible SVG area
95   d3svg.selectAll('rect.svg-border').data([null]).join('rect')
96     .attr('class', 'svg-border')
97     .attr('x', 0)
98     .attr('y', 0)
99     .attr('width', props.width)
100    .attr('height', props.height)
101    .attr('fill', 'none')
102    .attr('stroke', '#000')
103    .attr('stroke-width', 2);
104
105    // Create a group for all nodes/links to apply zoom/pan
106    const group = d3svg.append('g').attr('class', 'flaretree-group');

```



Links

Die Verbindungen zwischen den Knoten werden als SVG-Pfade gezeichnet. Diese sind für die Funktion `d3.linkHorizontal` kubische Bezier-Splines, die von der Quelle zum Zielknoten führen. Die Pfade werden so gezeichnet, dass sie den vertikalen Abstand zwischen den Knoten berücksichtigen. Die Start- und Endpunkte der Pfade werden aus den x- und y-Positionen der Knoten berechnet.

```

107 | group.append('g')
108 |   .selectAll('path')
109 |   .data(root.links())
110 |   .enter()
111 |   .append('path')
112 |   .attr('fill', 'none')
113 |   .attr('stroke', '#555')
114 |   .attr('stroke-width', 2)
115 |   .attr('d', d3.linkHorizontal<any,any>()
116 |     .x(d => d.y + yShift)
117 |     .y(d => verticalScale(d.x))
118 |   );

```

Die Knoten und deren Labels werden in einer Gruppe gezeichnet. Die Position der Knoten wird aus den x- und y-Positionen der Knoten berechnet. Die y-Position wird mit dem vertikalen Skalierungsfaktor skaliert, um den vertikalen Abstand zwischen den Knoten zu berücksichtigen. Die x-Position wird um den `yShift` verschoben, um sicherzustellen, dass der Wurzelknoten immer sichtbar ist.

```

119 | const node = group.append('g')
120 |   .selectAll('g')
121 |   .data(allNodes)
122 |   .enter()
123 |   .append('g')
124 |   .attr('transform', d => `translate(${(d.y ?? 0) + yShift},${verticalScale(d.x
    ?? 0)})`);

```

Die Knoten selbst werden als Kreise gezeichnet. Die Farbe der Knoten wird abhängig von der Sichtbarkeit des Subtrees gesetzt. Wenn der Subtree sichtbar ist, wird er grau gefüllt, andernfalls weiß. Die Knoten sind klickbar und die Subtrees können durch Klicken auf den Kreis ein- und ausgeblendet werden. Bei Mouseover wird ein Tooltip mit dem Namen des Knotens

```

125 | node.append('circle')
126 |   .attr('r', props.nodeRadius)
127 |   .attr('fill', d => {
128 |     return isVisible(d.data.name) ? '#ccc' : '#fff';
129 |   })
130 |   .attr('stroke', '#000')
131 |   .attr('stroke-width', 2)
132 |   .style('cursor', 'pointer')
133 |   .on('click', (event, d) => {
134 |     toggleVisibility(d.data.name);
135 |   })
136 |   .on('mouseover', (event, d) => {
137 |     showTooltip(d.data.name, event.pageX, event.pageY);

```

```

138     })
139     .on('mousemove', (event, d) => {
140       showTooltip(d.data.name, event.pageX, event.pageY);
141     })
142     .on('mouseout', () => {
143       hideTooltip();
144     });

```

Text wird in der Regel rechts von den Knoten platziert, außer für den Wurzelknoten, bei dem der Text links vom Knoten steht.

```

145     node.append('text')
146     .attr('dx', d => {
147       // For the root node, always place the label to the left
148       if (!d.parent) return -(props.nodeRadius + 6);
149       return d.children ? -(props.nodeRadius + 6) : (props.nodeRadius + 6);
150     })
151     .attr('text-anchor', d => {
152       // For the root node, always anchor right
153       if (!d.parent) return 'end';
154       return d.children ? 'end' : 'start';
155     })
156     .attr('dy', '0.35em')
157     .text(d => d.data.name)
158     .attr('font-size', 16)
159     .attr('fill', '#333');
160   }

```

Nach dem Mounten der Komponente wird die JSON-Datei `flare.json` geladen. Die Hierarchie wird erstellt und die Sichtbarkeit aller Knoten wird auf `true` gesetzt. Anschließend wird die Baumstruktur gezeichnet.

```

161   onMounted(async () => {
162     const rootData = await d3.json('/data/flare.json');
163     if (!rootData) return;
164     d3.hierarchy(rootData).descendants().forEach(
165       d => visibility.set(d.data.name, true));
166     renderTree();

```

Die Funktion `renderTree` wird sowohl beim Mounten der Komponente als auch bei Änderungen der Sichtbarkeit der Knoten aufgerufen. Dadurch wird sichergestellt, dass die Baumstruktur immer aktuell ist.^[1em]

Zusätzlich wird das Zoom- und Pan-Verhalten für die SVG-Zeichenfläche eingerichtet. Hierzu wird die `D3.js`-Zoom-Funktion verwendet, die es ermöglicht, die Baumstruktur zu zoomen und zu verschieben. Die Transformationen werden auf die Gruppe angewendet, die den gesamten Baum enthält.

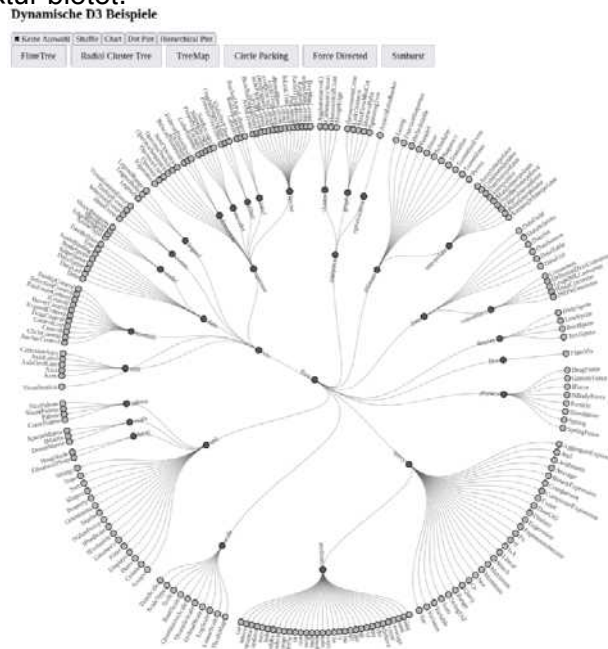
```

167   // Set up the SVG element
168   // Add zoom and pan behavior
169   const d3svg = d3.select(svg.value);
170   const zoom = d3.zoom<SVGSVGElement, unknown>()
171     .scaleExtent([0.5, 10])
172     .on('zoom', (event) => {
173       d3svg.select('g.flaretree-group')
174         .attr('transform', event.transform);
175     });
176   d3svg.call(zoom as any);
177 });
178 </script>

```

11.7.3. Radialer Cluster

Der radiale Cluster ist eine Variation der hierarchischen Baum-Darstellung. Anstatt in kartesischen Koordinaten wird der Baum in Polarkoordinaten dargestellt. Dabei werden die x-Werte als Winkel und die y-Werte als Radius interpretiert. In dieser Darstellung werden die Blattknoten auf einem Kreis angeordnet. Diese Anordnung ermöglicht es, die Hierarchieebenen klar zu unterscheiden und gleichzeitig die Beziehungen zwischen den Knoten zu visualisieren. Der radiale Cluster ist besonders nützlich für die Darstellung von großen und komplexen Datensätzen, da er eine klare und übersichtliche Struktur bietet.



Ein radialer Cluster der Daten aus `flare.json`.

components/RadialClusterTree.vue

Die Komponente `RadialClusterTree.vue` zeichnet die Baumstruktur in einer kreisförmigen Darstellung. Sie wird ohne Verwendung von Klassen umgesetzt. Auf die Implementierung von Interaktionen mit der Darstellung wird hier aus Zeitgründen ebenfalls verzichtet. Allerdings wird hier etwas mehr Wert auf die Typisierung gelegt.

```

1  <template>
2    <svg ref="svg" :width="width" :height="height"></svg>
3  </template>
4
5  <style scoped>
6    svg {
7      background: #fff;
8      display: block;
9    }
10 </style>
11
12 <script setup lang="ts">
13 import { ref, onMounted } from 'vue';
14 import * as d3 from 'd3';
15
16 const props = defineProps({
17   width: { type: Number, default: 1200 },
18   height: { type: Number, default: 1200 },
19   nodeRadius: { type: Number, default: 6 }
20 });

```

```

20 interface FlareNode {
21   name: string;
22   size?: number;
23   children?: FlareNode[];
24 }

```

Die Größe der SVG-Zeichenfläche wird über die Props `width` und `height` gesteuert. Der Radius der Knoten wird über die Prop `nodeRadius` gesteuert. Die Datenstruktur `FlareNode` beschreibt einen Knoten im Baum. Ein Knoten kann entweder ein Blattknoten mit einem `size`-Eintrag oder ein innerer Knoten mit einem `children`-Eintrag sein.

```

25 type HierarchyNode = d3.HierarchyNode<FlareNode>;
26 type HFlareNode = d3.HierarchyPointNode<FlareNode>;
27 type FlareLink = d3.HierarchyPointLink<FlareNode>;
28 type ClusterLayout = d3.ClusterLayout<FlareNode>;
29
30 const svg = ref<SVGSVGElement | null>(null);
31 const radius = Math.min(props.width, props.height) / 2 - 100;
32 const offset = 1.5 * props.nodeRadius; // Offset for text positioning

```

Die Typen `HierarchyNode`, `HFlareNode` und `FlareLink` werden für die Hierarchie-Datenstruktur verwendet. Der Radius der Baumstruktur wird auf die Hälfte der kleineren Dimension der SVG-Zeichenfläche minus 100 Pixel gesetzt. Der Offset für die Textpositionierung wird auf das 1,5-fache des Knotendurchmessers gesetzt.

Dendrogramm

Das Dendrogramm ist eine spezielle Form eines Baumes, bei dem alle Blattknoten in einer Ebene angeordnet sind. Auch alle Knoten der vorhergehenden Baumtiefe werden jeweils auf einer Ebene angeordnet. Damit wird der Baum weniger dicht gepackt als bei einem `TidyLayout`. Das Dendrogramm ist besonders nützlich, um die Hierarchieebenen klar zu unterscheiden und gleichzeitig die Beziehungen zwischen den Knoten zu visualisieren. Es wird häufig in der Bioinformatik verwendet, um phylogenetische Bäume darzustellen.

Dendrogramm in D3

In D3 wird ein Dendrogramm erzeugt, indem für die Hierarchie-Datenstruktur die Methode `d3.cluster()` anstatt `d3.tree()` verwendet wird.

Die Daten werden gelesen und in eine Hierarchie-Datenstruktur umgewandelt. Anschließend wird die Cluster-Layout-Funktion auf die Hierarchie angewendet, um die Knoten in einem radialen Layout anzuordnen. Knoten mit denselben Eltern werden näher beieinander angeordnet, während Knoten mit unterschiedlichen Eltern weiter auseinander liegen.

```

33 onMounted(async () => {
34   const filename = '/data/flare.json';
35   const data: FlareNode|undefined = await d3.json(filename);
36   if(!data) return;
37
38   const hierarchyData = d3.hierarchy<FlareNode>(data!)
39     .sort((a: HierarchyNode, b: HierarchyNode) =>
40       d3.ascending(a.data.name, b.data.name));
41   const cluster: ClusterLayout = d3.cluster()
42     .size([2 * Math.PI, radius])
43     .separation((a, b) => (a.parent === b.parent ? 1 : 2) / a.depth)
44     as ClusterLayout;
45   const root = cluster(hierarchyData);

```

Falls das SVG-Element noch nicht existiert, wird es erstellt. Die SVG-Zeichenfläche wird geleert, um sicherzustellen, dass keine vorherigen Zeichnungen vorhanden sind. Der Ursprung des Koordinatensystems wird auf die Mitte der SVG-Zeichenfläche gesetzt, damit der Baum zentriert dargestellt wird.

```

46 | const svgSel = d3.select(svg.value);
47 | svgSel.selectAll('*').remove();
48 | // Origin is at the center of the SVG
49 | const g = svgSel.append('g')
50 |   .attr('transform',
51 |     `translate(${props.width / 2},${props.height / 2})`);

```

Zunächst werden die Verbindungen zwischen den Knoten gezeichnet. Die Verbindungen werden mit der Methode `d3.linkRadial()` gezeichnet, die die radialen Koordinaten (Winkel und Radius) der Knoten verwendet. Die Farbe und Transparenz der Verbindungen wird ebenfalls festgelegt. Auch hier werden die Verbindungen als kubische Bézier-Kurven gezeichnet, um eine glatte Verbindung zwischen den Knoten zu erzeugen.

```

52 | // Draw links using d3.linkRadial
53 | g.append('g')
54 |   .attr('fill', 'none')
55 |   .attr('stroke', '#555')
56 |   .attr('stroke-opacity', 0.4)
57 |   .attr('stroke-width', 1.5)
58 |   .selectAll('path')
59 |   .data(root.links())
60 |   .join('path')
61 |   .attr('d', d3.linkRadial<FlareLink, HFlareNode>()
62 |     .angle(d => d.x)
63 |     .radius(d => d.y)
64 |   );

```

Die Kreise und Textbeschriftungen für die Knoten werden in einer gemeinsamen Gruppe platziert, um die Transformationen (Rotation und Translation) anzuwenden. Die Gruppe wird zuerst entlang des Radius verschoben und dann den Ursprung des Koordinatensystems rotiert.

```

65 | // Draw nodes: translate by y, then rotate by x
66 | const node = g.append('g')
67 |   .selectAll('g')
68 |   .data(root.descendants())
69 |   .join('g')
70 |   .attr('transform', d => `rotate(${d.x * 180 / Math.PI - 90}) translate(${d.y
    },0)`);

```

Der Gruppe werden Kreise zur Darstellung der Knoten und Textbeschriftungen hinzugefügt. Die Kreise werden mit einem Radius von `props.nodeRadius` gezeichnet und erhalten eine Füllfarbe, die abhängig davon ist, ob der Knoten Kinder hat oder nicht. Die Textbeschriftungen werden entsprechend der Position des Knotens im linken oder rechten Halbkreis gespiegelt und ausgerichtet. Der Offset wird verwendet, um die Beschriftungen außerhalb des Kreises zu platzieren.

```

71 | node.append('circle')
72 |   .attr('r', props.nodeRadius)
73 |   .attr('fill', d => d.children ? '#555' : '#BBB')
74 |   .attr('stroke', '#000')
75 |   .attr('stroke-width', 1.5);
76 |
77 | node.append('text')
78 |   .attr('transform', d => `rotate(${d.x > Math.PI ? 180 : 0})`)
79 |   .attr('dy', '0.32em')
80 |   // !== corresponds for booleans to XOR (exclusive or)
81 |   .attr('x', d => d.x < Math.PI === !d.children ? offset : -offset)
82 |   .attr('text-anchor', d => d.x < Math.PI === !d.children ? 'start' : 'end')
83 |   .attr('font-size', 13)
84 |   .attr('fill', '#333')
85 |   .text(d => d.data.name);
86 | });
87 | </script>

```

11.7.4. Treemap-Layout

Das Treemap-Layout ist eine Methode zur Darstellung hierarchischer Daten in Form von verschachtelten Rechtecken. Jedes Rechteck repräsentiert einen Knoten, und die Größe der Rechtecke entspricht dem Wert des Knotens. Diese Methode ermöglicht es, große Datenmengen auf kompakte Weise darzustellen und gleichzeitig die Hierarchie zu visualisieren. Treemaps sind besonders nützlich für die Analyse von Verhältnissen und Größenverhältnissen in großen Datensätzen.

Dynamische D3 Beispiele



Eine Treemap-Darstellung der Daten aus `flare.json`.

components/TreeMap.vue

Es wird eine Vue-Komponente erstellt, die das Treemap-Layout mit D3.js implementiert. Diese Komponente rendert ein SVG-Element, in dem die Rechtecke für die Treemap gezeichnet werden. Die Größe der Rechtecke wird basierend auf den Daten aus einer JSON-Datei berechnet, die hierarchische Informationen enthält.

```

1 <template>
2   <svg ref="svg" :width="width" :height="height"></svg>
3 </template>
4 <style scoped>
5   svg {
6     background: #fff;
7   }
8   .treemap-tooltip {
9     box-shadow: 0 2px 8px rgba(0,0,0,0.12);
10    transition: opacity 0.1s;
11  }
12 </style>

```

TypeScript Setup

In der Komponente werden zunächst die reaktiven Eigenschaften für die Breite und Höhe des SVG-Elements definiert. Diese Werte können beim Einbinden der Komponente angepasst werden. Das SVG-Element wird mit einem `ref` versehen, um später darauf zugreifen zu können.[1em] Dann werden die nötigen Typen für die D3.js-Hierarchie und Treemap-Layout definiert. Diese Typen helfen bei der Typensicherheit und Autovervollständigung in TypeScript.

```

13 <script setup lang="ts">
14 import { ref, onMounted } from 'vue';
15 import * as d3 from 'd3';
16
17 const props = defineProps({
18   width: { type: Number, default: 1200 },
19   height: { type: Number, default: 800 }
20 });
21
22 const svg = ref<SVGSVGElement | null>(null);
23
24 interface FlareNode {
25   name: string;
26   size?: number;
27   children?: FlareNode[];
28 }
29 type HierarchyNode = d3.HierarchyNode<FlareNode>;
30 type TreemapNode = d3.HierarchyRectangularNode<FlareNode>;
31 type TreemapLayout = d3.TreemapLayout<FlareNode>;
32 type ZoomBehavior = d3.ZoomBehavior<SVGSVGElement, unknown>;

```

Mach dem einlesen der Daten wird wieder eine D3-Hierarchie erstellt, die die Datenstruktur in eine Form bringt, die für das Treemap-Layout geeignet ist. Die Hierarchie wird so konfiguriert, dass sie die Größe der Knoten summiert und sortiert.

```

33 onMounted(async () => {
34   const filename = '/data/flare.json';
35   const data: FlareNode = await d3.json(filename) as FlareNode;
36   if (!data) {
37     console.error('Failed to load data from', filename);
38     return;
39   }
40
41   const root: HierarchyNode = d3.hierarchy<FlareNode>(data as any)
42     .sum(d => d.size || 0)
43     .sort((a, b) => (b.value || 0) - (a.value || 0));
44
45   const treemapLayout: TreemapLayout = d3.treemap<FlareNode>()
46     .size([props.width, props.height])
47     .padding(2);
48   treemapLayout(root);

```

Falls das SVG-Element noch nicht existiert, wird es erstellt. Anschließend wird die Zoom- und Pan-Funktionalität hinzugefügt, um die Interaktivität der Treemap zu ermöglichen.

```

49   const d3svg = d3.select(svg.value);
50   d3svg.selectAll('*').remove();
51
52   // Add zoom and pan behavior
53   const zoom: ZoomBehavior = d3.zoom<SVGSVGElement, unknown>()
54     .scaleExtent([0.5, 10])
55     .on('zoom', (event) => {
56       d3svg.select('g.treemap-group')
57         .attr('transform', event.transform);
58     });
59   d3svg.call(zoom as any);

```

Die nachfolgenden Tooltip-Funktionen sind analog zur Implementierung in der FlareTree-Komponente.

```

60   let tooltip: d3.Selection<HTMLDivElement, unknown, HTMLElement, any>
61     | null = null;
62
63   function showTooltip(name: string, x: number, y: number) {

```

```

64   if (!tooltip) {
65     tooltip = d3.select('body')
66       .append('div')
67       .attr('class', 'treemap-tooltip')
68       .style('position', 'absolute')
69       .style('pointer-events', 'none')
70       .style('background', '#fff')
71       .style('border', '1px solid #888')
72       .style('border-radius', '4px')
73       .style('padding', '4px 10px')
74       .style('font-size', '15px')
75       .style('color', '#222')
76       .style('z-index', '1000');
77   }

78   tooltip.style('display', 'block')
79     .style('left', `${x}px`)
80     .style('top', `${y - 36}px`)
81     .text(name);
82   }
83
84   function hideTooltip() {
85     if (tooltip) {
86       tooltip.style('display', 'none');
87     }
88   }

```

Das SVG-Element wird mit einem Group-Element umgeben, um alle Treemap-Knoten zu gruppieren. Dadurch kann die Zoom- und Pan-Funktionalität auf alle Knoten angewendet werden.

```

89   // Create a group for all treemap nodes to apply zoom/pan
90   const group = d3svg.append('g').attr('class', 'treemap-group');
91
92   const nodes = group.selectAll('g')
93     .data(root.leaves() as TreemapNode[])
94     .enter()
95     .append('g')
96     .attr('transform', d => `translate(${d.x0},${d.y0})`);

```

Die Rechtecke für die Treemap-Knoten werden gezeichnet. Jedes Rechteck erhält eine Breite und Höhe basierend auf den Koordinaten des Knotens. Die Farbe und der Rahmen der Rechtecke können angepasst werden, um die Lesbarkeit zu verbessern. Zudem werden Event-Listener für Mouseover, Mousemove und Mouseout hinzugefügt, um Tooltips anzuzeigen.

```

97   nodes.append('rect')
98     .attr('width', d => d.x1 - d.x0)
99     .attr('height', d => d.y1 - d.y0)
100    .attr('fill', '#69b3a2')
101    .attr('stroke', '#fff')
102    .on('mouseover', (event, d) => {
103      showTooltip(d.data.name, event.pageX, event.pageY);
104    })
105    .on('mousemove', (event, d) => {
106      showTooltip(d.data.name, event.pageX, event.pageY);
107    })
108    .on('mouseout', () => {
109      hideTooltip();
110    });

```

Die Rechtecke werden mit Texten versehen, die den Namen des Knotens anzeigen. Die Schriftgröße und Farbe können angepasst werden, um die Lesbarkeit zu verbessern. Zudem wird überprüft, ob

der Text in das Rechteck passt. Falls der Text zu breit ist, wird der Text ausgeblendet, steht aber über den Tooltip zur Verfügung.

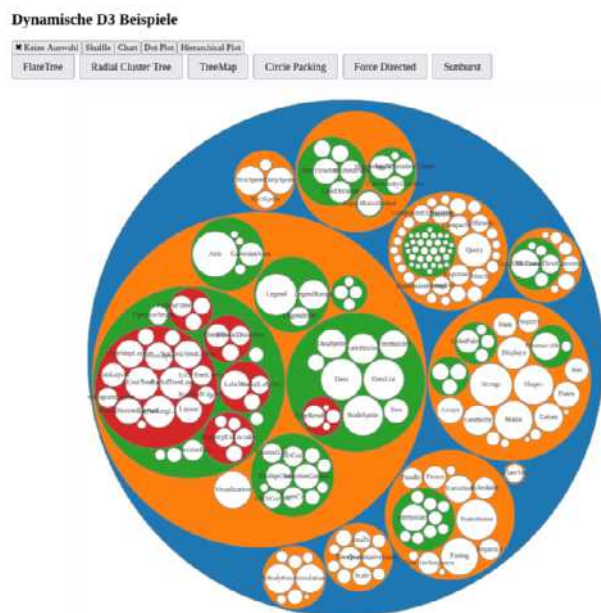
```

111 nodes.append('text')
112   .attr('x', 4)
113   .attr('y', 18)
114   .text(d => d.data.name)
115   .attr('font-size', 14)
116   .attr('fill', '#222')
117   .attr('pointer-events', 'none')
118   .attr('display', function(d) {
119     const textWidth = (this as SVGTextElement).getComputedTextLength();
120     const rectWidth = d.x1 - d.x0;
121     return textWidth + 8 > rectWidth ? 'none' : null;
122   });
123 });
124 </script>

```

11.7.5. Circle Packing

Circle Packing ist eine Methode zur Darstellung von hierarchischen Daten, bei der Knoten als Kreise dargestellt werden. Die Größe der Kreise entspricht dem Wert des Knotens, und die Position der Kreise wird so gewählt, dass sie sich nicht überlappen. Diese Methode ermöglicht es, hierarchische Beziehungen auf eine visuell ansprechende Weise darzustellen und gleichzeitig die Größe der Knoten zu berücksichtigen.



Eine Circle Packing-Darstellung der Daten aus `flare.json`.

components/CirclePacking.vue

Es wird eine Vue-Komponente erstellt, die das Circle Packing-Layout mit D3.js implementiert. Diese Komponente rendert ein SVG-Element, in dem die Kreise für das Circle Packing gezeichnet werden. Die Größe der Kreise wird basierend auf den Daten aus einer JSON-Datei berechnet, die hierarchische Informationen enthält.

```

1 <template>
2   <div ref="containerRef" class="circle-packing-container"></div>
3 </template>

```

Das FlareNode-Interface entspricht der Struktur der Daten, entnommene den vorhergehenden Beispielen. Die unterschiedlichen Tiefen des Baumes sollen in unterschiedlichen Farben dargestellt werden. Die Größe der Kreise wird aus dem Wert des Knotens berechnet oder aus den im Knoten enthaltenen Subbäumen. Die Größe der Kreise wird so gewählt, dass sie sich nicht überlappen.

```

4 <script setup lang="ts">
5 import { ref, onMounted } from 'vue';
6 import * as d3 from 'd3';
7
8 interface FlareNode {
9   name: string;
10  size?: number;
11  children?: FlareNode[];
12 }
13
14 const width = 900;
15 const height = 900;
16 const color = d3.scaleOrdinal(d3.schemeCategory10);
17
18 const containerRef = ref<HTMLElement | null>(null);

```

Mit `d3.hierarchy` wird wieder ein Model der eingelesenen hierarchischen JSON-Daten erstellt. Zur Visualisierung wird das Circle Packing-Layout verwendet, welches die Knoten als Kreise anordnet. Die Größe der Kreise wird aus den Werten der Knoten berechnet.

```

19 let svg: d3.Selection<SVGSVGElement, unknown, null, undefined>;
20 let node: d3.Selection<SVGSVGElement, d3.HierarchyCircularNode<any>, any, unknown>;
21 let rootPacked: d3.HierarchyCircularNode<any>;
22 let root: d3.HierarchyNode<FlareNode>;
23
24
25 onMounted(async () => {
26   const data = await d3.json('/data/flare.json') as FlareNode;
27   root = d3.hierarchy(data)
28     .sum(d => d.size || 0)
29     .sort((a, b) => (b.value || 0) - (a.value || 0));
30   const pack = d3.pack<FlareNode>()
31     .size([width, height])
32     .padding(3);
33   rootPacked = pack(root);

```

Das SVG-Element wird erzeugt und jeder Knoten wird als eine Gruppe dargestellt. Die Position der Gruppen wird so gewählt, dass sie den Mittelpunkt des jeweiligen Kreises repräsentiert.

```

34   svg = d3.select(containerRef.value)
35     .append('svg')
36     .attr('width', width)
37     .attr('height', height)
38     .attr('viewBox', `0 0 ${width} ${height}`)
39     .attr('style', 'background: #fff;');
40
41   node = svg.selectAll('g')
42     .data(rootPacked.descendants(), (d: any) => d.data.name)
43     .join('g')
44     .attr('transform', d => `translate(${d.x},${d.y})`) as any;

```

Zuletzt werden die Kreise und Texte für die Knoten hinzugefügt. Die Farbe der Kreise wird basierend auf der Tiefe des Knotens im Baum gewählt. Die Texte werden nur für Knoten mit einer ausreichenden Größe angezeigt, um Überlappungen zu vermeiden.

```

45   node.append('circle')
46     .attr('r', d => d.r)

```

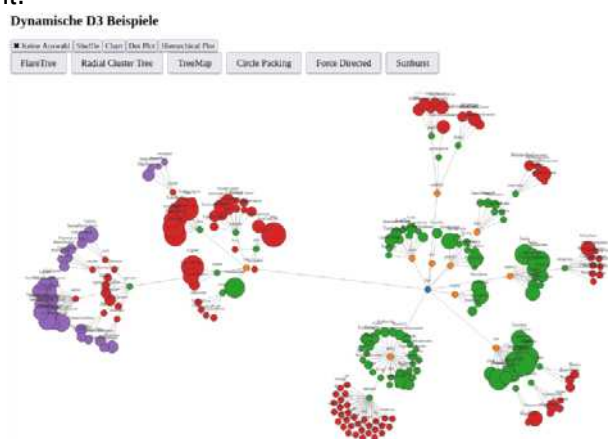
```

47 | .attr('fill', d => d.children ? color(d.depth.toString()) : '#fff')
48 | .attr('stroke', '#888')
49 | .attr('stroke-width', 1.2);
50 |
51 | node.append('text')
52 |   .filter(d => d.r > 15)
53 |   .attr('text-anchor', 'middle')
54 |   .attr('dominant-baseline', 'middle')
55 |   .attr('font-size', 12)
56 |   .attr('fill', '#333')
57 |   .text(d => d.data.name);
58 | });
59 | </script>
60 |
61 | <style scoped>
62 | .circle-packing-container {
63 |   width: 100%;
64 |   max-width: 900px;
65 |   margin: 0 auto;
66 |   background: #fff;
67 |   border-radius: 8px;
68 |   box-shadow: 0 2px 4px rgba(0,0,0,0.07);
69 |   padding: 1rem;
70 |   overflow-x: auto;
71 | }
72 | svg {
73 |   width: 100%;
74 |   height: auto;
75 |   display: block;
76 | }
77 | </style>

```

11.7.6. Force-Layout

Das Force-Layout ist eine Methode zur Darstellung von Netzwerken und Graphen, die auf physikalischen Kräften basiert. Es simuliert die Anziehung und Abstoßung zwischen Knoten, um eine ansprechende visuelle Anordnung zu erzeugen. Diese Methode ist besonders nützlich für die Darstellung von Beziehungen in komplexen Datensätzen, da sie eine intuitive und dynamische Visualisierung ermöglicht.



Eine Force-Layout-Darstellung der Daten aus `flare.json`.

components/ForceLayout.vue

Es wird eine Vue-Komponente erstellt, die das Force-Layout mit D3.js implementiert. Diese Komponente rendert ein SVG-Element, in dem die Knoten und Kanten des Netzwerks gezeichnet werden. Die Position der Knoten wird durch die Simulation physikalischer Kräfte bestimmt, was zu einer ansprechenden Anordnung führt.

```
1 <template>
2   <div ref="containerRef" class="force-directed-container"></div>
3 </template>
```

Die eingelesenen Daten entsprechen wieder der Struktur der vorhergehenden Beispiele. Es wird jedoch noch eine optionale `id` Eigenschaft hinzugefügt, die für die eindeutige Identifikation der Knoten verwendet wird.

```
4 <script setup lang="ts">
5 import { ref, onMounted } from 'vue';
6 import * as d3 from 'd3';
7
8 const width = 1200;
9 const height = 900;
10 const nodeRadius = 10;
11 const containerRef = ref<HTMLElement | null>(null);
12
13 interface FlareNode {
14   name: string;
15   size?: number;
16   children?: FlareNode[];
17   id?: string;
18 }
```

d3.SimulationNodeDatum

Bei `d3.SimulationNodeDatum` handelt es sich um eine Schnittstelle, die die Eigenschaften eines Knotens in einer D3-Simulation beschreibt. Diese Eigenschaften sind für die physikalische Simulation der Knotenpositionen und -bewegungen erforderlich.

```
19 interface SimulationNodeDatum {
20   index?: number; // Index of the node in the array
21   x?: number;     // Current x-position
22   y?: number;     // Current y-position
23   vx?: number;    // Current x-velocity
24   vy?: number;    // Current y-velocity
25   fx?: number | null; // Fixed x-position (if set, node is fixed)
26   fy?: number | null; // Fixed y-position (if set, node is fixed)
27 }
```

d3.SimulationLinkDatum

`d3.SimulationLinkDatum` beschreibt die Eigenschaften einer Kante in einer D3-Simulation. Diese Eigenschaften sind für die Definition der Verbindungen zwischen Knoten erforderlich, einschließlich der Quell- und Zielknoten.

```
28 interface SimulationLinkDatum<NodeType> {
29   source: NodeType | string; // Source node (can be a node or an ID)
30   target: NodeType | string; // Target node (can be a node or an ID)
31   index?: number;           // Index of the link in the array
32   strength?: number;        // Strength of the link (default is 1)
33   distance?: number;        // Desired distance between source and target
34 }
```

ForceNode

ForceNode erweitert das Interface `d3.SimulationNodeDatum`, um spezifische Eigenschaften für das Force-Layout zu definieren. ForceNode enthält zusätzliche Informationen wie die Tiefe des Knotens.

```

35 | interface ForceNode extends d3.SimulationNodeDatum {
36 |   id: string;
37 |   name: string;
38 |   size?: number;
39 |   depth: number;
40 | }

```

ForceLink

ForceLink benötigt hier keine weiteren Eigenschaften, daher beschränken wir es auf `d3.SimulationLinkDatum` mit ForceNode als Instanz des generischen Datentyps.

```

41 | type ForceLink = d3.SimulationLinkDatum<ForceNode>;

```

Die Funktion `flatten` wandelt die hierarchische Struktur der Knoten in eine flache Liste um, die für das Force-Layout verwendet werden kann. Jeder Knoten erhält eine eindeutige ID, die aus seinem Namen und einer zufälligen Zeichenkette generiert wird.

```

42 | function flatten(root: FlareNode): ForceNode[] {
43 |   const nodes: ForceNode[] = [];
44 |   function recurse(node: FlareNode, depth: number) {
45 |     // create unique ID by extending name with a 6-character random string from
46 |     // characters [0-9a-z]
47 |     node.id = node.id || node.name + Math.random().toString(36).slice(2, 8);
48 |     nodes.push({ id: node.id, name: node.name, size: node.size, depth });
49 |     if (node.children) node.children.forEach(child => recurse(child, depth + 1));
50 |   }
51 |   recurse(root, 0);
52 |   return nodes;
53 | }

```

Die Funktion `getLinks` erstellt die Verbindungen zwischen den Knoten, indem sie rekursiv durch die hierarchische Struktur der Knoten geht. Jede Verbindung wird als Objekt mit Quell- und Zielknoten-ID gespeichert.

```

53 | function getLinks(root: FlareNode): ForceLink[] {
54 |   const links: ForceLink[] = [];
55 |   function recurse(node: FlareNode) {
56 |     if (node.children) {
57 |       node.children.forEach(child => {
58 |         links.push({ source: node.id!, target: child.id! });
59 |         recurse(child);
60 |       });
61 |     }
62 |   }
63 |   recurse(root);
64 |   return links;
65 | }

```

Nachdem das DOM erzeugt wurde, wird die JSON-Datei `flare.json` geladen und die Knoten und Verbindungen werden extrahiert. Anschließend wird ein SVG-Element erstellt, in dem die Knoten und Kanten des Force-Layouts gezeichnet werden.

```

66 | onMounted(async () => {
67 |   const filename = '/data/flare.json';
68 |   const data = await d3.json(filename) as FlareNode;
69 |   const nodes = flatten(data);
70 |   const links = getLinks(data);

```

```

71
72 const svg = d3.select(containerRef.value)
73   .append('svg')
74   .attr('width', width)
75   .attr('height', height)
76   .attr('viewBox', `0 0 ${width} ${height}`)
77   .attr('style', 'background: #fff;');

```

Es wird eine Gruppe für das Zooming erstellt, die alle Elemente des Force-Layouts enthält.

```

78 // Add a group for zooming
79 const zoomGroup = svg.append('g');
80
81 // Add zoom behavior
82 svg.call(d3.zoom<SVGSVGElement, unknown>()
83   .scaleExtent([0.2, 4])
84   .on('zoom', (event) => {
85     zoomGroup.attr('transform', event.transform);
86   })
87 );

```

Die D3-Simulation wird initialisiert, um die Knoten und Kanten zu positionieren. Die Kräfte für die Knoten und Kanten werden definiert, einschließlich der Anziehungskraft zwischen den Knoten und der Abstoßungskraft, um Überlappungen zu vermeiden. Die Wahl der Parameter für die Kräfte beeinflusst die Anordnung der Knoten im Force-Layout. Daher erfordert es oft einige Experimente, um die besten Werte zu finden.

```

88 const simulation = d3.forceSimulation(nodes)
89   .force('link',
90     d3.forceLink(links)
91       .id(d => (d as ForceNode).id)
92       .distance(60)
93       .strength(1)
94   )
95   .force('charge', d3.forceManyBody().strength(-120))
96   .force('center', d3.forceCenter(width / 2, height / 2))
97   .force('collide', d3.forceCollide(nodeRadius + 2));

```

Die Kanten und Knoten werden in der SVG-Grafik gezeichnet. Die Kanten werden als Linien dargestellt, während die Knoten als Kreise gezeichnet werden. Die Größe der Knoten wird basierend auf ihrer Größe (falls vorhanden) skaliert, um eine visuelle Hierarchie darzustellen.

```

98 const link = zoomGroup.append('g')
99   .attr('stroke', '#aaa')
100   .attr('stroke-width', 1.5)
101   .selectAll('line')
102   .data(links)
103   .join('line');
104
105 const node = zoomGroup.append('g')
106   .attr('stroke', '#333')
107   .attr('stroke-width', 1.5)
108   .selectAll('circle')
109   .data(nodes)
110   .join('circle')
111   .attr('r', d => d.size
112     ? Math.max(nodeRadius, Math.sqrt(d.size) / 4)
113     : nodeRadius)
114   .attr('fill', d => d3.schemeCategory10[d.depth % 10])
115   .call((drag(simulation) as any));

```

Die Beschriftungen der Knoten werden als Text-Elemente hinzugefügt. Die Position der Beschriftungen wird relativ zu den Knoten festgelegt, um eine klare Lesbarkeit zu gewährleisten.

```

115   const label = zoomGroup.append('g')
116     .selectAll('text')
117     .data(nodes)
118     .join('text')
119     .attr('font-size', 12)
120     .attr('text-anchor', 'middle')
121     .attr('dy', -nodeRadius - 2)
122     .attr('fill', '#333')
123     .text(d => d.name);

```

Die Simulation wird gestartet und die Positionen der Knoten und Kanten werden in jedem Tick aktualisiert. Die Knoten und Kanten werden entsprechend ihrer aktuellen Positionen gezeichnet, um die dynamische Natur des Force-Layouts zu zeigen.

```

124   simulation.on('tick', () => {
125     link
126       .attr('x1', d => (d.source as ForceNode).x!)
127       .attr('y1', d => (d.source as ForceNode).y!)
128       .attr('x2', d => (d.target as ForceNode).x!)
129       .attr('y2', d => (d.target as ForceNode).y!);
130     node
131       .attr('cx', d => d.x!)
132       .attr('cy', d => d.y!);
133     label
134       .attr('x', d => d.x!)
135       .attr('y', d => d.y! - (d.size ? Math.max(nodeRadius, Math.sqrt(d.size) / 4)
136         : nodeRadius) - 2);
136   });

```

Drag-and-Drop für Knoten

Die Drag-and-Drop-Funktionalität wird hinzugefügt, um die Knoten interaktiv zu verschieben. Diese Funktion ermöglicht es den Benutzern, Knoten zu ziehen und ihre Positionen manuell anzupassen, was die Interaktivität des Force-Layouts erhöht.

Dabei ist zu beachten, dass die Knoten während des Ziehens ihre physikalische Anziehungskraft verlieren, um ein sanftes Ziehen zu ermöglichen. Nach dem Loslassen wird die Anziehungskraft wiederhergestellt.

Welche Rolle spielt Alpha?

Die α in der D3-Simulation steuert die „Temperatur“ der Simulation. Ein höherer Wert bedeutet, dass die Knoten stärker voneinander abgestoßen werden, was zu einer größeren Bewegung führt. Ein niedrigerer Wert führt zu einer stabileren Anordnung. Beim Starten der Drag-and-Drop-Interaktion wird `alphaTarget` auf einen höheren Wert gesetzt, um die Bewegung zu erleichtern, und nach dem Loslassen wird es wieder auf 0 gesetzt, um die Stabilität wiederherzustellen.

```

137   function drag(simulation: d3.Simulation<ForceNode, ForceLink>) {
138     function dragStarted(event: any, d: ForceNode) {
139       if (!event.active) simulation.alphaTarget(0.3).restart();
140       d.fx = d.x;
141       d.fy = d.y;
142     }
143     function dragged(event: any, d: ForceNode) {
144       d.fx = event.x;
145       d.fy = event.y;
146     }
147     function dragEnded(event: any, d: ForceNode) {
148       if (!event.active) simulation.alphaTarget(0);
149       d.fx = null;
150       d.fy = null;

```

```

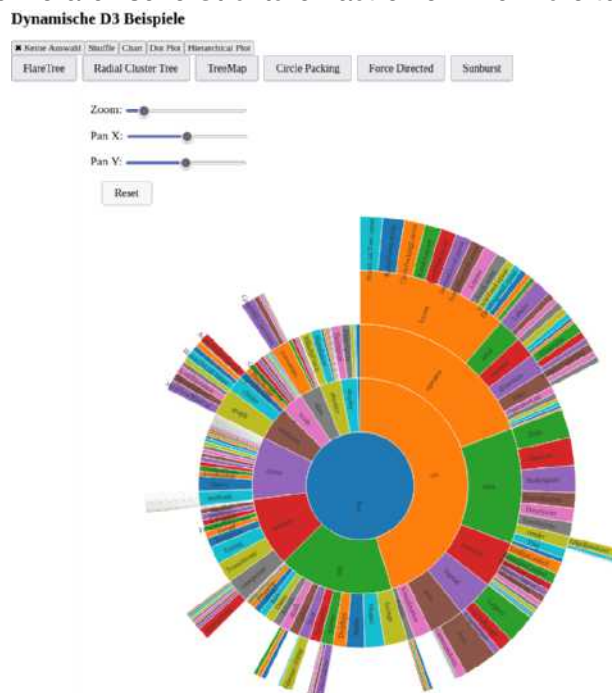
151     }
152     return d3.drag<SVGCircleElement, ForceNode>()
153       .on('start', dragStarted)
154       .on('drag', dragged)
155       .on('end', dragEnded);
156   }
157 });
158 </script>

159 <style scoped>
160 .force-directed-container {
161   width: 100%;
162   max-width: 1200px;
163   margin: 0 auto;
164   background: #fff;
165   border-radius: 8px;
166   box-shadow: 0 2px 4px rgba(0,0,0,0.07);
167   padding: 1rem;
168   overflow-x: auto;
169 }
170 svg {
171   width: 100%;
172   height: auto;
173   display: block;
174 }
175 </style>

```

11.7.7. Sunburst-Diagramm

Das Sunburst-Diagramm ist eine spezielle Form der Darstellung hierarchischer Daten. Es visualisiert die Beziehungen zwischen den Knoten in einer kreisförmigen Anordnung, wobei jeder Knoten als Segment eines Kreises dargestellt wird. Die Größe der Segmente repräsentiert die Größe der Datenpunkte, während die Position im Kreis die Hierarchieebene anzeigt. Diese Darstellung ermöglicht es, komplexe hierarchische Strukturen auf einen Blick zu erfassen und zu analysieren.



Eine Sunburst-Darstellung der Daten aus `flare.json`.

components/Sunburst.vue

Es wird eine Vue-Komponente erstellt, die das Sunburst-Diagramm mit D3.js implementiert. Diese Komponente rendert ein SVG-Element, in dem die Segmente des Sunburst-Diagramms gezeichnet werden. Die Größe der Segmente wird basierend auf den Daten aus einer JSON-Datei berechnet, die hierarchische Informationen enthält. Zoom und Pan werden diesmal mit Hilfe von Slidern und einem Reset-Button gesteuert.

```

1 <template>
2   <div class="sunburst-container">
3     <label>Zoom: <input type="range" min="0.5" :max="maxZoom" step="0.01" v-model.
        number="zoomLevel" /></label>
4     <label>Pan X: <input type="range" :min="-panLimitX" :max="panLimitX" v-model.
        number="panX" /></label>
5     <label>Pan Y: <input type="range" :min="-panLimitY" :max="panLimitY" v-model.
        number="panY" /></label>
6     <button @click="resetView" class="reset-btn">Reset</button>
7     <div ref="containerRef"></div>
8   </div>
9 </template>

```

Es werden die Rechtecke aus der Treemap-Komponente ersetzt, die Rechteckdaten aber als Daten als Kreissegmente interpretiert.

```

10 <script setup lang="ts">
11 import { ref, onMounted, watch, computed } from 'vue';
12 import * as d3 from 'd3';
13
14 interface FlareNode {
15   name: string;
16   size?: number;
17   children?: FlareNode[];
18 }
19
20 type HierarchyNode = d3.HierarchyRectangularNode<FlareNode>;
21 const width = 900;
22 const height = 900;
23 const radius = Math.min(width, height) / 2;
24 const color = d3.scaleOrdinal(d3.schemeCategory10);
25 const containerRef = ref<HTMLElement | null>(null);

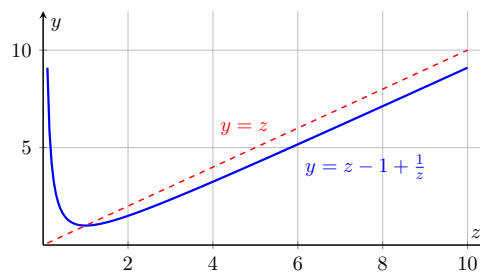
```

Zoom und Pan werden mit Hilfe von Slidern gesteuert. Der Zoom-Level wird auf einen Wert zwischen 0.5 und 5 begrenzt, um eine Überzoomung zu vermeiden. Die Pan-Limits werden dynamisch in Abhängigkeit vom Zoom-Level berechnet, um sicherzustellen, dass die Ansicht nicht über die Grenzen des Diagramms hinaus verschoben werden kann. Die Pan-Limits sind so definiert, dass sie den Radius des Diagramms berücksichtigen und verhindern, dass der Benutzer außerhalb des sichtbaren Bereichs scrollt.

```

26 const zoomLevel = ref(1);
27 const maxZoom = 5;
28 const panX = ref(0);
29 const panY = ref(0);
30 const panLimitX = computed(() => radius *
31   ((zoomLevel.value - 1) + 1/zoomLevel.value));
32 const panLimitY = computed(() => radius *
33   ((zoomLevel.value - 1) + 1/zoomLevel.value));

```



Der Pan-Limit für die X- und Y-Achse wird dynamisch in Abhängigkeit vom Zoom-Level z berechnet.

Es werden die benötigten Variablen definiert ohne diese zu initialisieren. Diese werden später im Skript verwendet, um das SVG-Element, den Arc-Generator, die Hierarchie und die Pfade für die Segmente des Sunburst-Diagramms zu erstellen. Die Variablen sind so definiert, dass sie den Typ der D3.js-Elemente und der Hierarchie-Knoten widerspiegeln, um die Typensicherheit in TypeScript zu gewährleisten.

```

34 | let svg: d3.Selection<SVGSVGElement, unknown, null, undefined>;
35 | let arc: d3.Arc<any, HierarchyNode>;
36 | let root: d3.HierarchyNode<FlareNode>;
37 | let partitioned: HierarchyNode
38 | let path: d3.Selection<SVGPathElement, HierarchyNode, SVGSVGElement, unknown>;
39 | let g: d3.Selection<SVGSVGElement, unknown, null, undefined>;

```

Die Funktion `updateSunburst` aktualisiert die Darstellung des Sunburst-Diagramms, wenn sich der Zoom-Level ändert. Sie passt den inneren und äußeren Radius der Segmente an und aktualisiert die Position und Größe der Textbeschriftungen entsprechend dem neuen Zoom-Level. Die Funktion wird aufgerufen, wenn sich der Zoom-Level ändert oder die Ansicht zurückgesetzt wird.

```

40 | function updateSunburst() {
41 |   if (!svg || !partitioned) return;
42 |   const k = zoomLevel.value;
43 |   const minFontSize = 10;
44 |   const maxFontSize = 28;
45 |   arc.innerRadius(d => d.y0 * k).outerRadius(d => d.y1 * k);
46 |   svg.selectAll('g.sunburst-arcs path')
47 |     .transition()
48 |     .duration(400)
49 |     .attr('d', arc as any);

```

Die Textbeschriftungen der Segmente werden ebenfalls aktualisiert, um die Lesbarkeit zu gewährleisten. Die Schriftgröße wird basierend auf dem Zoom-Level skaliert, um sicherzustellen, dass die Beschriftungen bei unterschiedlichen Zoom-Stufen gut lesbar sind. Die Position der Beschriftungen wird so angepasst, dass sie immer im Zentrum des jeweiligen Segments angezeigt werden.

```

50 |   svg.selectAll('g.sunburst-arcs text')
51 |     .transition()
52 |     .duration(400)
53 |     .attr('transform', d => {
54 |       const datum = d as d3.HierarchyRectangularNode<FlareNode>;
55 |       const angle = ((datum.x0 + datum.x1) / 2) * 180 / Math.PI - 90;
56 |       const r = (datum.y0 + datum.y1) / 2 * k;
57 |       return `rotate(${angle}) translate(${r},0) rotate(${angle > 90 ? 180 : 0})`;
58 |     })
59 |     .attr('font-size', d => {
60 |       const datum = d as d3.HierarchyRectangularNode<FlareNode>;
61 |       const base = 12;
62 |       const scaled = base * k;
63 |       return Math.max(minFontSize, Math.min(maxFontSize, scaled));
64 |     });
65 | }

```

Die Funktion `resetView` setzt den Zoom-Level und die Pan-Positionen auf ihre Standardwerte zurück. Diese Funktion wird aufgerufen, wenn der Benutzer den Reset-Button klickt. Zusätzlich wird ein Watcher eingerichtet, der die Pan-Positionen zurücksetzt, wenn sich der Zoom-Level ändert, um sicherzustellen, dass die Ansicht immer zentriert bleibt.

```

66 function resetView() {
67   zoomLevel.value = 1;
68   panX.value = 0;
69   panY.value = 0;
70 }
71
72 watch(zoomLevel, () => {
73   panX.value = 0;
74   panY.value = 0;
75 });

```

Die Daten werden eingelesen und mit Hilfe der Funktion `d3.partition` in eine Hierarchie umgewandelt. Diese Hierarchie wird dann partitioniert, um die Segmente des Sunburst-Diagramms zu erstellen. Die Partitionierung erfolgt so, dass die Größe der Segmente proportional zur Größe der Datenpunkte ist. Der Arc-Generator wird verwendet, um die Pfade für die Segmente zu erstellen, basierend auf den Start- und Endwinkeln sowie den inneren und äußeren Radien der Segmente. Die x-Koordinaten der Rechtecke werden als Winkel im Bogenmaß interpretiert, und die y-Koordinaten werden als Radien interpretiert, die den Abstand vom Zentrum des Diagramms angeben.

```

76 onMounted(async () => {
77   const filename = '/data/flare.json';
78   const data = await d3.json(filename) as FlareNode;
79   root = d3.hierarchy(data)
80     .sum(d => d.size || 0)
81     .sort((a, b) => (b.value || 0) - (a.value || 0));
82
83   partitioned = d3.partition<FlareNode>()
84     .size([2 * Math.PI, radius])(root);
85
86   arc = d3.arc<d3.HierarchyRectangularNode<FlareNode>>()
87     .startAngle(d => d.x0)
88     .endAngle(d => d.x1)
89     .innerRadius(d => d.y0 * zoomLevel.value)
90     .outerRadius(d => d.y1 * zoomLevel.value);

```

Die SVG-Elemente werden erstellt und die Segmente des Sunburst-Diagramms werden gezeichnet. Die Segmente werden mit Farben gefüllt, die auf den Namen der Knoten basieren, um eine visuelle Unterscheidung zu ermöglichen. Die Pfade der Segmente werden mit Titeln versehen, die den Namen und den Wert des jeweiligen Knotens anzeigen. Diese Titel erscheinen als Tooltips, wenn der Benutzer mit der Maus über die Segmente fährt.

```

91   svg = d3.select(containerRef.value)
92     .append('svg')
93     .attr('width', width)
94     .attr('height', height)
95     .attr('viewBox', `0 0 ${width} ${height}`)
96     .attr('style', 'background: #fff;');
97
98   g = svg.append('g')
99     .attr('class', 'sunburst-arcs')
100     .attr('transform', `translate(${width / 2},${height / 2})`);
101
102   path = g.selectAll('path')
103     .data(partitioned.descendants())
104     .join('path')

```

```

104     .attr('fill', d => color(
105         d.ancestors()
106         .map(d => d.data.name)
107         .join('/')
108     )
109 )
110 .attr('stroke', '#fff')
111 .attr('d', arc)
112 .append('title')
113 .text(d => `${d.data.name}\n${d.value}`)
114 .select(function() {
115     return this.parentNode as SVGPathElement;
116 });

117 // Add labels for large enough arcs
118 g.selectAll('text')
119   .data(partitioned.descendants().filter(d => (d.x1-d.x0) > 0.03))
120   .join('text')
121   .attr('transform', d => {
122       const angle = ((d.x0 + d.x1) / 2) * 180 / Math.PI - 90;
123       const r = (d.y0 + d.y1) / 2 * zoomLevel.value;
124       return `rotate(${angle}) translate(${r},0) rotate(${angle > 90 ? 180 : 0})`;
125   })
126   .attr('dx', '-10')
127   .attr('dy', '.35em')
128   .attr('font-size', 12)
129   .attr('fill', '#333')
130   .attr('text-anchor', 'middle')
131   .text(d => d.data.name);
132
133 panX.value = 0;
134 panY.value = 0;
135 });

```

Die Pan- und Zoom-Funktionalität wird mit Hilfe von Watchern implementiert, die auf Änderungen der entsprechenden Variablen reagieren. Wenn sich der Zoom-Level oder die Pan-Positionen ändern, wird die Transformation des SVG-Gruppenelements aktualisiert, um die neue Ansicht anzuzeigen. Dadurch wird das Sunburst-Diagramm entsprechend dem aktuellen Zoom- und Pan-Zustand neu gezeichnet.

```

136 watch([zoomLevel, panX, panY], () => {
137     if (g) {
138         g.attr('transform', `translate(${width / 2 + Number(panX.value)}, ${height / 2
139             + Number(panY.value)})`);
140     }
141     updateSunburst();
142 });
</script>

```

Zuletzt werden noch die CSS-Stile für die Sunburst-Komponente definiert. Diese Stile sorgen dafür, dass das Diagramm ansprechend aussieht und die Benutzeroberfläche benutzerfreundlich ist. Die Stile umfassen Einstellungen für den Container, die SVG-Elemente, die Beschriftungen und den Reset-Button.

```

143 <style scoped>
144 .sunburst-container {
145   width: 100%;
146   max-width: 900px;
147   margin: 0 auto;
148   background: #fff;
149   border-radius: 8px;
150   box-shadow: 0 2px 4px rgba(0,0,0,.07);
151   padding: 1rem;
152   overflow-x: auto;
153 }
154 svg {
155   width: 100%;
156   height: auto;
157   display: block;
158 }
159 label {
160   display: block;
161   margin-bottom: 1rem;
162   font-size: 1.1rem;
163 }
164 input[type="range"] {
165   width: 200px;
166   vertical-align: middle;
167 }
168 .reset-btn {
169   margin-bottom: 1rem;
170   margin-left: 1rem;
171   padding: 0.5rem 1.2rem;
172   font-size: 1rem;
173   border-radius: 6px;
174   border: 1px solid #bbb;
175   background: #f8f8f8;
176   cursor: pointer;
177 }
178 </style>

```

11.8. Darstellung von Landkarten

11.8.1. Einbindung

Es wird eine Komponente `MapSelector` zur Auswahl verschiedener Darstellungen von Karten erstellt. Die Komponente selbst muss in `pages/index.vue` eingebunden werden. Die Komponente selbst stellt eine Reihe von Buttons zur Auswahl unterschiedlicher Komponenten bereit.

components/MapSelector.vue

Zuerst werden die Buttons für die Auswahl der Kartenprojektionen und -typen erstellt. Die Buttons sind so gestaltet, dass sie den ausgewählten Typ hervorheben.

```

1 <template>
2   <div>
3     <div class="button-group">
4       <button :class="{selected: selected === 'projections'}"
5         @click="selected = 'projections'">Projections</button>
6       <button :class="{selected: selected === 'choropleth'}"
7         @click="selected = 'choropleth'">Choropleth</button>
8       <button :class="{selected: selected === 'bubblemap'}"
9         @click="selected = 'bubblemap'">BubbleMap</button>
10      <button :class="{selected: selected === 'germany'}"
11        @click="selected = 'germany'">Germany Map</button>
12    </div>

```

Je nach ausgewählter Karte wird die entsprechende Komponente geladen. Die Komponenten `Projections`, `Choropleth`, `BubbleMap` und `GermanyMap` müssen zuvor erstellt werden. Diese Komponenten sind für die Darstellung der jeweiligen Karten verantwortlich. Die Größe der Karten wird über die `width` und `height` Props gesteuert.

```

13   <div v-if="selected === 'projections'">
14     <Projections :width="1000" :height="1000" />
15   </div>
16   <div v-else-if="selected === 'choropleth'">
17     <Choropleth :width="1000" :height="750" />
18   </div>
19   <div v-else-if="selected === 'bubblemap'">
20     <BubbleMap :width="1000" :height="750" />

```

```

21 |     </div>
22 |     <div v-else-if="selected === 'germany'">
23 |       <GermanyMap :width="900" :height="900" />
24 |     </div>
25 |   </div>
26 | </template>

```

Im Script-Teil der Komponente wird die ausgewählte Karte als reaktive Variable `selected` definiert. Diese Variable wird verwendet, um die aktuell ausgewählte Karte zu verfolgen und die entsprechende Komponente anzuzeigen. Die anderen Kartenkomponenten werden importiert, um sie in der Template-Sektion verwenden zu können.

```

27 | <script setup lang="ts">
28 | import { ref } from 'vue';
29 | import Projections from './Projections.vue';
30 | import Choropleth from './Choropleth.vue';
31 | import BubbleMap from './BubbleMap.vue';
32 | import GermanyMap from './GermanyMap.vue';
33 |
34 |
35 | const selected = ref('projections');
36 | </script>

```

Die Styles für die Buttons und die Platzhalter für die Karten werden im Style-Tag definiert. Die Buttons haben eine einfache Gestaltung mit einem Hover-Effekt, der den ausgewählten Button hervorhebt. Der Platzhalter für die Karten wird angezeigt, wenn keine Karte ausgewählt ist.

```

37 | <style scoped>
38 | .button-group {
39 |   margin-bottom: 1rem;
40 | }
41 | .button-group button {
42 |   margin-right: 0.5rem;
43 |   padding: 0.5rem 1rem;
44 |   font-size: 1rem;
45 |   cursor: pointer;
46 | }
47 | .button-group button.selected {
48 |   background: #1976d2;
49 |   color: #fff;
50 |   border: 1px solid #1976d2;
51 | }
52 | .placeholder {
53 |   padding: 2rem;
54 |   color: #888;
55 |   background: #f8f8f8;
56 |   border: 1px dashed #ccc;
57 |   border-radius: 6px;
58 |   text-align: center;
59 | }
60 | </style>

```

11.8.2. Projektionen

Die Komponente `Projections` ermöglicht die Darstellung von Weltkarten in verschiedenen Projektionen. Die Nutzer können zwischen verschiedenen Projektionen wählen, um die Karte entsprechend anzupassen. Die verfügbaren Projektionen sind: Natural Earth, Mercator, Orthographic (Globus), Equirectangular und Azimuthal Equal Area.

Mit Ausnahme der orthografischen Projektion kann mit den Karten per Zoom und Pan interagiert werden. Bei der orthografischen Projektion kann die Karte durch Ziehen mit der Maus gedreht werden, um verschiedene Ansichten zu erhalten. In allen Karten wird das mit der Maus überfahrene Land hervorgehoben und der Name des Landes in einem Tooltip angezeigt.



Die Kartendaten werden aus einer TopoJSON-Datei geladen, die die Geometrien der Länder enthält. Diese Daten werden mit D3.js verarbeitet, um die Karte entsprechend der ausgewählten Projektion darzustellen. Die Karte wird in einem SVG-Element gerendert, das in einem Container platziert ist. Die Karte enthält eine Graticule, die die Längs- und Breitengrade anzeigt, sowie die Ländergrenzen. Die Länder werden in verschiedenen Farben dargestellt, und beim Überfahren mit der Maus wird der Name des Landes in einem Tooltip angezeigt. Die TopoJSON-Daten können von <https://github.com/topojson/world-atlas> heruntergeladen werden.

orthografische Projektion

Die Projektionen können aus einem Dropdown-Menü ausgewählt werden. Die Auswahl wird in der reaktiven Variable `selectedProjection` gespeichert. Die Karte wird entsprechend der ausgewählten Projektion aktualisiert, wenn sich die Größe des Containers ändert oder eine neue Projektion ausgewählt wird. Der angezeigte Tooltip wird in einem separaten `div`-Element angezeigt, das außerhalb des SVG-Elements positioniert ist, um eine bessere Sichtbarkeit zu gewährleisten.

```

1 <template>
2   <div class="projection-container">
3     <label>Projection:
4       <select v-model="selectedProjection">
5         <option v-for="proj in projections" :key="proj.value"
6           :value="proj.value">{{ proj.label }}</option>
7       </select>
8     </label>
9     <div ref="containerRef"></div>
10    <div ref="tooltipRef" class="country-tooltip" style="position: absolute;
11      pointer-events: none; display: none;"></div>
12  </div>
13 </template>

```

D3 kann nicht direkt mit TopoJSON arbeiten, daher wird die TopoJSON-Bibliothek verwendet. Diese wird mit

```

1 | bun add topojson-client
2 | bun add -D @types/topojson-client

```

installiert.

```

13 | <script setup lang="ts">
14 | import { ref, onMounted, watch, nextTick } from 'vue';
15 | import * as d3 from 'd3';
16 | import { feature } from 'topojson-client';
17 |
18 | const props = defineProps<{ width: number; height: number }>();
19 | const containerRef = ref<HTMLElement | null>(null);
20 | const tooltipRef = ref<HTMLElement | null>(null);
21 | const selectedProjection = ref('geoNaturalEarth1');

```

Es folgt eine Liste der verfügbaren Projektionen. Diese Liste wird im Dropdown-Menü angezeigt, um die Auswahl der Projektion zu erleichtern. Jede Projektion hat einen Wert und eine Bezeichnung, die im Menü angezeigt wird.

```

22 | const projections = [
23 |   { value: 'geoNaturalEarth1', label: 'Natural Earth' },
24 |   { value: 'geoMercator', label: 'Mercator' },
25 |   { value: 'geoOrthographic', label: 'Orthographic (Globe)' },
26 |   { value: 'geoEquirectangular', label: 'Equirectangular' },
27 |   { value: 'geoAzimuthalEqualArea', label: 'Azimuthal Equal Area' },
28 | ];

```

Es werden mehrere Variablen für die D3-Auswahl, die SVG-Elemente und die Projektion definiert. Diese Variablen werden später verwendet, um die Karte zu rendern und die Interaktivität zu ermöglichen. Die `currentRotation` Variable wird für die orthografische Projektion verwendet, um die Drehung der Karte zu steuern.

Die Variable werden mit den zugehörigen Typen deklariert, um die Typensicherheit zu gewährleisten.

```

29 | let svg: d3.Selection<SVGSVGElement, unknown, null, undefined>;
30 | let g: d3.Selection<SVGGElement, unknown, null, undefined>;
31 | let path: d3.GeoPath<any, d3.GeoPermissibleObjects>;
32 | let projection: d3.GeoProjection;
33 | let worldData: GeoJSON.FeatureCollection<GeoJSON.Geometry,
34 |   GeoJSON.GeoJsonProperties>;
35 | let currentRotation: [number, number, number] = [0, 0, 0];

```

Die vorhandenen Projektionen werden mit einer switch-Anweisung ausgewählt. Jede Projektion wird mit der D3-Funktion für die jeweilige Projektion erstellt und an die Größe des Containers angepasst. Die Projektionen werden so konfiguriert, dass die Erdkugel auf sie projiziert wird. Jede Projektionsmethode verursacht ihr eigene Artefakte. Die orthografische Projektion wird mit einer speziellen Konfiguration versehen, um die Drehung der Karte zu ermöglichen.

```

36 | function getProjection() {
37 |   const { width, height } = props;
38 |   switch (selectedProjection.value) {
39 |     case 'geoNaturalEarth1':
40 |       return d3.geoNaturalEarth1().fitSize(
41 |         [width, height], {type: 'Sphere'});
42 |
43 |     case 'geoMercator':
44 |       return d3.geoMercator().fitSize(
45 |         [width, height], {type: 'Sphere'});
46 |     case 'geoOrthographic':
47 |       return d3.geoOrthographic().fitExtent(
48 |         [[10, 10], [width-10, height-10]],
49 |         {type: 'Sphere'})
50 |         .clipAngle(90).rotate(currentRotation);
51 |     case 'geoEquirectangular':
52 |       return d3.geoEquirectangular().fitSize(
53 |         [width, height], {type: 'Sphere'});
54 |     case 'geoAzimuthalEqualArea':
55 |       return d3.geoAzimuthalEqualArea().fitSize(
56 |         [width, height], {type: 'Sphere'});
57 |     default:
58 |       return d3.geoNaturalEarth1().fitSize(
59 |         [width, height], {type: 'Sphere'});
60 |   }

```

In der Funktion `render` wird die Karte gerendert. Zuerst wird überprüft, ob die `worldData` geladen ist. Wenn nicht, wird die Funktion beendet. Andernfalls wird ein neues SVG-Element erstellt und in den Container eingefügt. Die Größe des SVG-Elements wird auf die in den Props angegebenen Werte gesetzt. Anschließend wird eine Gruppe (`g`) für die Karte erstellt und die Projektion sowie der Pfad für die Geometrien werden initialisiert.

```

61 function render() {
62   if (!worldData) return;
63   d3.select(containerRef.value).selectAll('svg').remove();
64   svg = d3.select(containerRef.value)
65     .append('svg')
66     .attr('width', props.width)
67     .attr('height', props.height)
68     .attr('viewBox', `0 0 ${props.width} ${props.height}`)
69     .attr('style', 'background: #e6f2ff;');
70
71   g = svg.append('g');
72   projection = getProjection();
73   path = d3.geoPath().projection(projection);

```

Nun werden die Längen- und Breitengrade (Graticule) gezeichnet. Die Graticule wird mit der D3-Funktion `geoGraticule10` erstellt, die eine feine Unterteilung der Längs- und Breitengrade bietet. Diese Graticule wird als Pfad in die Gruppe `g` eingefügt. Die Graticule wird mit einer hellgrauen Farbe und einer dünnen Linie dargestellt.

```

74 // Draw graticule
75 g.append('path')
76   .datum(d3.geoGraticule10())
77   .attr('fill', 'none')
78   .attr('stroke', '#bbb')
79   .attr('stroke-width', 1)
80   .attr('d', path as any);

```

Es werden die Länder der Welt gezeichnet. Die Länder werden aus den geladenen GeoJSON-Daten extrahiert und als Pfade in die Gruppe `g` eingefügt. Jedes Land wird mit einer hellblauen Farbe gefüllt und hat eine dünne schwarze Umrandung.

```

81 // Draw land
82 g.append('g')
83   .selectAll('path')
84   .data(worldData.features)
85   .join('path')
86   .attr('fill', '#b3cde0')
87   .attr('stroke', '#333')
88   .attr('stroke-width', 0.7)
89   .attr('d', path as any)

```

Die Länder werden mit einem Mouseover-Ereignis versehen, das den Namen des Landes in einem Tooltip anzeigt. Der Tooltip wird angezeigt, wenn die Maus über ein Land bewegt wird, und verschwindet, wenn die Maus das Land verlässt. Die Farbe des Landes ändert sich ebenfalls, um anzuzeigen, dass es ausgewählt wurde.

```

90 .on('mouseover', function(event, d) {
91   const name = d.properties && (d.properties.name ||
92     d.properties.NAME || d.properties.admin || '');
93   if (tooltipRef.value) {
94     tooltipRef.value.style.display = 'block';
95     tooltipRef.value.textContent = name;
96   }
97   d3.select(this).attr('fill', '#f7b267');
98 })
99 .on('mousemove', function(event) {
100   if (tooltipRef.value) {
101     tooltipRef.value.style.left = (event.pageX + 10) + 'px';
102     tooltipRef.value.style.top = (event.pageY - 10) + 'px';
103   }
104 })
105 .on('mouseout', function() {

```

```

106     if (tooltipRef.value) tooltipRef.value.style.display = 'none';
107     d3.select(this).attr('fill', '#b3cde0');
108   });

```

Zeichne die Umriss der Weltkugel. Dies wird durch das Hinzufügen eines Pfades mit der Form einer Kugel erreicht. Der Pfad wird mit einer dünnen schwarzen Linie gezeichnet, um die Grenzen der Kugel darzustellen. Diese Kugel dient als visuelle Referenz für die Projektion und hilft, die Perspektive der Karte zu verdeutlichen.

```

109   // Draw sphere outline
110   g.append('path')
111     .datum({type: 'Sphere'})
112     .attr('fill', 'none')
113     .attr('stroke', '#333')
114     .attr('stroke-width', 1.5)
115     .attr('d', path as any);

```

Für die orthografische Projektion wird eine spezielle Behandlung vorgenommen, um die Drehung der Karte zu ermöglichen. Hierbei wird ein Drag-Handler hinzugefügt, der es dem Nutzer ermöglicht, die Karte durch Ziehen mit der Maus zu drehen. Für andere Projektionen wird stattdessen ein Zoom-Handler verwendet, um die Interaktivität zu ermöglichen.

```

116   // Add zoom/pan/rotate
117   if (selectedProjection.value === 'geoOrthographic') {
118     let lastPos: [number, number] | null = null;
119     svg.call(d3.drag<SVGSVGElement, unknown>()
120       .on('start', (event) => {
121         lastPos = [event.x, event.y];
122       })
123       .on('drag', (event) => {
124         if (!lastPos) return;
125         const dx = event.x - lastPos[0];
126         const dy = event.y - lastPos[1];
127         lastPos = [event.x, event.y];
128         currentRotation[0] += dx / 2;
129         currentRotation[1] -= dy / 2;
130         projection.rotate(currentRotation);
131         g.selectAll('path').attr('d', path as any);
132         g.selectAll('g path').attr('d', path as any);
133       })
134     );
135   } else {
136     svg.call(d3.zoom<SVGSVGElement, unknown>()
137       .scaleExtent([1, 8])
138       .on('zoom', (event) => {
139         g.attr('transform', event.transform);
140       })
141     );
142   }
143 }

```

Die Kartendaten werden werden asynchron geladen, sobald die Komponente gemountet wird.

```

144 onMounted(async () => {
145   const topo = await d3.json('/data/countries-110m.json');
146   const fc = feature(topo as any, (topo as any).objects.countries);
147   if (fc && typeof fc === 'object' && 'features' in fc &&
148     Array.isArray((fc as any).features)) {
149     worldData = fc as unknown as GeoJSON.FeatureCollection<
150       GeoJSON.Geometry, GeoJSON.GeoJsonProperties>;
151   } else if (fc && typeof fc === 'object') {

```

```

152 |     worldData = {
153 |       type: 'FeatureCollection',
154 |       features: [fc as GeoJSON.Feature<
155 |         GeoJSON.Geometry, GeoJSON.GeoJsonProperties>]
156 |     };
157 |   } else {
158 |     worldData = { type: 'FeatureCollection', features: [] };
159 |   }
160 |   render();
161 | });

```

Die Karte wird aktualisiert, wenn sich die ausgewählte Projektion ändert oder wenn sich die Größe des Containers ändert. Dies geschieht durch die Verwendung von `watch` in Vue.js, um auf Änderungen der reaktiven Variablen zu reagieren. Wenn die orthografische Projektion ausgewählt ist, wird die aktuelle Drehung zurückgesetzt, um eine konsistente Ansicht zu gewährleisten.

```

162 | watch(selectedProjection, () => {
163 |   if (selectedProjection.value === 'geoOrthographic') {
164 |     currentRotation = [0, 0, 0];
165 |   }
166 |   render();
167 | });
168 |
169 | watch(() => [props.width, props.height], () => {
170 |   render();
171 | });
172 | </script>

```

Zuletzt werden die Styles für die Karte definiert. Diese Styles sorgen dafür, dass die Karte gut aussieht und die Interaktivität unterstützt wird.

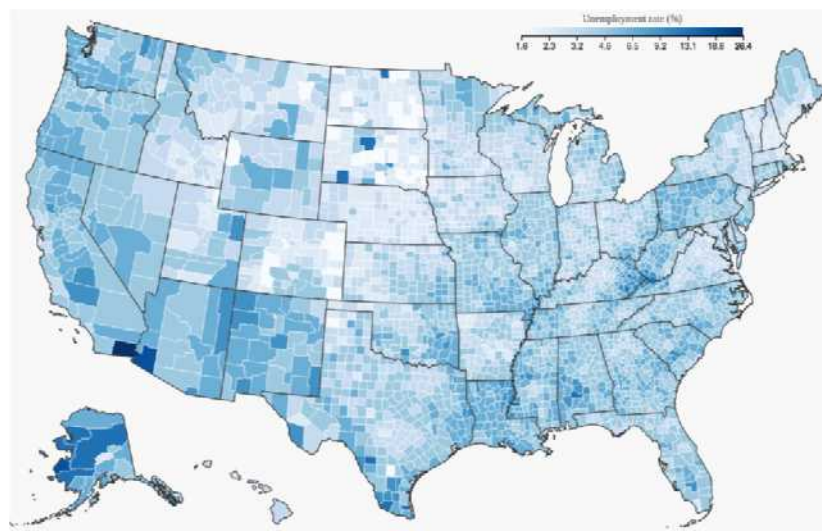
<pre> 173 <style scoped> 174 .projection-container { 175 width: 100%; 176 max-width: 100vw; 177 margin: 0 auto; 178 background: #fff; 179 border-radius: 8px; 180 box-shadow: 0 2px 4px rgba 181 (0,0,0,0.07); 182 padding: 1rem; 183 overflow-x: auto; 184 } 185 svg { 186 width: 100%; 187 height: auto; 188 display: block; </pre>	<pre> 189 select { 190 margin-left: 1rem; 191 font-size: 1rem; 192 } 193 .country-tooltip { 194 background: rgba(0,0,0,0.8); 195 color: #fff; 196 padding: 4px 8px; 197 border-radius: 4px; 198 font-size: 0.9rem; 199 pointer-events: none; 200 z-index: 10; 201 white-space: nowrap; 202 } 203 </style> </pre>
---	--

11.8.3. Choropleth-Karten

- Choropleth-Karten sind Karten, die geografische Regionen mit Farben darstellen
- Sie eignen sich gut für die Visualisierung von Verteilungen und Dichten
- Die Farben repräsentieren eine quantitative Variable
- Sie werden häufig in der Geografie und Sozialwissenschaften verwendet.

- In unserem Beispiel wird die Arbeitslosenquote in den USA im Jahr 2016 pro County dargestellt. Die Farbskala wird zur besseren Unterscheidung logarithmisch skaliert.
- Die Arbeitslosen-Daten stammen indirekt von der Bureau of Labor Statistics. Die Daten wurden direkt von observablehq.com übernommen.
- Die Landkarte der USA stammt von Github: topojson/us-atlas. Vorsicht: Beim gewählten Format wurde die AlbersUSA-Projektion bereits angewendet. Daher wird im Programm keine Projektion mehr benötigt.

Dynamische D3 Beispiele



Choropleth-Karte der USA mit Arbeitslosenquote im Jahr 2016

components/Choropleth.vue

Es wird eine speziell an die USA angepasste Albers-Projektion verwendet. Hier werden Alaska und Hawaii in der Karte links unten eingefügt.

```

1 <template>
2   <div class="choropleth-container">
3     <div ref="containerRef"></div>
4   </div>
5 </template>

6 <script setup lang="ts">
7   import { ref, onMounted } from 'vue';
8   import * as d3 from 'd3';
9   import { feature } from 'topojson-client';
10
11   const props = defineProps<{ width?: number; height?: number }>();
12   const width = props.width ?? 960;
13   const height = props.height ?? 600;
14
15   const containerRef = ref<HTMLElement | null>(null);
16
17   let svg: d3.Selection<SVGSVGElement, unknown, null, undefined>;
18   let g: d3.Selection<SVGGELEMENT, unknown, null, undefined>;

```

Es werden sowohl die Daten der Landkarte als auch die Daten der Arbeitslosenquote geladen. Die Daten der Landkarte sind im TopoJSON-Format, die Arbeitslosen-Daten im CSV-Format. Insbesondere folgen die Arbeitslosen-Daten dem FIPS-Format⁵, das eine eindeutige Identifikation von Counties in den USA ermöglicht. Die FIPS-Codes bestehen aus 5 Ziffern, wobei die ersten beiden Ziffern den Bundesstaat und die letzten drei Ziffern den County identifizieren.

```
19 onMounted(async () => {
20   const us = await d3.json('/data/counties-albers-10m.json');
21   const unemployment = await d3.csv('/data/unemployment-x.csv', d => ({
22     id: d.id,
23     state: d.state,
24     county: d.county,
25     rate: +d.rate
26   }));
27   // Map FIPS to full data object for tooltip
28   const dataById = new Map(unemployment.map(d => [d.id, d]));
```

Um die Datengrenzen für die logarithmische Farbskala zu bestimmen, werden nur die positiven Arbeitslosenquoten betrachtet. Die Skala wird in diskrete Bins unterteilt, um eine bessere visuelle Unterscheidung zu ermöglichen. Die Farben werden aus der D3-Farbpalette `schemeBlues` entnommen.

```
29 // Logarithmic color scale with discrete bins using scaleThreshold
30 const positive = unemployment.map(d => d.rate).filter(r => r>0);
31 const minRate = d3.min(positive) || 0.1;
32 const maxRate = d3.max(positive) || 1;
33 const n = d3.schemeBlues[9].length;
34 const thresholds = Array.from({length: n - 1}, (_, i) =>
35   minRate * Math.pow(maxRate / minRate, (i + 1) / (n - 1))
36 );
37 const color = d3.scaleThreshold<number, string>()
38   .domain(thresholds)
39   .range(d3.schemeBlues[9]);
40
41 if (us && typeof us === 'object' && 'objects' in us) {
42   // Log available object keys
43   // @ts-ignore
44   console.log('Available TopoJSON object keys:', Object.keys(us.objects));
45 }
```

Findet die Pfade der County-Grenzen in der TopoJSON-Datei. Diese Pfade werden später für die Darstellung der Landkarte verwendet. Die D3-Funktion `geoPath` wird ohne Projektion verwendet, da die Daten bereits in der Albers-Projektion vorliegen. `d3.geoPath` ist eine Funktion, die GeoJSON-Objekte in SVG-Pfade umwandelt. In diesem Fall wird sie verwendet, um die County-Grenzen in SVG-Pfade zu konvertieren, die dann auf der Karte gezeichnet werden.

```
46 const path = d3.geoPath(); // data is already projected
47 svg = d3.select(containerRef.value)
48   .append('svg')
49   .attr('width', width)
50   .attr('height', height)
51   .attr('viewBox', `0 0 ${width} ${height}`)
52   .attr('style', 'background: #f8f8f8;');
53 g = svg.append('g');
54 // Draw counties (choropleth fill)
55 const countiesFeature = feature(us as any, (us as any).objects.counties);
56 const countiesData = countiesFeature && 'features' in countiesFeature ?
   countiesFeature.features : [];
```

⁵Federal Information Processing Standards

Zwischendurch wird ein Tooltip-Element erstellt, das später für die Anzeige von Informationen zu den einzelnen Counties verwendet wird. Der Tooltip wird als `div`-Element erstellt und mit CSS-Stilen versehen, um eine ansprechende Darstellung zu gewährleisten. Er wird zunächst ausgeblendet und erscheint erst bei Mouseover-Ereignissen auf den Counties.

```

57 // Add tooltip div
58 const tooltip = d3.select('body').append('div')
59   .attr('class', 'choropleth-tooltip')
60   .style('position', 'absolute')
61   .style('pointer-events', 'none')
62   .style('background', 'rgba(255,255,255,0.95)')
63   .style('border', '1px solid #aaa')
64   .style('border-radius', '4px')
65   .style('padding', '6px 10px')
66   .style('font-size', '13px')
67   .style('color', '#222')
68   .style('display', 'none');

```

Die Counties werden gezeichnet, indem die Pfade der County-Grenzen aus den TopoJSON-Daten extrahiert und mit den Arbeitslosendaten verknüpft werden. Die Füllfarbe jedes County wird basierend auf der Arbeitslosenquote festgelegt. Wenn keine Daten für ein County vorhanden sind, wird eine graue Farbe verwendet.

```

69 g.append('g')
70   .selectAll('path')
71   .data(countiesData as any)
72   .join('path')
73   .attr('fill', (d: any) => {
74     const datum = dataById.get(d.id);
75     return datum && datum.rate !== null ? color(datum.rate) : '#eee';
76   })
77   .attr('stroke', '#fff')
78   .attr('stroke-width', 0.5)
79   .attr('d', path as any)

```

Die Counties werden mit Mouseover- und Mouseleave-Ereignissen versehen, um den Tooltip anzuzeigen und die Farbe des Counties zu ändern.

```

80 .on('mousemove', function(event: MouseEvent, d: any) {
81   const datum = dataById.get(d.id);
82   tooltip.style('display', 'block')
83     .html(datum
84       ? `<strong>${datum.county}, ${datum.state}</strong><br><strong>
85         Unemployment:</strong> ${datum.rate}%`
86       : `<strong>FIPS:</strong> ${d.id}<br><strong>Unemployment:</strong> N/A`);
87   .style('left', (event.pageX + 15) + 'px')
88   .style('top', (event.pageY - 28) + 'px');
89   d3.select(this).attr('stroke', '#222').attr('stroke-width', 1.5);
90 })
91 .on('mouseleave', function() {
92   tooltip.style('display', 'none');
93   d3.select(this).attr('stroke', '#fff').attr('stroke-width', 0.5);
94 });

```

Die Umrisse der Bundesstaaten werden ebenfalls gezeichnet, um die Grenzen zwischen diesen hervorzuheben. Dies geschieht durch das Zeichnen von Pfaden für die Bundesstaaten, die aus den TopoJSON-Daten extrahiert werden. Die Umrisse werden in einer grauen Farbe dargestellt, um sie von den Counties abzuheben.

```

94 // Draw state outlines
95 const statesFeature = feature(us as any, (us as any).objects.states);

```

```

96 | g.append('path')
97 |   .datum(statesFeature)
98 |   .attr('fill', 'none')
99 |   .attr('stroke', '#333')
100 |   .attr('stroke-width', 1.2)
101 |   .attr('pointer-events', 'none')
102 |   .attr('d', path as any);

```

Zoom und Pan wird initiiert, um die Karte interaktiv zu machen. Danach wird eine Legende hinzugefügt.

```

103 | // Zoom and pan
104 | svg.call(d3.zoom<SVGSVGElement, unknown>()
105 |   .scaleExtent([1, 8])
106 |   .on('zoom', (event) => {
107 |     g.attr('transform', event.transform);
108 |   })
109 | );
110 | // Add legend
111 | const legendWidth = 260;
112 | const legendHeight = 10;
113 | const legendMargin = 30;
114 | const legendSvg = svg.append('g')
115 |   .attr('transform', `translate(600,20)`);
116 |
117 | const legendScale = d3.scaleLog()
118 |   .domain([minRate, maxRate])
119 |   .range([0, legendWidth]);

```

Die Legende enthält eine Farbskala im Bereich der auftretenden Arbeitslosenquoten.

```

120 | // Gradient
121 | const defs = svg.append('defs');
122 | const linearGradient = defs.append('linearGradient')
123 |   .attr('id', 'legend-gradient');
124 | linearGradient.selectAll('stop')
125 |   .data(color.range().map((c, i, arr) => ({
126 |     offset: `${(i / (arr.length - 1)) * 100}%`,
127 |     color: c
128 |   })))
129 |   .join('stop')
130 |   .attr('offset', d => d.offset)
131 |   .attr('stop-color', d => d.color);
132 |
133 | legendSvg.append('rect')
134 |   .attr('width', legendWidth)
135 |   .attr('height', legendHeight)
136 |   .style('fill', 'url(#legend-gradient)');
137 |
138 | // Legend axis for thresholds
139 | const legendAxis = d3.axisBottom(legendScale)
140 |   .tickValues([minRate, ...thresholds, maxRate])
141 |   .tickFormat(d3.format('.1f'));
142 | legendSvg.append('g')
143 |   .attr('transform', `translate(0,${legendHeight})`)
144 |   .call(legendAxis);
145 |
146 | legendSvg.append('text')
147 |   .attr('x', legendWidth / 2)
148 |   .attr('y', -6)
149 |   .attr('text-anchor', 'middle')
150 |   .attr('font-size', 12)
151 |   .attr('fill', '#333');

```

```
151 |     .text('Unemployment rate (%)');  
152 | });  
153 | </script>
```

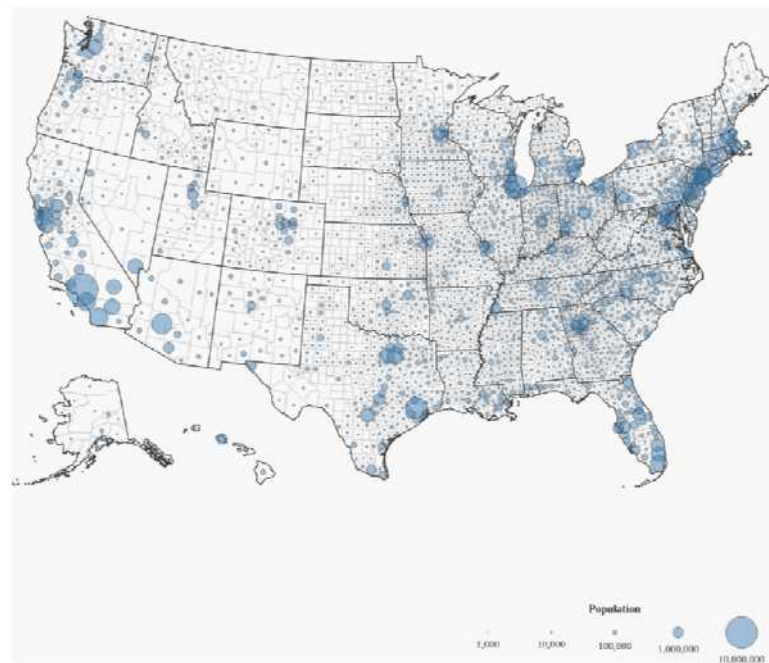
Zuletzt wird das CSS für den Choropleth-Container und die SVG-Elemente definiert.

```
154 | <style scoped>  
155 | .choropleth-container {  
156 |     width: 100%;  
157 |     max-width: 960px;  
158 |     margin: 0 auto;  
159 |     background: #fff;  
160 |     border-radius: 8px;  
161 |     box-shadow: 0 2px 4px rgba(0,0,0,0.07);  
162 |     padding: 1rem;  
163 |     overflow-x: auto;  
164 | }  
165 | svg {  
166 |     width: 100%;  
167 |     height: auto;  
168 |     display: block;  
169 | }  
170 | </style>
```

11.8.4. Bubble-Maps

- Bubble-Maps sind Karten, die Datenpunkte als Blasen darstellen
- Die Größe der Blasen repräsentiert eine quantitative Variable
- Sie eignen sich gut für die Visualisierung von Verteilungen und Dichten
- Hier wird eine Bubble-Map für die Bevölkerungszahl in den Counties der USA erstellt. Nicht für alle Counties sind Daten vorhanden.
- Die Blasen sind auf der Karte im Zentroid des Counties positioniert und ihre Fläche ist proportional zur Bevölkerungszahl.

Dynamische D3 Beispiele



Bubble-Map der Bevölkerungszahlen in den USA pro County

components/BubbleMap.vue

Die Komponente `BubbleMap` wird in der Datei `src/components/BubbleMap.vue` erstellt. Sie verwendet `D3.js`, um eine interaktive Karte mit Kreisen als Maß für die Bevölkerungszahl zu erstellen.

```

1 <template>
2   <div class="bubblemap-container">
3     <div ref="containerRef"></div>
4   </div>
5 </template>

```

Zuerst werden die benötigten Abhängigkeiten importiert und die grundlegenden Eigenschaften der Karte definiert.

```

6 <script setup lang="ts">
7 import { ref, onMounted } from 'vue';
8 import * as d3 from 'd3';
9 import { feature } from 'topojson-client';
10
11 const props = defineProps<{ width?: number; height?: number }>();
12 const width = props.width ?? 960;
13 const height = props.height ?? 600;
14
15 const containerRef = ref<HTMLElement | null>(null);
16
17 let svg: d3.Selection<SVGSVGElement, unknown, null, undefined>;
18 let g: d3.Selection<SVGGELEMENT, unknown, null, undefined>;

```

Nun werden die beiden Datenquellen geladen: die GeoJSON-Daten für die Counties und die Bevölkerungsdaten. Die Bevölkerungsdaten werden in eine Map umgewandelt, um den Zugriff zu erleichtern.

```

19 | onMounted(async () => {
20 |   const us = await d3.json('/data/counties-albers-10m.json');
21 |   const popRaw = await d3.json('/data/us_population2016.json');
22 |   const popData = (popRaw as any[]).slice(1); // Skip header row
23 |   // Build FIPS and population map
24 |   const popByFips = new Map<string, any>(popData.map((d: any[]) => {
25 |     const state = d[2].padStart(2, '0'); //state-id is 2 digits
26 |     const county = d[3].padStart(3, '0'); //county-id is 3 digits
27 |     const fips = state + county;
28 |     return [fips, {
29 |       population: +d[0],
30 |       name: d[1],
31 |       state: d[2],
32 |       county: d[3]
33 |     }]);
34 | }));

```

Erzeuge SVG und darin enthaltene Gruppe. Interpretiere die GeoJSON-Daten für die Counties.

```

35 |   // Use geoPath with no projection because data is already projected
36 |   const path = d3.geoPath();
37 |
38 |   svg = d3.select(containerRef.value)
39 |     .append('svg')
40 |     .attr('width', width)
41 |     .attr('height', height)
42 |     .attr('viewBox', `0 0 ${width} ${height}`)
43 |     .attr('style', 'background: #f8f8f8;');
44 |
45 |   g = svg.append('g');
46 |
47 |   // Draw counties (outline only)
48 |   const countiesFeature = feature(us as any, (us as any).objects.counties);
49 |   const countiesData = countiesFeature && 'features' in countiesFeature ?
     countiesFeature.features : [];

```

Ignoriere Counties ohne FIPS oder Bevölkerungszahl. Dann zeichne die Umrisse der Counties.

```

50 |   const missingOrZero = (countiesData as any[]).filter(d => {
51 |     const pop = popByFips.get(d.id);
52 |     return !pop || pop.population === 0;
53 |   });
54 |   if (missingOrZero.length > 0) {
55 |     console.log('Counties with non-matching FIPS or 0 population:', missingOrZero.
56 |       map(d => d.id));
57 |   }
58 |
59 |   g.append('g')
60 |     .selectAll('path')
61 |     .data(countiesData as any)
62 |     .join('path')
63 |     .attr('fill', 'none')
64 |     .attr('stroke', '#bbb')
65 |     .attr('stroke-width', 0.5)
66 |     .attr('d', path as any);

```

Identifiziere die Bundesstaaten und zeichne deren Umrisse. Dann erstelle eine Skala für die Blasengröße basierend auf den Bevölkerungszahlen.

```

66 |   // Draw state outlines
67 |   const statesFeature = feature(us as any, (us as any).objects.states);
68 |   g.append('path')
69 |     .datum(statesFeature)

```

```

70     .attr('fill', 'none')
71     .attr('stroke', '#333')
72     .attr('stroke-width', 1.2)
73     .attr('pointer-events', 'none')
74     .attr('d', path as any);
75 // Bubble size scale
76 const populations = Array.from(popByFips.values()).map((d: any) => d.population)
77 ;
78 const minPop = d3.min(populations) || 1;
79 const maxPop = d3.max(populations) || 1;
80 const rScale = d3.scaleSqrt()
81   .domain([minPop, maxPop])
82   .range([1, 20]);

```

Erzeuge ein Tooltip-Element, das beim Überfahren der Blasen angezeigt wird. Das Tooltip zeigt den County-Namen und die Bevölkerungszahl an.

```

82 // Tooltip
83 const tooltip = d3.select('body').append('div')
84   .attr('class', 'bubble-tooltip')
85   .style('position', 'absolute')
86   .style('pointer-events', 'none')
87   .style('background', 'rgba(255,255,255,0.95)')
88   .style('border', '1px solid #aaa')
89   .style('border-radius', '4px')
90   .style('padding', '6px 10px')
91   .style('font-size', '13px')
92   .style('color', '#222')
93   .style('display', 'none');

```

Zeichne die Kreise, die die Bevölkerungszahlen repräsentieren. Die Kreise werden im Zentroid des jeweiligen Counties positioniert und ihre Größe wird durch die Bevölkerungszahl skaliert.

```

94 // Draw bubbles
95 g.append('g')
96   .selectAll('circle')
97   .data(countiesData as any)
98   .join('circle')
99   .attr('cx', (d: any) => path.centroid(d)[0])
100  .attr('cy', (d: any) => path.centroid(d)[1])
101  .attr('r', (d: any) => {
102    const pop = popByFips.get(d.id) as any;
103    return pop ? rScale(pop.population) : 0;
104  })
105  .attr('fill', 'steelblue')
106  .attr('fill-opacity', 0.5)
107  .attr('stroke', '#222')
108  .attr('stroke-width', 0.5)

```

Installiere die Ereignis-Handler für die Mausbewegungen über den Kreisen. Beim Überfahren eines Kreises wird das Tooltip angezeigt und der Kreis hervorgehoben.

```

109 .on('mousemove', function(event: MouseEvent, d: any) {
110   const pop = popByFips.get(d.id) as any;
111   if (pop) {
112     tooltip.style('display', 'block')
113     .html(`<strong>${pop.name}</strong><br>Population: ${pop.population.
114       toLocaleString()}`)
115     .style('left', (event.pageX + 15) + 'px')
116     .style('top', (event.pageY - 28) + 'px');
117   }
118   d3.select(this).attr('stroke', '#000').attr('stroke-width', 1.5);
119 })

```

```

119 | .on('mouseleave', function() {
120 |     tooltip.style('display', 'none');
121 |     d3.select(this).attr('stroke', '#222').attr('stroke-width', .5);
122 | });

```

Es wird noch eine Legende für die Kreisgrößen hinzugefügt, die die Bevölkerungszahlen in verschiedenen Größenordnungen darstellt. Die SVG-Höhe wird angepasst, um Platz für die Legende zu schaffen.

```

123 | // Increase SVG height to fit legend
124 | const legendHeight = 80;
125 | svg.attr('height', height + legendHeight)
126 |     .attr('viewBox', `0 0 ${width} ${height + legendHeight}`);
127 |
128 | // Legend for bubble sizes: powers of 10
129 | const legendValues: number[] = [
130 |     1_000, 10_000, 100_000, 1_000_000, 10_000_000];
131 | const legendX: number = width / 2 + 100;
132 | const legendY: number = height + 40;
133 | const legendSpacing: number = 80;
134 | const legend = svg.append('g')
135 |     .attr('class', 'bubble-legend')
136 |     .attr('transform', `translate(${legendX}, ${legendY})`);
137 |
138 | legend.selectAll('circle')
139 |     .data(legendValues)
140 |     .join('circle')
141 |     .attr('cx', (_, i: number) => i * legendSpacing)
142 |     .attr('cy', 0)
143 |     .attr('r', (d: number) => rScale(d))
144 |     .attr('fill', 'steelblue')
145 |     .attr('fill-opacity', 0.5)
146 |     .attr('stroke', '#222')
147 |     .attr('stroke-width', 0.5);
148 |
149 | legend.selectAll('text')
150 |     .data(legendValues)
151 |     .join('text')
152 |     .attr('x', (_, i: number) => i * legendSpacing)
153 |     .attr('y', (d: number) => rScale(d) + 18)
154 |     .attr('text-anchor', 'middle')
155 |     .attr('font-size', 13)
156 |     .attr('fill', '#222')
157 |     .text((d: number) => d3.format(',')(d));
158 |
159 | legend.append('text')
160 |     .attr('x', legendSpacing * 2)
161 |     .attr('y', -25)
162 |     .attr('text-anchor', 'middle')
163 |     .attr('font-size', 14)
164 |     .attr('font-weight', 'bold')
165 |     .attr('fill', '#222')
166 |     .text('Population');

```

Erlaube Zoom und Pan auf der Karte. Die Karte kann gezoomt und verschoben werden, um verschiedene Bereiche zu betrachten. Die Karte kann maximal auf das 8-fache vergrößert werden.

```

165 | // Zoom and pan
166 | svg.call(d3.zoom<SVGSVGElement, unknown>()
167 |     .scaleExtent([1, 8])
168 |     .on('zoom', (event) => {
169 |         g.attr('transform', event.transform);

```

```

170 |     })
171 |   );
172 | });
173 | </script>

```

Die Formatierung des HTML-Templates und der CSS-Stile für die Bubble-Map-Komponente wird im `style`-Block definiert. Die Karte wird in einem Container mit einem weißen Hintergrund und abgerundeten Ecken dargestellt. Die SVG-Größe passt sich an die Containergröße an.

```

174 | <style scoped>
175 | .bubblemap-container {
176 |   width: 100%;
177 |   max-width: 960px;
178 |   margin: 0 auto;
179 |   background: #fff;
180 |   border-radius: 8px;
181 |   box-shadow: 0 2px 4px rgba(0,0,0,0.07);
182 |   padding: 1rem;
183 |   overflow-x: auto;
184 | }
185 | svg {
186 |   width: 100%;
187 |   height: auto;
188 |   display: block;
189 | }
190 | </style>

```

11.8.5. Deutsche Landkreise

Karten sind ein wichtiges Werkzeug, um geographische Daten zu visualisieren und zu analysieren. Dabei sollte jedoch beachtet werden, dass Karten schnell veralten können. Die Landkreis-Daten der vorgestellten Karte sind bereits einige Jahre alt und enthalten nicht aktuelle Änderungen, wie z.B. die Bildung des Landkreises Harz. Es gibt GeoJSON-Daten neueren Datums, jedoch wird für diese eine Recherche benötigt um die enthaltenen Marker zu verstehen und korrekt darzustellen.

- Die Karte zeigt die Landkreise Deutschlands
- Die Karte kann als Grundlage für weitere Analysen dienen
- Die Daten stammen von <https://github.com/isellsoap/deutschlandGeoJSON>
- Die Landkreisdaten sind aber bereits etwas älter (z.B. gibt es noch keinen Landkreis Harz)
- Neuere Daten stehen z.B. auf <https://opendatalab.de> zur Verfügung, erfordern aber mehr Recherche zur Nutzung.

Dynamische D3 Beispiele



Deutschlandkarte mit Landkreisen

components/GermanyMap.vue

Für die Deutschlandkarte genügt eine einfache Vue-Komponente.

```

1 <template>
2   <div class="germany-map-container">
3     <div ref="containerRef"></div>
4   </div>
5 </template>

```

Die dynamischen Teile der Komponente werden im Skript-Teil definiert. Hierbei wird TypeScript verwendet, um die Typen der Eigenschaften zu definieren.

```

6 <script setup lang="ts">
7 import { ref, onMounted } from 'vue';
8 import * as d3 from 'd3';
9
10 const props = defineProps<{ width?: number; height?: number }>();
11 const width = props.width ?? 800;
12 const height = props.height ?? 900;
13
14 const containerRef = ref<HTMLElement | null>(null);
15
16 let svg: d3.Selection<SVGSVGElement, unknown, null, undefined>;
17 let g: d3.Selection<SVGGELElement, unknown, null, undefined>;

```

Es werden die GeoJSON-Daten der Landkreise geladen und eine Mercator-Projektion erstellt, die die Karte an die Größe des Containers anpasst. Die Karte wird in einem SVG-Element gezeichnet.

```

18 onMounted(async () => {
19   const geo = await d3.json('/data/3_mittel.geo.json');
20   const projection = d3.geoMercator()
21     .fitSize([width, height], geo as any);
22   const path = d3.geoPath().projection(projection);
23
24   svg = d3.select(containerRef.value)
25     .append('svg')
26     .attr('width', width)
27     .attr('height', height)
28     .attr('viewBox', `0 0 ${width} ${height}`)
29     .attr('style', 'background: #f8f8f8;');
30
31   g = svg.append('g');

```

Im Tooltip werden Informationen zu den Landkreisen angezeigt, wenn der Mauszeiger über einen Landkreis bewegt wird. Der Tooltip wird als div-Element erstellt und mit CSS gestylt.

```

32 // Tooltip
33 const tooltip = d3.select('body').append('div')
34   .attr('class', 'germany-tooltip')
35   .style('position', 'absolute')
36   .style('pointer-events', 'none')
37   .style('background', 'rgba(255,255,255,0.95)')
38   .style('border', '1px solid #aaa')
39   .style('border-radius', '4px')
40   .style('padding', '6px 10px')
41   .style('font-size', '13px')
42   .style('color', '#222')
43   .style('display', 'none');

```

Es werden die aus GeoJSON ausgewählten Pfade gezeichnet. Dabei wird die Farbe der Landkreise auf ein helles Blau gesetzt und die Ränder auf einen dunkleren Grauton.

```

44 g.selectAll('path')
45   .data((geo as any).features)
46   .join('path')
47   .attr('d', path as any)

```

```

48 | .attr('fill', '#90caf9')
49 | .attr('stroke', '#333')
50 | .attr('stroke-width', 0.5)
51 |
52 | .on('mousemove', function(event, d) {
53 |   tooltip.style('left', (event.pageX + 15) + 'px')
54 |   .style('top', (event.pageY - 28) + 'px');
55 | })

```

Der Tooltip wird bei entsprechenden Maus-Ereignissen angezeigt. Hierbei wird die Farbe des Landkreises bei Mouseover geändert und der Tooltip mit den entsprechenden Informationen gefüllt.

```

56 | .on('mouseover', function (event, d) {
57 |   d3.select(this).attr('fill', '#1976d2');
58 |   const featureProps = (d as any).properties;
59 |   tooltip.style('display', 'block')
60 |     .html(
61 |       '<strong>Landkreis:</strong> ${featureProps.NAME_3}<br>' +
62 |       '<strong>Regierungsbezirk:</strong> ${featureProps.NAME_2}<br>' +
63 |       '<strong>Bundesland:</strong> ${featureProps.NAME_1}`
64 |     )
65 |     .style('left', (event.pageX + 15) + 'px')
66 |     .style('top', (event.pageY - 28) + 'px');
67 | })
68 | .on('mouseout', function () {
69 |   d3.select(this).attr('fill', '#90caf9');
70 |   tooltip.style('display', 'none');
71 | });

```

Die Zoom-Funktionalität wird mit D3's Zoom-API hinzugefügt. Diese ermöglicht es, die Karte zu zoomen und zu verschieben.

```

72 | svg.call(d3.zoom<SVGSVGElement, unknown>()
73 |   .scaleExtent([1, 8])
74 |   .on('zoom', (event) => {
75 |     g.attr('transform', event.transform);
76 |   })
77 | );
78 | });
79 | </script>

```

Zuletzt noch die Foramtierung des SVG-Elements und des Karten-Containers.

```

80 | <style scoped>
81 | .germany-map-container {
82 |   width: 100%;
83 |   max-width: 900px;
84 |   margin: 0 auto;
85 |   background: #fff;
86 |   border-radius: 8px;
87 |   box-shadow: 0 2px 4px rgba(0,0,0,0.07);
88 |   padding: 1rem;
89 |   overflow-x: auto;
90 | }
91 | svg {
92 |   width: 100%;
93 |   height: auto;
94 |   display: block;
95 | }
96 | </style>

```

Literatur

- [1] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 1. Auflage. Rheinwerk Computing. mit allen Beispielen zum Download. Bonn: Rheinwerk Verlag GmbH, 2016. 1229 Seiten. ISBN: 9783836238380.
- [2] Philip Ackermann. *Professionell entwickeln mit JavaScript. Design, Patterns, Praxistipps*. 1. Auflage. Rheinwerk Computing. Titelzusatz auf dem Umschlag: "Für Client und Server, Web und Enterprise - ideal für Umsteiger und Profis - inkl. ECMAScript 6 in der professionellen Praxis". Bonn: Rheinwerk-Verlag (vormals Galileo Press), 2015. 450 Seiten. ISBN: 9783836223799.
- [3] Katy Börner. *Atlas of knowledge. Anyone can map*. Literaturverzeichnis: Seite 178-205 und Indexes. Cambridge, Mass.: The MIT Press, 2015. xi, 211 Seiten. ISBN: 9780262028813.
- [4] Katy Börner. *Atlas of science. Visualizing what we know*. Literaturangaben. Cambridge, Mass. [u.a.]: MIT Press, 2010. XI, 254. ISBN: 9780262014458.
- [5] Katy Börner und David E. Polley. *Visual insights. A practical guide to making sense of data*. Includes bibliographical references and index. Cambridge, MA [u.a.]: MIT Press, 2014. Online-Ressource (310 pages). ISBN: 9780262320238.
- [6] Paul Bradshaw und Liisa Rohumaa. *The online journalism handbook. Skills to survive and thrive in the digital age*. @Longman practical journalism series. Includes bibliographical references and index. London [u.a.]: Routledge, 2013. Online-Ressource (XX, 203 S.) ISBN: 9781315834184.
- [7] Vicki Bruce, Patrick R. Green und Mark A. Georgeson. *Visual perception. Physiology, psychology, & ecology*. 4. ed. Rev. ed. of: Visual perception, physiology, psychology, and ecology. c1985. Hove [u.a.]: Psychology Press, 2003. XII, 483. ISBN: 1841692379.
- [8] Alberto Cairo. *The functional art. an introduction to information graphics and visualization*. Berkeley, Calif.: New Riders, 2013. XXI, 363. ISBN: 9780321834737.
- [9] Kyran Dale. *Data visualization with Python and JavaScript. Scrape, clean, explore & transform your data*. Beijing, 2016. XXXIII, 551 Seiten. ISBN: 9781491920510.
- [10] Bill Ferster. *Interactive Visualization*. MIT Press, 2013.
- [11] Stephen Few. *Information dashboard design. Displaying data for at-a-glance monitoring*. 2nd ed. Burlingame, CA: Analytics Press, 2013. VIII, 246. ISBN: 9781938377006.
- [12] Stephen Few. *Show me the numbers. Designing tables and graphs to enlighten*. 2. ed. Burlingame, CA: Analytics Press, 2012. XVIII, 351. ISBN: 9780970601971.
- [13] Ben Fry. *Visualizing data. [exploring and explaining data with the processing environment]*. 1. ed. Beijing [u.a.]: O'Reilly, 2008. XIII, 366. ISBN: 9780596514556.
- [14] Jonathan Gray u. a., Hrsg. *The data journalism handbook*. First edition. Theory in practice. "Strata: making data work"—Cover page [4]. Sebastopol, California: O'Reilly Media, 2012. Online-Ressource (242 pages). ISBN: 9781449330064.
- [15] Marijn Haverbeke. *Die Kunst der JavaScript-Programmierung. Eine moderne Einführung in die Sprache des Web*. Hrsg. von Volkmar Gronau. 1. Aufl. Aus dem Amerikan. übers. Heidelberg: dpunkt-Verl., 2012. IX, 229. ISBN: 9783898647878.
- [16] Marijn Haverbeke. *Eloquent javascript. A modern introduction to programming*. Second edition. Includes index. San Francisco, CA: No Starch Press, 2014. Online-Ressource (1 volume).
- [17] Raimar Heber. *Infografik. Gute Geschichten erzählen mit komplexen Daten*. 1. Auflage. Rheinwerk Design. Bonn: Rheinwerk Verlag GmbH, 2016. 301 Seiten. ISBN: 9783836231930.

- [18] Noah Iliinsky. *Designing Data Visualizations. Representing Informational Relationships*. Hrsg. von Julie Steele. Description based upon print version of record. Sebastopol: O'Reilly Media, Inc, 2011. Online-Ressource (112 p.) ISBN: 9781449312282.
- [19] Jay Jacobs und Bob Rudis. *Data driven security. Analysis, visualization and dashboards*. Safari Tech Books Online. Description based upon print version of record. Indianapolis, Ind.: Wiley, 2014. Online-Ressource (354 S.) ISBN: 9781118793725.
- [20] Marc Jansen und Till Adams. *OpenLayers. Webentwicklung mit dynamischen Karten und Geodaten*. München: Open Source Press, 2010. 344 S. ISBN: 9783937514925.
- [21] Joel Katz. *Designing information. Perception, human factors, and common sense*. Includes bibliographical references and index. Hoboken: Wiley, 2012. Online-Resource (224 S.) ISBN: 9781118341971.
- [22] Ritchie S. King. *Visual storytelling with D3. An introduction to data visualization in JavaScript*. Addison-Wesley data & analytics series. Includes index. Upper Saddle River, NJ [u.a.]: Addison-Wesley, 2015. XVIII, 264. ISBN: 9780321933171.
- [23] Andy Kirk. *Data visualisation. A handbook for data driven design*. First published. Literaturverzeichnis: Seiten 335-336. Los Angeles: SAGE, 2016. xiv, 347 Seiten. ISBN: 9781473912137.
- [24] Cole Nussbaumer Knaflitz. *Storytelling with data. A data visualization guide for business professionals*. Includes bibliographical references and index. Hoboken, New Jersey: Wiley, 2015. 1 Online-Ressource. ISBN: 9781119002260.
- [25] Carolyn Knight und Jessica Glaser. *Diagrams. Innovative solutions for graphic designers*. CD-ROM. Hove: RotoVision, 2009. 223 S. ISBN: 9782888930617.
- [26] Jörn Kohlhammer, Dirk U. Proff und Andreas Wiener. *Visual Business Analytics. Effektiver Zugang zu Daten und Informationen*. 1. Aufl. Edition TDWI. Includes bibliographical references and index. Heidelberg: dpunkt-Verl, 2013. Online-Ressource (XIII, 218 S.) ISBN: 9783864900440.
- [27] Christoph Körner. *Data Visualization with D3 and AngularJS*. Community Experience Distilled. Description based upon print version of record. Birmingham: Packt Publishing, 2015. Online-Ressource (278 p). ISBN: 9781784398484.
- [28] Dimitrios Kouzis-Loukas. *Learning Scrapy. Learn the art of efficient web scraping and crawling with Python*. Community experience distilled. Includes index. Birmingham, UK: Packt Publishing, 2016. 1 Online-Ressource (1 volume). ISBN: 9781784390914.
- [29] Richard Lawson. *Web Scraping with Python. Scrape data from any website with the power of Python*. Community experience distilled. Includes index. Birmingham: Packt Publishing, 2015. 1 Online-Ressource.
- [30] Manuel Lima. *Book of Trees. Visualizing Branches of Knowledge*. Description based upon print version of record. New York: Princeton Architectural Press, 2014. Online-Ressource (208 p). ISBN: 9781616892180.
- [31] Manuel Lima. *Visual complexity. Mapping patterns of information*. 1st ed. Includes bibliographical references and index. New York: Princeton Architectural Press, 2011. 272 p. ISBN: 9781568989365.
- [32] David MacCandless. *The visual miscellaneum. A colorful guide to the world's most consequential trivia*. New York, NY [u.a.]: Harper Design, 2009. 251 S. ISBN: 9780061748363.
- [33] Connie Malamed. *Visual language for designers. Principles for creating graphics that people understand*. Includes bibliographical references (p. 228). Beverly, Mass: Rockport Publishers, 2011. Online-Ressource (240 p). ISBN: 9781592535156.

- [34] David McCandless. *Information is beautiful*. [New ed., revised, recalculated and reimagined]. 1. publ. in 2009. London: Collins, 2012. 255 S. ISBN: 9780007492893.
- [35] David McCandless. *Knowledge is beautiful*. London: Collins, 2014. 255 S. ISBN: 9780007427925.
- [36] Elijah Meeks. *D3.js in action*. Shelter Island, NY: Manning, 2015. XVIII, 334 Seiten. ISBN: 9781617292118.
- [37] Isabel Meirelles. *Design for information. An introduction to the histories, theories, and best practices behind effective information visualizations*. Includes bibliographical references and index. Beverly, MA: Rockport Publishers, 2013. Online-Ressource (224 S.) ISBN: 9781592538065.
- [38] Igor Milovanovic. *Python data visualization cookbook. Over 70 recipes, based on the principal concepts of data visualization, to get you started with popular Python libraries*. Hrsg. von Dimitry Foures und Giuseppe Vettigli. Second edition. Quick answers to common problems. Birmingham: Packt Publishing, 2015. 1 Online Ressource (IX, 281 Seiten). ISBN: 9781784394943.
- [39] Ryan Mitchell. *Web scraping with Python. Collecting data from the modern web*. Includes index. Sebastopol, CA: O'Reilly, 2015. Online-Ressource (XIII, 238 S.)
- [40] Tamara Munzner. *Visualization analysis & design*. Hrsg. von Éamonn Maguire. A K Peters Visualization Series. Includes bibliographical references. Boca Raton, Florida: CRC Press, 2015. Online-Ressource (422 pages).
- [41] Scott Murray. *Interactive Data Visualization for the Web*. Safari Tech Books Online. Sebastopol, Calif.: O'Reilly, 2013. Online-Ressource (272 S.)
- [42] Markus Nix. *Visual simplicity. Die Darstellung großer Datenmengen*. Frankfurt am Main: entwickler.press, 2013. 240 S. ISBN: 9783868020991.
- [43] Jon Raasch, Graham Murray und Vadim Ogievetsky. *JavaScript and jQuery for data analysis and visualization*. Hrsg. von Joseph W. Lowery. Includes index. Indianapolis: Wrox, 2015. Online-Ressource (1 volume). ISBN: 9781118847060.
- [44] Thomas Rahlf. *Datendesign mit R. 100 Visualisierungsbeispiele*. 1. Aufl. Professional reference. Literaturverz. S. 415 - 418. München: Open Source Press, 2014. 426 S. ISBN: 9783955390945.
- [45] Cyrille Rossant. *IPython interactive computing and visualization cookbook. Over 100 hands-on recipes to sharpen your skills in high-performance numerical computing and data science with Python. "Quick answers to common problems". - Includes index*. Birmingham, UK: Packt Pub, 2014. Online-Ressource (512 S.) ISBN: 9781783284825.
- [46] Lindy Ryan. *The Visual Imperative. Creating a Visual Culture of Data Discovery*. 1. Aufl. Description based upon print version of record. [s.l.]: Elsevier Reference Monographs, 2016. 1 Online-Ressource (320 S.) ISBN: 0128038446.
- [47] Timothy Samara. *Design elements. A graphic style manual*. Numbering on alternate pages. Gloucester, Mass: Rockport Publishers, 2007. Online-Ressource (272 p). ISBN: 9781592532612.
- [48] Robert Spence. *Information Visualization. An Introduction*. 3rd ed. 2014. Cham: Springer International Publishing, 2014. Online-Ressource (XXI, 321 p. 328 illus., 214 illus. in color). ISBN: 9783319073415.
- [49] Julia Steele und Noah P. N. Iliinsky. *Beautiful visualization*. Safari Tech Books Online. Parallel als Druckausg. erschienen. Sebastopol, Calif.: O'Reilly, 2010. Online-Ressource (414 S.) ISBN: 9781449379865.
- [50] Stephen A. Thomas. *Data visualization with JavaScript*. Includes index. San Francisco, CA: No Starch Press, 2015. Online-Ressource (1 volume).

- [51] Sandro Tosi. *Matplotlib for Python Developers*. Packt Publishing, 2009.
- [52] Edward R. Tufte. *Beautiful evidence*. Literaturangaben. Cheshire, Conn.: Graphics Press LLC, 2007. 213 S. ISBN: 9780961392178.
- [53] Edward R. Tufte. *The visual display of quantitative information*. Cheshire, Conn.: Graphics Pr., 1984. 197 S.
- [54] Edward R. Tufte. *Visual explanations. Images and quantities, evidence and narrative*. 6. print., with revisions. Cheshire, Conn.: Graphics Press, 2003. 156 S. ISBN: 0961392126.
- [55] Edward Rolf Tufte. *Envisioning information*. 12. printing. Cheshire, Conn.: Graphics Press, 2008. 126 S. ISBN: 9780961392116.
- [56] Shai Vaingast. *Beginning Python Visualization*. Apress, 2009.
- [57] Shai Vaingast. *Beginning Python Visualization. Crafting Visual Transformation Scripts*. Second Edition. The expert's voice in Python. Includes bibliographical references and index. Berkeley, CA: Apress, 2014. Online-Ressource (XXI, 383 p. 76 illus). ISBN: 9781484200520.
- [58] Matthew O. Ward. *Interactive Data Visualization. Foundations, Techniques, and Applications*. Hrsg. von Georges Grinstein und Daniel Keim. Second edition. Description based upon print version of record. Boca Raton: CRC Press, 2015. 1 Online-Ressource (549 Seiten). ISBN: 9781482257380.
- [59] Colin Ware. *Information Visualization*. 3rd ed. Morgan Kaufmann, 2013.
- [60] Colin Ware. *Information visualization. Perception for design*. 2. ed., [Nachdr.] The @Morgan Kaufmann series in interactive technologies. Literaturverz. S. 405 - 449. Amsterdam: Elsevier [u.a.], 2009. XXVI, 486. ISBN: 9781558608191.
- [61] Colin Ware. *Visual Thinking for Design*. Morgan Kaufmann, 2008.
- [62] Hadley Wickham. *ggplot2. Elegant graphics for data analysis*. Use R! Cham: Springer, 2016. 1 Online-Ressource (XVI, 260 Seiten, 232 Illustrationen, 140 Illustrationen in Farbe). ISBN: 9783319242774.
- [63] Dona M. Wong. *The Wall Street Journal guide to information graphics. The dos and don'ts of presenting data, facts, and figures*. 1. ed. Includes index. New York [u.a.]: W.W. Norton & Co., 2010. 157 S. ISBN: 9780393072952.
- [64] Nathan Yau. *Data points. Visualization that means something*. Includes bibliographical references and index. Indianapolis, Ind.: Wiley, 2013. XIII, 300. ISBN: 9781118462195.
- [65] Nathan Yau. *Visualize This*. Wiley, 2011.
- [66] Nick Qi Zhu. *Data visualization with D3.js cookbook. Over 70 recipes to create dynamic data-driven visualization with D3.js*. Description based upon print version of record. Birmingham [u.a.]: Packt Publ., 2013. Online-Ressource. ISBN: 9781306055253.

Anhang

A. Scalable Vector Graphics

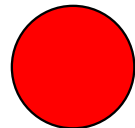
Scalable Vector Graphics (SVG) ist ein XML-basiertes Format zur Darstellung von zweidimensionalen Vektorgrafiken für das Web. SVGs sind besonders nützlich, da sie verlustfrei skalierbar sind und sich hervorragend mit HTML und CSS integrieren lassen.

SVG ist ein W3C-Standard und wird von allen gängigen Browsern unterstützt. Es bietet eine Vielzahl von Funktionen, darunter die Möglichkeit, Grafiken mit JavaScript zu animieren und mit CSS zu gestalten. SVG-Grafiken können auch in HTML-Dokumente eingebettet und mit CSS und JavaScript manipuliert werden.

A.1. Grundstruktur von SVG

Ein SVG-Dokument besteht aus Elementen, die grafische Objekte wie Formen, Linien und Texte darstellen. Die Grundstruktur eines SVG-Dokuments beginnt mit einem `<svg>` Tag, das die Zeichenfläche definiert:

```
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <circle cx="100" cy="100" r="80" stroke="black" stroke-width="3"
3     " fill="red" />
4 </svg>
```

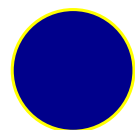


In diesem Beispiel wird ein roter Kreis mit einem schwarzen Rand auf einem 200x200 Pixel großen Zeichenbereich gezeichnet. Die Attribute `cx`, `cy` und `r` des `<circle>` Elements definieren die Position und den Radius des Kreises.

A.2. Gestaltung mit CSS

SVG erlaubt die Verwendung von CSS, um das Aussehen der Grafikelemente zu gestalten. Farben, Linienstärken und andere Stilattribute können direkt im SVG oder in einer externen CSS-Datei spezifiziert werden:

```
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2 <style>
3   circle {
4     fill: darkblue;
5     stroke: yellow;
6     stroke-width: 4;
7   }
8 </style>
9   <circle cx="100" cy="100" r="80" />
10 </svg>
```



Hier sorgt das CSS dafür, dass der Kreis blau gefüllt ist und eine schwarze Linie mit einer Breite von 2 Pixeln hat.

Ebenso kann Text gestaltet werden:

```

1 <svg width="300" height="200">
2   <style>
3     .title {
4       font-family: Arial, sans-serif;
5       font-size: 48px;
6       font-weight: bold;
7       fill: #3498db;
8       text-anchor: middle;
9     }
10
11    .subtitle {
12      font-family: "Times New Roman", serif;
13      font-size: 32px;
14      font-style: italic;
15      fill: #34495e;
16      text-anchor: middle;
17    }
18
19    text:hover {
20      fill: #e74c3c;
21      text-decoration: underline;
22      cursor: pointer;
23    }
24  </style>
25
26  <text x="100" y="80" class="title">
27    SVG Text with CSS</text>
28  <text x="100" y="120" class="subtitle">
29    Styled using CSS classes</text>
30 </svg>

```

SVG Text with CSS
Styled using CSS classes

Die hover-Pseudoklasse wird verwendet, um den Text bei Mausberührung zu verändern. Dies funktioniert in SVG genauso wie in HTML, jedoch nicht in PDFs, da SVG dort nicht dargestellt wird und die entsprechenden SVG-Darstellungen zuerst in ein anderes Format konvertiert werden müssen.

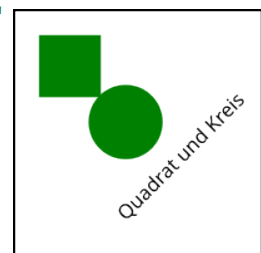
A.3. Gruppierungen und Transformationen

Um komplexere Grafiken zu erstellen, können Elemente in SVG gruppiert werden. Dies wird mit dem <g> Element erreicht, das es erlaubt, mehrere Elemente zusammenzufassen und gemeinsam zu transformieren:

```

1 <svg width="200" height="200" viewBox="-1 -1 202 202">
2   <rect x="0" y="0" width="200" height="200" fill="none" stroke="
3     black" stroke-width="2" />
4   <g fill="green" transform="translate(20,20)">
5     <rect x="0" y="0" width="50" height="50" />
6     <circle cx="70" cy="70" r="30" />
7     <text x="0" y="0" fill="black" font-size="12pt"
8       transform="translate(70,150) rotate(-45)">Quadrat und
9     Kreis</text>
10  </g>
11 </svg>

```



In diesem Beispiel werden ein Rechteck und ein Kreis in einem <g> Element gruppiert, das beide Objekte um 20 Einheiten nach rechts und unten verschiebt. Die Transformation wird durch das transform Attribut definiert.

Transformationen

Transformationen in SVG ermöglichen es, Grafikelemente zu verschieben, zu drehen, zu skalieren und zu spiegeln. Die folgenden Transformationen sind verfügbar:

- `translate(x, y)`: Verschiebt das Element um die angegebenen x- und y-Koordinaten.
- `rotate(angle, cx, cy)`: Rotiert das Element um den Winkel (in Grad) um den Punkt (cx, cy).
- `scale(sx, sy)`: Skaliert das Element um den Faktor sx in x-Richtung und sy in y-Richtung.
- `skewX(angle)` und `skewY(angle)`: Neigt das Element um den angegebenen Winkel.

Transformationen können einzeln oder in Kombination angewendet werden, um komplexe Effekte zu erzielen. Sie werden jedoch von rechts nach links angewendet, d.h. die letzte Transformation wird zuerst angewendet.

A.4. Fortgeschrittene SVG-Techniken

SVG bietet auch komplexere Features, wie Gradienten, Muster, Masken und Filter:

- Gradienten: Erlauben die Erstellung von Farbverläufen. Lineare und radiale Gradienten können verwendet werden, um visuelle Tiefe und interessante Farbverläufe zu erzeugen.
- Muster: Kacheln aus kleinen Grafiken, die über ein Element wiederholt werden.
- Masken und Filter: Dienen zur Erzeugung von Effekten wie Unschärfe, Schatten oder Farbverschiebungen.

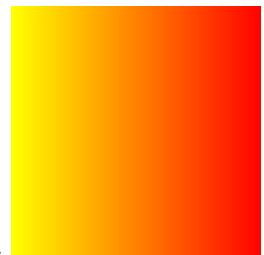
A.4.1. Lineare Gradienten

Lineare Gradienten verlaufen entlang einer geraden Linie und zeigen einen Übergang von einer Farbe zur anderen an. Ein linearer Gradient wird im SVG-Code mit dem `<linearGradient>`-Element definiert.

```

1 <svg width="200" height="200">
2   <defs>
3     <linearGradient id="grad1" x1="0%" y1="0%" x2="100%" y2="
      0%">
4       <stop offset="0%" style="stop-color:rgb(255,255,0);
      stop-opacity:1" />
5       <stop offset="100%" style="stop-color:rgb(255,0,0);
      stop-opacity:1" />
6     </linearGradient>
7   </defs>
8   <rect x="10" y="10" width="180" height="180" fill="url(#grad1
      )" />
9 </svg>

```



Dieses Beispiel zeigt einen gelb-roten Farbverlauf, der horizontal von links nach rechts verläuft. Die `'stop'`-Elemente definieren die Farben und Positionen im Verlauf. Der `'offset'`-Wert reicht von 0% bis 100% und bestimmt, wo der Übergang im Gradient auftreten soll.

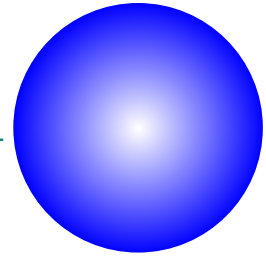
A.4.2. Radiale Gradienten

Diese breiten sich konzentrisch von einem zentralen Punkt nach außen aus. Sie werden mit dem `<radialGradient>`-Element definiert und ermöglichen ebenfalls die Angabe von Farben, die in Kreisen angeordnet sind.

```

1 <svg width="200" height="200">
2   <defs>
3     <radialGradient id="grad2" cx="50%" cy="50%" r="50%" fx="
4       50%" fy="50%">
5       <stop offset="0%" style="stop-color:rgb(255,255,255);
6         stop-opacity:1" />
7       <stop offset="100%" style="stop-color:rgb(0,0,255);stop-
8         opacity:1" />
9     </radialGradient>
10  </defs>
11  <circle cx="100" cy="100" r="90" fill="url(#grad2)" />
12 </svg>

```



Zum Erstellen eines Verlaufs definieren 'stop'-Elemente die Farben und deren Positionen im Verlauf. Die Offset-Werte reichen von 0% bis 100% und bestimmen, wo der Übergang im Gradient auftreten soll.

A.4.3. Masken

In SVG können Masken verwendet werden, um Teile von Elementen auszublenden oder freizulegen, wodurch interessante visuelle Effekte entstehen. Eine Maske wird mit dem <mask> Element definiert und kann auf andere Elemente angewendet werden. Hier ist ein Beispiel, das zeigt, wie eine Maske in SVG verwendet werden kann:

```

1 <svg width="200" height="200">
2   <!-- Definition der Maske -->
3   <defs>
4     <mask id="circle-mask">
5       <!-- Weiße Kreis ist sichtbar -->
6       <rect x="0" y="0" width="100%" height="100%" fill="black" /
7       >
8       <circle cx="100" cy="100" r="60" fill="white" />
9     </mask>
10  </defs>
11  <g>
12    <!-- Rechteck ohne Maske -->
13    <rect x="0" y="0" width="100" height="100" fill="blue" />
14    <!-- Rechteck mit Maske -->
15    <rect x="0" y="0" width="100" height="100" fill="blue" mask="
16      url(#circle-mask)" transform="translate(0, 120)" />
17  </g>
18 </svg>

```



- Das <defs> Element enthält die Definition der Maske, die für andere SVG-Elemente verwendet werden kann.
- Die Maske, die von id="circle-mask" identifiziert wird, hat einen schwarzen Hintergrund (<rect> mit fill="black"), der alles ausblendet, und einen weißen Kreis (<circle> mit fill="white"), der den sichtbaren Bereich definiert.
- Schwarz macht Bereiche in der Maske unsichtbar.
- Weiß macht Bereiche in der Maske sichtbar.
- Ein blaues Rechteck wird gezeichnet, das die gleiche Größe wie der gesamte SVG-Bereich hat.
- Das mask="url(#circle-mask)" Attribut auf dem Rechteck wendet die definierte Maske auf das Rechteck an, sodass nur der Bereich innerhalb des weißen Kreises sichtbar bleibt.

Mit Masken in SVG kann man also gezielt Teile eines Elements ausblenden oder freilegen, was nützlich ist, um komplexe visuelle Effekte und interessante Grafiken zu erstellen.

A.4.4. Filter

```

1 <svg width="200" height="200">
2   <defs>
3     <!-- Definition des Filters -->
4     <filter id="blurFilter" x="0" y="0">
5       <feGaussianBlur in="SourceGraphic" stdDeviation="5" />
6     </filter>
7   </defs>
8
9   <rect x="10" y="10" width="180" height="180" fill="blue" />
10  <!-- Wende Filters an -->
11  <rect x="10" y="10" width="180" height="180" fill="blue" filter
12    ="url(#blurFilter)" transform="translate(0,200)"/>
</svg>

```



1. Filter-Definition: Der `<filter>`-Block innerhalb des `<defs>`-Tags definiert einen Filter mit einer ID (`blurFilter`). Innerhalb dieses Blocks wird der `<feGaussianBlur>`-Primitive verwendet, um den Weichzeichnungseffekt zu erzeugen. Der `stdDeviation`-Wert legt die Stärke der Unschärfe fest.
2. Verwendung des Filters: Das `<rect>`-Element repräsentiert ein Rechteck. Durch das `filter`-Attribut wird der zuvor definierte `blurFilter` auf das Rechteck angewendet.

Filter in SVG ermöglichen es, visuelle Effekte wie Unschärfe, Schatten und Farbverschiebungen auf Grafikelemente anzuwenden. Diese Techniken können verwendet werden, um Grafiken und Diagramme visuell ansprechender zu gestalten und interessante Effekte zu erzielen.

A.5. Erstellung von Linienzügen

A.5.1. Polyline

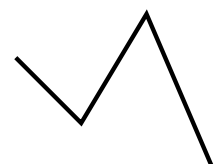
In SVG können komplexe Formen durch Polylines dargestellt werden. Diese Elemente bieten mehr Flexibilität als einfache geometrische Formen, da sie komplexe Pfade und Linienverläufe ermöglichen.

Das `<polyline>` Element wird verwendet, um eine durchgehende Linie aus mehreren Punkten zu erstellen. Es verbindet die angegebenen Punkte in der angegebenen Reihenfolge:

```

1 <svg width="200" height="200">
2   <polyline points="10,30 40,60 70,10 100,80"
3     fill="none"
4     stroke="black"
5     stroke-width="2" />
6 </svg>

```



In diesem Beispiel verbindet die Polyline die Punkte (10, 30), (40, 60), (70, 10) und (100, 80) miteinander. Die Attribute `fill=none` sorgen dafür, dass der Innenbereich nicht gefüllt wird, was für Linienzüge üblich ist. `stroke` bestimmt die Farbe der Linie und `stroke-width` die Breite.

A.5.2. Pfade

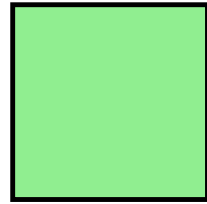
Ein `<path>` Element in SVG verwendet das `d`-Attribut, um komplexe Formen und Linien durch eine Reihe von Zeichenbefehlen zu definieren. Diese Befehle beschreiben Bewegungen, Linien, Kurven

und andere Aspekte der Grafik, die gezeichnet werden soll. Hier ist ein einfaches Beispiel für ein `<path>` Element:

```

1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <path d="M10 10
3           L90 10
4           L90 90
5           L10 90
6           Z"
7       fill="lightgreen"
8       stroke="black"
9       stroke-width="2" />
10 </svg>

```



In diesem Beispiel zeigt das `<path>` Element, wie man ein Quadrat zeichnet:

- `M10 10` ist der Move-To-Befehl (M) und legt den Anfangspunkt des Pfades auf die Koordinate (10, 10) fest.
- `L90 10` ist ein Line-To-Befehl (L), der eine Linie vom aktuellen Punkt (10, 10) zu (90, 10) zeichnet.
- `L90 90` fügt eine Linie von (90, 10) nach (90, 90) hinzu.
- `L10 90` verbindet die Punkte von (90, 90) nach (10, 90), um die dritte Seite des Quadrats zu definieren.
- `Z` schließt den Pfad zurück zum Startpunkt (10, 10), um das Quadrat zu vervollständigen.

Diese Pfaddefinition beschreibt die vier Seiten des Quadrats durch eine Reihe von geraden Linien. Das Quadrat ist mit einer hellgrünen Farbe gefüllt und hat einen schwarzen Rahmen mit einer Breite von 2 Pixeln. Das `<path>` Element ist besonders flexibel und kann für viel komplexere und detaillierte Formen als in diesem einfachen Beispiel verwendet werden.

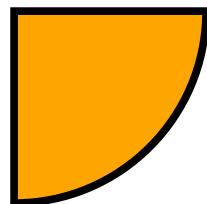
A.5.3. Kreissegment

Ein Kreissegment kann in SVG mit dem `<path>` Element dargestellt werden. Dazu verwendet man den Befehl für Kreisbögen, `A`, um einen Bogen zu zeichnen, und kombiniert diesen mit Linien, um einen geschlossenen Bereich zu formen.

```

1 <svg width="200" height="200">
2   <path d="M100,100
3           L 150,100
4           A 50,50 0 0,1 100,150
5           Z"
6       fill="orange"
7       stroke="black"
8       stroke-width="2" />
9 </svg>

```



- `M100,100` bewegt den Startpunkt zum Zentrum des Kreises.
- `L150,100` zieht eine Linie von diesem Punkt zum Rand des Kreises.
- `A 50,50 0 0,1 100,150` definiert einen Bogen mit einem Radius von 50, der sich zum Punkt (100,150) erstreckt.
- `Z` schließt den Pfad, indem es eine Linie zurück zum Startpunkt (100,100) zieht.

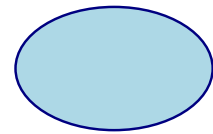
A.5.4. Ellipse

Eine Ellipse kann direkt mit dem `<ellipse>` Element in SVG gezeichnet werden. Dieses Element erfordert die Definition des Mittelpunkts sowie die Radien in x- und y-Richtung.

```

1 <svg width="200" height="200">
2   <ellipse cx="100" cy="100" rx="80" ry="50"
3           fill="lightblue"
4           stroke="navy"
5           stroke-width="2" />
6 </svg>

```



- `cx="100"` und `cy="100"` legen den Mittelpunkt der Ellipse fest.
- `rx="80"` gibt den horizontalen Radius an, während `ry="50"` den vertikalen Radius definiert.
- Die Ellipse wird mit einem hellblauen Farbton gefüllt und von einem navyblauen Rand umgeben.

A.5.5. Kreisbögen mit Arc-Anweisungen

Um eine Arc-Anweisung in SVG mit dem `<path>` Element zu verwenden, wird der A-Befehl innerhalb des `d` Attributes genutzt. Die Syntax für den Befehl A ist wie folgt: `A rx ry x-axis-rotation large-arc-flag sweep-flag x y` Hierbei sind die Parameter wie folgt definiert:

- `rx` und `ry` sind die x- und y-Radien des Bogens.
- `x-axis-rotation` ist die Rotation des Bogens um die x-Achse (in Grad).
- `large-arc-flag` bestimmt, ob der längere (1) oder der kürzere Bogen (0) gezeichnet wird.
- `sweep-flag` bestimmt die Richtung des Bogens: im Uhrzeigersinn (1) oder gegen den Uhrzeigersinn (0).
- `x` und `y` sind die absoluten Koordinaten des Endpunktes des Bogens.
- Der Startpunkt des Bogens wird durch den vorherigen Pfadpunkt definiert.

Hier ist ein Beispiel, das einen Bogen von einem Punkt zu einem anderen zeichnet:

```

1 <svg width="200" height="200">
2   <path d="M50,50
3         A 50,50 0 0,1 150,50"
4         fill="none"
5         stroke="black"
6         stroke-width="2" />
7 </svg>

```



- `M50,50` bewegt den Startpunkt des Pfads zum Punkt (50,50).
- `A 50,50 0 0,1 150,50` zeichnet einen Bogen:
 - `50,50` sind die x- und y-Radien des Bogens.
 - `0` für `x-axis-rotation` bedeutet, dass es keine Achsrotation gibt.
 - `0` für `large-arc-flag` bedeutet, dass der kürzere Bogen gewählt wird.
 - `1` für `sweep-flag` bedeutet, dass der Bogen im Uhrzeigersinn gezeichnet wird.

- 150,50 sind die Koordinaten des Endpunktes des Bogens.

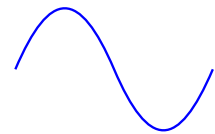
Dieser Bogen beginnt am Punkt (50,50) und endet am Punkt (150,50), wobei die Größe des Bogens durch die Radien (beide 50) und die Richtung durch die Sweep-Flag bestimmt wird. Der Bogen ist nach unten geöffnet.

A.5.6. Splines und Bézierkurven

Für geschwungene Linien werden im SVG-Kontext häufig Bézierkurven verwendet. Diese sind besonders nützlich, um sanfte, kurvige Linien zu erstellen. Das `<path>` Element ist das leistungsfähigste SVG-Element zum Zeichnen von Kurven und komplexen Formen. Es verwendet ein `d`-Attribut, das eine Reihe von Befehlen enthält.

Ein einfacher Bézierpfad kann wie folgt aussehen:

```
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <path d="M10 80 C 40 10, 65 10, 95 80 S 150 150, 180 80"
3         fill="none"
4         stroke="blue"
5         stroke-width="2" />
6 </svg>
```



In diesem Beispiel definiert das `d`-Pfadattribut eine komplexe Kurve:

- `M10 80` bewegt den "Stift" an die Startposition (10, 80).
- `C 40 10, 65 10, 95 80` erstellt eine Kubische Bézierkurve beginnend bei (10, 80) mit Kontrollpunkten bei (40, 10) und (65, 10), endend bei (95, 80).
- `S 150 150, 180 80` definiert eine fortgeführte glatte Kurve bis zum Punkt (180, 80), genutzt wird der symmetrische Kontrollpunkt zu dem vorhergehenden.

A.5.7. Quadratische Bézierkurven

SVG unterstützt auch quadratische Bézierkurven durch den `Q`-Befehl. Dieser verwendet einen einzigen Kontrollpunkt:

```
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <path d="M10 80 Q 52.5 10, 95 80 T 180 80 Z"
3         fill="none"
4         stroke="green"
5         stroke-width="2" />
6 </svg>
```



In diesem Fall wird:

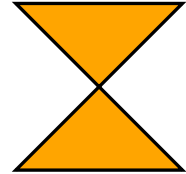
- `M10 80` den Startpunkt festgelegt.
- `Q 52.5 10, 95 80` erstellt eine Quadratische Bézierkurve mit einem Kontrollpunkt bei (52.5, 10).
- `T 180 80` definiert eine fortgeführte glatte quadratische Kurve zum Zielpunkt (180, 80) unter Nutzung des Kontrollpunktes symmetrisch zu (52.5, 10).
- `Z` schließt den Pfad.

Diese Konzepte erlauben es, detaillierte und ansprechende grafische Darstellungen in SVG zu erstellen, unterstützen die Darstellung von Datenlinien und Diagrammen und geben Ihnen die Kontrolle über die Feinheiten der Linienführung und Form.

A.5.8. Geschlossene und gefüllte Linienzüge

Linienzüge können auch geschlossen und gefüllt werden, um komplexere Formen zu erstellen. In SVG kann ein Linienzug mit dem `<polygon>` Element oder durch das Verwenden des Z-Befehls in `<path>` geschlossen werden.

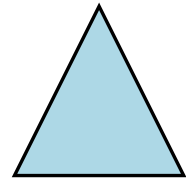
```
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <polygon points="150,50 50,150 150,150 50,50"
3     fill="orange"
4     stroke="black"
5     stroke-width="2" />
6 </svg>
```



In diesem Beispiel wird ein `<polygon>` verwendet, um ein geschlossenes Polygon zwischen den Punkten (150, 50), (50, 150), (150, 150) und (50,50) zu erstellen, das mit der Farbe Orange gefüllt und mit einer schwarzen Linie umrandet ist. Trotz Überschneidung der Linien wird der Bereich nur einmal gefüllt.

Alternativ kann ein geschlossener Linienzug auch mit dem `<path>` Element mit einem Z-Befehl erstellt werden, der den Pfad zum Ausgangspunkt zurückführt:

```
1 <svg width="200" height="200" xmlns="http://www.w3.org/2000/svg">
2   <path d="M50 150 L150 150 L100 50 Z"
3     fill="lightblue"
4     stroke="black"
5     stroke-width="2" />
6 </svg>
```



Hier wird ein Dreieck erstellt. Der Pfad beginnt bei (50, 150), geht zu (150, 150), dann zu (100, 50) und schließlich mit dem Z-Befehl zurück zum Startpunkt (50, 150). Dies füllt den erzeugten Bereich mit Hellblau.

B. Web Scraping mit Puppeteer

B.1. Einführung

B.1.1. Definition und Zweck von Web-Scraping

- Definition: Extraktion von Daten aus Webseiten
- Zweck: Automatisierung von Datenerfassung
- Anwendungsfälle: Preisvergleiche, Marktforschung, Datenvisualisierung

Visuelle Inspektion

Web Scraping ermöglicht die automatisierte Extraktion von Daten aus Webseiten. Hierzu ist es jedoch in Regel nötig die Struktur der Webseite zu kennen und die Daten zu identifizieren. Daher ist eine visuelle Inspektion des Browser-Codes der Webseite oft hilfreich.

B.1.2. Ethische und rechtliche Aspekte

Respektierung der Nutzungsbedingungen

Web Scraping sollte in Übereinstimmung mit den Nutzungsbedingungen der Website erfolgen. Daher sollte die Webseite vor dem Scraping auf entsprechende Hinweise und Einschränkungen überprüft werden.

Datenschutz und Privatsphäre

Insbesondere dürfen personenbezogene Daten nicht ohne Einwilligung der betroffenen Personen scraped werden. Es ist wichtig, die Datenschutzbestimmungen zu beachten und sicherzustellen, dass die extrahierten Daten nicht missbräuchlich verwendet werden.

robots.txt

Viele Websites haben eine robots.txt-Datei, die Anweisungen für Webcrawler enthält. Diese Datei ist normalerweise unter `example.com/robots.txt` zu finden. Sie gibt an, welche Bereiche der Website nicht gecrawlt werden sollen.

Beachte jedoch, dass `robots.txt` keine rechtliche Verpflichtung darstellt, sondern eine Richtlinie. Dennoch ist es höflich und oft erwartet, diese zu respektieren.

Begrenzung der Anfragen

Um die Server der Website nicht zu überlasten, sollte die Anzahl der Anfragen begrenzt werden. Dies kann durch Pausen zwischen den Anfragen oder durch die Verwendung von Proxies erreicht werden.

Ethische Überlegungen

Es ist wichtig, ethische Grundsätze zu beachten und sicherzustellen, dass das Scraping nicht missbräuchlich oder schädlich ist. Dies umfasst die Verwendung der Daten, die Einhaltung der Datenschutzbestimmungen und die Vermeidung von Schäden für die Website oder ihre Benutzer.

B.2. Was ist Puppeteer?

Einführung

Puppeteer ist eine Node.js-Bibliothek, die eine API zum Steuern von Headless Chrome oder Chromium-Browsern bereitstellt. Mit Puppeteer können Webseiten geöffnet, navigiert und manipuliert werden, um Daten zu extrahieren oder Aktionen auszuführen.

B.2.1. Vorteile und Unterschied

Vorteile

- Einfache und intuitive API
- Unterstützung für moderne Webtechnologien
- Automatisierung von Interaktionen und Workflows
- Integration in Node.js-Umgebung

Unterschiede zu anderen Web-Scraping-Tools

Im Vergleich zu anderen Web-Scraping-Tools, wie BeautifulSoup oder Selenium, bietet Puppeteer eine bessere Integration in die Node.js-Umgebung, eine einfachere API und eine bessere Unterstützung für moderne Webtechnologien wie JavaScript und AJAX.

Primärer Verwendungszweck

Puppeteer wurde ursprünglich für die Testautomatisierung für Web-Clients entwickelt, kann aber auch für Web-Scraping, Web-Monitoring und andere Anwendungen verwendet werden.

B.2.2. Installation und Setup

Installation von Puppeteer

Puppeteer kann über bun oder pnpm installiert werden:

```
1 | bun add puppeteer
```

B.2.3. Erste Schritte und grundlegende Konfiguration

Erstellen einer neuen Instanz von Puppeteer

Um Puppeteer zu verwenden, muss zunächst eine neue Instanz erstellt werden:

```
1 | import puppeteer from 'puppeteer';  
2 |  
3 | (async () => {  
4 |   const browser = await puppeteer.launch();  
5 |   const page = await browser.newPage();  
6 |   // Hier können Befehle ausgeführt werden  
7 |   await browser.close();  
8 | })();
```

B.2.4. Grundlagen von Puppeteer

Starten und Steuern eines Browsers

- `puppeteer.launch()`: Startet einen neuen Browser
- `browser.newPage()`: Erstellt eine neue Seite im Browser
- `browser.close()`: Schließt den Browser

Öffnen und Navigieren von Webseiten

- `page.goto(url)`: Öffnet eine Webseite
- `page.reload()`: Lädt die aktuelle Seite neu
- `page.goBack()`: Navigiert zur vorherigen Seite
- `page.goForward()`: Navigiert zur nächsten Seite

B.2.5. Web Scraping mit Puppeteer

Extrahieren von Daten aus Webseiten

- Verwendung von Selektoren (CSS, XPath)
- Extrahieren von Text, Bildern, Links, etc.
- Beispiel: Scrapen von Tabellendaten

Verwendung von Selektoren (CSS, XPath)

Puppeteer bietet verschiedene Methoden zum Suchen von Elementen auf der Seite:

- `page.$(selector)`: Sucht ein Element mit dem angegebenen CSS- oder XPath-Selektor
- `page.$$ (selector)`: Sucht alle Elemente mit dem angegebenen CSS- oder XPath-Selektor
- `page.evaluate(selector => document.querySelector(selector).innerText, selector)`
Extrahiert den Textinhalt eines Elements
- `page.evaluate(selector => document.querySelector(selector).getAttribute('src'), selector)`: Extrahiert den Wert eines Attributs
- `page.evaluate(() => Array.from(document.querySelectorAll('a')).map(a => a.href))`: Extrahiert die URLs aller Links auf der Seite

Umgang mit dynamischen Inhalten

Puppeteer kann auch mit dynamischen Inhalten umgehen, die durch JavaScript oder AJAX geladen werden. Dazu kann die `page.waitForSelector(selector)`-Methode verwendet werden, um auf das Laden eines Elements zu warten, bevor es extrahiert wird.

B.3. Datenaufbereitung für Visualisierungen

Reinigung und Strukturierung der extrahierten Daten

- Entfernen von Duplikaten und leeren Werten
- Konvertierung von Datentypen
- Aggregation und Zusammenfassung von Daten
- Filterung und Sortierung von Daten

Entfernen von Duplikaten und leeren Werten

Um die Qualität der Daten zu verbessern, sollten Duplikate und leere Werte entfernt werden. Dies kann durch Filtern der Daten oder durch Gruppierung und Aggregation erreicht werden.

Beispiel 87:

```
1 | const uniqueData = data.filter((value, index, self) => self.indexOf(value) ===  
  |   index);  
2 | const cleanData = data.filter(value => value !== '');
```

Konvertierung von Datentypen

Bei Web-Scraping erhält man üblicherweise alle Daten als Strings (Zeichenketten). Für Datenvisualisierungen müssen diese oft in numerische Werte, Datumsangaben oder boolesche Werte umgewandelt werden. Diese Konvertierung erfolgt in JavaScript nachdem die Daten extrahiert wurden.

Beispiel 88:

```
1 | const stringPrice = '€42.99'; // String zu Zahl konvertieren  
2 | const numericPrice = parseFloat(stringPrice.replace('€', ''));  
3 |  
4 | const dateString = '2024-03-10'; // String zu Datum konvertieren  
5 | const dateObject = new Date(dateString);  
6 |  
7 | const availability = productInfo.textContent.includes('In Stock'); // String zu  
  |   Boolean: true/false  
8 |  
9 | // Batch-Konvertierung von Arrays  
10 | const priceStrings = ['€10.99', '€24.50', '€5.75'];  
11 | const numericPrices = priceStrings.map(str =>  
12 |   parseFloat(str.replace('€', '').replace(',', '.'))  
13 | );
```

Aggregation und Zusammenfassung von Daten

Um die Daten für die Visualisierung vorzubereiten, können sie aggregiert und zusammengefasst werden. Dies kann durch Gruppierung, Sortierung und Berechnung von Statistiken wie Durchschnitt, Summe oder Anzahl erreicht werden.

1. Daten mit puppeteer extrahieren
2. Daten mit reduce aggregieren
3. Aggregierte Daten Zusammenfassen (Mittelwerte, Statistk, etc.)

Beispiel 89:

```
1 | import puppeteer from 'puppeteer';  
2 |  
3 | (async () => {  
4 |   const browser = await puppeteer.launch();  
5 |   const page = await browser.newPage();  
6 |   await page.goto('https://example.com/products');  
7 |  
8 |   // Extract data from the page  
9 |   const data = await page.evaluate(() => {  
10 |     return Array.from(  
11 |       document.querySelectorAll('.product')).map(product => ({
```

```
12     category: product.querySelector('.category').textContent,  
13     price: parseFloat(product.querySelector('.price')  
14         .textContent.replace('€', ''))  
15     });  
16 });  
17 await browser.close();  
18 console.log(data);  
19 const aggregatedData = data.reduce((acc, item) => {  
20     if (!acc[item.category]) {  
21         acc[item.category] = { total: 0, count: 0 };  
22     }  
23     acc[item.category].total += item.price;  
24     acc[item.category].count += 1;  
25     return acc;  
26 }, {});  
27  
28 const summarizedData = Object.keys(aggregatedData).map(category => ({  
29     category, // category: "variableninhalt",  
30     total: aggregatedData[category].total,  
31     count: aggregatedData[category].count,  
32     average: aggregatedData[category].total / aggregatedData[category].count  
33 }));  
34 console.log(summarizedData);  
35 })();
```

C. Testing

C.1. JavaScript Testing Frameworks

Testing

- Testing ist wichtig, um sicherzustellen, dass der Code korrekt funktioniert.
- Es gibt verschiedene Arten von Tests:
 - Unit-Tests: Testen einzelne Funktionen oder Module.
 - Integrationstests: Testen das Zusammenspiel mehrerer Module.
 - End-to-End-Tests: Testen die gesamte Anwendung aus der Sicht des Benutzers.
- In JavaScript gibt es viele Frameworks für das Testen:
 - Jest
 - Mocha
 - Jasmine
 - Cypress

D. Geodaten in D3.js

D3 benutzt verschiedene Formate, um geografische Daten darzustellen und zu visualisieren. Zu den gängigsten Formaten gehören GeoJSON, TopoJSON und Shapefile. Diese Formate ermöglichen es, geografische Objekte wie Punkte, Linien und Polygone zu kodieren und in D3.js-Visualisierungen zu integrieren.

D.1. GIS Dateiformate

D.1.1. GeoJSON

Das GeoJSON-Format ist ein offenes Standardformat für die Kodierung von geografischen Datenstrukturen. Es basiert auf JSON und ermöglicht die Darstellung von Punkten, Linien, Polygonen und anderen geografischen Objekten. GeoJSON wird häufig in Webanwendungen verwendet, um Karten und geografische Informationen darzustellen. D3.js bietet integrierte Unterstützung für GeoJSON, was die Visualisierung von geografischen Daten erleichtert. GeoJSON-Dateien enthalten Informationen über die Geometrie von Objekten sowie deren Attribute. Sie sind leichtgewichtig und gut für die Verwendung in Webanwendungen geeignet. GeoJSON-Dateien bestehen aus einem JSON-Objekt, das mindestens die folgenden Eigenschaften enthält:

- **type: FeatureCollection**: Gibt an, dass es sich um eine Sammlung von geografischen Objekten handelt.
- **features: []**: Ein Array von Features, die die geografischen Objekte repräsentieren.
- Jedes Feature enthält:
 - **type: Feature**: Gibt an, dass es sich um ein einzelnes geografisches Objekt handelt.
 - **geometry**: : Ein Objekt, das die Geometrie des Features beschreibt (z.B. Punkt, Linie, Polygon).
 - **properties**: : Ein Objekt, das zusätzliche Attribute des Features enthält.

Eine GeoJSON-Datei kann mit der Funktion `d3.json()` in D3.js geladen werden. Die Geometrie der Features kann dann mit der Funktion `d3.geoPath()` in SVG-Pfade umgewandelt werden, um sie auf einer Karte darzustellen. GeoJSON ist besonders nützlich für die Darstellung von geografischen Daten in Webanwendungen, da es leichtgewichtig und einfach zu handhaben ist.

```
1 d3.json("data.geojson").then(function(data) {  
2   const path = d3.geoPath();  
3   d3.select("svg")  
4     .selectAll("path")  
5     .data(data.features)  
6     .enter()  
7     .append("path")  
8     .attr("d", path)  
9     .attr("fill", "steelblue")  
10    .attr("stroke", "black");  
11 });
```

D.1.2. TopoJSON

TopoJSON ist eine Erweiterung von GeoJSON, die darauf abzielt, die Dateigröße zu reduzieren und die Effizienz der Datenübertragung zu verbessern. Es speichert nicht nur die Geometrie von geografischen Objekten, sondern auch deren Topologie, was bedeutet, dass gemeinsame Kanten

zwischen Polygonen nur einmal gespeichert werden. Dies führt zu einer geringeren Dateigröße und schnelleren Ladezeiten in Webanwendungen.

D3.js unterstützt TopoJSON nativ, was die Visualisierung von komplexen geografischen Daten erleichtert. TopoJSON-Dateien enthalten Informationen über die Geometrie von Objekten sowie deren Topologie und Attribute. Sie sind besonders nützlich für die Darstellung von Karten mit vielen Polygonen, wie z.B. Länder- oder Bundesstaatengrenzen. TopoJSON-Dateien bestehen aus einem JSON-Objekt, das mindestens die folgenden Eigenschaften enthält:

- **type: Topology**: Gibt an, dass es sich um eine TopoJSON-Datei handelt.
- **objects**: : Ein Objekt, das die verschiedenen geografischen Objekte enthält.
- **arcs**: []: Ein Array von Bögen, die die Geometrie der Objekte beschreiben.
- **transform**: : Ein Objekt, das Informationen über die Transformation der Geometrie enthält (z.B. Skalierung und Verschiebung).
- Jedes Objekt in **objects** enthält:
 - **type: "GeometryCollection"**: Gibt an, dass es sich um eine Sammlung von Geometrien handelt.
 - **geometries**: []: Ein Array von Geometrien, die die geografischen Objekte repräsentieren.
 - **properties**: : Ein Objekt, das zusätzliche Attribute des Objekts enthält.

Ein TopoJSON-Datei wird von D3 mit der Funktion `d3.json()` geladen dann mit der Funktion `d3.geoPath()` in SVG-Pfade umgewandelt werden, um sie auf einer Karte darzustellen:

```

1 d3.json("data/topojson-file.json").then(function(topoData) {
2   const geoData = topojson.feature(topoData, topoData.objects.collectionName);
3   const path = d3.geoPath();
4
5   svg.selectAll("path")
6     .data(geoData.features)
7     .enter()
8     .append("path")
9     .attr("d", path)
10    .attr("fill", "steelblue")
11    .attr("stroke", "black");
12 });

```

D.1.3. Shapefile

Shapefile ist ein weit verbreitetes Dateiformat für geografische Informationssysteme (GIS). Es besteht aus mehreren Dateien, die zusammen eine geografische Datenstruktur repräsentieren. Shapefiles enthalten Informationen über Punkte, Linien und Polygone sowie deren Attribute. Sie sind in vielen GIS-Anwendungen und -Software weit verbreitet und bieten eine robuste Möglichkeit, geografische Daten zu speichern und auszutauschen. Shapefiles bestehen in der Regel aus mehreren Dateien:

.shp: Enthält die Geometrie der geografischen Objekte.

.shx: Enthält den Index für die Geometrie, um schnellen Zugriff auf die Daten zu ermöglichen.

.dbf: Enthält die Attributdaten der geografischen Objekte in tabellarischer Form.

- .prj: Enthält die Projektion und Koordinatensysteminformationen für die Geometrie.
- .cpg: Enthält die Zeichencodierung für die Attributdaten.
- .sbx Enthält räumliche Indizes für die Geometrie, um die Abfragegeschwindigkeit zu verbessern.
- .xml: Enthält Metadaten über das Shapefile, wie z.B. die Quelle und die Erstellungszeit.
- .qix: Enthält einen räumlichen Index für die Geometrie, um die Abfragegeschwindigkeit zu verbessern.

Shapefiles werden nicht direkt in D3 unterstützt. Zuerst muss eine Umwandlung in GeoJSON oder TopoJSON erfolgen.

D.2. GDAL

GDAL (Geospatial Data Abstraction Library) ist eine Open-Source-Bibliothek, die eine Vielzahl von Geodatenformaten unterstützt. Sie ermöglicht das Lesen, Schreiben und Konvertieren von Geodaten in verschiedenen Formaten, einschließlich GeoJSON, Shapefile und vielen anderen. GDAL wird häufig in GIS-Anwendungen verwendet, um geografische Daten zu verarbeiten und zu analysieren. Insbesondere erlaubt das Kommandozeilenprogramm `ogr2ogr` die Konvertierung von Shapefiles in GeoJSON, was für die Verwendung in D3.js-Visualisierungen nützlich ist.

D.3. Umwandlung GeoJSON nach TopoJSON

Die Umwandlung von GeoJSON nach TopoJSON kann mit dem Tool `geo2topo` durchgeführt werden. Dieses Tool ermöglicht es, GeoJSON-Daten in das TopoJSON-Format zu konvertieren, um die Dateigröße zu reduzieren und die Effizienz der Datenübertragung zu verbessern. Die Umwandlung erfolgt in der Regel über die Kommandozeile mit dem Befehl:

```
1 | geo2topo input.geojson > output.topojson
```

Der Befehl steht im Projekt nach der Installation von

```
1 | bun add topojson-server topojson-client topojson-simplify
```

im Verzeichnis `node_modules/.bin/geo2topo` zur Verfügung.